

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Welcome to 6.004!



Figure by MIT OpenCourseWare.

I thought this course was called
"Computation Structures"

Handouts: Lecture Slides, Calendar, *Info sheet*

Course Mechanics

Unlike other big courses, you'll have

NO evening quizzes

NO final exam

NO weekly graded problem sets

Instead, you'll face

Repository of tutorial problems
(with answers)

FIVE quizzes, based on these problems
(in Friday sections)

EIGHT labs + on-line lab questions + Design Contest
(all labs and olqs must be completed to pass!)

ALGORITHMIC assignment of your grade!

How do you build systems with > 1 G components?



Personal Computer:
Hardware & Software



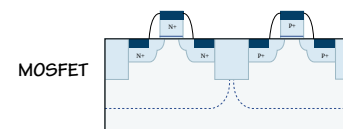
Circuit Board:
~1-8 / system
1-2G devices



Integrated Circuit:
~8-16 / PCB
.25M-1G devices



Module:
~8-64 / IC
.1M-1M devices



MOSFET

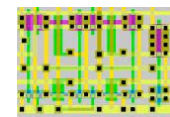
Scheme for
representing
information



Figure by MIT OpenCourseWare.



Gate:
~2-16 / Cell
8 devices



Cell:
~1K-10K / Module
16-64 devices

What do we see?

- **Structure**
 - hierarchical design:
 - limited complexity at each level
 - reusable building blocks
- **Interfaces**
 - Key elements of system engineering; typically outlive the technologies they interface
 - Isolate technologies, allow evolution
 - Major abstraction mechanism
- **What makes a good system design?**
 - “Bang for the buck”: minimal mechanism, maximal function
 - reliable in a wide range of environments
 - accommodates future technical improvements

Our plan of attack...



- ♦ Understand how things work, *bottom-up*
- ♦ Encapsulate our understanding using appropriate **abstractions**
- ♦ Study organizational principles: *abstractions, interfaces, APIs.*
- ♦ Roll up our sleeves and design at each level of hierarchy
- ♦ Learn engineering tricks
 - history
 - systematic approaches
 - algorithms
 - diagnose, fix, and avoid bugs



First up: INFORMATION

Whirlwind, MIT Lincoln Labs
<http://www.chick.net/wizards/whirlwind.html>

If we want to design devices to manipulate, communicate and store information then we need to quantify information so we can get a handle on the engineering issues. Goal:

Two photographs removed due to copyright restrictions.
Please see <http://www.chick.net/wizards/images/whirlwind.jpg> and
<http://www.chick.net/wizards/images/wwtea.gif>.

good implementations

- Easy-to-use
- Efficient
- Reliable
- Secure
- ...
- Low-level physical representations
- High-level symbols and sequences of symbols

What is “Information”?

information, *n.* Knowledge communicated or received concerning a particular fact or circumstance.

Tell me something new...

Information resolves uncertainty.
Information is simply that which cannot be predicted.

The less predictable a message is, the more information it conveys!

Quantifying Information

(Claude Shannon, 1948)

Suppose you're faced with N equally probable choices, and I give you a fact that narrows it down to M choices. Then I've given you

Information is measured in bits
(binary digits) = number of 0/1's
required to encode choice(s)

$\log_2(N/M)$ bits of information

Examples:

- ♦ information in one coin flip: $\log_2(2/1) = 1$ bit
- ♦ roll of 2 dice: $\log_2(36/1) = 5.2$ bits
- ♦ outcome of a Red Sox game: 1 bit
(well, actually, are both outcomes equally probable?)

Encoding

- ♦ Encoding describes the process of *assigning representations to information*
- ♦ Choosing an appropriate and efficient encoding is a real engineering challenge
- ♦ Impacts design at many levels
 - Mechanism (devices, # of components used)
 - Efficiency (bits used)
 - Reliability (noise)
 - Security (encryption)

Next lecture: encoding a bit.

What about *longer* messages?

Fixed-length encodings

If all choices are *equally likely* (or we have no reason to expect otherwise), then a fixed-length code is often used. Such a code will use at least enough bits to represent the information content.

ex. Decimal digits $10 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

4-bit BCD (binary coded decimal)

$\log_2(10) = 3.322 < 4\text{bits}$

ex. ~86 English characters =

{A-Z (26), a-z (26), 0-9 (10), punctuation (11), math (9), financial (4)}

7-bit ASCII (American Standard Code for Information Interchange)

$\log_2(86) = 6.426 < 7\text{bits}$

Encoding numbers

It is straightforward to encode positive integers as a sequence of bits. Each bit is assigned a weight. Ordered from right to left, these weights are increasing powers of 2. The value of an n-bit number encoded in this fashion is given by the following formula:

$$v = \sum_{i=0}^{n-1} 2^i b_i$$

$$2^{11} 2^{10} 2^9 2^8 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0$$

$$011111010000 = 2000_{10}$$

7 d 0

Oftentimes we will find it convenient to cluster groups of bits together for a more compact notation. Two popular groupings are clusters of 3 bits and 4 bits.

03720

Octal - base 8

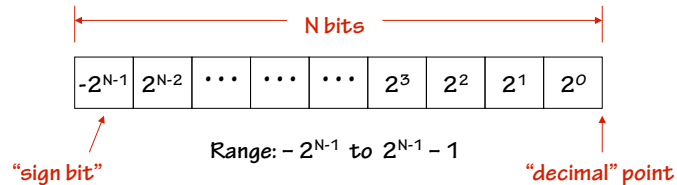
000 - 0
001 - 1
010 - 2
011 - 3
100 - 4
101 - 5
110 - 6
111 - 7

0x7d0

Hexadecimal - base 16

0000 - 0 1000 - 8
0001 - 1 1001 - 9
0010 - 2 1010 - a
0011 - 3 1011 - b
0100 - 4 1100 - c
0101 - 5 1101 - d
0110 - 6 1110 - e
0111 - 7 1111 - f

Signed integers: 2's complement



8-bit 2's complement example:

$$11010110 = -2^7 + 2^6 + 2^4 + 2^2 + 2^1 = -128 + 64 + 16 + 4 + 2 = -42$$

If we use a two's complement representation for signed integers, the same binary addition mod 2^n procedure will work for adding positive and negative numbers (don't need separate subtraction rules). The same procedure will also handle unsigned numbers!

By moving the implicit location of "decimal" point, we can represent fractions too:

$$1101.0110 = -2^3 + 2^2 + 2^0 + 2^{-2} + 2^{-3} = -8 + 4 + 1 + 0.25 + 0.125 = -2.625$$

When choices aren't equally probable

When the choices have different probabilities (p_i), you get more information when learning of a unlikely choice than when learning of a likely choice

$$\text{Information from choice}_i = \log_2(1/p_i) \text{ bits}$$

$$\text{Average information from a choice} = \sum p_i \log_2(1/p_i)$$

Example

choice _i	p _i	log ₂ (1/p _i)
"A"	1/3	1.58 bits
"B"	1/2	1 bit
"C"	1/12	3.58 bits
"D"	1/12	3.58 bits

Average information

$$= (.333)(1.58) + (.5)(1) + (2)(.083)(3.58) = 1.626 \text{ bits}$$

Can we find an encoding where transmitting 1000 choices is close to 1626 bits on the average? Using two bits for each choice = 2000 bits

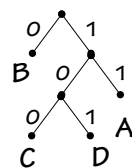
Variable-length encodings

(David Huffman, MIT 1950)

Use shorter bit sequences for high probability choices, longer sequences for less probable choices

choice _i	p _i	encoding
"A"	1/3	11
"B"	1/2	0
"C"	1/12	100
"D"	1/12	101

B C A B A D
010011011101



Huffman Decoding Tree

$$\begin{aligned} \text{Average information} &= (.333)(2) + (.5)(1) + (2)(.083)(3) \\ &= 1.666 \text{ bits} \end{aligned}$$

Transmitting 1000 choices takes an average of 1666 bits... better but not optimal

To get a more efficient encoding (closer to information content) we need to encode **sequences of choices**, not just each choice individually. This is the approach taken by most file compression algorithms...

Data Compression

Key: re-encoding to remove redundant information: match data rate to actual information content.

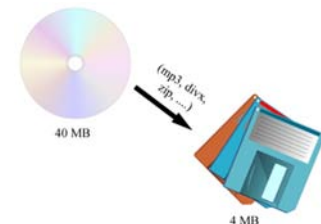


Figure by MIT OpenCourseWare.

"Outside of a dog, a book is man's best friend. Inside of a dog, it's too dark to read..."

-Groucho Marx

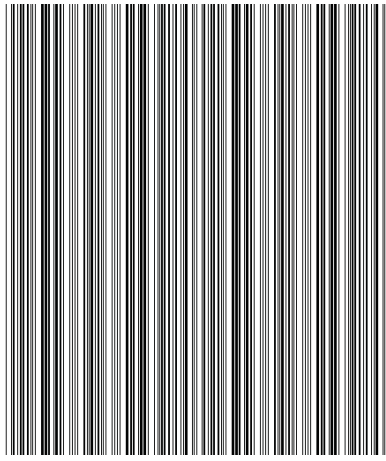
Ideal: No redundant info - Only unpredictable bits transmitted. Result appears *random*!

LOSSLESS: can 'uncompress', get back original.

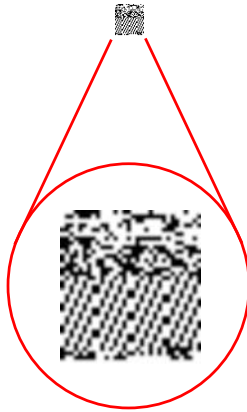
A84b!*m9@+M(p

"Able was I ere I saw Elba."*1024

Uncompressed: 27648 bytes



Compressed: 138 bytes



Does recompression work?

If ZIP compression of a 40MB Bible yields a 4MB ZIP file, what happens if we compress *that*?

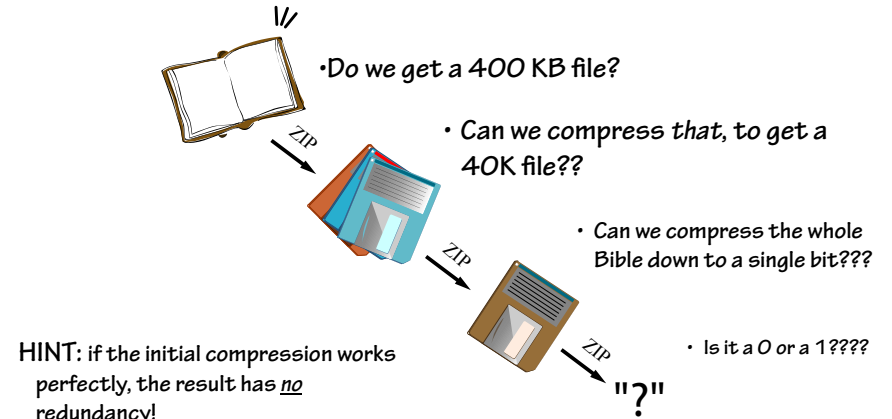


Figure by MIT OpenCourseWare.

Is redundancy *always* bad?

Encoding schemes that attempt to match the information content of a data stream are minimizing redundancy. They are *data compression* techniques.

However, sometimes the goal of encoding information is to *increase redundancy*, rather than remove it. Why?

- Make the information easy to manipulate (fixed-sized encodings)
- Make the data stream resilient to noise (error detecting and correcting codes)

Error detection and correction

Suppose we wanted to reliably transmit the result of a single coin flip:



Heads: "0"



Tails: "1"

This is a prototype of the "bit" coin for the new information economy. Value = 12.5¢

Further suppose that during transmission a *single-bit error* occurs, i.e., a single "0" is turned into a "1" or a "1" is turned into a "0".

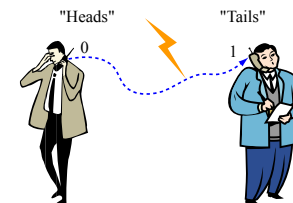


Figure by MIT OpenCourseWare.

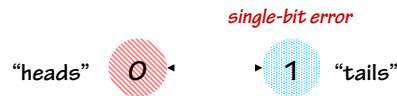
Hamming Distance

(Richard Hamming, 1950)

HAMMING DISTANCE: The number of digit positions in which the corresponding digits of two encodings of the same length are different

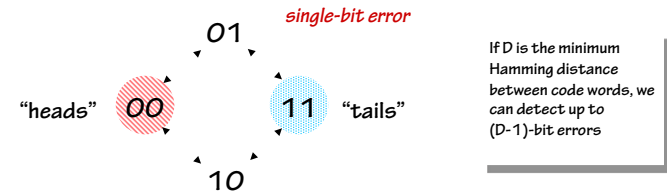
The Hamming distance between a valid binary code word and the same code word with single-bit error is 1.

The problem with our simple encoding is that the two valid code words ("0" and "1") also have a Hamming distance of 1. So a single-bit error changes a valid code word into another valid code word...



Error Detection

What we need is an encoding where a single-bit error doesn't produce another valid code word.

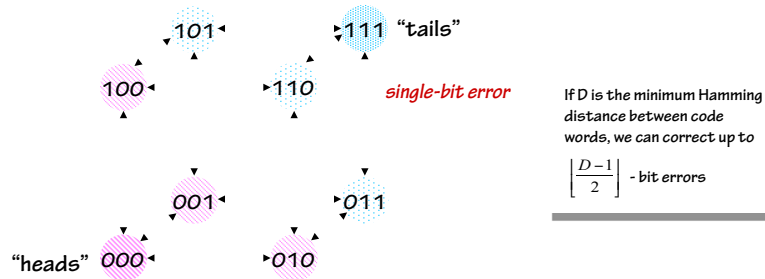


If D is the minimum Hamming distance between code words, we can detect up to (D-1)-bit errors

We can add single-bit error detection to any length code word by adding a **parity bit** chosen to guarantee the Hamming distance between any two valid code words is at least 2. In the diagram above, we're using "even parity" where the added bit is chosen to make the total number of 1's in the code word even.

Can we correct detected errors? Not yet...

Error Correction



If D is the minimum Hamming distance between code words, we can correct up to

$$\left\lfloor \frac{D-1}{2} \right\rfloor \text{ - bit errors}$$

By increasing the Hamming distance between valid code words to 3, we guarantee that the sets of words produced by single-bit errors don't overlap. So if we detect an error, we can perform **error correction** since we can tell what the valid code was before the error happened.

- Can we safely detect double-bit errors while correcting 1-bit errors?
- Do we always need to triple the number of bits?

The right choice of codes can solve hard problems

Reed-Solomon (1960)

First construct a polynomial from the data symbols to be transmitted and then send an over-sampled plot of the polynomial instead of the original symbols themselves - spread the information out so it can be recovered from a subset of the transmitted symbols.

Particularly good at correcting bursts of erasures (symbols known to be incorrect)

Used by CD, DVD, DAT, satellite broadcasts, etc.

Viterbi (1967)

A dynamic programming algorithm for finding the most likely sequence of hidden states that result in a sequence of observed events, especially in the context of hidden Markov models.

Good choice when soft-decision information is available from the demodulator.

Used by QAM modulation schemes (eg, CDMA, GSM, cable modems), disk drive electronics (PRML)

Summary

- Information resolves uncertainty
- Choices equally probable:
 - N choices down to M $\rightarrow \log_2(N/M)$ bits of information
 - use fixed-length encodings
 - encoding numbers: 2's complement signed integers
- Choices not equally probable:
 - choice_i with probability $p_i \rightarrow \log_2(1/p_i)$ bits of information
 - average number of bits = $\sum p_i \log_2(1/p_i)$
 - use variable-length encodings
- To detect D-bit errors: Hamming distance > D
- To correct D-bit errors: Hamming distance > 2D

Next time:

- encoding information electrically
- the digital abstraction
- combinational devices

Hand in Information Sheets!

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

The Digital Abstraction

1. Making bits concrete
2. What makes a good bit
3. Getting bits under contract

Handouts: Lecture Slides

Concrete encoding of information

To this point we've discussed encoding information using bits. But where do bits come from?

If we're going to design a machine that manipulates information, how should that information be physically encoded?

He said to his friend, "If the British march
By land or sea from the town to-night,
Hang a lantern aloft in the belfry arch
Of the North Church tower as a signal light,--
One if by land, and two if by sea;
And I on the opposite shore will be,
Ready to ride and spread the alarm
Through every Middlesex village and farm,
For the country folk to be up and to arm."

What makes a good bit?

- cheap (we want a lot of them)
- stable (reliable, repeatable)
- ease of manipulation
(access, transform, combine, transmit, store)

Substrates for computation

We can build upon almost any physical phenomenon...

Wait!
Those last ones
might have potential...

lanterns
dominos
engraved stone tablets
Billiard balls
E. Coli
polarization of a photon

But, since we're EE's...

Stick with things we know about:

voltages	phase
currents	frequency

This semester we'll use **voltages** to encode information. But the best choice depends on the intended application...

Voltage pros:

- easy generation, detection
- lots of engineering knowledge
- potentially low power in steady state

zero

Voltage cons:

- easily affected by environment
- DC connectivity required?
- R & C effects slow things down

Representing information with voltage

Representation of each point (x, y) on a B&W Picture:

0 volts: BLACK
1 volt: WHITE
0.37 volts: 37% Gray
etc.

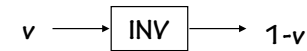
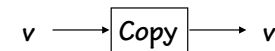


Representation of a picture:
Scan points in some prescribed
raster order... generate voltage
waveform

How much information
at each point?

Information Processing = Computation

First let's introduce some processing blocks:



Why have processing blocks?

The goal of modular design:

Abstraction

What does that mean anyway:

- Rules simple enough for a 6-3 to follow...
- Understanding BEHAVIOR
without knowing IMPLEMENTATION
- Predictable **composition** of functions
- Tinker-toy assembly
- Guaranteed behavior,
under REAL WORLD circumstances

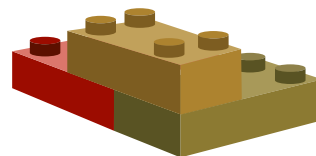
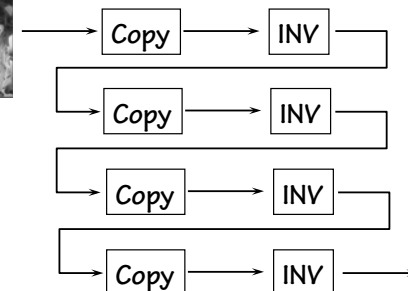


Figure by MIT OpenCourseWare.

Let's build a system!



input



(Reality)



output

Why did our system fail?

Why doesn't reality match theory?

1. COPY Operator doesn't work right
2. INVERSION Operator doesn't work right
3. Theory is imperfect
4. Reality is imperfect
5. Our system architecture stinks

ANSWER: **all** of the above!

Noise and inaccuracy are inevitable; we can't reliably reproduce infinite information-- we must **design our system to tolerate some amount of error** if it is to process information reliably.

The Key to System Design

A system is a structure that is guaranteed to exhibit a specified behavior, assuming all of its components obey their specified behaviors.

How is this achieved?

Contracts!

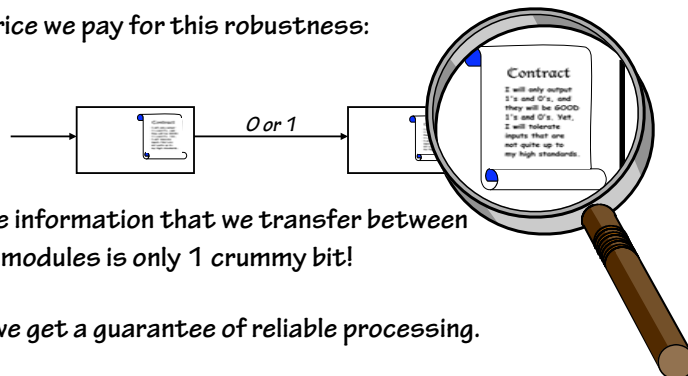
Every system component will have clear obligations and responsibilities. If these are maintained we have every right to expect the system to behave as planned. If contracts are violated all bets are off.

The Digital Panacea ...

Why digital?

... because it keeps the contracts simple!

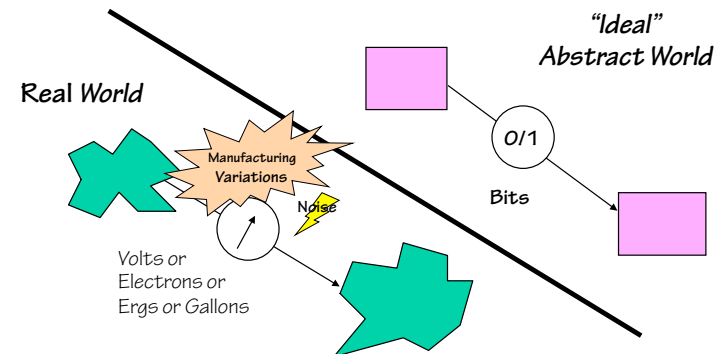
The price we pay for this robustness:



All the information that we transfer between modules is only 1 crummy bit!

But, we get a guarantee of reliable processing.

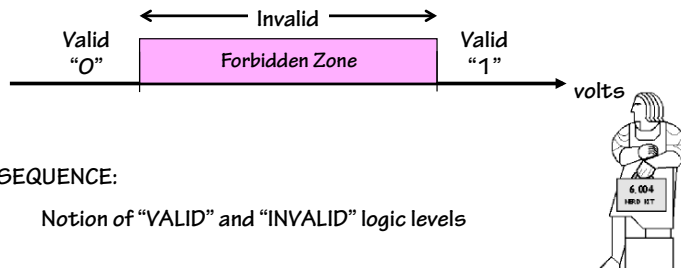
The Digital Abstraction



Keep in mind that the world is not digital, we would simply like to engineer it to behave that way. Furthermore, we must use **real physical phenomena** to implement digital designs!

Using Voltages “Digitally”

- Key idea: don’t allow “0” to be mistaken for a “1” or vice versa
- Use the same “uniform representation convention” for *every* component and wire in our digital system
- To implement devices with high reliability, we outlaw “close calls” via a representation convention which forbids a range of voltages between “0” and “1”.



CONSEQUENCE:

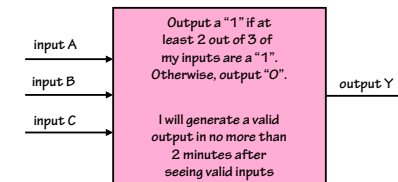
Notion of “VALID” and “INVALID” logic levels

A Digital Processing Element

A combinational device is a circuit element that has

Static discipline

- one or more digital inputs
- one or more digital outputs
- a functional specification that details the value of each output for every possible combination of valid input values
- a timing specification consisting (at minimum) of an upper bound t_{pd} on the required time for the device to compute the specified output values from an arbitrary set of stable, valid input values



A Combinational Digital System



A set of interconnected elements is a combinational device if

- each circuit element is combinational
- every input is connected to exactly one output or to some vast supply of constant 0’s and 1’s
- the circuit contains no directed cycles

Why is this true?

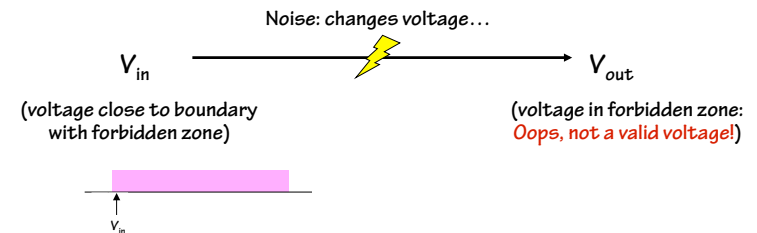
Given an acyclic circuit meeting the above constraints, we can derive functional and timing specs for the input/output behavior from the specs of its components!

We’ll see lots of examples soon. But first, we need to build some combinational devices to work with...



Wires: theory vs. practice

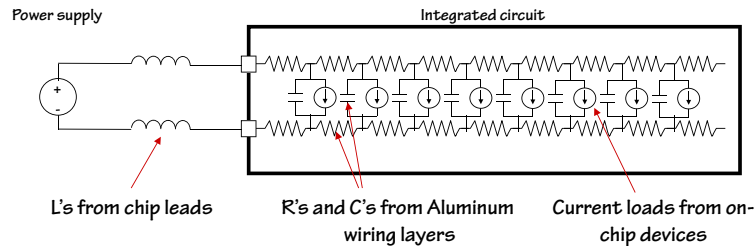
Does a wire obey the static discipline?



Questions to ask ourselves:

*In digital systems, where does noise come from?
How big an effect are we talking about?*

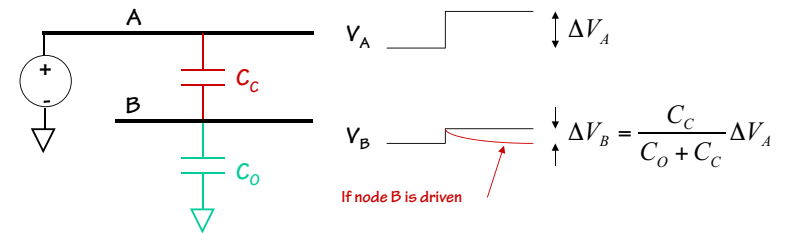
Power Supply Noise



ΔV from:

- **IR drop**
(between gates: 30mV, within module: 50mV, across chip: 350mV)
- **L(dI/dt) drop**
(use extra pins and bypass caps to keep within 250mV)
- **LC ringing** triggered by current "steps"

Crosstalk

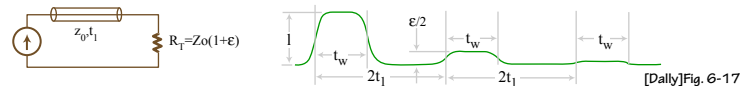


This situation frequently happens on integrated circuits where there are many overlapping wiring layers. In a modern integrated circuit ΔV_A might be 2.5V, $C_o = 20\text{fF}$ and $C_c = 10\text{fF} \rightarrow \Delta V_B = 0.83\text{V}$! Designers often try to avoid these really bad cases by careful routing of signals, but some crosstalk is unavoidable.

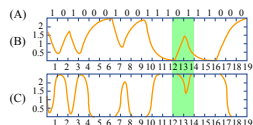
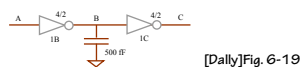
Sequential Interference

ΔV from energy storage left over from earlier signaling on the wire:

- **transmission line discontinuities**
(reflections off of impedance mismatches and terminations)



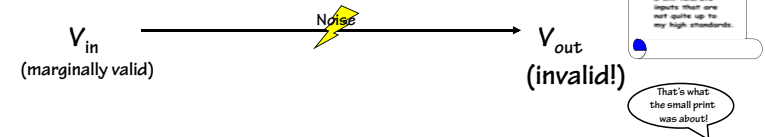
- **charge storage in RC circuit**
(narrow pulses are lost due to incomplete transitions)
- **RLC ringing** (triggered by voltage "steps")



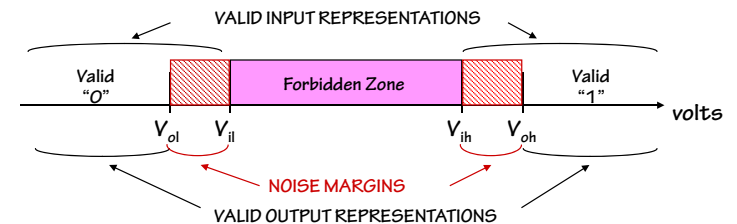
Fix: slower operation, limiting voltage swings and slew rates

Needed: Noise Margins!

Does a wire obey the static discipline?

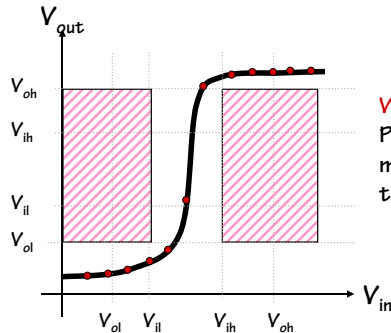


No! A combinational device must restore marginally valid signals. It must accept marginal inputs and provide unquestionable outputs (i.e., to leave room for noise).



A Buffer

A simple *combinational device*:



Voltage Transfer Characteristic (VTC):
Plot of V_{out} vs. V_{in} where each measurement is taken after any transients have died out.

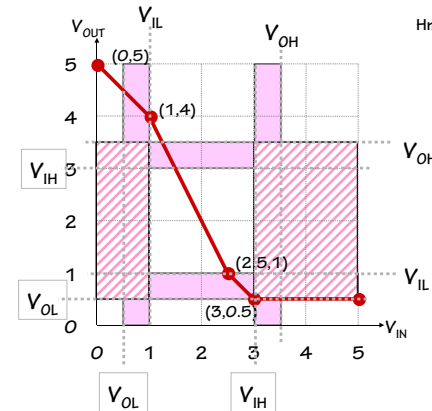
Note: VTC does not tell you anything about how fast a device is— it measures static behavior not dynamic behavior

Static Discipline requires that the VTC avoid the shaded regions (aka “forbidden zones”), which correspond to *valid* inputs but *invalid* outputs.

Net result: combinational devices must have **GAIN > 1** and be **NONLINEAR**.

Can this be a combinational device?

Suppose that you measured the voltage transfer curve of the device shown below.
Could we build a logic family using it as a single-input combinational device?



Hmmm, it had better be an **INVERTER**...

The device must be able to actually produce the desired output level. Thus, V_{OL} can be no lower than 0.5 V.

Try $V_{OL} = 0.5$ V

V_{IH} must be high enough to produce V_{OL}

Try $V_{IH} = 3$ V

Now, choose noise margins – find an N and set

$$V_{OH} = V_{IH} + N$$

$$V_{IL} = V_{OL} + N$$

Such that

V_{IH} IN generates V_{OL} or less out; AND

V_{IL} IN generates V_{OH} or more out.

Try $N = 0.5$ V

Summary

- Use voltages to encode information
- “Digital” encoding
 - valid voltage levels for representing “0” and “1”
 - forbidden zone avoids mistaking “0” for “1” and vice versa
 - gives rise to notion of signal VALIDITY.
- Noise
 - Want to tolerate real-world conditions: NOISE.
 - Key: tougher standards for output than for input
 - devices must have gain and have a non-linear VTC
- Combinational devices
 - Each logic family has Tinkertoy-set simplicity, modularity
 - predictable composition: “parts work → whole thing works”
 - static discipline
 - digital inputs, outputs; restore marginal input voltages
 - complete functional spec
 - valid inputs lead to valid outputs in bounded time

Next time: Building Logic w/ Transistors



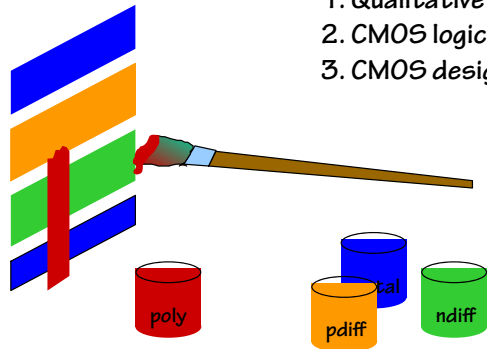
MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

CMOS Technology

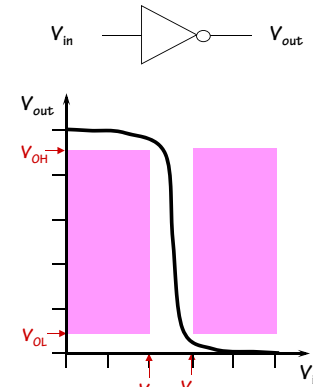


1. Qualitative MOSFET model
2. CMOS logic gates
3. CMOS design issues

NEXT WEEK:

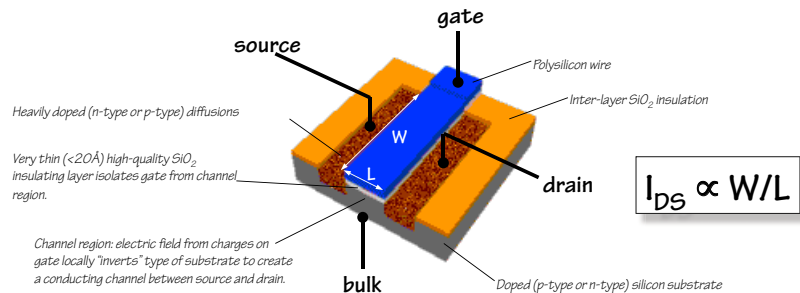
- TUE: no lecture
- THU: Lab 1 due!
- FRI: QUIZ 1!!!

Combinational Device Wish List



- ✓ Design our system to tolerate some amount of error
⇒ Add positive noise margins
⇒ VTC: gain > 1 & nonlinearity
- ✓ Lots of gain ⇒ big noise margin
- ✓ Cheap, small
- ✓ Changing voltages will require us to dissipate power, but if no voltages are changing, we'd like zero power dissipation
- ✓ Want to build devices with useful functionality (what sort of operations do we want to perform?)

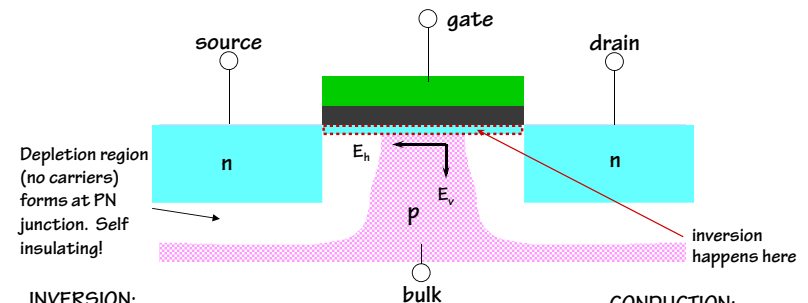
MOSFETS: Gain & non-linearity



MOSFETs (metal-oxide-semiconductor field-effect transistors) are four-terminal voltage-controlled switches. Current flows between the diffusion terminals if the voltage on the gate terminal is large enough to create a conducting "channel", otherwise the mosfet is off and the diffusion terminals are not connected.

FETs as switches

The four terminals of a Field Effect Transistor (gate, source, drain and bulk) connect to conductors that generate a complicated set of electric fields in the channel region which depend on the relative voltages of each terminal.

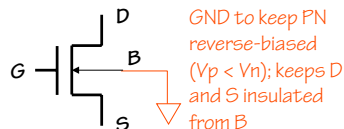
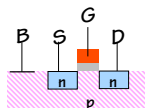


INVERSION:
A sufficiently strong vertical field will attract enough electrons to the surface to create a conducting n-type channel between the source and drain. The gate voltage when the channel first forms is called the **threshold voltage** -- the mosfet switch goes from "off" to "on".

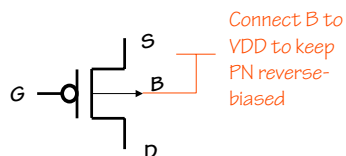
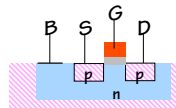
CONDUCTION:
If a channel exists, a horizontal field will cause a drift current from the drain to the source.

FETs come in two flavors

NFET: n-type source/drain diffusions in a p-type substrate. Positive threshold voltage; inversion forms n-type channel



PFET: p-type source/drain diffusions in a n-type substrate. Negative threshold voltage; inversion forms p-type channel.



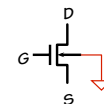
The use of both NFETs and PFETs – complimentary transistor types – is a key to CMOS (complementary MOS) logic families.

CMOS Recipe

If we follow two rules when constructing CMOS circuits then we can model the behavior of the mosfets as simple switches:

Rule #1: **only use NFETs in pulldown circuits** (paths from output node to GND)

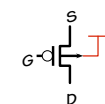
Rule #2: **only use PFETs in pullup circuits** (paths from output node to V_{DD})



NFET Operating regions:

"off": $V_G < V_{TH,NFET}$

"on": $V_G > V_{TH,NFET}$

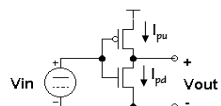


PFET Operating regions:

"off": $V_G > V_{DD} + V_{TH,PFET}$

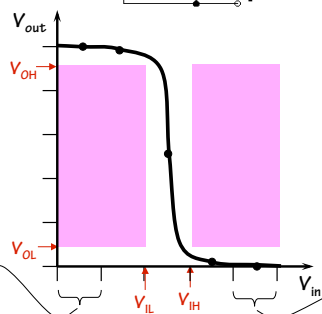
"on": $V_G < V_{DD} + V_{TH,PFET}$

CMOS Inverter VTC



Steady state reached when V_{out} reaches value where $I_{pu} = I_{pd}$

When V_{IN} is low, the nfet is off and the pfet is on, so current flows into the output node and V_{OUT} eventually reaches V_{DD} ($= V_{OH}$) at which point no more current will flow.



When V_{IN} is high, the pfet is off and the nfet is on, so current flows out of the output node and V_{OUT} eventually reaches GND ($= V_{OL}$) at which point no more current will flow.

When V_{IN} is in the middle, both the pfet and nfet are "on" and the shape of the VTC depends on the details of the devices' characteristics. CMOS gates have very high gain in this region (small changes in V_{IN} produce large changes in V_{OUT}) and the VTC is almost a step function.

Beyond Inverters: Complementary pullups and pulldowns

Now you know what the "C" in CMOS stands for!

We want **complementary** pullup and pulldown logic, i.e., the pulldown should be "on" when the pullup is "off" and vice versa.

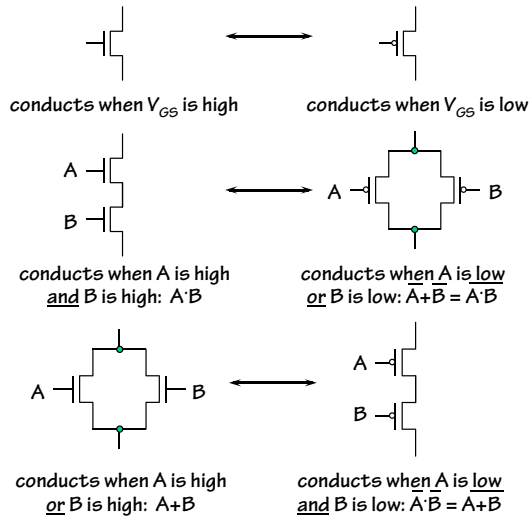
pullup	pulldown	$F(A_1, \dots, A_n)$
on	off	driven "1"
off	on	driven "0"
on	on	driven "X"
off	off	no connection

Since there's plenty of capacitance on the output node, when the output becomes disconnected it "remembers" its previous voltage -- at least for a while. The "memory" is the load capacitor's charge. Leakage currents will cause eventual decay of the charge (that's why DRAMs need to be refreshed!).

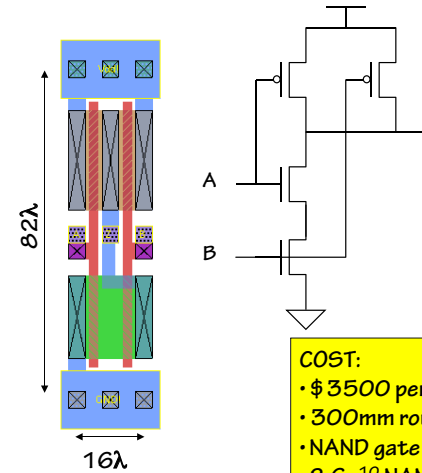
What a nice V_{OH} you have...

Thanks, it runs in the family...

CMOS complements



A pop quiz!



What function does this gate compute?

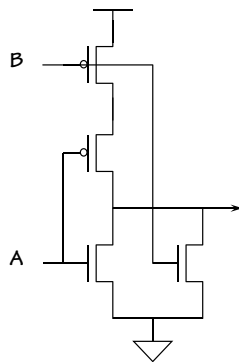
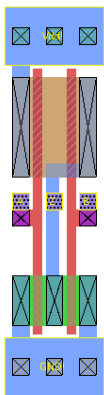
A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

NAND

COST:

- \$3500 per 300mm wafer
- 300mm round wafer = $\pi(150e^{-3})^2 = .07m^2$
- NAND gate = $(82)(16)(45e^{-9})^2 = 2.66e^{-12}m^2$
- $2.6e^{10}$ NAND gates/wafer (= 100 billion FETS!)
- marginal cost of NAND gate: **132n\$**

Here's another...



What function does this gate compute?

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

NOR

General CMOS gate recipe

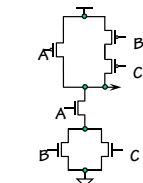
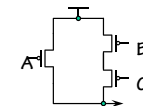
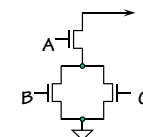
Step 1. Figure out pulldown network that does what you want, e.g.,

$$\overline{F} = A \cdot (B + C)$$

(What combination of inputs generates a low output)

Step 2. Walk the hierarchy replacing nfets with pfets, series subnets with parallel subnets, and parallel subnets with series subnets

Step 3. Combine pfet pullup network from Step 2 with nfet pulldown network from Step 1 to form fully-complementary CMOS gate.

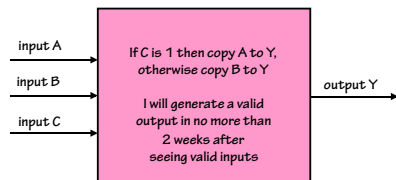


So, what's the big deal?

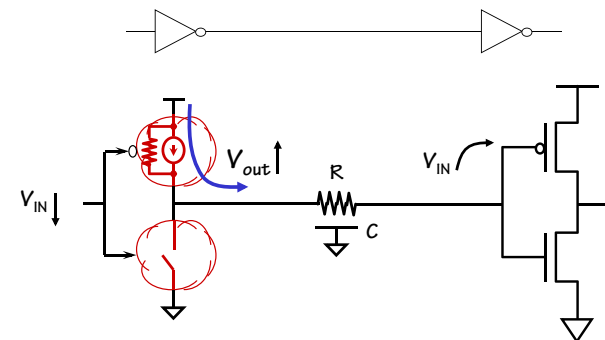
A Quick Review

- A **combinational device** is a circuit element that has
 - one or more digital *inputs*
 - one or more digital *outputs*
 - a *functional specification* that details the value of each output for every possible combination of valid input values
 - a *timing specification* consisting (at minimum) of an upper bound t_{PD} on the required time for the device to compute the specified output values from an arbitrary set of stable, valid input values

Static discipline



Big Issue 1: Wires



Today (i.e., 100nm):

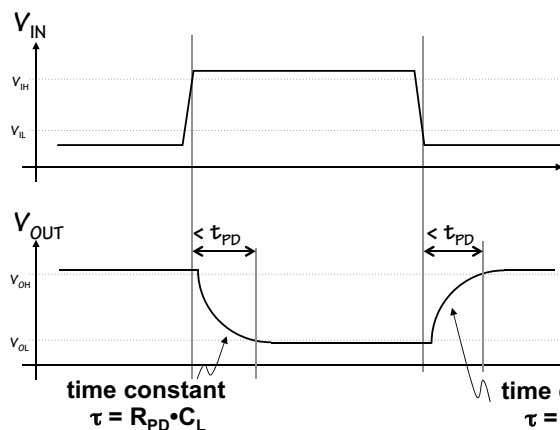
$$\tau_{RC} \approx 50\text{ps/mm}$$

Implies > 1 ns to traverse a 20mm x 20mm chip
This is a long time in a 2GHz processor

Due to unavoidable delays...

Propagation delay (t_{PD}):

An UPPER BOUND on the delay from valid inputs to valid outputs.



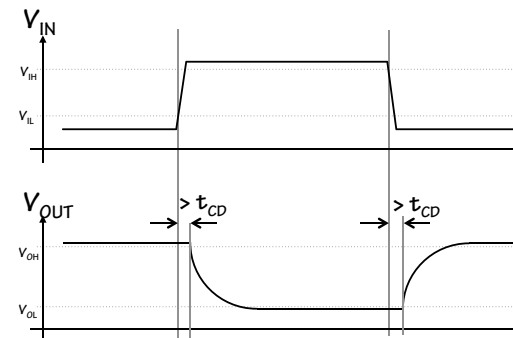
GOAL:
minimize
propagation
delay!

ISSUE:
keep
Capacitances
low and
transistors
fast

Contamination Delay

an optional, additional timing spec

INVALID inputs take time to propagate, too...



Do we really need t_{CD} ?

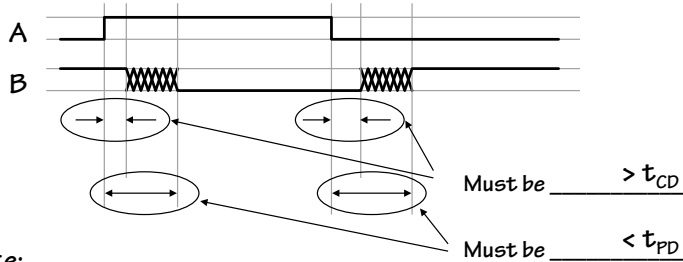
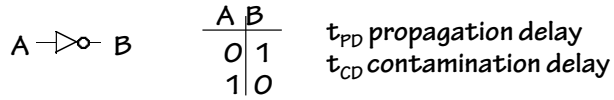
Usually not... it'll be important when we design circuits with registers (coming soon!)

If t_{CD} is not specified, safe to assume it's 0.

CONTAMINATION DELAY, t_{CD}

A LOWER BOUND on the delay from any invalid input to an invalid output

The Combinational Contract

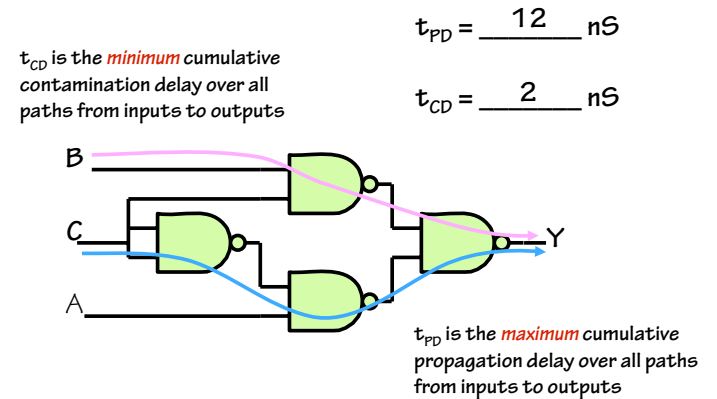


Note:

1. No Promises during
2. Default (conservative) spec: $t_{CD} = 0$

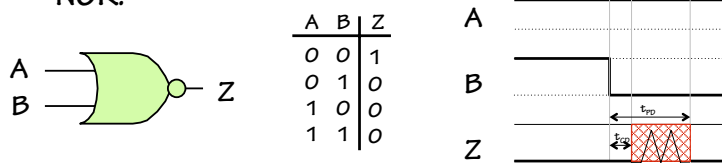
Acyclic Combinational Circuits

If NAND gates have a $t_{PD} = 4\text{ nS}$ and $t_{CD} = 1\text{ nS}$



Oh yeah... one last issue

NOR:



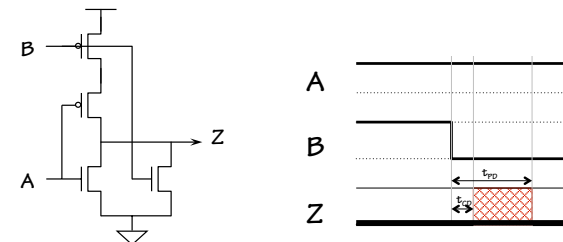
Recall the rules for *combinational devices*:

Output guaranteed to be valid when **all** inputs have been valid for at least t_{PD} , and, outputs may become invalid no earlier than t_{CD} after an input changes!

Many gate implementations--e.g., CMOS--adhere to even tighter restrictions.

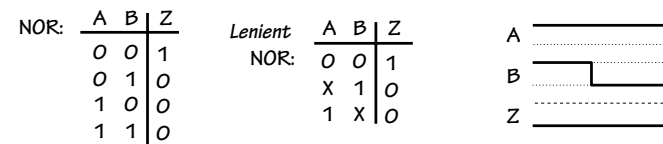
What happens in this case?

CMOS NOR:

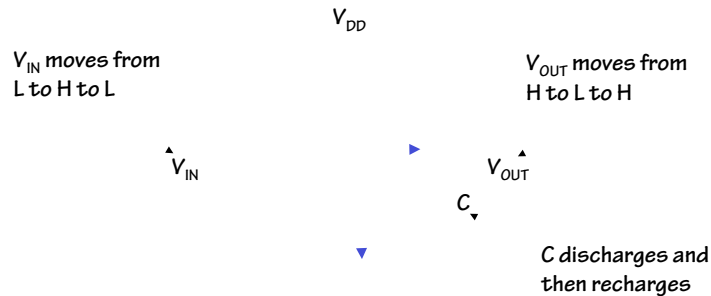


LENIENT Combinational Device:

Output guaranteed to be valid when **any** combination of inputs sufficient to determine output value has been valid for at least t_{PD} . Tolerates transitions -- and invalid levels -- on irrelevant inputs!



Big Issue 2: Power



$$\text{Energy dissipated} = C V_{DD}^2 \text{ per cycle}$$

$$\text{Power consumed} = f n C V_{DD}^2 \text{ per chip}$$

where f = frequency of charge/discharge
 n = number of gates /chip

Unfortunately...

Modern chips (UltraSparc III, Power4, Itanium 2) dissipate from 80W to 150W with a $V_{DD} \approx 1.2V$ (Power supply current is ≈ 100 Amps)

Cooling challenge is like making the filament of a 100W incandescent lamp cool to the touch!

Worse yet...

- Little room left to reduce V_{DD}
- nC and f continue to grow

Hey: could we
 Somehow recycle
 the charge?

MUST computation consume energy?

(a tiny digression...)

How energy-efficient can we make a gate? It seems that switching the input to a NAND gate will always dissipate some energy...

▼ <http://www.research.ibm.com/journal/rd/4-4-1/landaueril.pdf>

Landauer's Principle (1961): *discarding information is what costs energy!*

▼ <http://www.research.ibm.com/journal/rd/1-76/ibmrd1706G.pdf>

Bennett (1973): Use reversible logic gates, not NAND, and there's no lower bound to energy use!

The fundamental physical limits of computation, Bennett & Landauer, Scientific American. Vol. 253, pp. 48-56. July 1985

A	B	C	
0	0	1	NAND GATE: 2 bits → 1 bit (information Loss!)
0	1	1	
1	0	1	
1	1	0	

A	B	P	Q	
0	0	0	0	FEYNMAN GATE: 2 bits → 2 bits (information Preserving!)
0	1	0	1	
1	0	1	1	
1	1	1	0	

Bennett, Fredkin, Feynman, others: Computer systems constructed from info-preserving elements.

Theory: NO lower bound on energy use!

Practice: Research frontier (qubits, etc.)

Summary

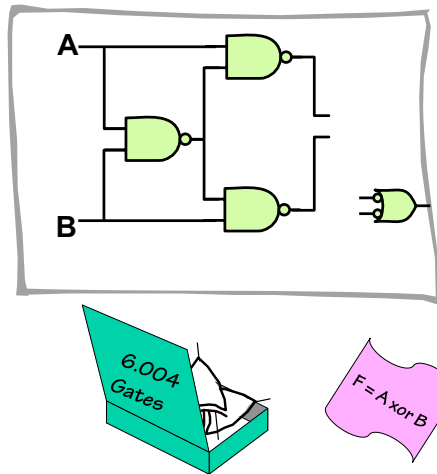
- CMOS
 - Only use NFETs in pulldowns, PFETs in pullups → mosfets behave as voltage-controlled switches
 - Series/parallel Pullup and pulldown switch circuits are complementary
 - CMOS gates are naturally inverting (rising input transition can only cause falling output transition, and vice versa).
 - "Perfect" VTC (high gain, $V_{OH} = V_{DD}$, $V_{OL} = GND$) means large noise margins and no static power dissipation.
- Timing specs
 - t_{PD} : upper bound on time from valid inputs to valid outputs
 - t_{CD} : lower bound on time from invalid inputs to invalid outputs
 - If not specified, assume $t_{CD} = 0$
 - Lenient gates: output unaffected by some input transitions
- Next time: logic simplification, other canonical forms

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Synthesis of Combinational Logic

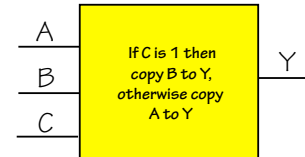


Lab 1 is due Thursday 2/19
Quiz 1 is a week from Friday (in section)

Functional Specifications

There are many ways of specifying the function of a combinational device, for example:

Argh... I'm tired of word games



Truth Table

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Concise alternatives:

truth tables are a concise description of the combinational system's function.

Boolean expressions form an algebra in whose operations are AND (multiplication), OR (addition), and inversion (overbar).

$$Y = \overline{C}\overline{B}A + \overline{C}BA + C\overline{B}\overline{A} + CBA$$

Any combinational (Boolean) function can be specified as a truth table or an equivalent **sum-of-products** Boolean expression!

Here's a Design Approach

Truth Table

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

1) Write out our functional spec as a truth table

2) Write down a Boolean expression with terms covering each '1' in the output:

$$Y = \overline{C}\overline{B}A + \overline{C}BA + C\overline{B}\overline{A} + CBA$$

3) Wire up the gates, call it a day, and declare success!

-it's systematic!
-it works!
-it's easy!
-are we done yet???

This approach will always give us Boolean expressions in a particular form: **SUM-OF-PRODUCTS**

Straightforward Synthesis

We can implement

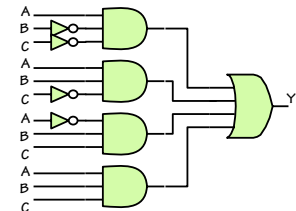
SUM-OF-PRODUCTS
with just three levels of logic.

INVERTERS/AND/OR

Propagation delay --

No more than 3 gate delays

(assuming gates with an arbitrary number of inputs)



Basic Gate Repertoire

Are we sure we have all the gates we need?

Just how many two-input gates are there?

AND		OR		NAND		NOR	
AB	Y	AB	Y	AB	Y	AB	Y
00	0	00	0	00	1	00	1
01	0	01	1	01	1	01	0
10	0	10	1	10	1	10	0
11	1	11	1	11	0	11	0

Hmmmm... all of these have 2-inputs (no surprise)

... each with 4 combinations, giving 2^2 output cases

$$2^{2^2} = 2^4 = 16$$

How many ways are there of assigning 4 outputs? _____

There are only so many gates

There are only 16 possible 2-input gates

... some we know already, others are just silly

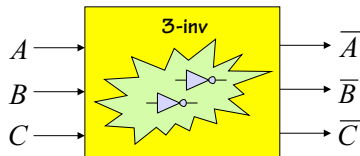
[illegible]

How many of these gates can be implemented using a single CMOS gate?

CMOS gates are inverting; we can always respond positively to positive transitions by cascaded gates. But suppose our logic yielded cheap *positive* functions, while inverters were expensive...

Logic Geek Party Games

You have plenty of ANDs and ORs, but only 2 inverters. Can you invert more than 2 independent inputs?



CHALLENGE: Come up with a combinational circuit using ANDs, ORs, and at most 2 inverters that inverts A, B, and C!

Such a circuit exists. What does that mean?

- If we can invert 3 signals using 2 inverters, can we use 2 of the pseudo-inverters to invert 3 more signals?
- Do we need only 2 inverters to make ANY combinational circuit?

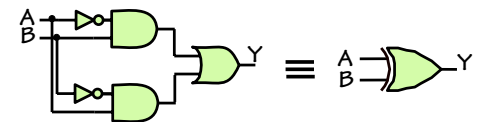
Hint: there's a subtle difference between our 3-inv device and three combinational inverters!

Is our 3-inv device *LENIENT*?

Fortunately, we can get by with a few basic gates...

AND, OR, and NOT are sufficient... (cf Boolean Expressions):

B>A		XOR	
AB	Y	AB	Y
00	0	00	0
01	1	01	1
10	0	10	1
11	0	11	0



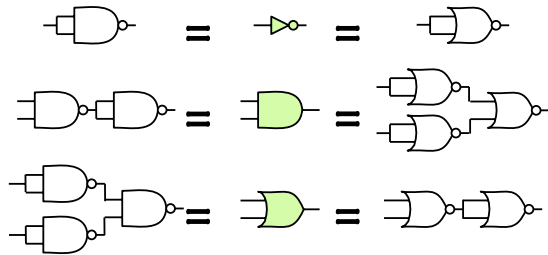
That is just
DeMorgan's
Theorem!

$\overline{A}B = \overline{A+B}$
 $\overline{A+B} = \overline{A}B$

How many different gates do we really need?

One will do!

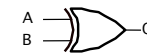
NANDs and NORs are universal:



Ah!, but what if we want more than 2-inputs

Stupid Gate Tricks

Suppose we have some 2-input XOR gates:

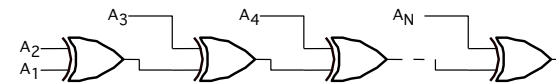


$$t_{pd} = 1$$

$$t_{cd} = 0$$

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

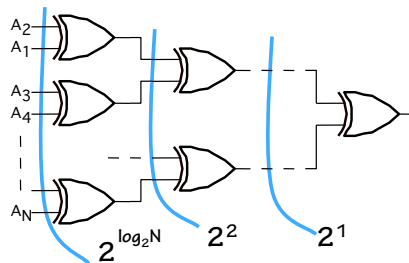
And we want an N-input XOR:



$$t_{pd} = O(\underline{N}) \text{ -- WORST CASE.}$$

Can we compute N-input XOR faster?

I think that I shall never see a circuit lovely as...



N-input TREE has $O(\underline{\log N})$ levels...

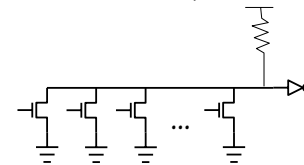
Signal propagation takes $O(\underline{\log N})$ gate delays.

Question: Can EVERY N-Input Boolean function be implemented as a tree of 2-input gates?

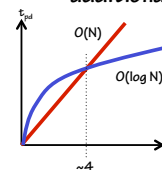
Are Trees Always Best?

Alternate Plan: Large Fan-in gates

- ♦ N pulldowns with complementary pullups
- ♦ Output HIGH if any input is HIGH = "OR"



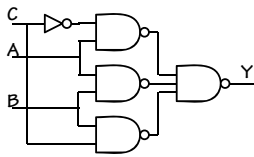
- ♦ Propagation delay: $O(N)$ since each additional MOSFET adds C



Don't be misled by the "big O" stuff... the constants in this case can be much smaller... so for small N this plan might be the best.

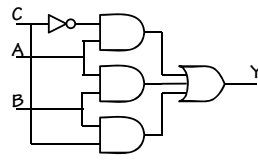
Practical SOP Implementation

NAND-NAND



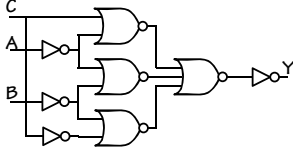
$$\overline{AB} = \overline{A+B}$$

"Pushing Bubbles"

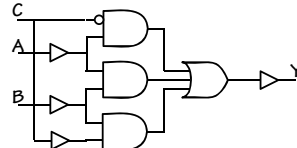


$$xyz = \overline{\overline{x} + \overline{y} + \overline{z}}$$

NOR-NOR



$$\overline{\overline{AB}} = \overline{A+B}$$



$$\overline{x+y} = \overline{x}\overline{y}$$

You might think all these extra inverters would make this structure less attractive. However, quite the opposite is true.

Logic Simplification

Can we implement the same function with fewer gates?

Before trying we'll add a few more tricks in our bag.

BOOLEAN ALGEBRA:

OR rules:

$$a + 1 = 1, a + 0 = a, a + a = a$$

AND rules:

$$a1 = a, a0 = 0, aa = a$$

Commutative:

$$a + b = b + a, ab = ba$$

Associative:

$$(a + b) + c = a + (b + c), (ab)c = a(bc)$$

Distributive:

$$a(b+c) = ab + ac, a + bc = (a+b)(a+c)$$

Complements:

$$a + \overline{a} = 1, a\overline{a} = 0$$

Absorption:

$$a + ab = a, a + \overline{a}b = a + b$$

$$a(a+b) = a, a(\overline{a}+b) = ab$$

Reduction:

$$\overline{ab + \overline{a}b} = \overline{b}, (a+b)(\overline{a}+b) = b$$

DeMorgan's Law:

$$\overline{a+b} = \overline{a}\overline{b}, \overline{ab} = \overline{a} + \overline{b}$$

Boolean Minimization:

An Algebraic Approach

Lets (again!) simplify

$$Y = \overline{C}BA + C\overline{B}A + CBA + \overline{C}BA$$

Using the identity

$$\alpha A + \alpha \overline{A} = \alpha$$

For any expression α and variable A:

$$Y = \overline{C}BA + C\overline{B}A + CBA + \overline{C}BA$$

$$Y = \overline{C}BA + CB + \overline{C}BA$$

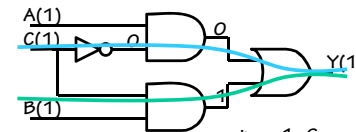
$$Y = \overline{C}A + CB$$

Can't he come up with a new example???

Hey, I could write A program to do That!

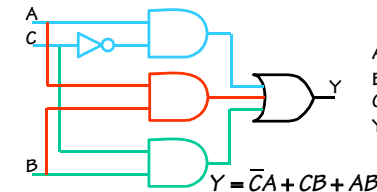
A Case for Non-Minimal SOP

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



$$Y = \overline{C}A + CB$$

NOTE: The steady state behavior of these circuits is identical. They differ in their transient behavior.



Truth Tables with “Don’t Cares”

One way to reveal the opportunities for a more compact implementation is to rewrite the truth table using “don’t cares” (--) to indicate when the value of a particular input is irrelevant in determining the value of the output.

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

➔

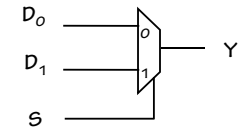
C	B	A	Y
0	--	0	0
0	--	1	1
1	0	--	0
1	1	--	1
--	0	0	0
--	1	1	1

$\rightarrow \bar{C}A$

$\rightarrow CB$

$\rightarrow BA$

We’ve been designing a “mux”

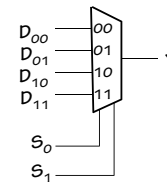


2-input Multiplexer

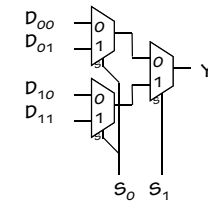
Truth Table

C	B	A	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

MUXes can be generalized to 2^k data inputs and k select inputs ...

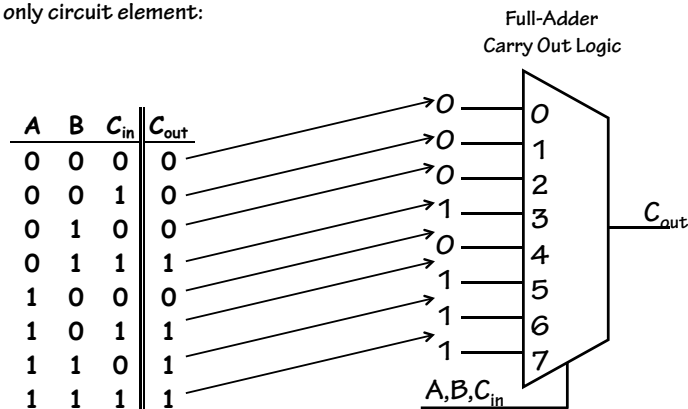


... and implemented as a tree of smaller MUXes:

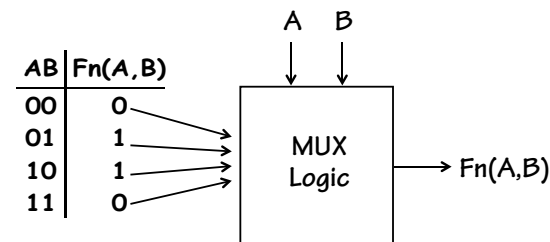


Systematic Implementations of Combinational Logic

Consider implementation of some arbitrary Boolean function, $F(A,B,C)$... using a MULTIPLEXER as the only circuit element:



General Table Lookup Synthesis



Generalizing:

In theory, we can build any 1-output combinational logic block with multiplexers.

For an N-input function we need a 2^N input mux.

BIG Multiplexers?

How about 10-input function? 20-input?

Muxes are UNIVERSAL!

1 0 1
0 1 1
A

Y = A NOT Y

0 1 1
1 0 1
A

Y = A AND B

1 0 1
0 1 1
A

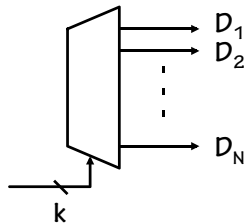
Y = A OR B

In future technologies muxes might be the “natural gate”.



What does that one do?

A New Combinational Device



DECODER:
k SELECT inputs,
 $N = 2^k$ DATA OUTPUTS.
Selected D_j HIGH;
all others LOW.

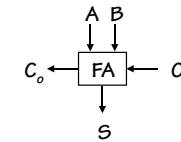
Have I mentioned that HIGH is a synonym for '1' and LOW means the same as '0'.

NOW, we are well on our way to building a general purpose table-lookup device.

We can build a 2-dimensional ARRAY of decoders and selectors as follows ...

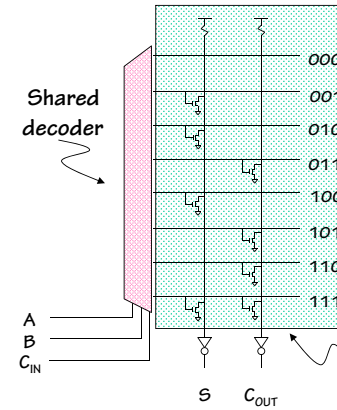
Read-only memories (ROMs)

Full Adder



A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Each column is large fan-in "OR" as described on slide #12. Note location of pulldowns correspond to a "1" output in the truth table!

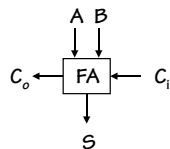


For K inputs, decoder produces 2^K signals, only 1 of which is asserted at a time -- think of it as one signal for each possible product term.

One selector for each output

Read-only memories (ROMs)

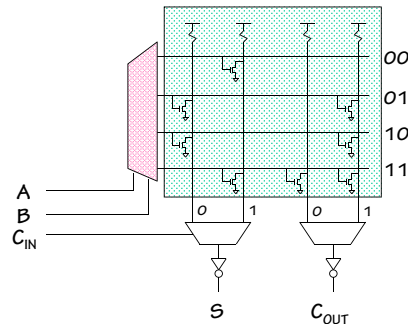
Full Adder



A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

LONG LINES slow down propagation times...

The best way to improve this is to build square arrays, using some inputs to drive output selectors (MUXes):



2D Addressing: Standard for ROMs, RAMs, logic arrays...

Logic According to ROMs

ROMs ignore the structure of combinational functions ...

- Size, layout, and design are independent of function
- Any Truth table can be "programmed" by minor reconfiguration:

- Metal layer (masked ROMs)
- Fuses (Field-programmable PROMs)
- Charge on floating gates (EPROMs)
- ... etc.

ROMs tend to generate "glitchy" outputs. WHY?

Model: LOOK UP value of function in truth table...

Inputs: "ADDRESS" of a T.T. entry

ROM SIZE = # TT entries...

... for an N-input boolean function, size = $2^N \times \text{\#outputs}$

Summary

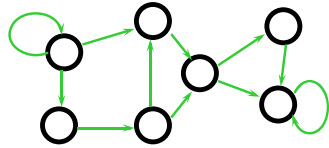
- **Sum of products**
 - Any function that can be specified by a truth table or, equivalently, in terms of AND/OR/NOT (Boolean expression)
 - “3-level” implementation of any logic function
 - Limitations on number of inputs (fan-in) increases depth
 - SOP implementation methods
 - NAND-NAND, NOR-NOR
- **Muxes used to build table-lookup implementations**
 - Easy to change implemented function -- just change constants
- **ROMs**
 - Decoder logic generates all possible product terms
 - Selector logic determines which p'terms are or'ed together

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Sequential Logic: adding a little state



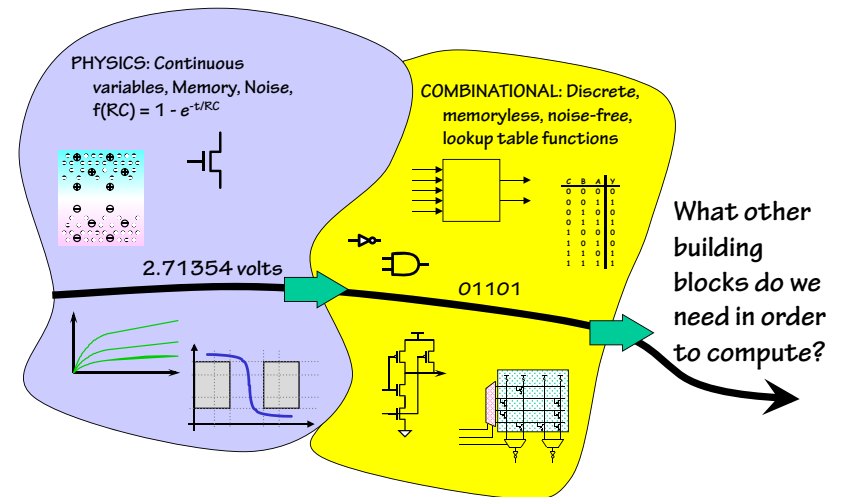
Lab #1 is due tonight

(checkoff meeting by next Thursday).

QUIZ #1 Tomorrow!

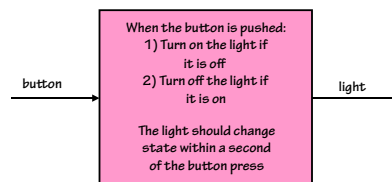
(covers thru L4/R5)

6.004: Progress so far...



Something We Can't Build (Yet)

What if you were given the following design specification:

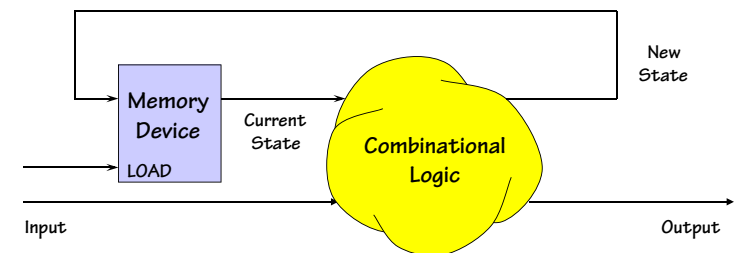


What makes this circuit so different from those we've discussed before?

1. "State" – i.e. the circuit has memory
2. The output was changed by an input "event" (pushing a button) rather than an input "value"

Digital State

One model of what we'd like to build



Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output
- Combinational Logic computes
 - NEXT state (from input, current state)
 - OUTPUT bit (from input, current state)
- State changes on LOAD control input

Needed: Storage

Combinational logic is *stateless*:

valid outputs always reflect current inputs.

To build devices with state, we need components which *store* information (e.g., state) for subsequent access.

ROMs (and other combinational logic) store information “wired in” to their truth table

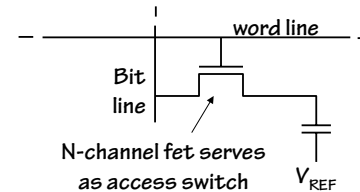
Read/Write memory elements are required to build devices capable of changing their contents.

How can we store – and subsequently access -- a bit?

- Mechanics: holes in cards/tapes
- Optics: Film, CDs, DVDs, ...
- Magnetic materials
- Delay lines; moonbounce
- Stored charge

Storage: Using Capacitors

We’ve chosen to encode information using voltages and we know from 6.002 that we can “store” a voltage as charge on a capacitor:



To write:

Drive bit line, turn on access fet, force storage cap to new voltage

To read:

precharge bit line, turn on access fet, detect (small) change in bit line voltage

Pros:

- ♦ compact – low cost/bit (on BIG memories)

Cons:

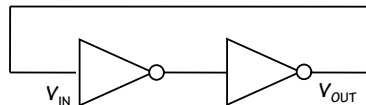
- ♦ complex interface
- ♦ stable? (noise, ...)
- ♦ it leaks! ⇒ refresh

Suppose we refresh CONTINUOUSLY?

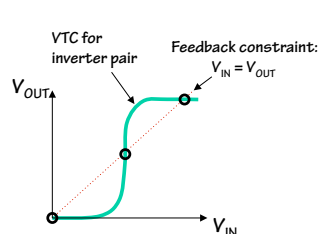
Storage: Using Feedback

IDEA: use **positive feedback** to maintain storage indefinitely.

Our logic gates are built to restore marginal signal levels, so noise shouldn’t be a problem!



Result: a **bistable storage element**



Three solutions:

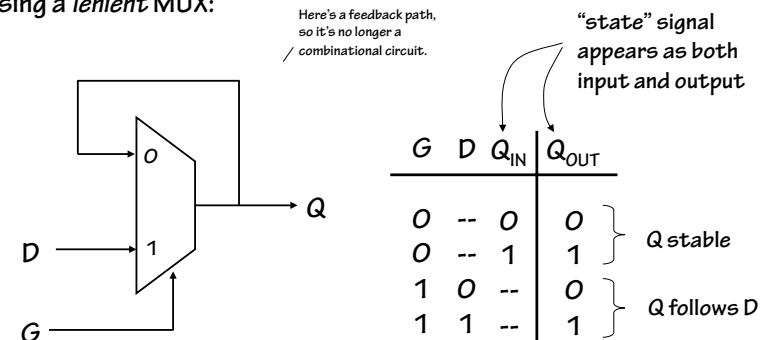
- ♦ two end-points are **stable**
- ♦ middle point is **unstable**

Not affected by noise

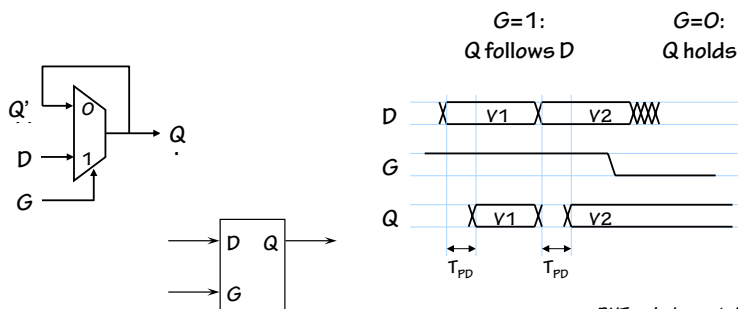
We’ll get back to this!

Settable Storage Element

It’s easy to build a settable storage element (called a **latch**) using a *lenient* MUX:



New Device: D Latch

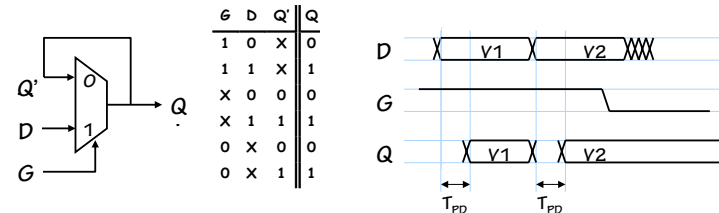


$G=1$: Q Follows D, independently of Q'

$G=0$: Q Holds stable Q' , independently of D

BUT... A change in D or G contaminates Q, hence Q' ... how can this possibly work?

A Plea for Lenience...



Assume LENIENT Mux, propagation delay of T_{PD}

Then output valid when

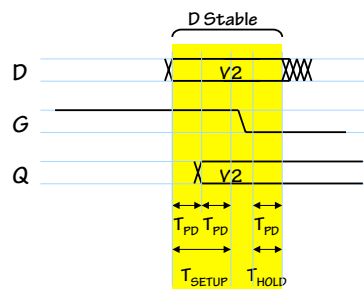
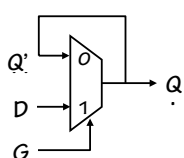
- $G=1$, D stable for T_{PD} , independently of Q' ; or
- $Q'=D$ stable for T_{PD} , independently of G; or
- $G=0$, Q' stable for T_{PD} , independently of D



Does lenience guarantee a working latch?

What if D and G change at about the same time...

... with a little discipline



To reliably latch V2:

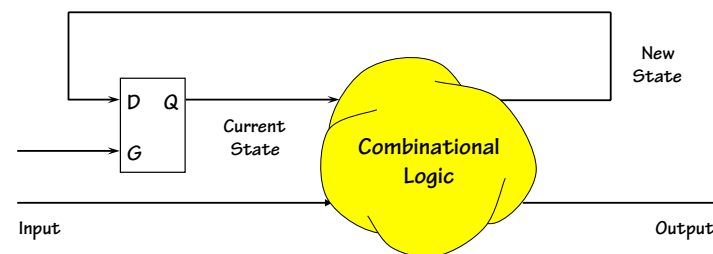
- Apply V2 to D, holding $G=1$
- After T_{PD} , V2 appears at $Q=Q'$
- After another T_{PD} , Q' & D both valid for T_{PD} ; will hold $Q=V2$ independently of G
- Set $G=0$, while Q' & D hold $Q=D$
- After another T_{PD} , $G=0$ and Q' are sufficient to hold $Q=V2$ independently of D

Dynamic Discipline for our latch:

$T_{SETUP} = 2T_{PD}$: interval prior to G transition for which D must be stable & valid

$T_{HOLD} = T_{PD}$: interval following G transition for which D must be stable & valid

Lets try it out!

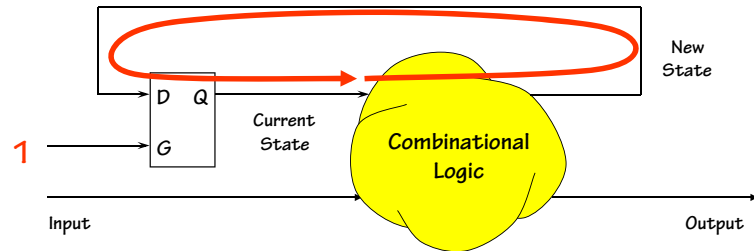


Plan: Build a Sequential Circuit with one bit of STATE -

- Single latch holds CURRENT state
- Combinational Logic computes
 - NEXT state (from input, current state)
 - OUTPUT bit (from input, current state)
- State changes when $G = 1$ (briefly!)

What happens when $G=1$?

Combinational Cycles



When $G=1$, latch is *Transparent*...

... provides a combinational path from D to Q.

Can't work without tricky timing constraints on $G=1$ pulse:

- Must fit within contamination delay of logic
- Must accommodate latch setup, hold times

Want to signal an *INSTANT*, not an *INTERVAL*...

Looks like a stupid
Approach to me...

Flakey Control Systems

Here's a strategy
for saving 3 bucks
on the Sumner
Tunnel!



Figure by MIT OpenCourseWare.

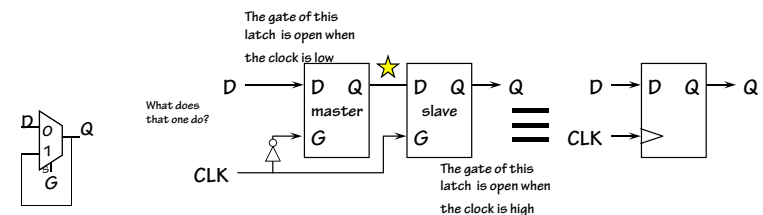
Escapement Strategy

The Solution:
Add two gates
and only open
one at a time.



Figure by MIT OpenCourseWare.

Edge-triggered Flip Flop



Observations:

Transitions mark
instants, not intervals

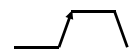
- ♦ only one latch “transparent” at any time:

- ♦ master closed when slave is open
- ♦ slave closed when master is open

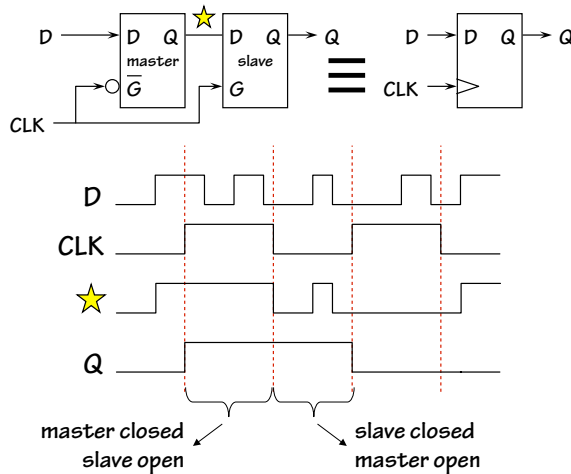
→ no combinational path through flip flop

(the feedback path in one of the master or slave latches is always active)

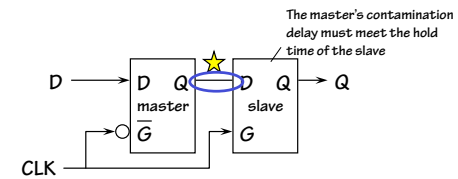
- ♦ Q only changes shortly after $0 \rightarrow 1$ transition of CLK, so flip flop **appears** to be “triggered” by rising edge of CLK



Flip Flop Waveforms



Um, about that hold time...

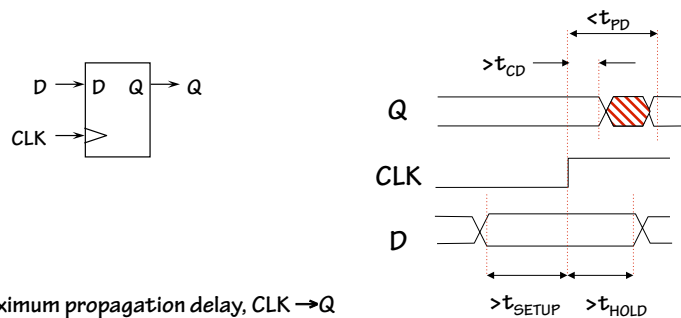


Consider HOLD TIME requirement for slave:

- Negative ($1 \rightarrow 0$) clock transition $\rightarrow$ slave freezes data:
 - SHOULD be no output glitch, since master held constant data; BUT
 - master output contaminated by change in G input!
- HOLD TIME of slave not met, UNLESS we assume sufficient contamination delay in the path to its D input!

Accumulated t_{CD} thru inverter, $G \rightarrow Q$ path of master must cover slave t_{HOLD} for this design to work!

Flip Flop Timing - I



t_{PD} : maximum propagation delay, CLK $\rightarrow$ Q

t_{CD} : minimum contamination delay, CLK $\rightarrow$ Q

t_{SETUP} : setup time

guarantee that D has propagated through feedback path before master closes

t_{HOLD} : hold time

guarantee master is closed and data is stable before allowing D to change

Single-clock Synchronous Circuits

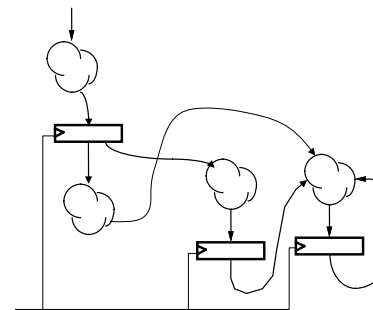


Does that symbol register?

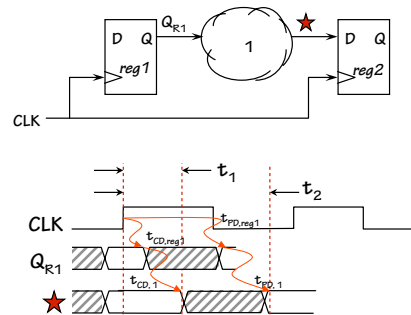
We'll use Flip Flops and *Registers* - groups of FFs sharing a clock input - in a highly constrained way to build digital systems:

Single-clock Synchronous Discipline

- No combinational cycles
- Single clock signal shared among all clocked devices
- Only care about value of register data inputs just before rising edge of clock
- Period greater than every combinational delay
- Change saved state after noise-inducing logic transitions have stopped!



Flip Flop Timing - II



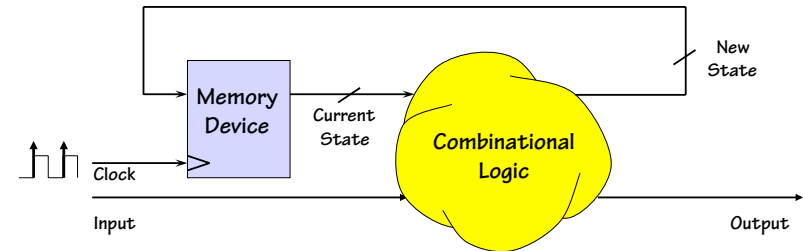
Questions for register-based designs:

- ♦ how much time for useful work (i.e. for combinational logic delay)?
- ♦ does it help to guarantee a minimum t_{CD} ? How 'bout designing registers so that $t_{CD,reg} > t_{HOLD,reg}$?
- ♦ what happens if CLK signal doesn't arrive at the two registers at exactly the same time (a phenomenon known as "clock skew")?

$$t_1 = t_{CD,reg1} + t_{CD,1} > t_{HOLD,reg2}$$

$$t_2 = t_{PD,reg1} + t_{PD,1} < t_{CLK} - t_{SETUP,reg2}$$

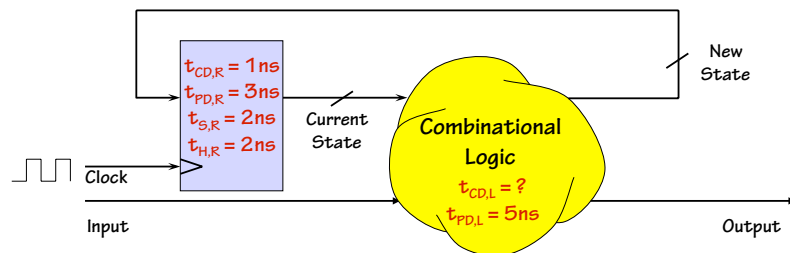
Model: Discrete Time



Active Clock Edges punctuate time ---

- Discrete Clock periods
- Discrete State Variables
- Discrete specifications (simple rules - eg tables - relating outputs to inputs, state variables)
- ABSTRACTION: Finite State Machines (next lecture!)

Sequential Circuit Timing



Questions:

- Constraints on T_{CD} for the logic? $> 1 \text{ ns}$
- Minimum clock period? $> 10 \text{ ns } (T_{PD,R} + T_{PD,L} + T_{S,R})$
- Setup, Hold times for Inputs? $T_S = T_{PD,L} + T_{S,R}$
 $T_H = T_{H,R} - T_{CD,L}$

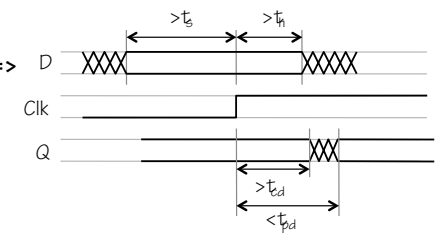
This is a simple *Finite State Machine* ... more next lecture!!

Summary

"Sequential" Circuits (with memory):

Basic memory elements:

- Feedback, detailed analysis => basic level-sensitive devices (eg, latch)
- 2 Latches => Flop
- Dynamic Discipline: constraints on input timing



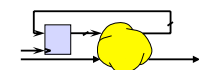
Synchronous 1-clock logic:

- Simple rules for sequential circuits
- Yields clocked circuit with T_S, T_H constraints on input timing



Finite State Machines

Next Lecture Topic!

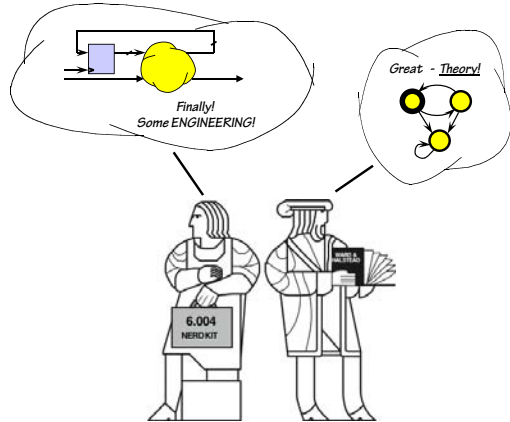


MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

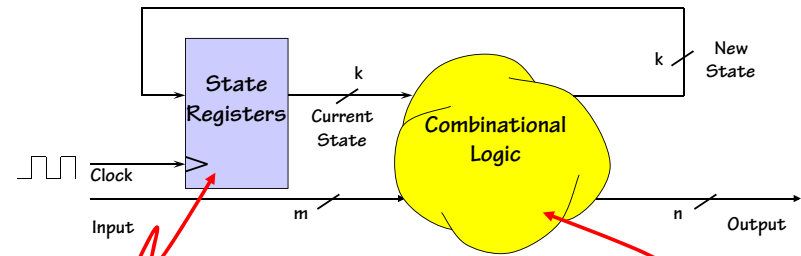
For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

(Synchronous) Finite State Machines



Lab 2 is due Thursday

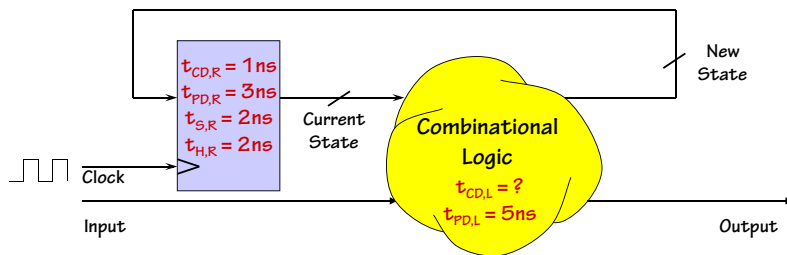
Our New Machine



- Engineered cycles
- Works only if dynamic discipline obeyed
- Remembers k bits for a total of 2^k unique combinations

- Acyclic graph
- Obeys static discipline
- Can be exhaustively enumerated by a truth table of 2^{k+m} rows and $k+n$ output columns

Must Respect Timing Assumptions!



Questions:

- Constraints on T_{CD} for the logic?
 - Minimum clock period?
 - Setup, Hold times for Inputs?
- $$t_{CD,R} (1 \text{ ns}) + t_{CD,L} (?) > t_{H,R} (2 \text{ ns})$$
- $$t_{CD,L} > 1 \text{ ns}$$
- $$t_{CLK} > t_{PD,R} + t_{PD,L} + t_{S,R} > 10 \text{ ns}$$
- $$t_S = t_{PD,L} + t_{S,R} = 7 \text{ ns}$$
- $$t_H = t_{H,R} - t_{CD,L} = 1 \text{ ns}$$

We know how fast it goes... But what can it do?

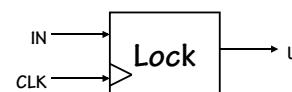
A simple sequential circuit...

Lets make a digital binary *Combination Lock*:

Specification:

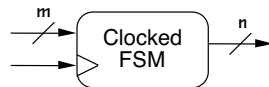
- One input ("0" or "1")
- One output ("Unlock" signal)
- UNLOCK is 1 if and only if:

Last 4 inputs were the "combination": 0110



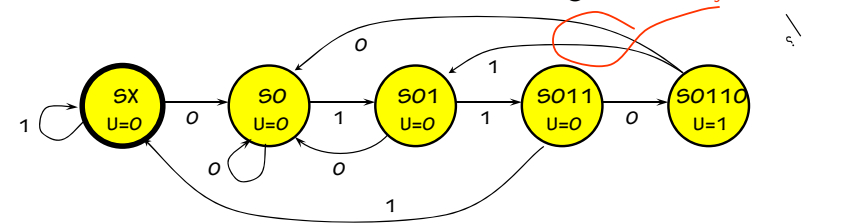
How many registers do I need?

Abstraction *du jour*: Finite State Machines



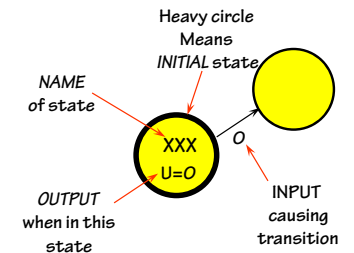
- A FINITE STATE MACHINE has
- k STATES: $S_1 \dots S_k$ (one is "initial" state)
- m INPUTS: $I_1 \dots I_m$
- n OUTPUTS: $O_1 \dots O_n$
- Transition Rules $s'(s, I)$ for each state s and input I
- Output Rules $Out(s)$ for each state s

State Transition Diagram

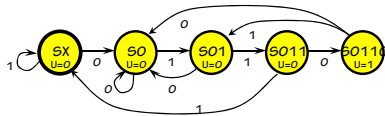


Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries



Yet Another Specification



IN	Current State	Next State	Unlock
0	SX 000	S0 0010	
1	SX 000	SX 0000	
0	S0 001	SX 0010	
1	S0 001	S01 0110	
0	S01 011	S0 0010	
1	S01 011	S011 0100	
0	S011 010	S0110 000	
1	S011 010	SX 0000	
0	S0110 100	S0 0011	
1	S0110 100	S01 0111	

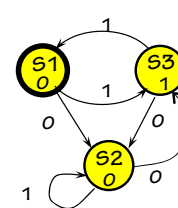
The assignment of codes to states can be arbitrary, however, if you choose them carefully you can greatly reduce your logic requirements.

All state transition diagrams can be described by truth tables...

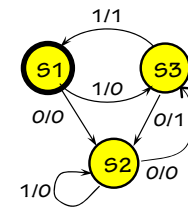
Binary encodings are assigned to each state (a bit of an art)

The truth table can then be simplified using the reduction techniques we learned for combinational logic

Valid State Diagrams



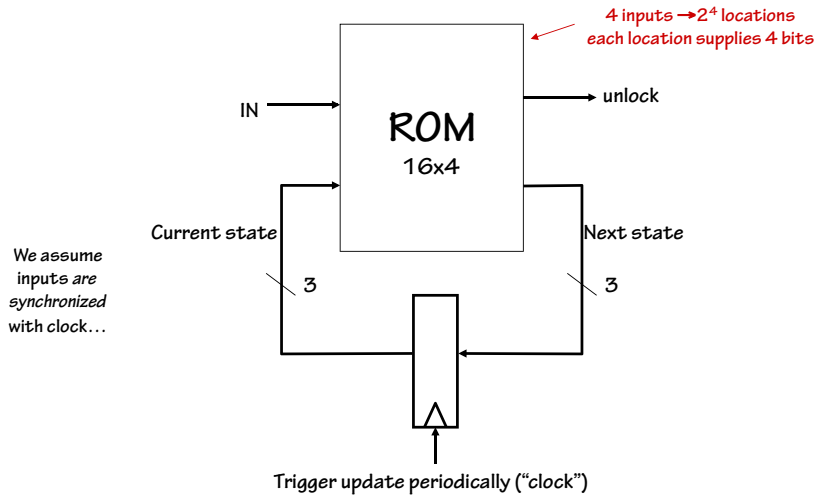
MOORE Machine:
Outputs on States



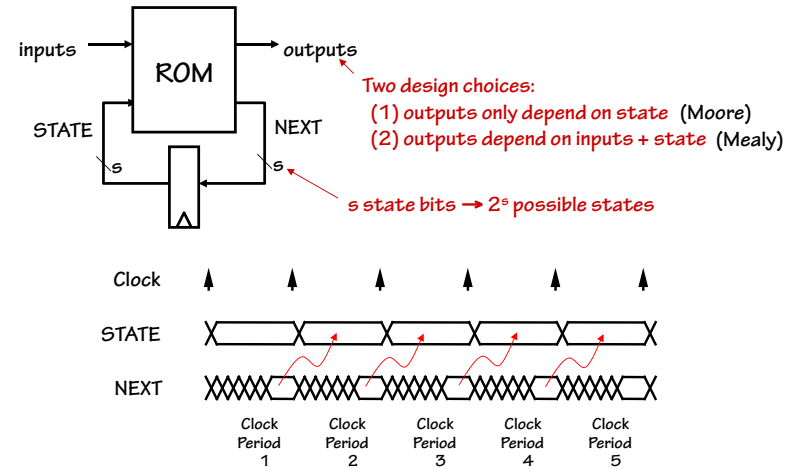
MEALY Machine:
Outputs on Transitions

- Arcs leaving a state must be:
 - (1) **mutually exclusive**
 - can't have two choices for a given input value
 - (2) **collectively exhaustive**
 - every state must specify what happens for each possible input combination. "Nothing happens" means arc back to itself.

Now put it in Hardware!



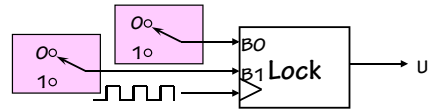
Discrete State, Time



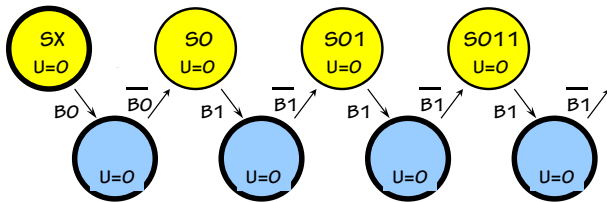
Asynchronous Inputs - I

Our example assumed a single clock transition per input. What if the “button pusher” is unaware of, or not synchronized with, the clock?

What if each button input is an asynchronous 0/1 level? How do we prevent a single button press, e.g., from making several transitions?



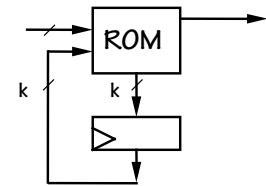
But what
About the
Dynamic
Discipline?



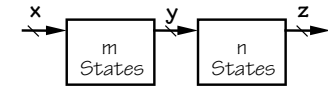
Use *intervening states* to synchronize button presses!

FSM Party Games

1. What can you say about the number of states?



2. Same question:



3. Here's an FSM. Can you discover its rules?

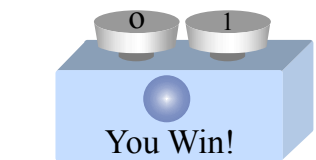
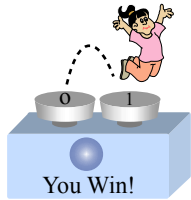


Figure by MIT OpenCourseWare.

What's My Transition Diagram?



0=OFF,
1=ON?

"1111" =
Surprise!

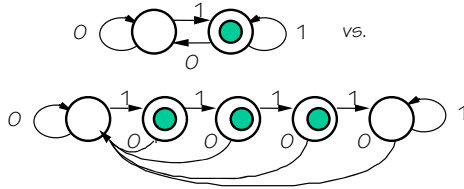


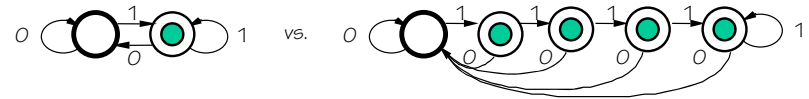
Figure by MIT OpenCourseWare.

- If you know NOTHING about the FSM, you're never sure!
- If you have a BOUND on the number of states, you can discover its behavior:

K-state FSM: Every (reachable) state can be reached in $< k$ steps.

BUT ... states may be **equivalent**!

FSM Equivalence



ARE THEY DIFFERENT?

NOT in any practical sense! They are EXTERNALLY INDISTINGUISHABLE, hence interchangeable.

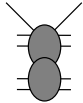
FSMs **EQUIVALENT** iff every input sequence yields identical output sequences.

ENGINEERING GOAL:

- HAVE an FSM which *works*...
- WANT simplest (ergo cheapest) equivalent FSM.

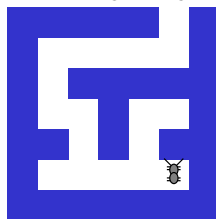
Lets build an Ant

8 legs?



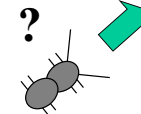
- SENSORS: antennae L and R, each 1 if in contact with something.
- ACTUATORS: Forward Step F, ten-degree turns TL and TR (left, right).

GOAL: Make our ant smart enough to get out of a maze like:

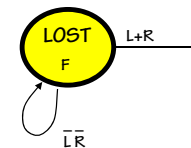


STRATEGY: "Right antenna to the wall"

Lost in space

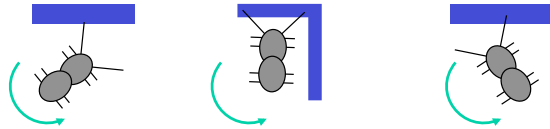


Action: Go forward until we hit something.

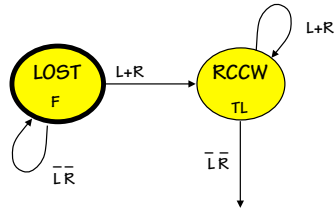


"lost" is the initial state

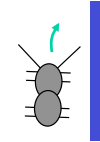
Bonk!



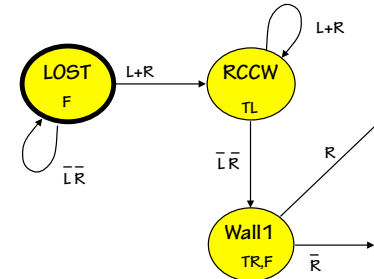
Action: Turn left (CCW) until we don't touch anymore



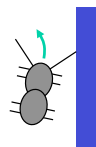
A little to the right...



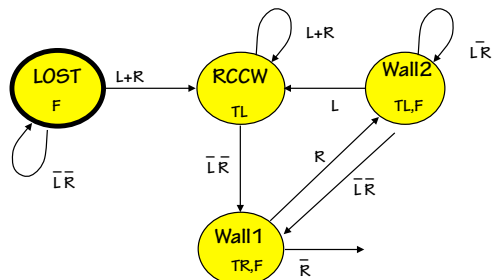
Action: Step and turn right a little, look for wall



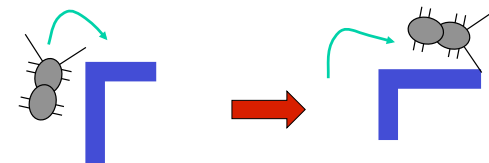
Then a little to the left



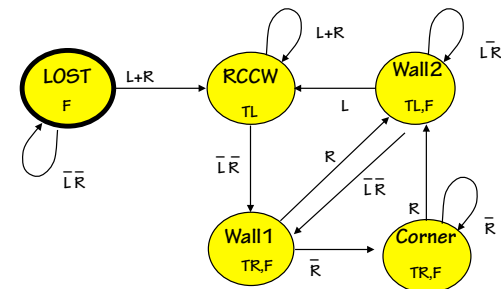
Action: Step and turn left a little, till not touching (again)



Dealing with corners



Action: Step and turn right until we hit perpendicular wall



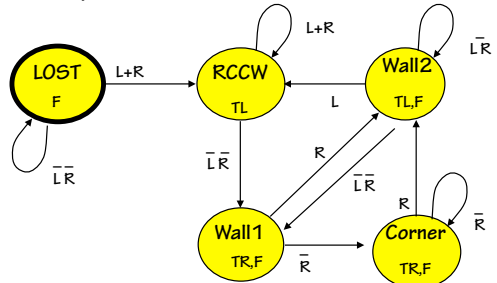
Equivalent State Reduction

Observation: $S_i \equiv S_j$ if

1. States have identical outputs; AND
2. Every input $\rightarrow$ equivalent states.

Reduction Strategy:

Find pairs of equivalent states, MERGE them.



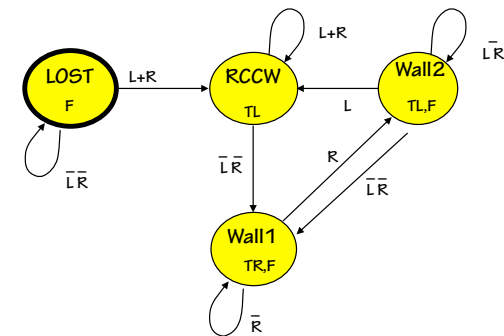
6.004 - Spring 2009

2/24/09

LOG - FSMs 21

An Evolutionary Step

Merge equivalent states Wall1 and Corner into a single new, combined state.



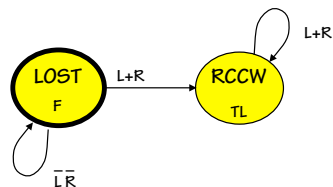
Behaves exactly as previous (5-state) FSM, but requires half the ROM in its implementation!

6.004 - Spring 2009

2/24/09

LOG - FSMs 22

Building the Transition Table



S	L	R	S'	TR	TL	F
00	0	0	00	0	0	1
00	1	-	01	0	0	1
00	0	1	01	0	0	1
01	1	-	01	0	1	0
01	0	1	01	0	1	0

6.004 - Spring 2009

2/24/09

LOG - FSMs 23

Implementation Details

	S	L	R	S'	TR	TL	F
LOST	00	0	0	00	0	0	1
	00	1	-	01	0	0	1
	00	0	1	01	0	0	1
	01	1	-	01	0	1	0
RCCW	01	0	1	01	0	1	0
	01	0	0	10	0	1	0
WALL1	10	-	0	10	1	0	1
	11	1	-	01	0	1	1
WALL2	11	0	0	10	0	1	1
	11	0	1	11	0	1	1

Complete Transition table

S1'	S1S0			
	00	01	11	10
00	0	1	1	1
LR 01	0	0	1	1
11	0	0	0	1
10	0	0	0	1

$$S'_1 = S_1 \overline{S_0} + \overline{L} S_1 + \overline{L} R S_0$$

S0'	S1S0			
	00	01	11	10
00	0	0	0	0
LR 01	1	1	1	1
11	1	1	1	1
10	1	1	1	0

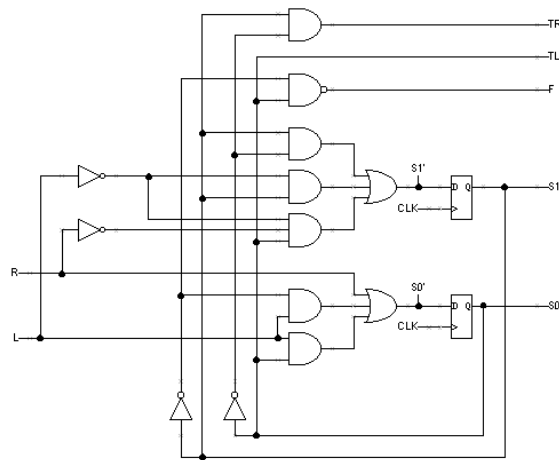
$$S'_0 = R + \overline{L} S_1 + L S_0$$

6.004 - Spring 2009

2/24/09

LOG - FSMs 24

Ant Schematic



Roboant®

FSM state table

Maze selection

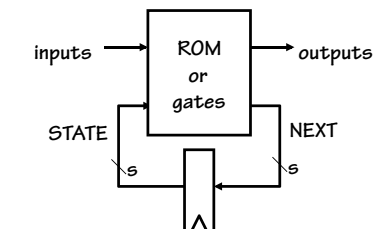
Plan view of maze

Simulation controls

Status display

Featuring the new Mark-II ant: can add (M), erase (E), and sense (S) marks along its path.

Housekeeping issues...



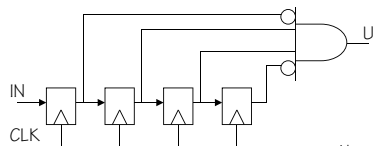
1. Initialization? Clear the memory?

2. Unused state encodings?

- waste ROM (use PLA or gates)
- what does it mean?
- can the FSM recover?

3. Choosing encoding for state?

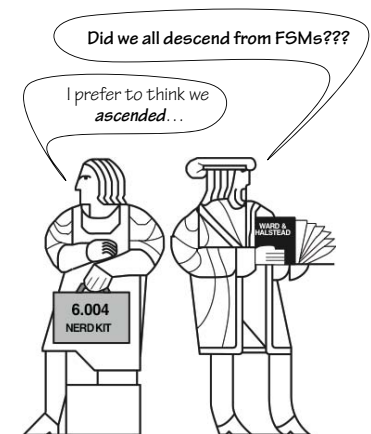
4. Synchronizing input changes with state update?



Now, that's a funny looking state machine

Twisting you Further...

- **MORE THAN ANTS:**
Swarming, flocking, and schooling can result from collections of very simple FSMs
 - **PERHAPS MOST PHYSICS:**
Cellular automata, arrays of simple FSMs, can more accurately model fluids than numerical solutions to PDEs
 - **WHAT IF:**
We replaced the ROM with a RAM and have outputs that modify the RAM?
- ... You'll see FSMs for the rest of your life!



MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

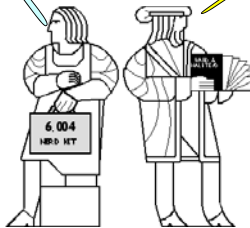
Synchronization, Metastability and Arbitration

Did you vote for Bush or Gore?

Didn't have enough time to decide.

Well, which hole did you punch?

Both, but not very hard...



"If you can't be just,
be arbitrary"

- Wm Burroughs, *Naked Lunch*
- US Supreme Court 12/00

Due tonight:

- ☐ Lab #2
- ☐ Lab #1 checkoff meeting

6.004 - Spring 2009

2/26/09

modified 2/23/09 09:30

L07 - Synchronization 1

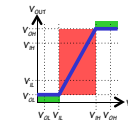
The Importance of being Discrete

We avoid possible errors by disciplines that avoid asking the tough questions – using a *forbidden zone* in both voltage and time dimensions:

Digital Values:

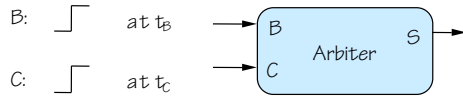
Problem: Distinguishing voltages representing “1” from “0”

Solution: *Forbidden Zone*: avoid using similar voltages for “1” and “0”



The Asynchronous Arbiter: a classic problem

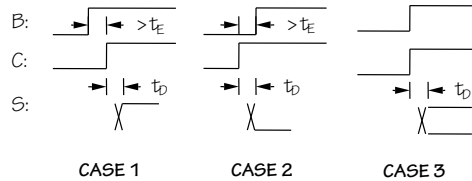
UNSOLVABLE



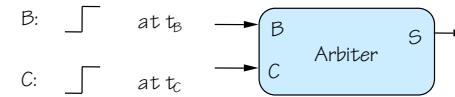
For NO finite value of t_E and t_D is this spec realizable, even with reliable components!

Arbiter specifications:

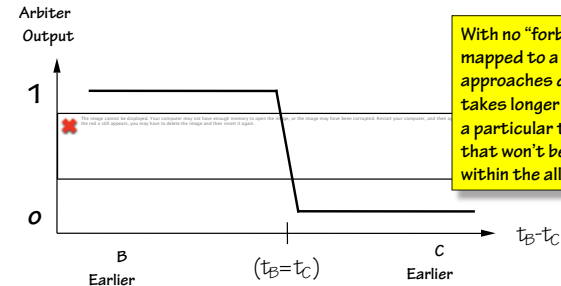
- finite t_D (decision time)
- finite t_E (allowable error)
- value of S at time $t_C + t_D$:
 1 if $t_B < t_C - t_E$
 0 if $t_B > t_C + t_E$
 $0, 1$ otherwise



Violating the Forbidden Zone



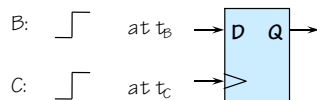
Issue: Mapping the continuous variable $(t_B - t_C)$ onto the discrete variable S in bounded time.



With no "forbidden zone," all inputs have to be mapped to a valid output. As the input approaches discontinuities in the mapping, it takes longer to determine the answer. Given a particular time bound, you can find an input that won't be mapped to a valid output within the allotted time.

Unsolvability that can't be true...

Lets just use a D Flip Flop:



DECISION TIME is T_{PD} of flop.

ALLOWABLE ERROR is $\max(t_{SETUP}, t_{HOLD})$

Our logic:

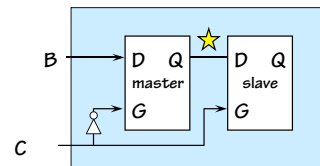
T_{PD} after T_C , we'll have

$Q=0$ iff $t_B + t_{SETUP} < t_C$

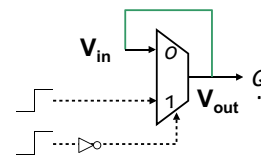
$Q=1$ iff $t_C + t_{HOLD} < t_B$

$Q=0$ or 1 otherwise.

We're lured by the digital abstraction into assuming that Q must be either 1 or 0 . But lets look at the input latch in the flip flop when B and C change at about the same time...

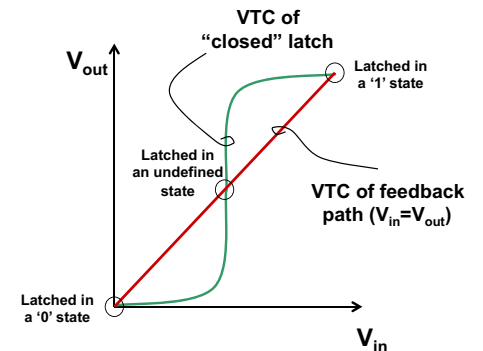


The Mysterious Metastable State



Recall that the latch output is the solution to two simultaneous constraints:

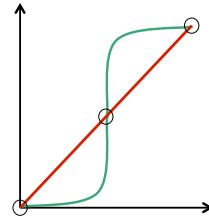
1. The VTC of path thru MUX; and
2. $V_{in} = V_{out}$.



In addition to our expected stable solutions, we find an unstable equilibrium in the forbidden zone called the "Metastable State"

Metastable State: Properties

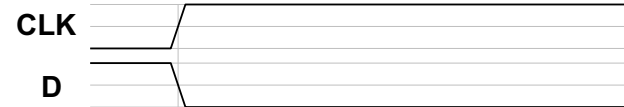
1. It corresponds to an *invalid* logic level – the switching threshold of the device.
2. Its an *unstable* equilibrium; a small perturbation will cause it to accelerate toward a stable 0 or 1.
3. It will settle to a valid 0 or 1... eventually.
4. BUT – depending on how close it is to the $V_{in}=V_{out}$ “fixed point” of the device – it may take arbitrarily long to settle out.
5. EVERY bistable system exhibits at least one metastable state!



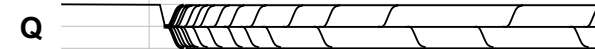
EVERY bistable system?
 Yep, every last one.
 Coin flip??
 Could land on edge.
 Horse race??
 Photo finish.
 Presidential Election??
 (Where's this twit
 been hiding???)

Observed Behavior: typical metastable symptoms

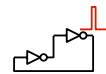
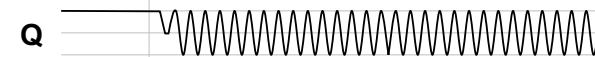
Following a clock edge on an asynchronous input:



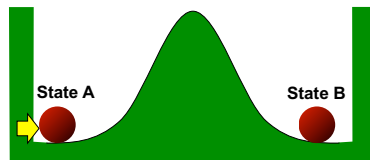
We may see exponentially-distributed metastable intervals:



Or periods of high-frequency oscillation (if the feedback path is long):



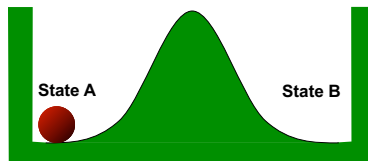
Mechanical Metastability



Metastable State

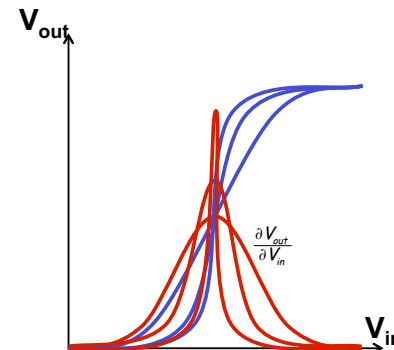
If we launch a ball up a hill we expect one of 3 possible outcomes:

- a) Goes over
- b) Rolls back
- c) Stalls at the apex



That last outcome is not stable.
 - a gust of wind
 - Brownian motion
 - it doesn't take much

How do balls relate to digital logic?



Our hill is analogous to the *derivative* of the VTC (Voltage Transfer Curve)... at the metastable point, the derivative (slope) is ZERO.

Notice that the higher the gain thru the transition region, the steeper the peak of the hill... making it harder to get into a metastable state...

We can decrease the probability of getting into the metastable state, but – assuming continuous models of physics – we can't eliminate the slope=0 point!

The Metastable State:

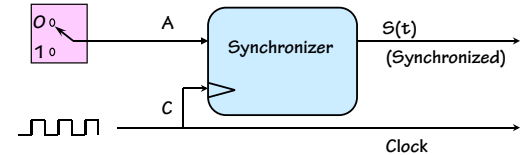
Why is it an inevitable risk of synchronization?

- Our active devices always have a fixed-point voltage, V_M , such that $V_{IN}=V_M$ implies $V_{OUT} = V_M$
- Violation of dynamic discipline puts our feedback loop at some voltage V_O near V_M
- The rate at which V progresses toward a stable "0" or "1" value is proportional to $(V - V_M)$
- The time to settle to a stable value depends on $(V_O - V_M)$; its theoretically infinite for $V_O = V_M$
- Since there's no lower bound on $(V_O - V_M)$, there's no upper bound on the settling time.
- Noise, uncertainty complicate analysis (but don't help).

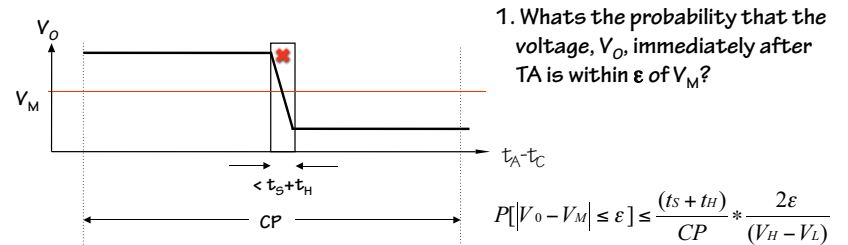
Sketch of analysis... I.

Assume asynchronous $O \rightarrow 1$
at T_A , clock period CP:

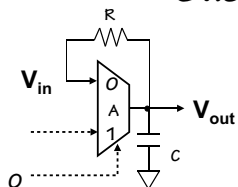
Whats the FF output voltage,
 V_O , immediately after T_A ?



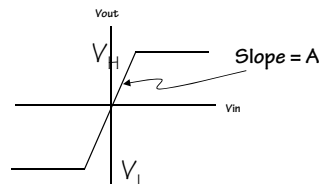
Potential trouble comes when V_O is near the metastable point, V_M ...



Sketch of analysis... II.



We can model our combinational cycle as an amplifier with gain A and saturation at V_H, V_L



2. For V_{out} near V_M , $V_{out}(t)$ is an exponential whose time constant reflects RC/A :

$$V_{out}(t) - V_M \cong \epsilon e^{t(A-1)/RC}$$

$$\cong \epsilon e^{t/\tau}$$

3. Given interval T , we can compute a minimum value of $\epsilon = |V_O - V_M|$ that will guarantee validity after T :

$$\epsilon(T) = (V_H - V_M) e^{-T/\tau}$$

4. Probability of metastability after T is computed by probability of a V_O yielding $\epsilon(T)$...

$$P_M(T) \cong P[|V_O - V_M| < \epsilon(T)]$$

$$\cong K e^{-T/\tau}$$

Failure Probabilities vs Delay

Making conservative assumptions about the distribution of V_O and system time constants, and assuming a 100 MHz clock frequency, we get results like the following:

Delay	P(Metastable)	Average time between failures
31 ns	3×10^{-16}	1 year
33.2 ns	3×10^{-17}	10 years
100 ns	10^{-45}	10^{30} years!

[For comparison:

Age of oldest hominid fossil: 5×10^6 years
Age of earth: 5×10^9 years]

Lesson: Allowing a bit of settling time is an easy way to avoid metastable states in practice!

The Metastable State:

a brief history

Antiquity: Early recognition	Buriden's Ass, and other fables...
Denial: Early 70s	Widespread disbelief. Early analyses documenting inevitability of problem rejected by skeptical journal editors.
Folk Cures: 70s-80s	Popular pastime: Concoct a "Cure" for the problem of "synchronization failure". Commercial synchronizer products.
Reconciliation: 80s-90s	Acceptance of the reality: synchronization takes time. Interesting special case solutions.

Ancient Metastability

Metastability is the occurrence of a persistent invalid output... an unstable equilibria.

The idea of Metastability is not new:

The Paradox of Buridan's Ass

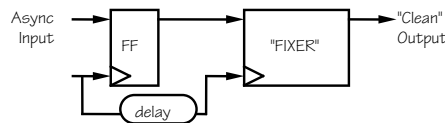
Buridan, Jean (1300-58), French Scholastic philosopher, who held a theory of determinism, contending that the will must choose the greater good. Born in Bethune, he was educated at the University of Paris, where he studied with the English Scholastic philosopher William of Ockham (whom you might recall from his razor business). After his studies were completed, he was appointed professor of philosophy, and later rector, at the same university. Buridan is traditionally, but probably incorrectly, associated with a philosophical dilemma of moral choice called "Buridan's ass."

In the problem an ass starves to death between two alluring bundles of hay because it does not have the will to decide which one to eat.

Folk Cures

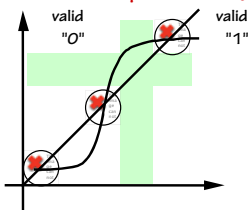
the "perpetual motion machine" of digital logic

Bad Idea # 1: Detect metastable state & Fix



Bug: detecting metastability is itself subject to metastable states, i.e., the "fixer" will fail to resolve the problem in bounded time.

Bad Idea #2: Define the problem away by making metastable point a valid output



Bug: the memory element will flip some valid "0" inputs to "1" after a while.

Many other bad ideas - involving noise injection, strange analog circuitry, ... have been proposed.

There's no easy solution

... so, embrace the confusion.

"Metastable States":

- Inescapable consequence of bistable systems
- Eventually a metastable state will resolve itself to valid binary level.
- However, the recovery time is UNBOUNDED ... but influenced by parameters (gain, noise, etc)
- Probability of a metastable state falls off EXPONENTIALLY with time -- modest delay after state change can make it very unlikely.

Our STRATEGY; since we can't eliminate metastability, we will do the best we can to keep it from contaminating our designs

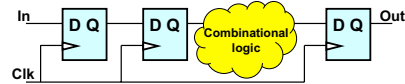
Modern Reconciliation:

delay buys reliability

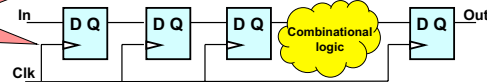
Synchronizers, extra flip flops between the asynchronous input and your logic, are the best insurance against metastable states.

The higher the clock rate, the more synchronizers should be considered.

A metastable state here will probably resolve itself to a valid level before it gets into my circuit.

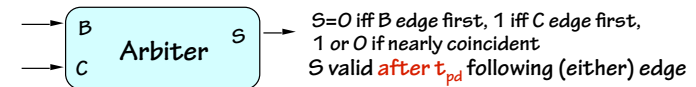


And one here will almost certainly get resolved.

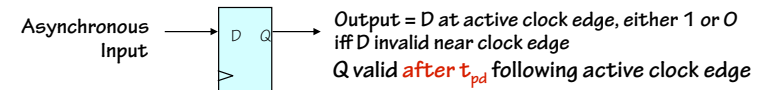


Things we CAN'T build

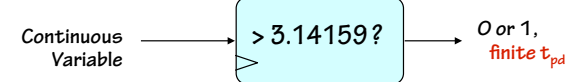
1. Bounded-time Asynchronous Arbiter:



2. Bounded-time Synchronizer:

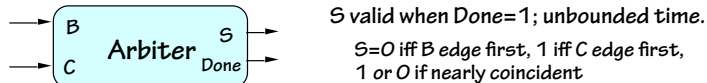


3. Bounded-time Analog Comparator:

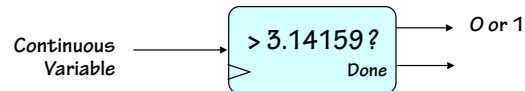


Some things we CAN build

1. Unbounded-time Asynchronous Arbiter:

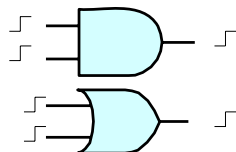


2. Unbounded-time Analog Comparator:



After arbitrary interval, decides whether input at time of last active clock edge was above/below threshold.

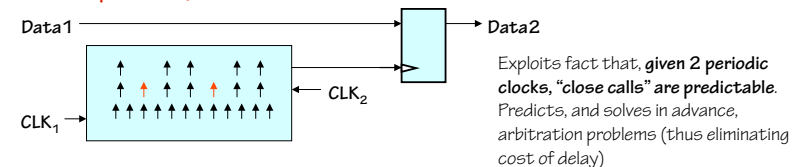
3. Bounded-time combinational logic:



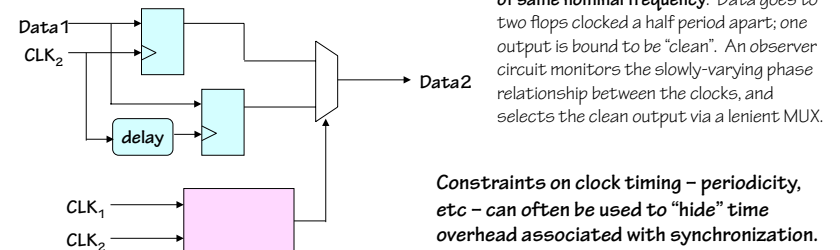
Produce an output transition within a fixed propagation delay of first (or second) transition on the input.

Interesting Special Case Hacks

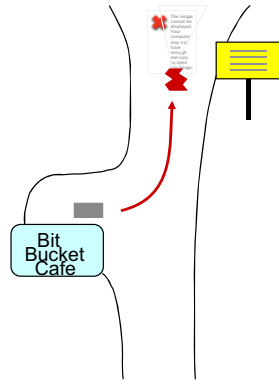
Predictive periodic synchronization:



Mesochronous communication:



Every-day Metastability - I



Ben Bitdiddle tries the famous "6.004 defense":

Ben leaves the Bit Bucket Café and approaches fork in the road. He hits the barrier in the middle of the fork, later explaining "I can't be expected to decide which fork to take in bounded time!".

Is the accident Ben's fault?

"Yes; he should have stopped until his decision was made."

Judge R. B. Trator, MIT '86

Every-day Metastability - II

GIVEN:

- Normal traffic light: GREEN, YELLOW, RED sequence
- 55 MPH Speed Limit
- Sufficiently long YELLOW, GREEN periods
- Analog POSITION input
- digital RED, YELLOW, GREEN inputs
- digital GO output

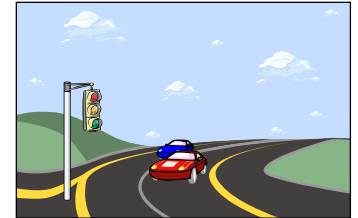


Image by MIT OpenCourseWare.

Can one reliably obey....

- LAW #1: DON'T CROSS LINE while light is RED.
GO = GREEN
- LAW #2: DON'T BE IN INTERSECTION while light is RED.

PLAUSIBLE STRATEGIES:

A. Move at 55. At calculated distance D from light, sample color (using an unbounded-time synchronizer). GO ONLY WHEN stable GREEN.

B. Stop 1 foot before intersection. On positive GREEN transition, gun it.

Summary

*The most difficult decisions
are those that matter the least.*

As a system designer...

Avoid the problem altogether, where possible

- Use single clock, obey dynamic discipline
- Avoid state. Combinational logic has no metastable states!

Delay after sampling asynchronous inputs: a fundamental cost of synchronization

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Pipelining

what Seymour Cray taught the laundry industry



Due Thursday: Lab #3

Forget circuits... lets solve a "Real Problem"

INPUT:
dirty laundry



OUTPUT:
6 more weeks



Device: Washer

Function: Fill, Agitate, Spin

Washer_{PD} = 30 mins



Device: Dryer

Function: Heat, Spin

Dryer_{PD} = 60 mins

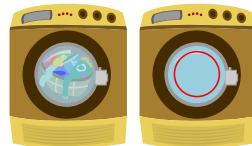
Figure by MIT OpenCourseware.

One load at a time

Everyone knows that the real reason that MIT students put off doing laundry so long is not because they procrastinate, are lazy, or even have better things to do.

The fact is, doing one load at a time is not smart.

Step 1:



Step 2:



Figure by MIT OpenCourseware.

$$\begin{aligned} \text{Total} &= \text{Washer}_{PD} + \text{Dryer}_{PD} \\ &= \underline{90} \text{ mins} \end{aligned}$$

Doing N loads of laundry

Here's how they do laundry at Harvard, the "combinational" way.

(Of course, this is just an urban legend. No one at Harvard actually *does* laundry. The butlers all arrive on Wednesday morning, pick up the dirty laundry and return it all pressed and starched in time for afternoon tea)



Image by MIT OpenCourseWare.

Step 1:



Step 2:



Step 3:



Step 4:



Figure by MIT OpenCourseware.

$$\begin{aligned} \text{Total} &= N * (\text{Washer}_{PD} + \text{Dryer}_{PD}) \\ &= \underline{N * 90} \text{ mins} \end{aligned}$$

Doing N Loads... the MIT way

MIT students “pipeline”
the laundry process.

That’s why we wait!

Actually, it’s more like $N \cdot 60 + 30$ if we account for the startup transient correctly. When doing pipeline analysis, we’re mostly interested in the “steady state” where we assume we have an infinite supply of inputs.

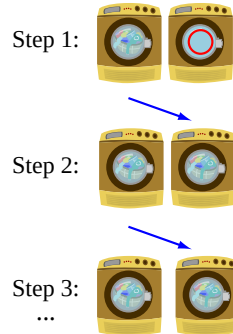


Figure by MIT OpenCourseware.

$$\text{Total} = N \cdot \text{Max}(\text{Washer}_{PD}, \text{Dryer}_{PD})$$

$$= N \cdot 60 \text{ mins}$$

Performance Measures

Latency:

The delay from when an input is established until the output associated with that input becomes valid.

$$\begin{aligned} \text{(Harvard Laundry)} &= \frac{90}{120} \text{ mins} \\ \text{(MIT Laundry)} &= \frac{120}{120} \text{ mins} \end{aligned}$$

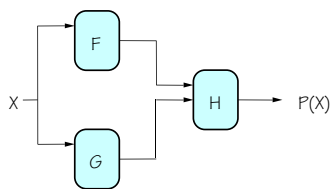
Assuming that the wash is started as soon as possible and waits (wet) in the washer until dryer is available.

Throughput:

The rate at which inputs or outputs are processed.

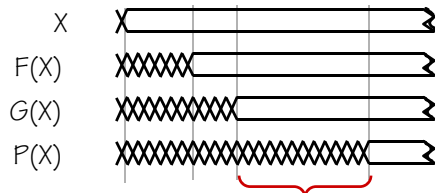
$$\begin{aligned} \text{(Harvard Laundry)} &= \frac{1}{90} \text{ outputs/min} \\ \text{(MIT Laundry)} &= \frac{1}{60} \text{ outputs/min} \end{aligned}$$

Okay, back to circuits...



For combinational logic:
latency = t_{PD} ,
throughput = $1/t_{PD}$.

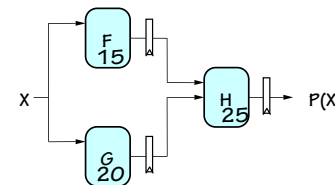
We can’t get the answer faster, but are we making effective use of our hardware at all times?



F & G are “idle”, just holding their outputs stable while H performs its computation

Pipelined Circuits

use registers to hold H’s input stable!

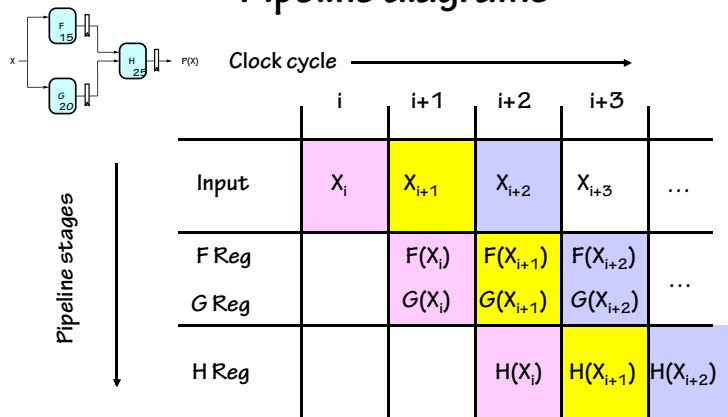


Now F & G can be working on input X_{i+1} while H is performing its computation on X_i . We’ve created a 2-stage **pipeline**: if we have a valid input X during clock cycle j, P(X) is valid during clock cycle j+2.

Suppose F, G, H have propagation delays of 15, 20, 25 ns and we are using ideal zero-delay registers:

	latency	throughput
unpipelined	45	1/45
2-stage pipeline	50	1/25
	worse	better

Pipeline diagrams



The results associated with a particular set of input data moves *diagonally* through the diagram, progressing through one pipeline stage each clock cycle.

Pipeline Conventions

DEFINITION:

a **K-Stage Pipeline** ("K-pipeline") is an acyclic circuit having exactly K registers on every path from an input to an output.

a COMBINATIONAL CIRCUIT is thus an 0-stage pipeline.

CONVENTION:

Every pipeline stage, hence every K-Stage pipeline, has a register on its **OUTPUT** (not on its input).

ALWAYS:

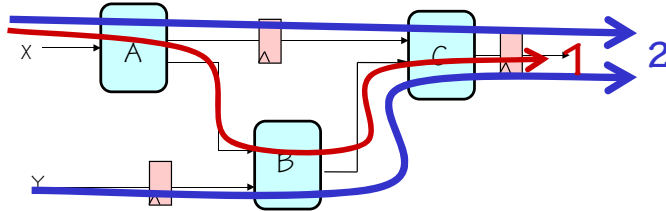
The CLOCK common to all registers must have a period sufficient to cover propagation over combinational paths PLUS (input) register t_{PD} PLUS (output) register t_{SETUP} .

The LATENCY of a K-pipeline is K times the period of the clock common to all registers.

The THROUGHPUT of a K-pipeline is the frequency of the clock.

Ill-formed pipelines

Consider a BAD job of pipelining:



For what value of K is the following circuit a K-Pipeline? **ANS: none**

Problem:

Successive inputs get mixed: e.g., $B(A(X_{i+1}), Y_i)$. This happened because some paths from inputs to outputs have 2 registers, and some have only 1!

This CAN'T HAPPEN on a well-formed K pipeline!

A pipelining methodology

Step 1:

Draw a line that crosses every output in the circuit, and mark the endpoints as terminal points.

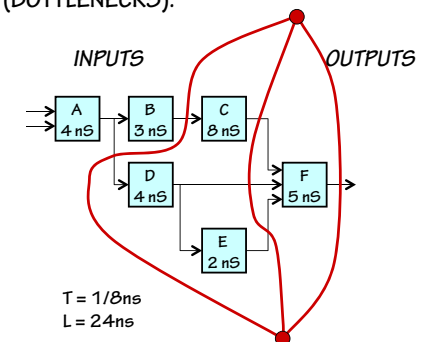
Step 2:

Continue to draw new lines between the terminal points across various circuit connections, ensuring that every connection crosses each line in the same direction. These lines demarcate pipeline stages.

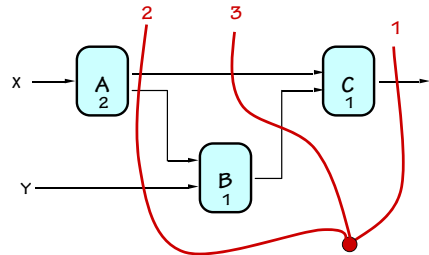
Adding a pipeline register at every point where a separating line crosses a connection will always generate a valid pipeline.

STRATEGY:

Focus your attention on placing pipelining registers around the slowest circuit elements (BOTTLENECKS).



Pipeline Example



OBSERVATIONS:

- 1-pipeline improves neither L or T.
- T improved by breaking long combinational paths, allowing faster clock.
- Too many stages cost L, don't improve T.
- Back-to-back registers are often required to keep pipeline well-formed.

	LATENCY	THROUGHPUT
0-pipe:	4	1/4
1-pipe:	4	1/4
2-pipe:	4	1/2
3-pipe:	6	1/2

Pipelining Summary

Advantages:

- Allows us to increase thruput, by breaking up long combinational paths and (hence) increasing clock frequency

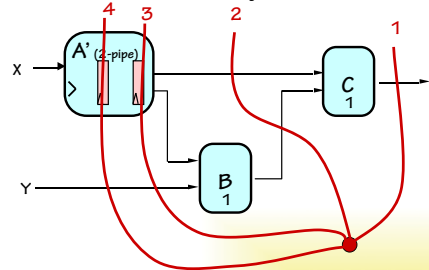
Disadvantages:

- May increase latency...
- Only as good as the weakest link: slowest step constrains system thruput.

This bottleneck is the only problem

Isn't there a way around this "weak link" problem?

Pipelined Components



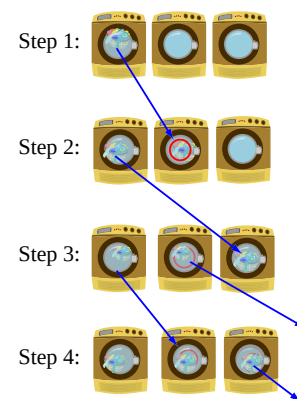
4-stage pipeline, thruput=1

but... but...
How can I pipeline a clothes dryer???

Pipelined systems can be hierarchical:

- Replacing a slow combinational component with a k-pipe version may increase clock frequency
- Must account for new pipeline stages in our plan

How do 6.004 Aces do Laundry?



They work around the bottleneck.
First, they find a place with twice as many dryers as washers.

Throughput = 1/30 loads/min

Latency = 90 mins/load

Figure by MIT OpenCourseware.

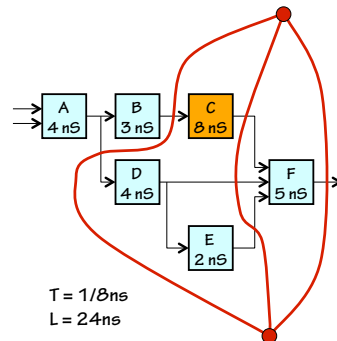
Back to our bottleneck...

Recall our earlier example...

- C – the slowest component – limits clock period to 8 ns.
- HENCE throughput limited to 1/8ns.

We could improve throughput by

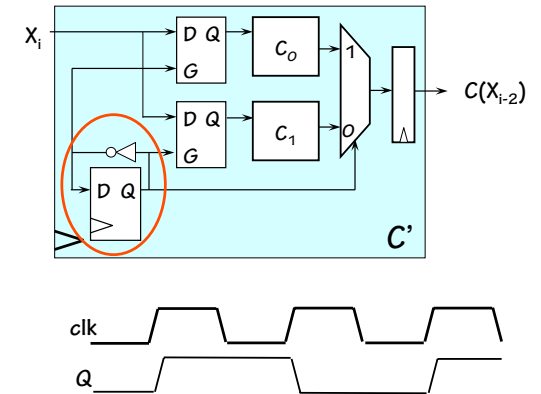
- Finding a pipelined version of C;
- OR ...
- *interleaving* multiple copies of C!



Circuit Interleaving

We can simulate a pipelined version of a slow component by replicating the critical element and alternate inputs between the various copies.

This is a simple 2-state FSM that alternates between 0 and 1 on each clock

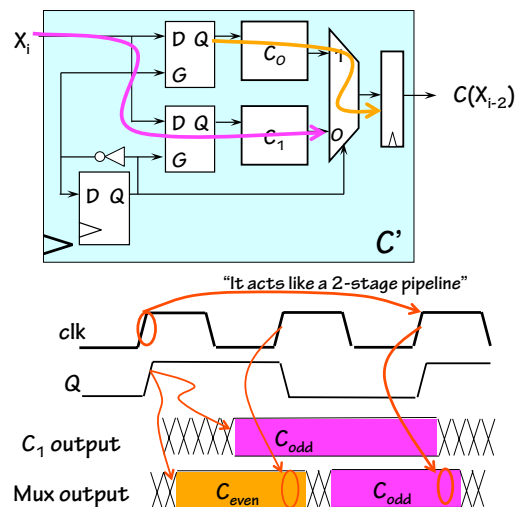


Circuit Interleaving

We can simulate a pipelined version of a slow component by replicating the critical element and alternate inputs between the various copies.

When Q is 1 the lower path is combinational (the latch is open), yet the output of the upper path will be enabled onto the input of the output register ready for the NEXT clock edge.

Meanwhile, the other latch maintains the input from the last clock.

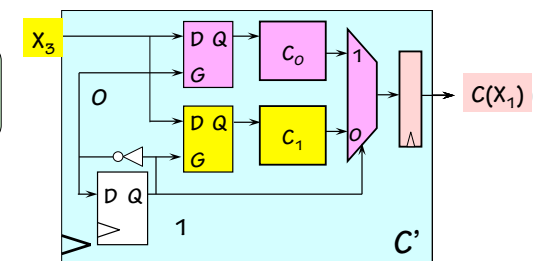
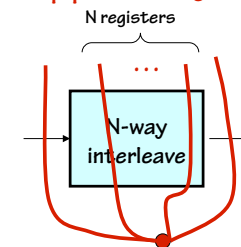


Circuit Interleaving

2-Clock Martinizing

"In by t_i , out by t_{i+2} "

N-way interleaving is equivalent to N pipeline Stages...



Latency = 2 clocks

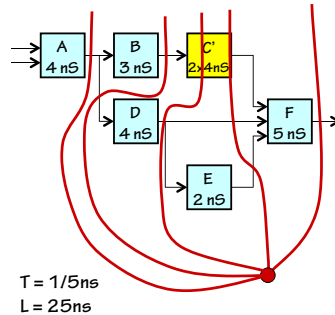
- Clock period 0: X_0 presented at input, propagates thru upper latch, C_0 .
- Clock period 1: X_1 presented at input, propagates thru lower latch, C_1 . $C_0(X_0)$ propagates to register inputs.
- Clock period 2: X_2 presented at input, propagates thru upper latch, C_0 . $C_0(X_0)$ loaded into register, appears at output.

Combining techniques

We can combine interleaving and pipelining. Here, C' interleaves two C elements with a propagation delay of 8 nS.

The resulting C' circuit has a throughput of 1/4 nS, and latency of 8 nS. This can be considered as an extra pipelining stage that passes through the middle of the C' module. One of our separation lines must pass through this pipeline stage.

By combining interleaving with pipelining we move the bottleneck from the C element to the F element.



And a little parallelism...



We can combine interleaving and pipelining with parallelism.

Throughput = $\frac{2}{30} \approx 1/15$ load/min

Latency = 90 min

Figure by MIT OpenCourseware.

Control Structure Approaches

Synchronous

ALL computation "events" occur at active edges of a periodic clock: time is divided into fixed-size discrete intervals.

RIGID

Globally Timed

Timing dictated by centralized FSM according to a fixed schedule.

Asynchronous

Events -- eg the loading of a register -- can happen at arbitrary times.

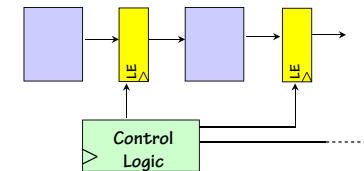
Laid Back

Locally Timed

Each module takes a START signal, generates a FINISHED signal. Timing is dynamic, data dependent.

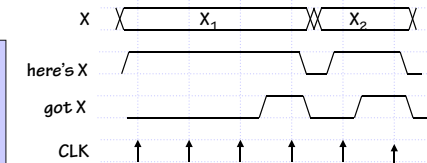
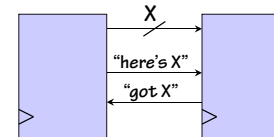
Control Structure Alternatives

Synchronous, globally-timed:
Control signals (e.g., load enables)
From FSM controller

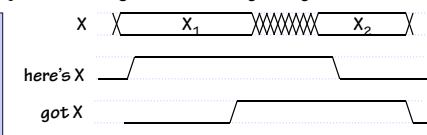
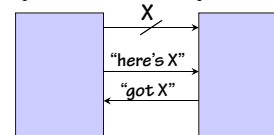


Synchronous, locally-timed:
Local circuitry, "handshake" controls

flow of data:

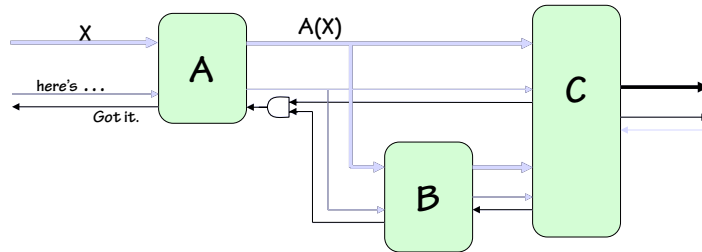


Asynchronous, locally-timed system using transition signaling:



Self-timed Example

a glimpse of an asynchronous, locally-time discipline



Elegant, timing-independent design:

- Each component specifies its own time constraints
- Local adaptation to special cases (eg, multiplication by 0)
- Module performance improvements automatically exploited
- Can be made asynchronous (no clock at all!) or synchronous

Control Structure Taxonomy

Easy to design but fixed-sized interval can be wasteful (no data-dependencies in timing)

Large systems lead to very complicated timing generators... just say no!

	Synchronous	Asynchronous
Globally Timed	Centralized clocked FSM generates all control signals.	Central control unit tailors current time slice to current tasks.
Locally Timed	Start and Finish signals generated by each major subsystem, synchronously with global clock.	Each subsystem takes asynchronous Start, generates asynchronous Finish (perhaps using local clock).

The best way to build large systems that have independently-timed components.

The "next big idea" for the last several decades: a lot of design work to do in general, but extra work is worth it in special cases

Summary

- Latency (L) = time it takes for given input to arrive at output
- Throughput (T) = rate at each new outputs appear
- For combinational circuits: $L = t_{PD}$ of circuit, $T = 1/L$
- For K-pipelines ($K > 0$):
 - always have register on output(s)
 - K registers on every path from input to output
 - Inputs available shortly after clock i, outputs available shortly after clock (i+K)
 - $T = 1/(t_{PD,REG} + t_{PD} \text{ of slowest pipeline stage} + t_{SETUP})$
 - more throughput → split slowest pipeline stage(s)
 - use replication/interleaving if no further splits possible
 - $L = K / T$
 - pipelined latency $\geq$ combinational latency

MIT OpenCourseWare
<http://ocw.mit.edu>

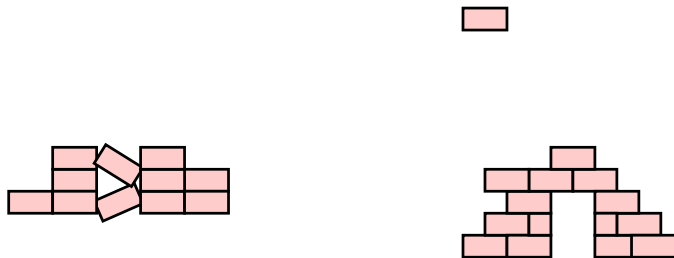
6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

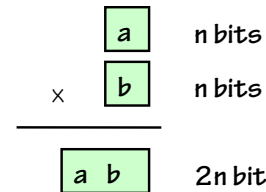
Cost/Performance Tradeoffs: a case study

Digital Systems Architecture 1.01



Lab #3 due tonight!

Binary Multiplication



since $(2^n - 1)^2 < 2^{2n}$

EASY PROBLEM: design combinational circuit to multiply tiny (1-, 2-, 3-bit) operands...

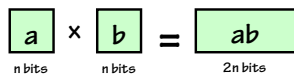
HARD PROBLEM: design circuit to multiply BIG (32-bit, 64-bit) numbers

We can make *big* multipliers out of *little* ones!

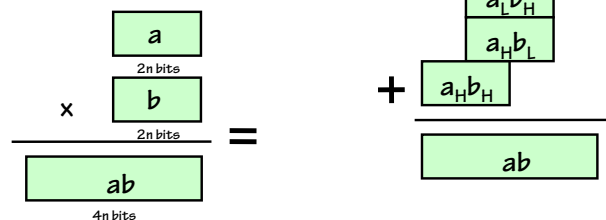
Engineering Principle:
Exploit **STRUCTURE** in problem.

Making a 2n-bit multiplier using n-bit multipliers

Given n-bit multipliers:



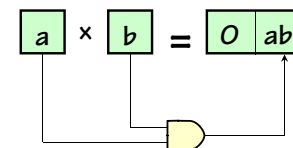
Synthesize 2n-bit multipliers:



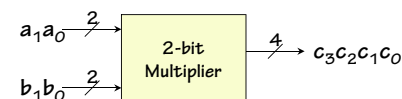
Our Basis:

n=1: minimalist starting point

Multiplying two 1-bit numbers is pretty simple:



Of course, we could start with optimized combinational multipliers for larger operands; e.g.

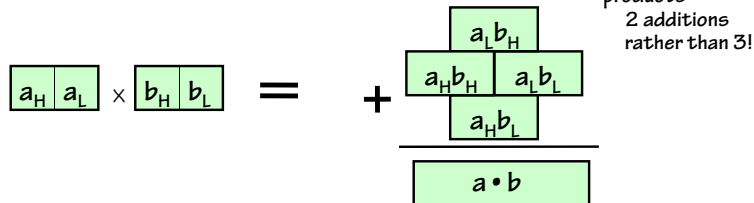


the logic gets more complex, but some optimizations are possible...

Our induction step:

2n-bit by 2n-bit multiplication:

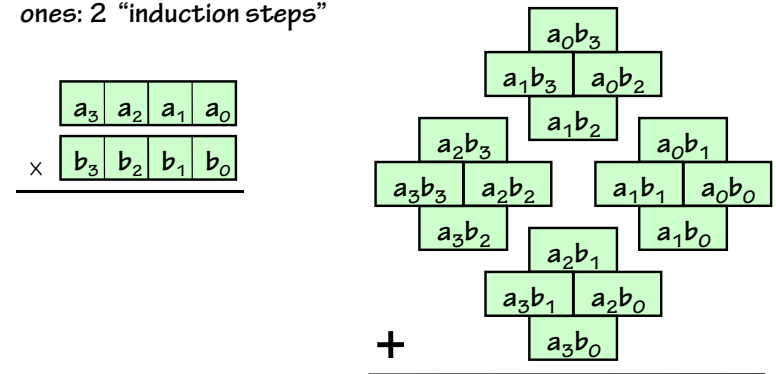
1. Divide multiplicands into n-bit pieces
2. Form 2n-bit partial products, using n-bit by n-bit multipliers.
3. Align appropriately
4. Add.



Induction: we can use the same structuring principle to build a 4n-bit multiplier from our newly-constructed 2n-bit ones...

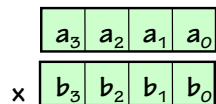
Brick Wall view of partial products

Making 4n-bit multipliers from n-bit ones: 2 "induction steps"

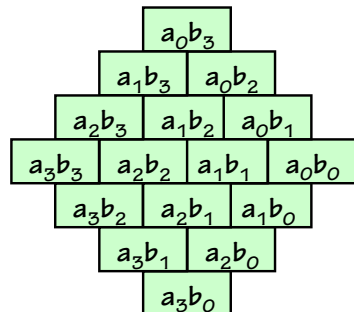


Multiplier Cookbook: Chapter 1

Given problem:

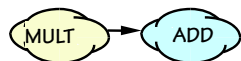


Step 1: Form (& arrange) Partial Products:

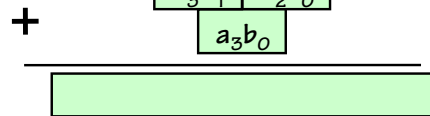


Subassemblies:

- Partial Products
- Adders



Step 2: Sum



Performance/Cost Analysis

"Order Of" notation:

"g(n) is of order f(n)" $g(n) = \Theta(f(n))$

$g(n) = \Theta(f(n))$ if there exist $C_2 \geq C_1 > 0$, such that for all but finitely many integral $n \geq 0$

$$C_1 \cdot f(n) \leq g(n) \leq C_2 \cdot f(n)$$

$g(n) = O(f(n))$

$\Theta(\dots)$ implies both inequalities; $O(\dots)$ implies only the second.

Example:

$$n^2 + 2n + 3 = \Theta(n^2)$$

since

$$n^2 \leq (n^2 + 2n + 3) \leq 2n^2$$

"almost always"

Partial Products:

Things to Add:

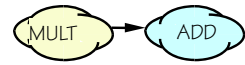
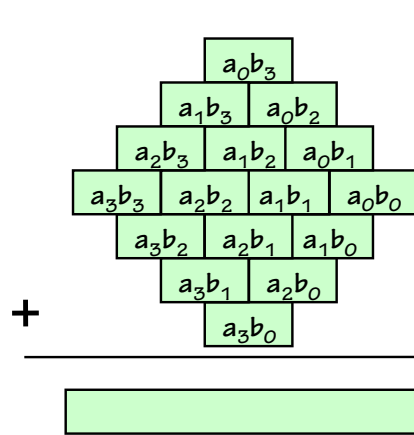
Adder Width:

Hardware Cost:

$$\begin{aligned} n^2 &= \Theta(n^2) \\ 2n-2 &= \Theta(n) \\ 2n &= \Theta(n) \\ ? &= \Theta(n^2) \end{aligned}$$

Latency: $O(n^2) ??$

Observations:

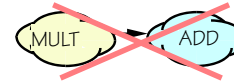


$\Theta(n^2)$ partial products.
 $\Theta(n^2)$ full adders.
 Hmm.

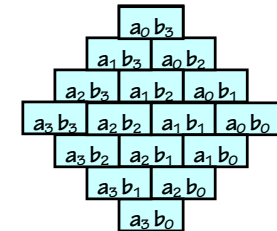
Repackaging Function

Engineering Principle #2:

Put the Solution where the Problem is.



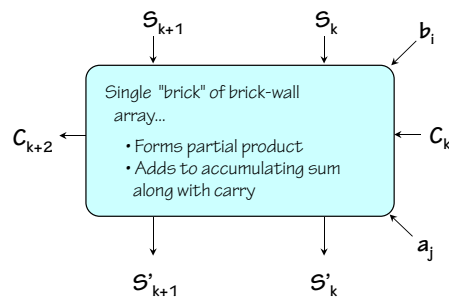
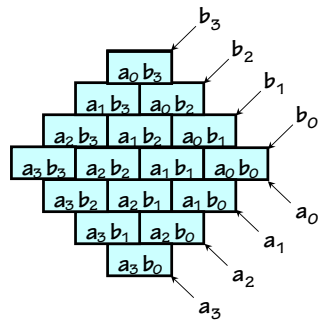
$\Theta(n^2)$ partial products.
 $\Theta(n^2)$ full adders.



How about n^2 blocks, each doing a little multiplication and a little addition?

Goal:

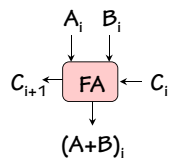
Array of Identical Multiplier Cells



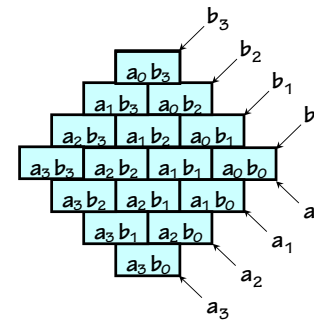
Necessary Component: Full Adder

Takes 2 addend bits plus carry bit. Produces sum and carry output bits.

CASCADE to form an n-bit adder.



Design of 1-bit multiplier "Brick":



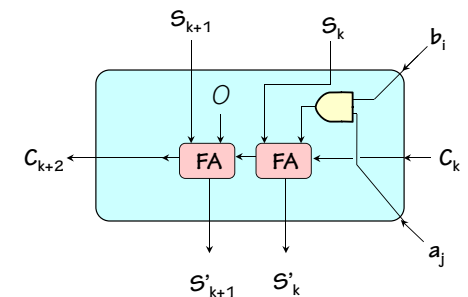
Array Layout:

- operand bits bused diagonally
- Carry bits propagate right-to-left
- Sum bits propagate down

Brick design:

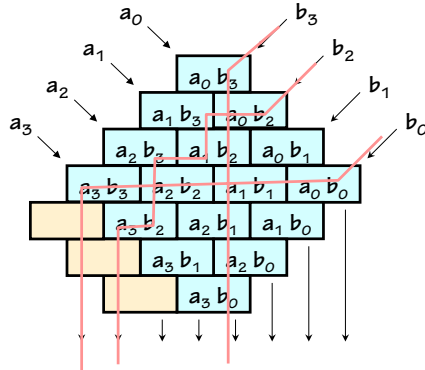
- AND gate forms 1x1 product
- 2-bit sum propagates from top to bottom
- Carry propagates to left

Wastes some gates... but consider (say) optimized 4x4-bit brick!



Latency revisited

Here's our combinational multiplier:



What's its propagation delay?

Naive (but valid) bound:

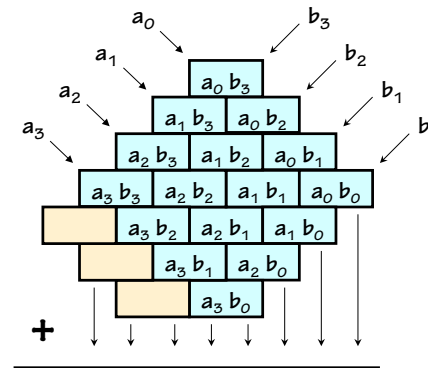
- $O(n)$ additions
- $O(n)$ time for each addition
- Hence $O(n^2)$ time required

On closer inspection:

- Propagation only toward left, bottom
- Hence longest path bounded by length + width of array:
 $O(n+n) = O(n)!$

Multiplier Cookbook: Chapter 2

Combinational Multiplier:



Hardware for n by n bits: $\Theta(n^2)$

Latency: $\Theta(n)$

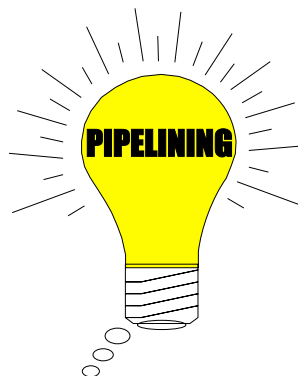
Throughput: $\Theta(1/n)$

Note: lots of tricks are available to make a faster combinational multiplier...

Combinational Multiplier: best bang for the buck?

Suppose we have LOTS of multiplications.

Can we do better from a cost/performance standpoint?

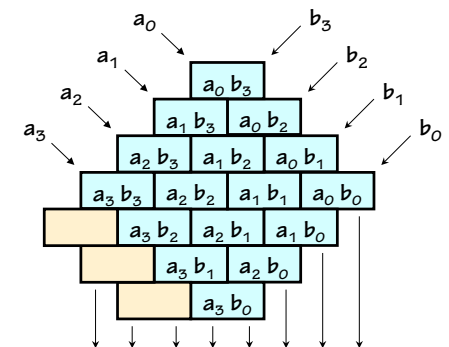


The Pipelining Bandwagon... where do I get on?

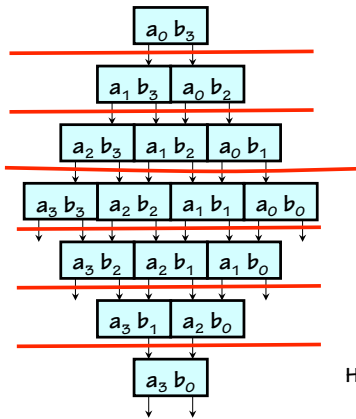
WE HAVE:

- Pipeline rules - "well formed pipelines"
- Plenty of registers
- Demand for higher throughput.

What do we do? Where do we define stages?



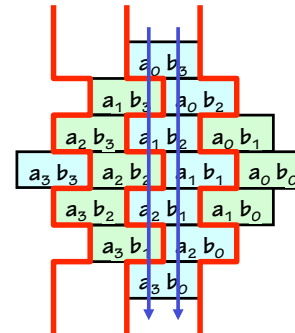
Stupid Pipeline Tricks



gotta break
that long
carry chain!

Stages: $\Theta(n)$
Clock Period: $\Theta(n)$
Hardware cost for n by n bits: $\Theta(n^2)$
Latency: $\Theta(n^2)$
Throughput: $\Theta(1/n)$

Even Stupider Pipeline Tricks



WORSE idea:

- Doesn't break long combinational paths
- NOT a well-formed pipeline...
... different register counts on alternative paths
... data crosses stage boundaries in both directions!

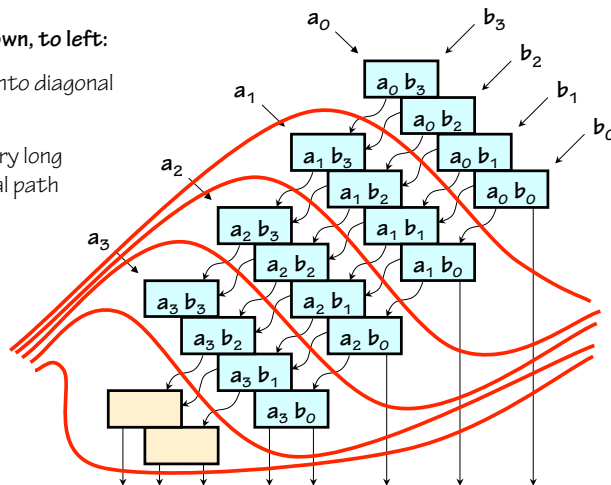
Back to basics:

what's the point of pipelining, anyhow?

Breaking $O(n)$ combinational paths

LONG PATHS go down, to left:

- Break array into diagonal slices
- Segment every long combinational path



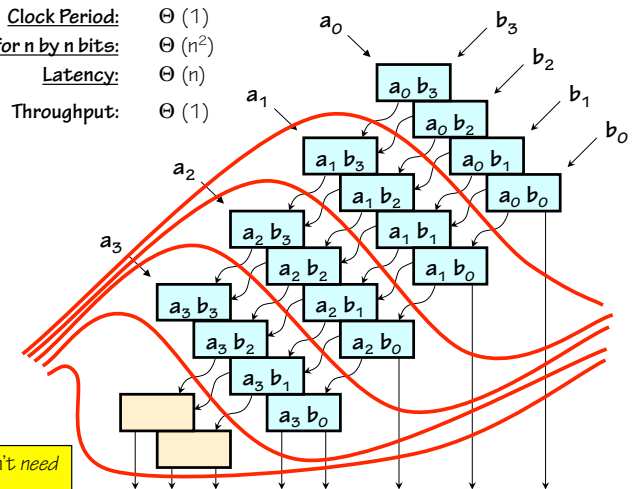
GOAL: $\Theta(n)$ stages; $\Theta(1)$ clock period!

Multiplier Cookbook: Chapter 3

Stages: $\Theta(n)$
Clock Period: $\Theta(1)$
Hardware cost for n by n bits: $\Theta(n^2)$
Latency: $\Theta(n)$
Throughput: $\Theta(1)$

- Well-formed pipeline (carefull!)
- Constant (high!) throughput, independently of operand size.

... but suppose we don't need the throughput?

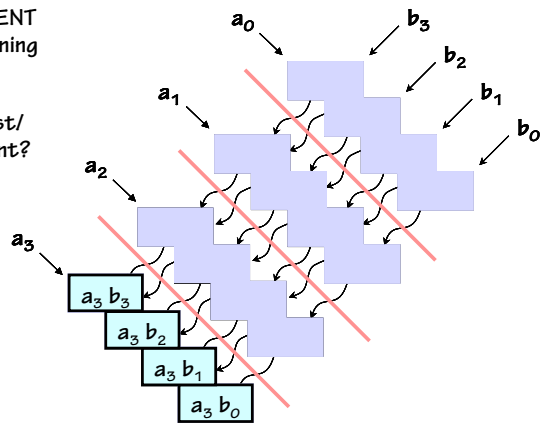


Moving down the cost curve...

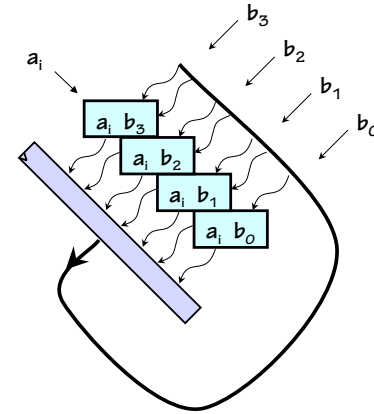
Suppose we have INFREQUENT multiplications... pipelining doesn't help us.

Can we do better from a cost/performance standpoint?

Hmmm, do I really need all these extras?



Multiplier Cookbook: Chapter 4



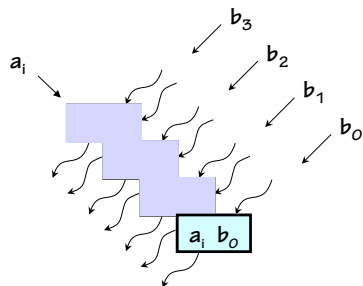
Sequential Multiplier:

- Re-uses a single n-bit "slice" to emulate each pipeline stage
- a operand entered serially
- Lots of details to be filled in...

Stages: 1
 Clock Period: $\Theta(1)$ (constant!)
 Hardware cost for n by n bits: $\Theta(n)$
 Latency: $\Theta(n)$
 Throughput: $\Theta(1/n)$

(Ridiculous?) Extremes Dept...

Cost minimization: how far can we go?



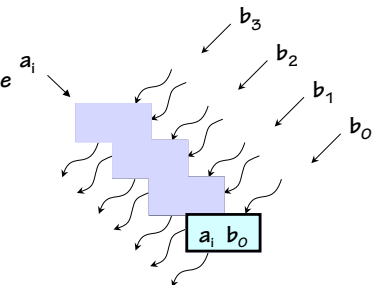
Suppose we want to minimize hardware (at any cost)...

- Consider bit-serial!
 - Form and add 1-bit partial product per clock
 - Reuse single "brick" for each bit b_j of slice;
 - Re-use slice for each bit of a operand

Multiplier Cookbook: Chapter 5

Bit Serial multiplier:

- Re-uses a single brick to emulate an n-bit slice
- both operands entered serially
- $O(n^2)$ clock cycles required
- Needs additional storage (typically from existing registers)



Stages: $\Theta(1/n)$
 Clock Period: $\Theta(1)$ (constant)
 Hardware cost for n by n bits: $\Theta(1) + ?$
 Latency: $\Theta(n^2)$
 Throughput: $\Theta(1/n^2)$

Summary:

Scheme:	\$	Latency	Thruput
Combinational	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1/n)$
N-pipe	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$
Slice-serial	$\Theta(n)$	$\Theta(n)$	$\Theta(1/n)$
Bit-serial	$\Theta(1)^*$	$\Theta(n^2)$	$\Theta(1/n^2)$

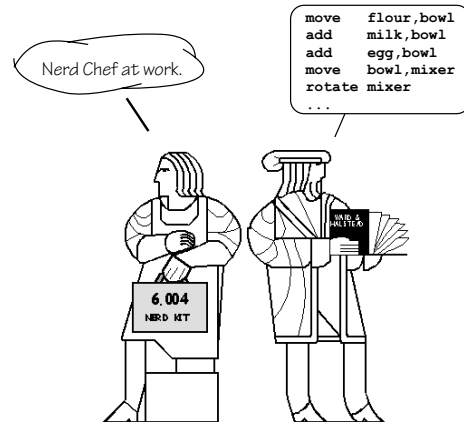
Lots more multiplier technology: fast adders, Booth Encoding, column compression, ...

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

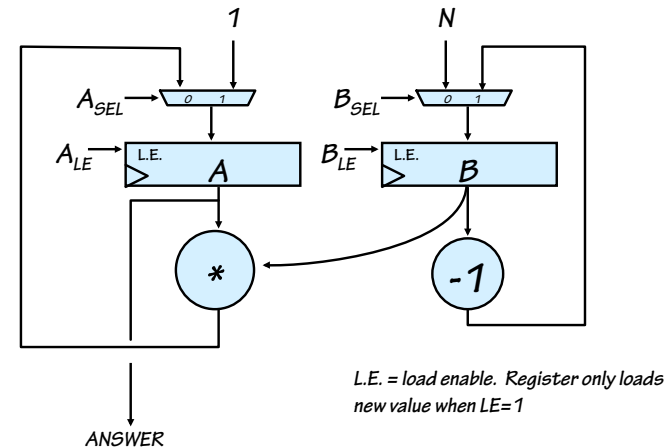
Designing an Instruction Set



Quiz 2 FRIDAY

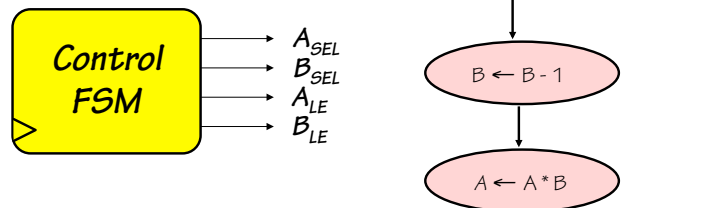
Let's Build a Simple Computer

Data path for computing $N*(N-1)$

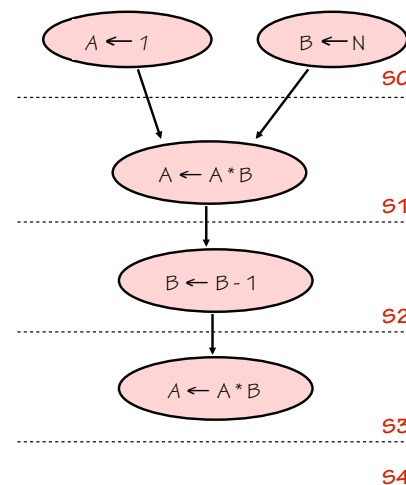


A Programmable Control System

Computing $N*(N-1)$ with this data path is a multi-step process. We can control the processing at each step with a FSM. If we allow different control sequences to be loaded into the control FSM, then we allow the machine to be programmed.



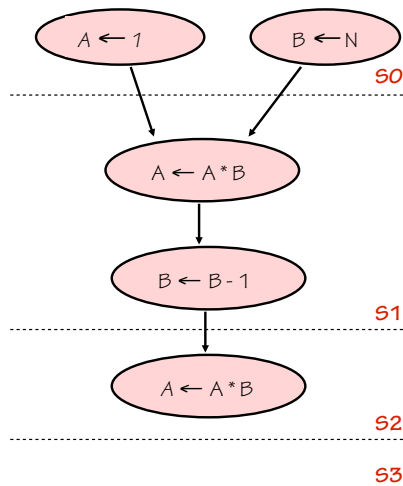
A First Program



Once more, writing a control program is nothing more than filling in a table:

S_N	S_{N+1}	A_{sel}	A_{LE}	B_{sel}	B_{LE}
0	1	1	1	0	1
1	2	0	1	0	0
2	3	0	0	1	1
3	4	0	1	0	0
4	4	0	0	0	0

An Optimized Program



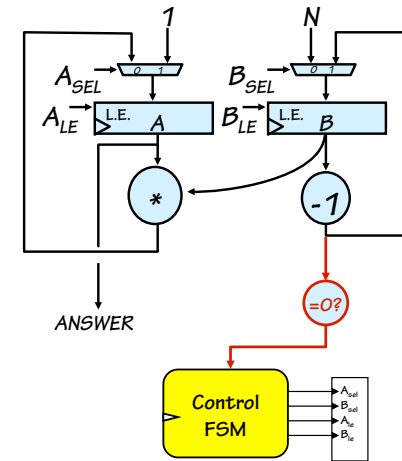
Some parts of the program can be computed simultaneously:

S_N	S_{N+1}	A_{sel}	A_{LE}	B_{sel}	B_{LE}
0	1	1	1	0	1
1	2	0	1	1	1
2	3	0	1	0	0
3	3	0	0	0	0

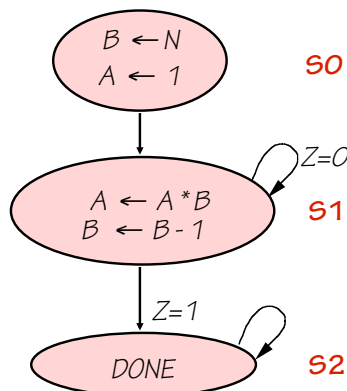
Computing Factorial

The advantage of a programmable control system is that we can reconfigure it to compute new functions.

In order to compute $N!$ we will need to add some new logic and an input to our control FSM:



Control Structure for Factorial



Programmability allows us to reuse data paths to solve new problems. What we need is a general purpose data path, which can be used to efficiently solve most problems as well as an easier way to control it.

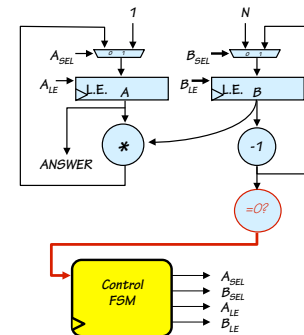
Z	S_N	S_{N+1}	A_{sel}	A_{LE}	B_{sel}	B_{LE}
-	0	1	1	1	0	1
0	1	1	0	1	1	1
1	1	2	0	1	1	1
-	2	2	0	0	0	0

A Programmable Engine

We've used the same data paths for computing $N*(N-1)$ and Factorial; there are a variety of other computations we might implement simply by re-programming the control FSM.

Although our little machine is programmable, it falls short of a practical general-purpose computer - and fails the Turing Universality test - for three primary reasons:

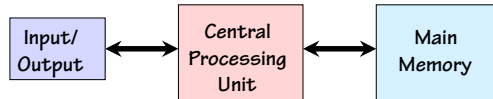
1. It has very limited storage: it lacks the "expandable" memory resource of a Turing Machine.
2. It has a tiny repertoire of operations.
3. The "program" is fixed. It lacks the power, e.g., to generate a new program and then execute it.



A General-Purpose Computer

The von Neumann Model

Many architectural approaches to the general purpose computer have been explored. The one on which nearly all modern, practical computers is based was proposed by John von Neumann in the late 1940s. Its major components are:



Ah, an FSM!
Like an Infinite Tape?
Hmm, guess J. von N was an engineer after all!

Central Processing Unit (CPU): containing several registers, as well as logic for performing a specified set of operations on their contents.

Memory: storage of N words of W bits each, where W is a fixed architectural parameter, and N can be expanded to meet needs.

I/O: Devices for communicating with the outside world.

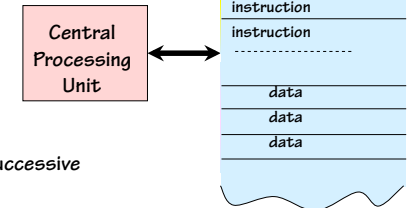
The Stored Program Computer

The von Neumann architecture easily addresses the first two limitations of our simple programmable machine example:

- A richer repertoire of operations, and
- An expandable memory.

But how does it achieve *programmability*?

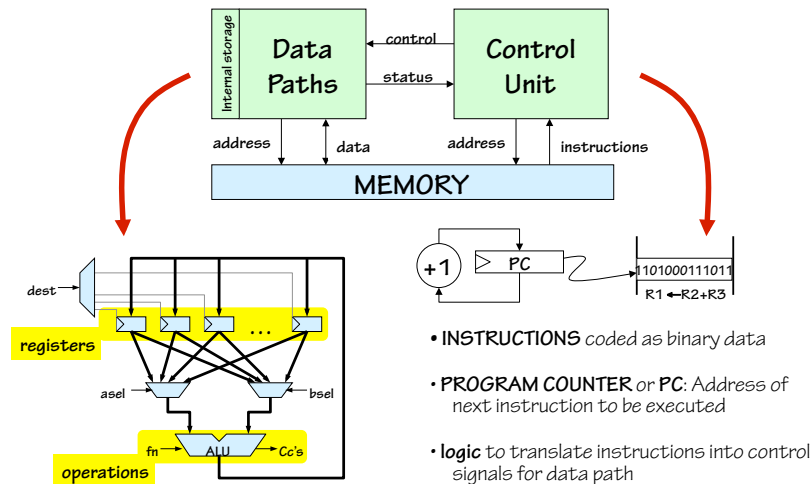
Key idea: Memory holds not only data, but *coded instructions* that make up a program.



CPU fetches and executes - interprets - successive instructions of the program ...

- Program is simply data for the interpreter, specifying what computation to perform
- Single expandable resource pool - main memory - constrains both data and program size.

Anatomy of a von Neumann Computer



Instruction Set Architecture

Coding of instructions raises some interesting choices...

- Tradeoffs: performance, compactness, programmability
- Uniformity. Should different instructions
 - Be the same size?
 - Take the same amount of time to execute?
 - Trend: Uniformity. Affords simplicity, speed, pipelining.
- Complexity. How many different instructions? What level operations?
 - Level of support for particular software operations: array indexing, procedure calls, "polynomial evaluate", etc
 - "Reduced Instruction Set Computer" (RISC) philosophy: simple instructions, optimized for speed

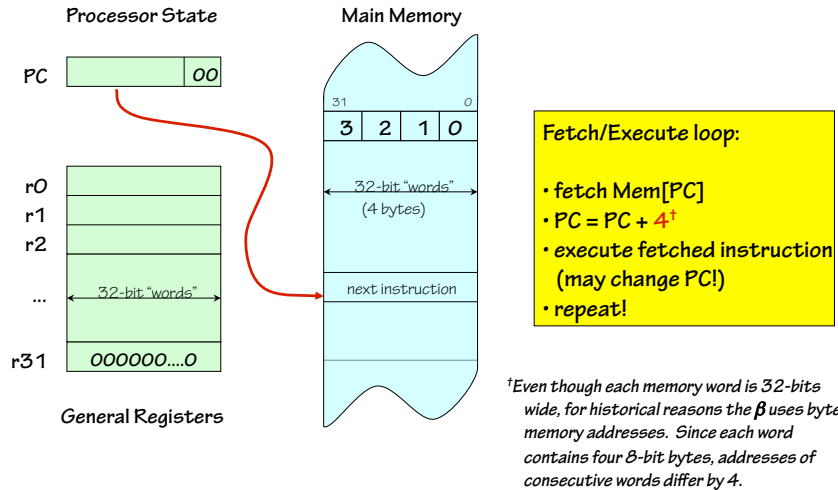
Mix of engineering & Art...

Trial (by simulation) is our best technique for making choices!

Our representative example: the β architecture!

β Programming Model

a representative, simple, contemporary RISC



β Instruction Formats

All Beta instructions fit in a single 32-bit word, whose fields encode combinations of

- a 6-bit OPCODE (specifying one of < 64 operations)
- several 5-bit OPERAND locations, each one of the 32 registers
- an embedded 16-bit constant ("literal")

There are two instruction formats:

- Opcode, 3 register operands (2 sources, destination)

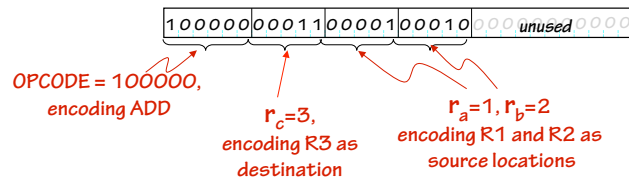


- Opcode, 2 register operands, 16-bit literal constant



β ALU Operations

Sample coded operation: ADD instruction



32-bit hex: 0x80611000

What we prefer to write: ADD (r1, r2, r3) ^{“assembly language”}

ADD (ra, rb, rc):

Reg[rc] = Reg[ra] + Reg[rb]

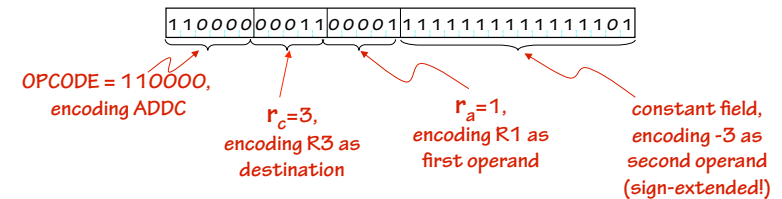
“Add the contents of ra to the contents of rb; store the result in rc”

Similar instructions for other ALU operations:

arithmetic: ADD, SUB, MUL, DIV
compare: CMPEQ, CMPLT, CMPLE
boolean: AND, OR, XOR
shift: SHL, SHR, SAR

β ALU Operations with Constant

ADDC instruction: adds constant, register contents:



Symbolic version: ADDC (r1, -3, r3)

ADDC (ra, const, rc):

Reg[rc] = Reg[ra] + sxt(const)

“Add the contents of ra to const; store the result in rc”

Similar instructions for other ALU operations:

arithmetic: ADDC, SUBC, MULC, DIVC
compare: CMPEQC, CMPLTC, CMPLEC
boolean: ANDC, ORC, XORC
shift: SHLC, SHRC, SARC

Do We Need Built-in Constants?

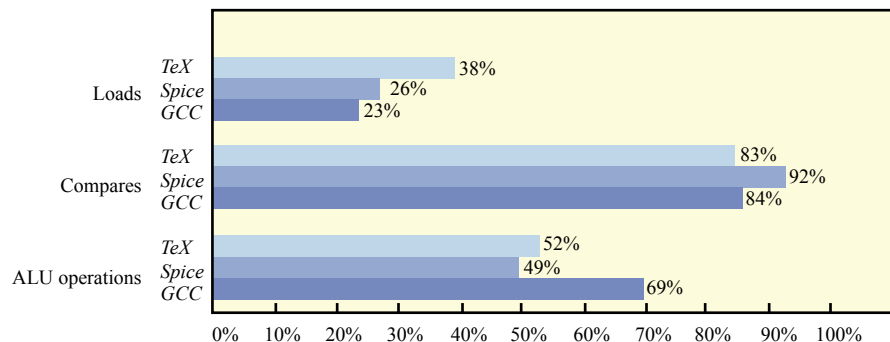


Figure by MIT OpenCourseWare.

Percentage of the operations that use a constant operand

One way to answer architectural questions is to evaluate the consequences of different choices using carefully chosen representative benchmarks (programs and/or code sequences). Make choices that are “best” according to some metric (cost, performance, ...).

Baby's First Beta Program

(fragment)

Suppose we have N in $r1$, and want to compute $N*(N-1)$, leaving the result in $r2$:

```
SUBC(r1, 1, r2) | put N-1 into r2
MUL(r2, r1, r2) | leave N*(N-1) in r2
```

These two instructions do what our little ad-hoc machine did. Of course, limiting ourselves to registers for storage falls short of our ambitions... it amounts to the finite storage limitations of an FSM!

Needed: instruction-set support for reading and writing locations in main memory...

β Loads & Stores

OPCODE	r_c	r_a	16-bit signed constant
--------	-------	-------	------------------------

LD(r_a , const, r_c) $\text{Reg}[r_c] = \text{Mem}[\text{Reg}[r_a] + \text{sxt}(\text{const})]$

“Fetch into r_c the contents of the memory location whose address is C plus the contents of r_a ”

Abbreviation: LD(C , r_c) for LD($R31$, C , r_c)

ST(r_c , const, r_a) $\text{Mem}[\text{Reg}[r_a] + \text{sxt}(\text{const})] = \text{Reg}[r_c]$

“Store the contents of r_c into the memory location whose address is C plus the contents of r_a ”

Abbreviation: ST(r_c , C) for ST(r_c , C , $R31$)

BYTE ADDRESSES, but only 32-bit word accesses to word-aligned addresses are supported. Low two address bits are ignored!

Storage Conventions

- Variables live in memory
- Operations done on registers
- Registers hold Temporary values

1000:	n
1004:	r
1008:	x
100C:	y
1010:	

translates to

or, more humanely, to

```
int x, y;
y = x * 37;
```

Compilation approach: LOAD, COMPUTE, STORE

```
LD(r31, 0x1008, r0)
MULC(r0, 37, r0)
ST(r0, 0x100C, r31)
```

```
x=0x1008
y=0x100C
LD(x, r0)
MULC(r0, 37, r0)
ST(r0, y)
```

Ra defaults to R31 (0)

Common “Addressing Modes”

β can do these with appropriate choices for Ra and const

- **Absolute:** “constant”
 - Value = Mem[constant]
 - Use: accessing static data
- **Indirect (aka Register deferred):** “(Rx)”
 - Value = Mem[Reg[x]]
 - Use: pointer accesses
- **Displacement:** “constant(Rx)”
 - Value = Mem[Reg[x] + constant]
 - Use: access to local variables
- **Indexed:** “(Rx + Ry)”
 - Value = Mem[Reg[x] + Reg[y]]
 - Use: array accesses (base+index)
- **Memory indirect:** “@(Rx)”
 - Value = Mem[Mem[Reg[x]]]
 - Use: access thru pointer in mem
- **Autoincrement:** “(Rx)+”
 - Value = Mem[Reg[x]]; Reg[x]++
 - Use: sequential pointer accesses
- **Autodecrement:** “-(Rx)”
 - Value = Reg[x]--; Mem[Reg[x]]
 - Use: stack operations
- **Scaled:** “constant(Rx)[Ry]”
 - Value = Mem[Reg[x] + c + d*Reg[y]]
 - Use: array accesses (base+index)

Argh! Is the complexity worth the cost?
Need a cost/benefit analysis!

Memory Operands: Usage

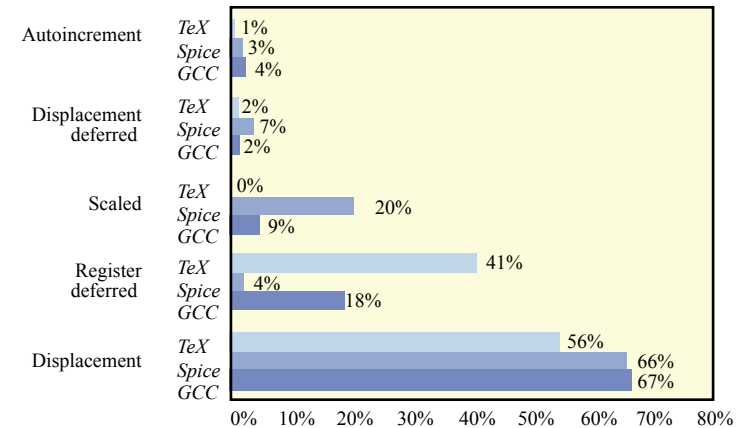


Figure by MIT OpenCourseWare.

Usage of different memory operand modes

Capability so far: Expression Evaluation

Translation of an Expression:

```
int x, y;
y = (x-3)*(y+123456)
```

```
x:    long(0)
y:    long(0)
c:    long(123456)
```

```
...
LD(x, r1)
SUBC(r1, 3, r1)
LD(y, r2)
LD(c, r3)
ADD(r2, r3, r2)
MUL(r2, r1, r1)
ST(r1, y)
```

- VARIABLES are allocated storage in main memory
- VARIABLE references translate to LD or ST
- OPERATORS translate to ALU instructions
- SMALL CONSTANTS translate to ALU instructions w/ built-in constant
- “LARGE” CONSTANTS translate to initialized variables

NB: Here we assume that variable addresses fit into 16-bit constants!

Can We Run Every Algorithm?

Model thus far:

- Executes instructions sequentially –
- Number of operations executed = number of instructions in our program!

Good news: programs can’t “loop forever”!

- Halting problem* is solvable for our current Beta subset!

*more next week!

NOT Universal*

Bad news: can’t compute Factorial:

- Only supports bounded-time computations;
- Can’t do a loop, e.g. for Factorial!

Needed: ability to change the PC.

Beta Branch Instructions

The Beta's branch instructions provide a way of conditionally changing the PC to point to some nearby location...

... and, optionally, remembering (in Rc) where we came from (useful for procedure calls).



NB: "offset" is a SIGNED CONSTANT encoded as part of the instruction!

BEQ(ra, label, rc): Branch if equal

BNE(ra, label, rc): Branch if not equal

PC = PC + 4;
Reg[rc] = PC;
if (REG[ra] == 0)
PC = PC + 4*offset;

PC = PC + 4;
Reg[rc] = PC;
if (REG[ra] != 0)
PC = PC + 4*offset;

offset = (label - <addr of BNE/BEQ>) / 4 - 1
= up to 32767 instructions before/after BNE/BEQ

Now we can do Factorial...

Synopsis (in C):

- Input in n, output in ans
- r1, r2 used for temporaries
- follows algorithm of our earlier data paths.

```
int n, ans;
r1 = 1;
r2 = n;
while (r2 != 0) {
    r1 = r1 * r2;
    r2 = r2 - 1;
}
ans = r1;
```

Beta code, in assembly language:

```
n:      long(123)
ans:    long(0)
...
ADDC(r31, 1, r1)    | r1 = 1
LD(n, r2)           | r2 = n
loop:  BEQ(r2, done, r31) | while (r2 != 0)
      MUL(r1, r2, r1)      | r1 = r1 * r2
      SUBC(r2, 1, r2)      | r2 = r2 - 1
      BEQ(r31, loop, r31) | Always branches!
done:  ST(r1, ans, r31)    | ans = r1
```

Summary

- Programmable data paths provide some algorithmic flexibility, just by changing control structure.
- Interesting control structure optimization questions – e.g., what operations can be done simultaneously?
- von Neumann model for general-purpose computation: need
 - support for sufficiently powerful operation repertoire
 - Expandable Memory
 - Interpreter for program stored in memory
- ISA design requires tradeoffs, usually based on benchmark results: art, engineering, evaluation & incremental optimizations
- Compilation strategy
 - runtime "discipline" for software implementation of a general class of computations
 - Typically enforced by compiler, run-time library, operating system. We'll see more of these!

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Machine Language, Assemblers, and Compilers

Long, long, time ago, I can still remember
how mnemonics used to make me smile...
And I knew that with just the opcode names
that I could play those BSIM games
and maybe hack some macros for a while.
But 6.004 gave me shivers
with every lecture they delivered.
Bad news at the door step,
I couldn't read one more spec.
I can't remember if I tried
to get Factorial optimized,
But something touched my nerdy pride
the day my Beta died.
And I was singing...

When I find my code in tons of trouble,
Friends and colleagues come to me,
Speaking words of wisdom:
"Write in C."



References (on web site):

β Documentation
BSIM reference
Notes on C Language

Quiz 2 TOMORROW!

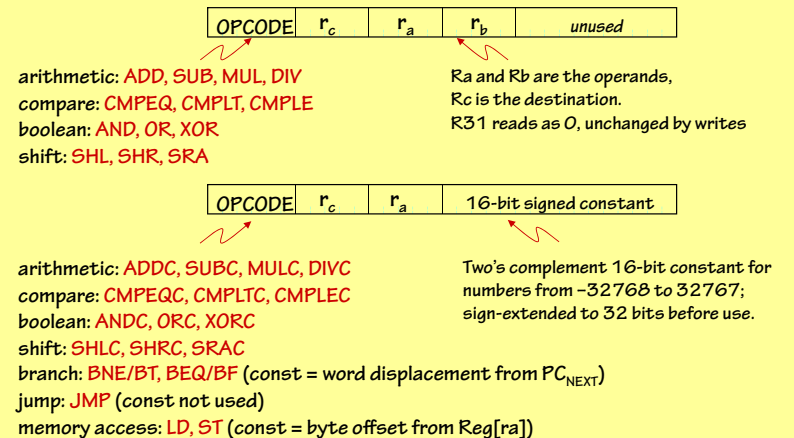
6.004 - Spring 2009

3/12/09

modified February 23, 09

L11 - Machine Language 1

β Machine Language: 32-bit instructions



How can we improve the programmability of the Beta?

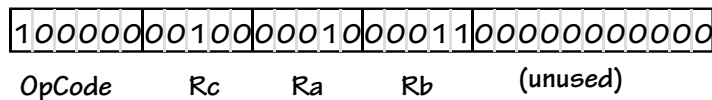
6.004 - Spring 2009

3/12/09

L11 - Machine Language 2

Encoding Binary Instructions

32-bit (4-byte) ADD instruction:



Means, to BETA, $Reg[4] = Reg[2] + Reg[3]$

But, most of us would prefer to write

ADD (R2, R3, R4) (ASSEMBLER)

or, better yet,

a = b+c; (High Level Language)

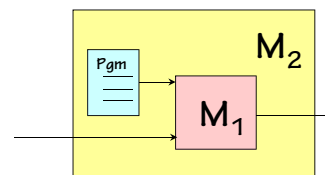
Software Approaches: INTERPRETATION, COMPILATION

6.004 - Spring 2009

3/12/09

L11 - Machine Language 3

Interpretation

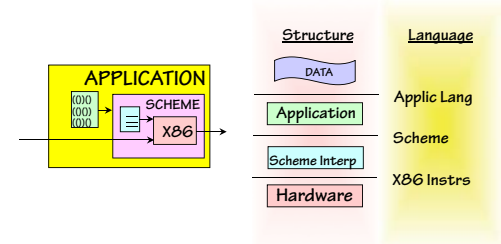


Turing's model of Interpretation:

- Start with some hard-to-program universal machine, say M_1
- Write a single program for M_1 which mimics the behavior of some easier machine, say M_2
- Result: a "virtual" M_2

"Layers" of interpretation:

- Often we use several layers of interpretation to achieve desired behavior, eg:
- X86 (Pentium), running
 - Scheme, running
 - Application, interpreting
 - Data.



6.004 - Spring 2009

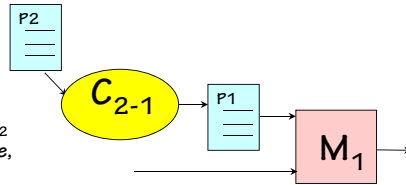
3/12/09

L11 - Machine Language 4

Compilation

Model of Compilation:

- Given some hard-to-program machine, say M_1 ...
- Find some easier-to-program language L_2 (perhaps for a more complicated machine, M_2); write programs in that language
- Build a translator (compiler) that translates programs from M_2 's language to M_1 's language. May run on M_1 , M_2 , or some other machine.



Interpretation & Compilation: two tools for improving programmability ...

- Both allow changes in the programming model
- Both afford programming applications in platform (e.g., processor) independent languages
- Both are widely used in modern computer systems!

Interpretation vs Compilation

There are some characteristic differences between these two powerful tools...

	Interpretation	Compilation
How it treats input "x+2"	computes x+2	generates a program that computes x+2
When it happens	During execution	Before execution
What it complicates/slows	Program Execution	Program Development
Decisions made at	Run Time	Compile Time

Major design choice we'll see repeatedly:
do it at Compile time or at Run time?

Software: Abstraction Strategy

Initial steps: compilation tools

Assembler (UASM):
symbolic representation
of machine language

Hides: bit-level representations,
hex locations, binary values

Compiler (C): symbolic
representation of
algorithm

Hides: Machine instructions,
registers, machine
architecture

Subsequent steps: interpretive tools

Operating system

Hides: Resource (memory, CPU,
I/O) limitations and details

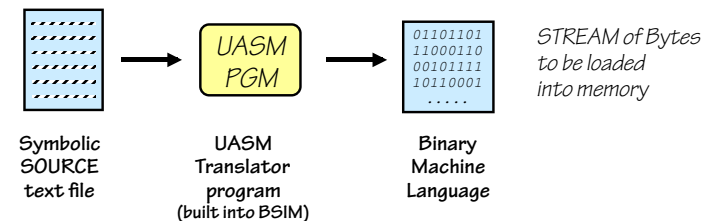
Apps (e.g., Browser)

Hides: Network; location; local
parameters

Abstraction step 1:

A Program for Writing Programs

UASM - the 6.004 (Micro) Assembly Language



UASM:

- A Symbolic LANGUAGE for representing strings of bits
- A PROGRAM ("assembler" = primitive compiler) for translating UASM source to binary.

UASM Source Language

A UASM SOURCE FILE contains, in symbolic text, values of successive bytes to be loaded into memory... e.g. in

```
37  -3  255      decimal (default);

0b100101        binary (note the "Ob" prefix);

0x25            hexadecimal (note the "Ox" prefix);
```

Values can also be expressions; eg, the source file

```
37+0b10-0x10    24-0x1    4*0b110-1    0xF7&0x1F
```

generates 4 bytes of binary output, each with the value **23**!

Symbolic Gestures

We can also define SYMBOLS for use in source programs:

```
x = 0x1000
y = 0x1004

| Symbolic names for registers:
R0 = 0
R1 = 1
...
R31 = 31
```

A variable location
| Another variable

Special variable "." (period) means next byte address to be filled:

```
. = 0x100                                | Assemble into 100
  1  2  3  4
five = .                                | Symbol "five" is 0x104
  5  6  7  8
. = .+16                                | Skip 16 bytes
  9 10 11 12
```

A "bar" denotes
the beginning of
a comment...
The remainder
of the line is
ignored

Labels (Symbols for Addresses)

LABELS are symbols that represent memory addresses.
They can be set with the following special syntax:

x: is an abbreviation for "x = ."

An Example--

```
---- MAIN MEMORY ----
1000: 09 04 01 00
1004: 31 24 19 10
1008: 79 64 51 40
100c: E1 C4 A9 90
1010: 10 ... ..

          . = 0x1000
sqr:      0 1 4 9
          16 25 36 49
          64 81 100 121
          144 169 196 225
slen:     .-sqr

          3 2 1 0
```

Mighty Macroinstructions

Macros are parameterized abbreviations, or shorthand

```
| Macro to generate 4 consecutive bytes:
.macro consec(n)  n  n+1  n+2  n+3

| Invocation of above macro:
consec(37)
```

Has same effect as:

```
37    38    39    40
```

Here are macros for breaking multi-byte data types into byte-sized chunks

```
| Assemble into bytes, little-endian (least-sig byte 1st)
.macro WORD(x)  x%256  (x/256)%256
.macro LONG(x)  WORD(x)  WORD(x >> 16)
```

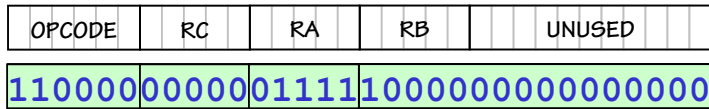
```
. = 0x100
LONG(0xdeadbeef)
```

Has same effect as:

```
Mem: 0xef 0xbe 0xad 0xde
      0x100 0x101 0x102 0x103
```

Boy, that's hard to read.
Maybe, those big-endian
types do have a point.

Assembly of Instructions



```

| Assemble Beta op instructions
.macro betaop(OP,RA,RB,RC) {
    .align 4
    LONG((OP<<26)+(RC<<32)<<21)+((RA<<32)<<16)+((RB<<32)<<11))
}

| Assemble Beta opc instructions
.macro betaopc(OP,RA,CC,RC) {
    .align 4
    LONG((OP<<26)+(RC<<32)<<21)+((RA<<32)<<16)+(CC % 0x10000))
}

| Assemble Beta branch instructions
.macro betabr(OP,RA,RC,LABEL)    betaopc(OP,RA,((LABEL-(. +4))>>2),RC)

For Example:
    ADDC(R15, -32768, R0) --> betaopc(0x30,15,-32768,0)

```

“align 4” ensures instructions will begin on word boundary (i.e., address = 0 mod 4)

Arrgh!

Finally, Beta Instructions

BETA Instructions:

```

.macro ADD(RA,RB,RC)    betaop(0x20,RA,RB,RC)
.macro ADDC(RA,C,RC)    betaopc(0x30,RA,C,RC)
.macro AND(RA,RB,RC)    betaop(0x28,RA,RB,RC)
.macro ANDC(RA,C,RC)    betaopc(0x38,RA,C,RC)
.macro MUL(RA,RB,RC)    betaop(0x22,RA,RB,RC)
.macro MULC(RA,C,RC)    betaopc(0x32,RA,C,RC)
...
.macro LD(RA,CC,RC)    betaopc(0x18,RA,CC,RC)
.macro LD(CC,RC)    betaopc(0x18,R31,CC,RC)
.macro ST(RC,CC,RA)    betaopc(0x19,RA,CC,RC)
.macro ST(RC,CC)    betaopc(0x19,R31,CC,RC)
...
.macro BEQ(RA,LABEL,RC) betabr(0x1D,RA,RC,LABEL)
.macro BEQ(RA,LABEL)    betabr(0x1D,RA,r31,LABEL)
.macro BNE(RA,LABEL,RC) betabr(0x1E,RA,RC,LABEL)
.macro BNE(RA,LABEL)    betabr(0x1E,RA,r31,LABEL)

```

Convenience macros so we don't have to specify R31...

(from beta.uasm)

Example Assembly

```

ADDC(R3,1234,R17)
    ↓ expand ADDC macro with RA=R3, C=1234, RC=R17
betaopc(0x30,R3,1234,R17)
    ↓ expand betaopc macro with OP=0x30, RA=R3, CC=1234, RC=R17
.align 4
LONG((0x30<<26)+(R17<<32)<<21)+((R3<<32)<<16)+(1234 % 0x10000))
    ↓ expand LONG macro with X=0xC22304D2
WORD(0xC22304D2)    WORD(0xC22304D2 >> 16)
    ↓ expand first WORD macro with X=0xC22304D2
0xC22304D2%256    (0xC22304D2/256)%256    WORD(0xC223)
    ↓ evaluate expressions, expand second WORD macro with X=0xC223
0xD2    0x04    0xC223%256    (0xC223/256)%256
    ↓ evaluate expressions
0xD2    0x04    0x23    0xC2

```

Don't have it? Fake it!

Convenience macros can be used to extend our assembly language:

```

.macro MOVE(RA,RC)    ADD(RA,R31,RC)    | Reg[RC] <- Reg[RA]
.macro CMOVE(CC,RC)    ADDC(R31,C,RC)    | Reg[RC] <- C

.macro COM(RA,RC)    XORC(RA,-1,RC)    | Reg[RC] <- ~Reg[RA]
.macro NEG(RB,RC)    SUB(R31,RB,RC)    | Reg[RC] <- -Reg[RB]
.macro NOP()    ADD(R31,R31,R31)    | do nothing

.macro BR(LABEL)    BEQ(R31,LABEL)    | always branch
.macro BR(LABEL,RC)    BEQ(R31,LABEL,RC)    | always branch
.macro CALL(LABEL)    BEQ(R31,LABEL,LP)    | call subroutine
.macro BF(RA,LABEL,RC)    BEQ(RA,LABEL,RC)    | 0 is false
.macro BF(RA,LABEL)    BEQ(RA,LABEL)
.macro BT(RA,LABEL,RC)    BNE(RA,LABEL,RC)    | 1 is true
.macro BT(RA,LABEL)    BNE(RA,LABEL)

| Multi-instruction sequences
.macro PUSH(RA)    ADDC(SP,4,SP)    ST(RA,-4,SP)
.macro POP(RA)    LD(SP,-4,RA)    ADDC(SP,-4,SP)

```

(from beta.uasm)

Abstraction step 2:

High-level Languages

Most algorithms are naturally expressed at a high level. Consider the following algorithm:

```
struct Employee
{ char *Name; /* Employee's name. */
  long Salary; /* Employee's salary. */
  long Points; /* Brownie points. */

/* Annual raise program. */
Raise(struct Employee P[100])
{ int i = 0;
  while (i < 100)
  { struct Employee *e = &P[i];
    e->Salary =
      e->Salary + 100 + e->Points;
    e->Points = 0; /* Start over! */
    i = i+1;
  }
}
```

Reference: C handout (6.004 web site)

We've used (and will continue to use throughout 6.004) C, a "mature" and common systems programming language. Modern popular alternatives include C++, Java, Python, and many others.

Why use these, not assembler?

- readable
- concise
- unambiguous
- portable (algorithms frequently outlast their HW platforms)
- Reliable (type checking, etc)

How Compilers Work

Contemporary compilers go far beyond the macro-expansion technology of UASM. They

- Perform sophisticated analyses of the source code
- Invoke arbitrary algorithms to generate efficient object code for the target machine
- Apply "optimizations" at both source and object-code levels to improve run-time efficiency.

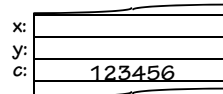
Compilation to *unoptimized* code is pretty straightforward... following is a brief glimpse.



Compiling Expressions

C code:

```
int x, y;
y = (x-3)*(y+123456)
```



Beta assembly code:

```
x:    LONG(0)
y:    LONG(0)
c:    LONG(123456)
...

LD(x, r1)
SUBC(r1, 3, r1)
LD(y, r2)
LD(c, r3)
ADD(r2, r3, r2)
MUL(r2, r1, r1)
ST(r1, y)
```

- **VARIABLES** are assigned memory locations and accessed via LD or ST instructions
- **OPERATORS** translate to ALU instructions
- **SMALL CONSTANTS** translate to "literal-mode" ALU instructions
- **LARGE CONSTANTS** translate to initialized variables

Data Structures: Arrays

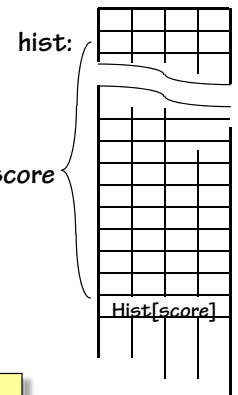
The C source code

```
int Hist[100];
...
Hist[score] += 1;
```

might translate to:

```
hist:    . = .+4*100 | Leave room for 100 ints
...
<score in r1>
MULC(r1, 4, r2) | index -> byte offset
LD(r2, hist, r0) | hist[score]
ADDC(r0, 1, r0) | increment
ST(r0, hist, r2) | hist[score]
```

Memory:



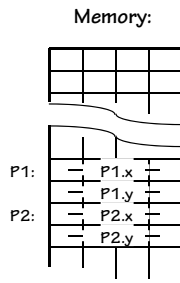
Address:
CONSTANT base address +
VARIABLE offset computed from index

Data Structures: Structs

```
struct Point
{ int x, y;
} P1, P2, *p;
...
P1.x = 157;
...
p = &P1;
p->y = 157;
```

might translate to:

```
P1: .+=8
P2: .+=8
x=0          | Offset for x component
y=4          | Offset for y component
...
CMOVE(157,r0) | r0 <- 157
ST(r0,P1+x)   | P1.x = 157
...
<p in r3>
ST(r0,y,r3)   | p->y = 157;
```



Address:
VARIABLE base address +
CONSTANT component offset

Conditionals

C code:
`if (expr)`
{
 STUFF
}

C code:
`if (expr)`
{
 STUFF1
}
`else`
{
 STUFF2
}

Beta assembly:
(compile *expr* into rx)
`BF(rx, Lendif)`
(compile *STUFF*)
Lendif:

Beta assembly:
(compile *expr* into rx)
`BF(rx, Lelse)`
(compile *STUFF1*)
`BR(Lendif)`
Lelse:
(compile *STUFF2*)
Lendif:

There are little tricks that come into play when compiling conditional code blocks. For instance, the statement:

```
if (y > 32)
{
    x = x + 1;
}
```

there's no >32 instruction!

compiles to:

```
LD(y, R1)
CMPLC(R1, 32, R1)
BT(R1, Lendif)
ADDC(R2, 1, R2)
Lendif:
```

Loops

C code:
`while (expr)`
{
 STUFF
}

Beta assembly:
Lwhile:
(compile *expr* into rx)
`BF(rx, Lendwhile)`
(compile *STUFF*)
`BR(Lwhile)`
Lendwhile:

Alternate Beta assembly:
`BR(Ltest)`
Lwhile:
(compile *STUFF*)
Ltest:
(compile *expr* into rx)
`BT(rx, Lwhile)`
Lendwhile:

Move the test to the end of the loop and branch there the first time thru... saves a branch

Compilers spend a lot of time optimizing in and around loops.

- moving all possible computations outside of loops
- "unrolling" loops to reduce branching overhead
- simplifying expressions that depend on "loop variables"

Our Favorite Program

```
int n = 20, r;
r = 1;
while (n > 0)
{
    r = r*n;
    n = n-1;
}

n: LONG(20)
r: LONG(0)
start:
    ADDC(r31, 1, r0)
    ST(r0, r)
loop:
    LD(n, r1)
    CMPLT(r31, r1, r2)
    BF(r2, done)
    LD(r, r3)
    LD(n, r1)
    MUL(r1, r3, r3)
    ST(r3, r)
    LD(n, r1)
    SUBC(r1, 1, r1)
    ST(r1, n)
    BR(loop)
done:
```

Cleverness:
None...
straightforward compilation
(11 instructions in loop...)

Optimizations are what make compilers complicated interesting!

Optimizations

```

int n = 20, r;
n: LONG(20)
r: LONG(0)

start:
  ADDC(r31, 1, r0)
  ST(r0, r)
  LD(n, r1)      | keep n in r1
  LD(r, r3)      | keep r in r3

loop:
  CMPLT(r31, r1, r2)
  BF(r2, done)
  MUL(r1, r3, r3)
  SUBC(r1, 1, r1)
  BR(loop)

done:
  ST(r1, n)      | save final n
  ST(r3, r)      | save final r

while (n > 0)
{
  r = r*n;
  n = n-1;
}

```

Cleverness:
We move LDs/STs
out of loop!
(Still, 5 instructions in loop...)

Really Optimizing...

```

int n = 20, r;
n: LONG(20)
r: LONG(0)

start:
  LD(n, r1)      | keep n in r1
  ADDC(r31, 1, r3) | keep r in r3
  BEQ(r1, done)  | why?

loop:
  MUL(r1, r3, r3)
  SUBC(r1, 1, r1)
  BNE(r1, loop)

done:
  ST(r1, n)      | save final n
  ST(r3, r)      | save final r

while (n > 0)
{
  r = r*n;
  n = n-1;
}

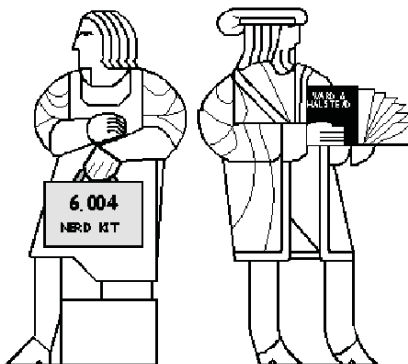
```

Cleverness:
We avoid overhead
of conditional!
(Now 3 instructions in loop...)

UNFORTUNATELY,

$20! = 2,432,902,008,176,640,000 > 2^{61}$ (overflows!)
but $12! = 479,001,600 = 0x1c8cfc00$

Coming Attractions: Procedures & Stacks



MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

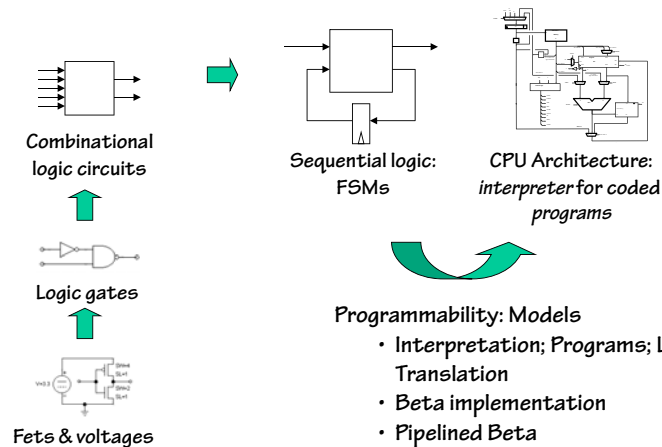
Programmability

from silicon to bits

0110100110
1110010010
0011100011

the Big Ideas
of
Computer Science

6.004 Roadmap



Programmability: Models

- Interpretation; Programs; Languages; Translation
- Beta implementation
- Pipelined Beta
- Software conventions
- Memory architectures

FSMs as Programmable Machines

ROM-based FSM sketch:

Given i , s , and o , we need a ROM organized as:

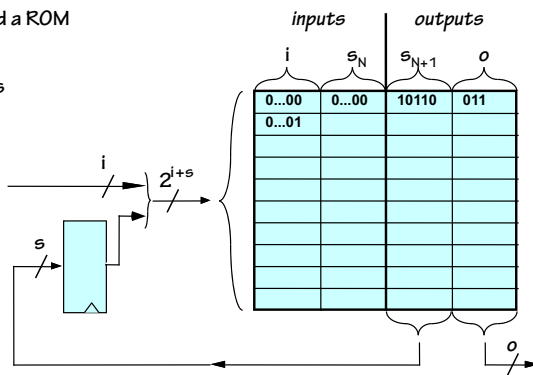
2^{i+s} words $\times$ $(o+s)$ bits

So how many possible i -input, o -output, FSMs with s -state bits exist?

$$2^{(o+s)2^{i+s}}$$

(some may be equivalent)

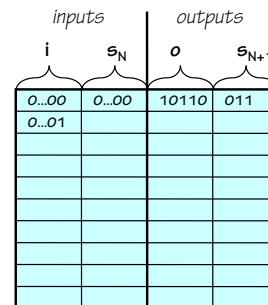
An FSM's behavior is completely determined by its ROM contents.



Big Idea #1: FSM Enumeration

GOAL: List all possible FSMs in some canonical order.

- INFINITE list, but
- Every FSM has an entry and an associated index.



i	s	o	FSM#	Truth Table
1	1	1	1	00000000
1	1	1	2	00000001
...	...	...	...	...
1	1	1	256	11111111
2	2	2	257	000000...000000
2	2	2	258	000000...000001
...	...	...	...	...
3	3	3	...	000000...000000
4	4	4	...	000000...000000

What if
 $s=2$,
 $i=o=1$??

Every possible FSM can be associated with a number. We can discuss the i^{th} FSM

Some Perennial Favorites...

FSM ₈₃₇	modulo 3 counter
FSM ₁₀₇₇	4-bit counter
FSM ₁₅₃₇	lock for 6.004 Lab
FSM ₈₉₁₄₃	Steve's digital watch
FSM ₂₂₆₉₈₄₆₉₈₈₄	Intel Pentium CPU – rev 1
FSM ₇₈₄₃₆₂₇₈₃	Intel Pentium CPU – rev 2
FSM ₇₂₆₉₈₄₃₆₅₆₃₇₈₃	Intel Pentium II CPU

Reality: The integer indexes of actual FSMs are much bigger than the examples above. They must include enough information to constitute a complete description of each device's unique structure.

Models of Computation

The roots of computer science stem from the study of many alternative mathematical “models” of computation, and study of the classes of computations they could represent.

An elusive goal was to find an “ultimate” model, capable of representing all practical computations...

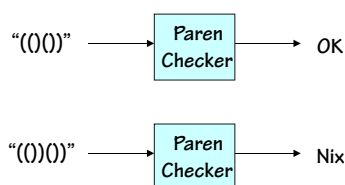
*We've got FSMs ...
what else do we need?*

- switches
- gates
- combinational logic
- memories
- FSMs

Are FSMs the
ultimate digital
computing device?

FSM Limitations

Despite their usefulness and flexibility, there exist common problems that cannot be computed by FSMs. For instance:



Well-formed Parentheses Checker:

Given any string of coded left & right parens, outputs 1 if it is balanced, else 0.

Simple, easy to describe.

Is this device equivalent to one of our enumerated FSMs???

NO!

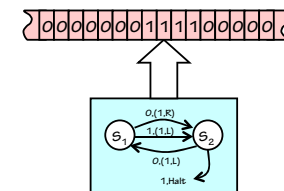
PROBLEM: Requires ARBITRARILY many states, depending on input. Must "COUNT" unmatched LEFT parens. An FSM can only keep track of a finite number of unmatched parens: for every FSM, we can find a string it can't check.

Big Idea #2: Turing Machines

Alan Turing was one of a group of researchers studying alternative models of computation.

He proposed a conceptual model consisting of an FSM combined with an infinite digital tape that could be read and written at each step.

Turing's model (like others of the time) solves the "FINITE" problem of FSMs.



A Turing machine Example

Turing Machine Specification

- Doubly-infinite tape
- Discrete symbol positions
- Finite alphabet – say $\{0, 1\}$
- Control FSM

INPUTS:

Current symbol

OUTPUTS:

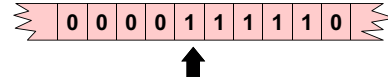
write 0/1

move Left/Right

- Initial Starting State $\{S_0\}$
- Halt State $\{Halt\}$

A Turing machine, like an FSM, can be specified with a truth table. The following Turing Machine implements a unary (base 1) incrementer.

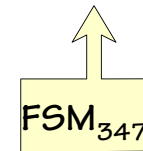
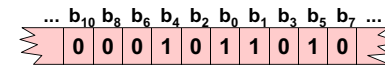
Current State	Input	Next State	Write Tape	Move Tape
S_0	1	S_0	1	R
S_0	0	S_1	1	L
S_1	1	S_1	1	L
S_1	0	HALT	0	R



OK, but how about *real* computations... like $fact(n)$?

Turing Machine Tapes as Integers

Canonical names for bounded tape configurations:



That's just Turing Machine 347 operating on tape 51

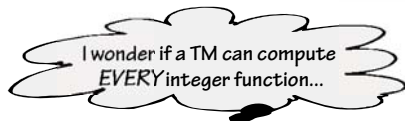
Encoding: starting at current position, build a binary integer taking successively higher-order bits from right and left sides. If nonzero region is bounded, eventually all 1's will be incorporated into the resulting integer representation.

TMs as Integer Functions

Turing Machine T_i operating on Tape x ,
where $x = \dots b_8 b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$

$$y = T_i[x]$$

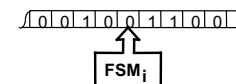
x : input tape configuration
 y : output tape configuration



Meanwhile,
Turing's buddies
Were busy too...

Alternative models of computation

Turing Machines [Turing]



Recursive Functions [Kleene]

$$F(0, x) \equiv x$$

$$F(1+y, x) \equiv 1+F(x, y)$$

```
(define (fact n)
  (... (fact (- n 1)) ...)
```

Kleene

Turing

Lambda calculus [Church, Curry, Rosser...]

$$\lambda x. \lambda y. xxy$$

$$(\text{lambda } (x) (\text{lambda } (y) (x \ (x \ y))))$$

Church

Production Systems [Post, Markov]

$$\alpha \rightarrow \beta$$

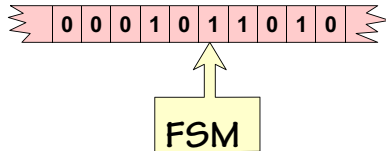
IF pulse=0 THEN
patient=dead

Post

The 1st Computer Industry Shakeout

Here's a TM that computes SQUARE ROOT!

how am I going to beat that?



And the battles raged

Here's a Lambda Expression that does the same thing...

$(\lambda (x) \dots)$

... and here's one that computes the n^{th} root for ANY n !

$(\lambda (x \ n) \dots)$

maybe if I gave away a microwave oven with every Turing Machine...

CONTEST: Which model computes more functions?

RESULT: an N-way TIE!

Big Idea #3: Computability

FACT: Each model studied is capable of computing exactly the same set of integer functions!

Proof Technique:

Constructions that translate between models

BIG IDEA:

Computability, independent of computation scheme chosen

unproved, but universally accepted...

Church's Thesis:

Every discrete function computable by ANY realizable machine is computable by some Turing machine.

Computable Functions

$f(x)$ *computable* $\iff$ for some k , all x :
 $f(x) = T_k[x] \equiv f_k(x)$

Equivalently: $f(x)$ computable on Cray, Pentium, in C, Scheme, Java, ...

Representation Tricks:

- Multi-argument functions? to compute $f_k(x,y)$, use $\langle x,y \rangle =$ integer whose even bits come from x , and whose odd bits come from y ;
whence $f_k(x,y) = T_k[\langle x,y \rangle]$
- Data types: Can encode characters, strings, floats, ... as integers.
- Alphabet size: use groups of N bits for 2^N symbols

Enumeration of Computable functions

Conceptual table of ALL Turing Machine behaviors...

VERTICAL AXIS: Enumeration of TM's (computable functions)

HORIZONTAL AXIS: Enumeration of input tapes.

ENTRY AT (n, m): Result of applying mth TM to argument n

INTEGER k: TM halts, leaving k on tape.

★ : TM never halts.

f_i	$f_i(0)$	$f_i(1)$	$f_i(2)$	...	$f_i(n)$	...
f_0	37	23	★	...	33	...
f_1	42	★	111	...	12	...
f_2	★	★	★	...	★	...
...	...	...	...	...	...	...
f_m	0	★	831	...	$f_m(n)$	...
...	...	...	...	...	...	...

aren't all well-defined
integer functions
computable?

NO!

there are simply
too many integer
functions to fit in
our enumeration!

Uncomputable Functions

Unfortunately, not every well-defined integer function is computable. The most famous such function is the so-called Halting function, $f_H(k, j)$, defined by:

$f_H(k, j) = 1$ if $T_k[j]$ halts;
0 otherwise.

$f_H(k, j)$ determines whether the kth TM halts when given a tape containing j.

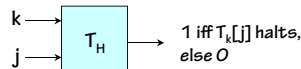
THEOREM: f_H is different from every function in our enumeration of computable functions; hence it cannot be computed by any Turing Machine.

PROOF TECHNIQUE: "Diagonalization" (after Cantor, Gödel)

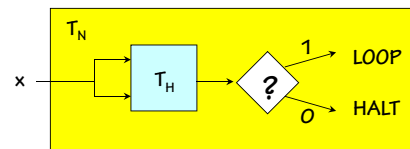
- If f_H is computable, it is equivalent to some TM (say, T_H).
- Using T_H as a component, we can construct another TM whose behavior differs from every entry in our enumeration and hence must not be computable.
- Hence f_H cannot be computable.

Why f_H is uncomputable

If f_H is computable, it is equivalent to some TM (say, T_H):



Then T_N (N for "Nasty"), which must be computable if T_H is:



$T_N[x]$: LOOPS if $T_x[x]$ halts;
HALTS if $T_x[x]$ loops

Finally, consider giving N as an argument to T_N :

$T_N[N]$: LOOPS if $T_N[N]$ halts;
HALTS if $T_N[N]$ loops



T_N can't be
computable, hence
 T_H can't either!

Footnote: Diagonalization

(clever proof technique used by Cantor, Gödel, Turing)

If T_H exists, we can use it to construct T_N . Hence T_N is computable if T_H is.
(informally we argue by Church's Thesis; but we can show the actual T_N construction, if pressed)

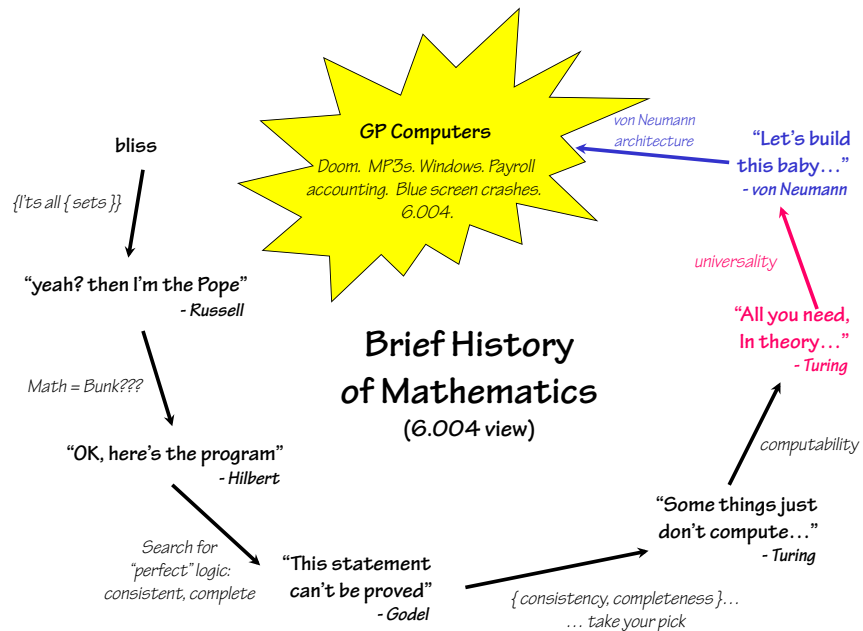
Why T_N can't be computable:

f_i	$f_i(0)$	$f_i(1)$	$f_i(2)$	...	$f_i(n)$	...
f_0	★	23	★	...	33	...
f_1	42	★	111	...	12	...
f_2	★	★	★	...	★	...
...	...	...	...	...	...	...
f_m	0	★	831	...	$f_m(n)$	...
...	...	...	...	...	...	...

T_N differs from every
computable function
for at least one
argument – along the
diagonal of our table.
Hence T_N can't be
among the entries in
our table!

Hence no such T_H can be constructed, even in theory.

Computable Functions:
A TINY SUBSET of all
Integer functions!



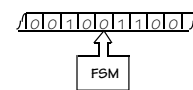
6.004 - Spring 2009

3/17/09

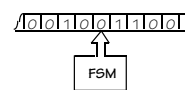
L12 - Models of Computation 21

meanwhile... Turing machines Galore!

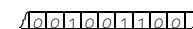
"special-purpose"
Turing Machines....



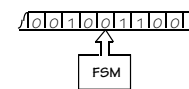
Multiplication



Sorting



Factorization



Primality Test

Is there an
alternative to
Infinitely many,
ad-hoc
Turing Machines?

6.004 - Spring 2009

3/17/09

L12 - Models of Computation 22

The Universal Function

OK, so there are uncomputable functions – infinitely many of them, in fact.

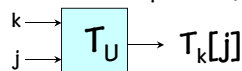
Here's an interesting candidate to explore: the Universal function, U , defined by

$$U(k, j) = T_k[j]$$

Could this be computable???

it sure would be neat
to have a single,
general-purpose
machine...

SURPRISE! U is computable by a Turing Machine:



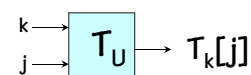
In fact, there are infinitely many such machines. Each is capable of performing any computation that can be performed by any TM!

6.004 - Spring 2009

3/17/09

L12 - Models of Computation 23

Big Idea #4: Universality



What's going on here?

k encodes a "program" – a description of some arbitrary machine.

j encodes the input data to be used.

T_U interprets the program, emulating its processing of the data!

KEY IDEA: Interpretation.
Manipulate coded representations of computing machines, rather than the machines themselves.

Turing Universality: The *Universal Turing Machine* is the paradigm for modern general-purpose computers! (cf: earlier *special-purpose* computers)

- Basic threshold test: Is your machine *Turing Universal*? If so, it can emulate every other Turing machine!
- Remarkably low threshold: UTMs with handfuls of states exist.
- Every modern computer is a UTM (given enough memory)
- To show your machine is Universal: demonstrate that it can emulate some known UTM.

6.004 - Spring 2009

3/17/09

L12 - Models of Computation 24

Coded Algorithms: Key to CS

data vs hardware

Algorithms as data: enables

COMPILERS: analyze, optimize, transform behavior

$$T_{\text{COMPILER-X-to-Y}}[P_X] = P_Y, \text{ such that}$$
$$T_X[P_X, z] = T_Y[P_Y, z]$$

LANGUAGE DESIGN: Separate specification
from implementation

- C, Java, JSIM, Linux, ... all run on X86, PPC, Sun, ...
- Parallel development paths:
 - Language/Software design
 - Interpreter/Hardware design

SOFTWARE ENGINEERING:

Composition, iteration,
abstraction of coded behavior

$$F(x) = g(h(x), p((q(x))))$$

Summary

Formal models (computability, Turing Machines, Universality)
provide the basis for modern computer science:

- Fundamental limits (what can't be done, even given plenty of memory and time)
- Fundamental equivalence of computation models
- Representation of algorithms as data, rather than machinery
- Programs, Software, Interpreters, Compilers, ...

They leave many practical dimensions to deal with:

- Costs: Memory size, Time Performance
- Programmability

Next step: Design of a practical interpreter!

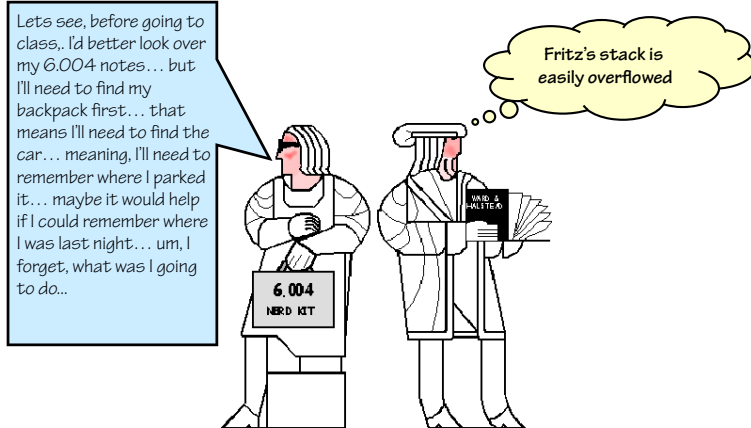
MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Stacks and Procedures



Lab 4 due tonight!

Where we left things last week...

```
int fact(int n)
{
    int r = 1;
    while (n>0) {
        r = r*n;
        n = n-1;
    }
    return r;
}

fact(4);
```

Procedures & Functions

- Reusable code fragments that are **called** as needed
- Single "named" entry point
- Parameterizable
- Local state (variables)
- Upon completion control is transferred back to **caller**

Procedure Linkage: First Try

```
int fact(int n)
{
    if (n>0)
        return n*fact(n-1);
    else
        return 1;
}

fact(4);
```

Oh no, not recursion! Didn't we get enough of that in 6.001?

fact(4) = 4*fact(3)
fact(3) = 3*fact(2)
fact(2) = 2*fact(1)
fact(1) = 1*fact(0)
fact(0) = 1

Proposed convention:

- pass arg in R1
- pass return addr in R28
- return result in R0
- questions:
 - nargs > 1?
 - preserve regs?

Let's just use some registers. We've got plenty...

Procedure Linkage: First Try

```
int fact(int n)
{
    if (n>0)
        return n*fact(n-1);
    else
        return 1;
}

fact(3);
```

```
fact:
    CMPLC(r1,0,r0)
    BT(r0,else)
    MOVE(r1,r2) | save n
    SUBC(r2,1,r1)
    BR(fact,r28)
    MUL(r0,r2,r0)
    BR(rtn)
else: CMOVE(1,r0)
rtn:  JMP(r28,r31)

main: CMOVE(3,r1)
      BR(fact,r28)
      HALT()
```

Proposed convention:

- pass arg in R1
- pass return addr in R28
- return result in R0
- questions:
 - nargs > 1?
 - preserve regs?

Need: $O(n)$ storage locations!

Revisiting Procedure's Storage Needs

Basic Overhead for Procedures/Functions:

- Arguments
 $f(x,y,z)$ or perhaps... $\sin(a+b)$
- Return Address back to caller
- Results to be passed back to caller.

In C it's the caller's job to evaluate its arguments as expressions, and pass their resulting *values* to the callee... Thus, a variable name is just a simple case of an expression.

Temporary Storage:

intermediate results during expression evaluation.

$(a+b)*(c+d)$

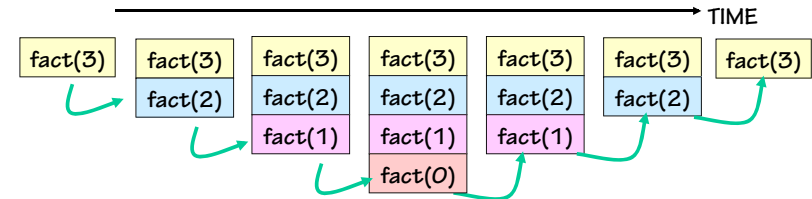
Local variables:

```
{ int x, y;
  ... x ... y ...;
}
```

Each of these is specific to a particular *activation* of a procedure; collectively, they may be viewed as the procedure's *activation record*.

Lives of Activation Records

```
int fact(int n)
{ if (n > 0) return n*fact(n-1);
  else return 1;
}
```



A procedure call creates a new activation record. Caller's record is preserved because we'll need it when call finally returns.

Return to previous activation record when procedure finishes, permanently discarding activation record created by call we are returning from.

Insight (ca. 1960): We need a STACK!

Suppose we allocated a SCRATCH memory for holding temporary variables. We'd like for this memory to grow and shrink as needed. And, we'd like it to have an easy management policy.

One possibility is a

STACK

A last-in-first-out (LIFO) data structure.



Figure by MIT OpenCourseWare.

Some interesting properties of stacks:

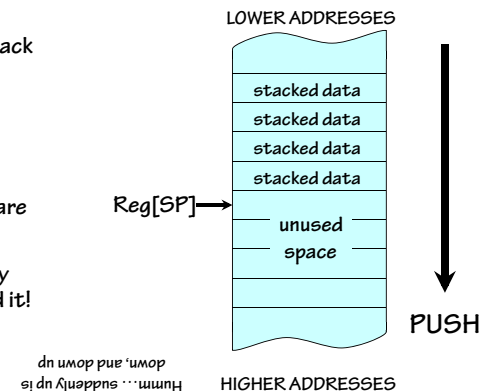
- Low overhead: Allocation, deallocation by simply adjusting a pointer.
- Basic PUSH, POP discipline: strong constraint on deallocation order.
- Discipline matches procedure call/return, block entry/exit, interrupts, etc.

Stack Implementation

CONVENTIONS:

- Dedicate a register for the Stack Pointer (SP), R29.
- Builds UP (towards higher addresses) on push
- SP points to first **UNUSED** location; locations below SP are allocated (protected).
- Discipline: can use stack at any time; but leave it as you found it!
- Reserve a block of memory well away from our program and its data

We use only *software conventions* to implement our stack (many architectures dedicate hardware)



Other possible implementations include stacks that grow "down", SP points to top of stack, etc.

Stack Management Macros

PUSH (RX) : push Reg[x] onto stack

Reg[SP] = Reg[SP] + 4;
Mem[Reg[SP]-4] = Reg[x]

ADDC(R29, 4, R29)
ST(RX, -4, R29)

POP (RX) : pop the value on the top of the stack into Reg[x]

Reg[x] = Mem[Reg[SP]-4]
Reg[SP] = Reg[SP] - 4;

LD(R29, -4, RX)
ADDC(R29, -4, R29)

Why?

ALLOCATE (k) : reserve k WORDS of stack

Reg[SP] = Reg[SP] + 4*k

ADDC(R29, 4*k, R29)

DEALLOCATE (k) : release k WORDS of stack

Reg[SP] = Reg[SP] - 4*k

SUBC(R29, 4*k, R29)

Fun with Stacks

We can squirrel away variables for later. For instance, the following code fragment can be inserted anywhere within a program.

```
|
| Argh!!! I'm out of registers Scotty!!
|
PUSH(R0)      | Frees up R0
PUSH(R1)      | Frees up R1
LD(R31, dilithium_xtals, R0)
LD(R31, seconds_til_explosion, R1)
suspense: SUBC(R1, 1, R1)
            BNE(R1, suspense, R31)
            ST(R0, warp_engines, R31)
POP(R1)       | Restores R1
POP(R0)       | Restores R0
```

Data is popped off the stack in the opposite order that it is pushed on

AND Stacks can also be used to solve other problems...

Solving Procedure Linkage "Problems"

A reminder of our storage needs:

- 1) We need a way to *pass arguments* into procedures
- 2) Procedures need their own *LOCAL variables*
- 3) Procedures need to *call other procedures*
- 4) Procedures might *call themselves* (Recursion)

BUT FIRST, WE'LL COMMIT SOME MORE REGISTERS:

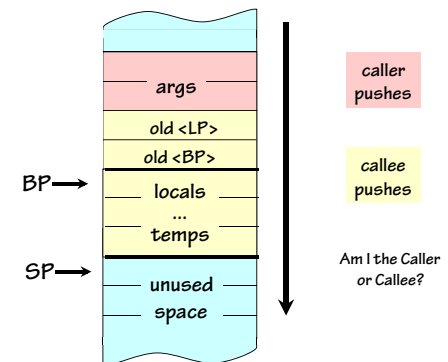
r27 = **BP**. Base ptr, points into stack to the local variables of callee
r28 = **LP**. Linkage ptr, return address to caller
r29 = **SP**. Stack ptr, points to 1st unused word

PLAN: CALLER puts args on stack, calls via something like BR(CALLEE, LP)
leaving return address in LP.

"Stack frames" as activation records

The CALLEE will use the stack for all of the following storage needs:

1. saving the RETURN ADDRESS back to the caller
2. saving the CALLER's base ptr
3. Creating its own local/ temp variables



In theory it's possible to use SP to access stack frame, but offsets will change due to PUSHs and POPs. For convenience we use BP so we can use constant offsets to find, e.g., the first argument.

Stack Frame Details

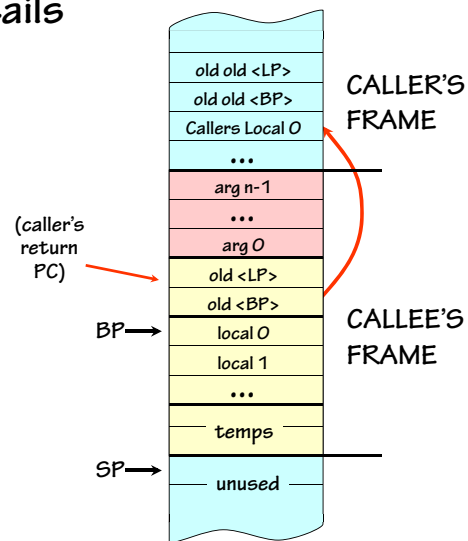
The CALLER passes arguments to the CALLEE on the stack in REVERSE order

F(1,2,3,4) is translated to:

```

ADDC (R31, 4, R0)
PUSH (R0)
ADDC (R31, 3, R0)
PUSH (R0)
ADDC (R31, 2, R0)
PUSH (R0)
ADDC (R31, 1, R0)
PUSH (R0)
BEQ (R31, F, LP)
    
```

Why push args in REVERSE order???



Order of Arguments

Why push args onto the stack in reverse order?

1) It allows the BP to serve double duties when accessing the local frame

To access k^{th} local variable ($k \geq 0$)

```
LD (BP, k*4, rx)
```

or

```
ST (rx, k*4, BP)
```

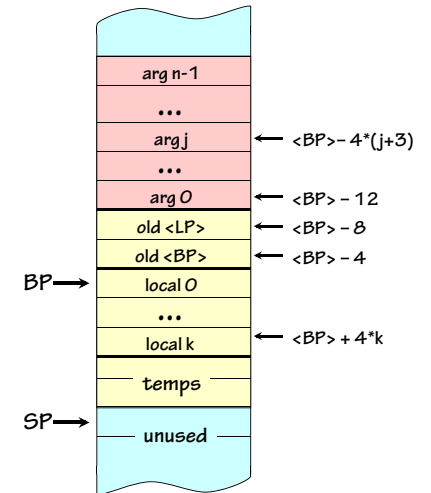
To access j^{th} argument ($j \geq 0$):

```
LD (BP, -4*(j+3), rx)
```

or

```
ST (rx, -4*(j+3), BP)
```

2) The CALLEE can access the first few arguments without knowing how many arguments have been passed!



Procedure Linkage: The Contract

The CALLER will:

- Push args onto stack, in reverse order.
- Branch to callee, putting return address into LP.
- Remove args from stack on return.

The CALLEE will:

- Perform promised computation, leaving result in R0.
- Branch to return address.
- Leave stacked data intact, including stacked args.
- Leave regs (except R0) unchanged.

Procedure Linkage

typical "boilerplate" templates

Calling Sequence

```

PUSH (arg_n)      | push args, last arg first
...
PUSH (arg_1)
BEQ (R31, f, LP)   | Call f.
DEALLOCATE (n)     | Clean up!
...               | (f's return value in r0)
    
```

Entry Sequence

```

f:  PUSH (LP)       | Save LP and BP
    PUSH (BP)       | in case we make new calls.
    MOVE (SP, BP)   | set BP=frame base
    ALLOCATE (nlocals) | allocate locals
    (push other regs) | preserve any regs used
    
```

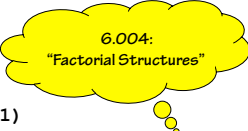
Return Sequence

```

(pop other regs)   | restore regs
MOVE (val, R0)     | set return value
MOVE (BP, SP)      | strip locals, etc
POP (BP)           | restore CALLER's linkage
POP (LP)           | (the return address)
JMP (LP, R31)      | return.
    
```

Where's the Deallocate?

Our favorite procedure...

fact: <pre> PUSH (LP) PUSH (BP) MOVE (SP, BP) PUSH (r1) LD (BP, -12, r1) BNE (r1, big) ADDC (r31, 1, r0) BR (rtn) </pre>	<pre> save linkages new frame base preserve regs r1 ← n if (n != 0) else return 1; else return 1; </pre>	<pre> int fact(int n) { if (n != 0) return n*fact(n-1); else return 1; } </pre>
big: <pre> SUBC (r1, 1, r1) PUSH (r1) BR (fact, LP) DEALLOCATE (1) LD (BP, -12, r1) MUL (r1, r0, r0) </pre>	<pre> r1 ← (n-1) push arg1 fact(n-1) pop arg1 r0 ← n r0 ← n*fact(n-1) </pre>	
rtn: <pre> POP (r1) MOVE (BP, SP) POP (BP) POP (LP) JMP (LP, R31) </pre>	<pre> restore regs Why? restore links return. </pre>	

Recursion?

But of course!

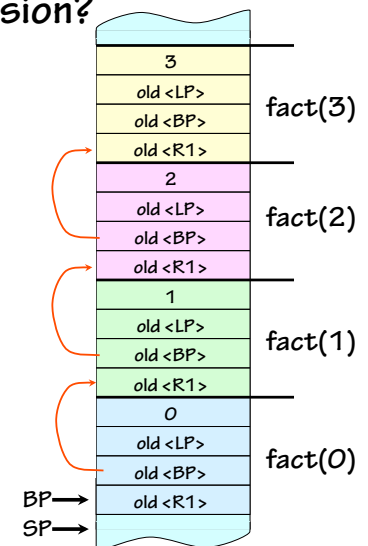
- Frames allocated for each recursive call...
- De-allocated (in inverse order) as recursive calls return.

Debugging skill:

"stack crawling"

- Given code, stack snapshot - figure out what, where, how, who...
- Follow old <BP> links to parse frames
- Decode args, locals, return locations, etc etc etc

Particularly useful on 6.004 quizzes!

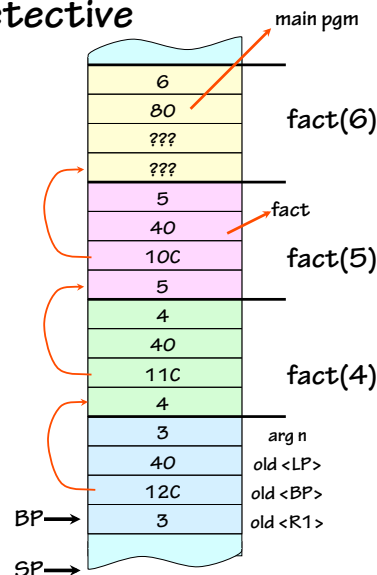


Stack Detective

fact(n) is called. During the calculation, the computer is stopped with the PC at **0x40**; the stack contents are shown (in hex).

- What's the argument to the most recent call to fact? **3**
- What's the argument to the original call to fact? **6**
- What's the location of the original calling (BR) instruction? **80 - 4 = 7C**
- What instruction is about to be executed? **DEALLOCATE(1)**
- What value is in BP? **13C**
- What value is in SP? **13C + 4 + 4 = 144**
- What value is in R0? **fact(2) = 2**
- What follows the call to fact(n)?

another call to fact. Its the only program these guys have.



Man vs. Machine

Here's a C program which was fed to the C compiler*. Can you generate code as good as it did?

```

int ack(int i, int j)
{
  if (i == 0) return j+j;
  if (j == 0) return i+1;
  return ack(i-1, ack(i, j-1));
}

```

* GCC Port courtesy of Cotton Seed, Pat LoPresti, & Mitch Berger; available on Athena:

```

Athena% attach 6.004
Athena% gcc-beta -S -O2 file.c

```

Tough Problems

1. NON-LOCAL variable access, particularly in nested procedure definitions.

"FUNarg" problem of LISP:

```
((lambda (x)
  (lambda (y) (+ x y)))
  3)
```

4) Python:

```
def f(x):
    def g(y): return x+y
    return g
z = f(3)(4)
```

```
1 (int x)
{
  int g(int y)
  {
    return x+y;
  }
  return g;
}
z = f(3)(4);
```

Conventional solutions:

- Environments, closures.
- "static links" in stack frames, pointing to frames of statically enclosing blocks. This allows a run-time discipline which correctly accesses variables in enclosing blocks.

BUT... enclosing block may no longer exist (as above!).

(C avoids this problem by outlawing nested procedure declarations!)

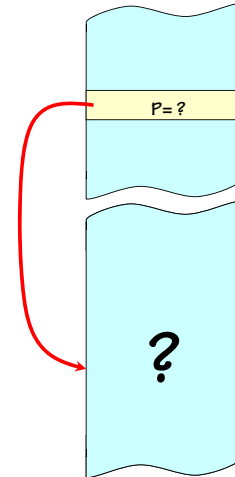
2. "Dangling References" - - -

Dangling References

```
int *p; /* a pointer */
```

```
int h(x)
{
    int y = x*3;
    p = &y;
    return 37;
}
```

```
h(10);
print(*p);
```



What do we expect???

*Randomness. Crashes. Smoke. Obscenities.
Furious calls to Redmond, WA.*

Dangling References:

different strokes...

C and C++: real tools, real dangers.

"You get what you deserve".

Java / Scheme / Python / ...: kiddie scissors only.

- No "ADDRESS OF" operator: language restrictions *forbid* constructs which could lead to dangling references.
- Automatic storage management: garbage collectors, reference counting: local variables allocated from a "heap" rather than a stack.

"Safety" as a language/runtime property: guarantees against stray reads, writes.

- Tension: (manual) algorithm-specific optimization opportunities vs. simple, uniform, non-optimal storage management
- Tough language/compiler problem: abstractions, compiler technology that provides simple safety yet exploits efficiency of stack allocation.

Next Time: Building a Beta

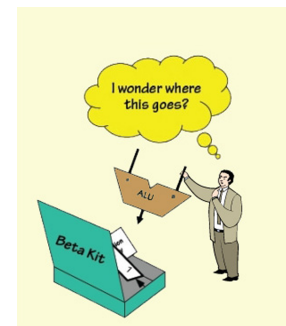


Figure MIT OpenCourseWare.

```
ack:  PUSH (LP)
      PUSH (BP)
      MOVE (SP, BP)
      PUSH (R1)
      PUSH (R2)
      LD (BP, -12, R2)
      LD (BP, -16, R0)
_L4:  SHLC (R0, 1, R1)
      BEQ (R2, _L1)
      ADDC (R2, 1, R1)
      BEQ (R0, _L1)
      SUBC (R2, 1, R1)
      SUBC (R0, 1, R0)
      PUSH (R0)
      PUSH (R2)
      BR (ack, LP)
      DEALLOCATE (2)
      MOVE (R1, R2)
      BR (_L4)
_L1:  MOVE (R1, R0)
      POP (R2)
      POP (R1)
      POP (BP)
      POP (LP)
      JMP (LP)
```

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Building the Beta



Figure by MIT OpenCourseWare.

Lab #5 due Thursday

CPU Design Tradeoffs

Maximum Performance: measured by the numbers of instructions executed per second

Minimum Cost: measured by the size of the circuit.

Best Performance/Price: measured by the ratio of MIPS to size. In power-sensitive applications MIPS/Watt is important too.

Performance Measure

Millions of Instructions per Second

$$\text{MIPS} = \frac{\text{Clock Frequency (MHz)}}{\text{C.P.I.}}$$

Clocks per instruction

PUSHING PERFORMANCE ...

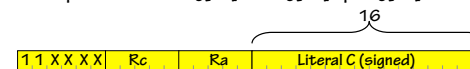
TODAY: 1 cycle/inst.

LATER: more MHz via pipelining

The Beta ISA



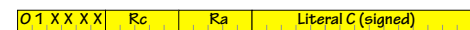
Operate class: $\text{Reg}[Rc] \leftarrow \text{Reg}[Ra] \text{ op } \text{Reg}[Rb]$



Operate class: $\text{Reg}[Rc] \leftarrow \text{Reg}[Ra] \text{ op } \text{SXT}(C)$

Opcodes, both formats:

ADD SUB MUL* DIV* *optional
CMFEQ CMPLT CMPLT
AND OR XOR
SHL SHR SRA



LD: $\text{Reg}[Rc] \leftarrow \text{Mem}[\text{Reg}[Ra] + \text{SXT}(C)]$
ST: $\text{Mem}[\text{Reg}[Ra] + \text{SXT}(C)] \leftarrow \text{Reg}[Rc]$
JMP: $\text{Reg}[Rc] \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{Reg}[Ra]$
BEQ: $\text{Reg}[Rc] \leftarrow \text{PC} + 4; \text{if } \text{Reg}[Ra] = 0 \text{ then } \text{PC} \leftarrow \text{PC} + 4 + 4 * \text{SXT}(C)$
BNE: $\text{Reg}[Rc] \leftarrow \text{PC} + 4; \text{if } \text{Reg}[Ra] \neq 0 \text{ then } \text{PC} \leftarrow \text{PC} + 4 + 4 * \text{SXT}(C)$
LDR: $\text{Reg}[Rc] \leftarrow \text{Mem}[\text{PC} + 4 + 4 * \text{SXT}(C)]$

Instruction classes distinguished by OPCODE:
OP
OPC
MEM
MEM
Transfer of Control

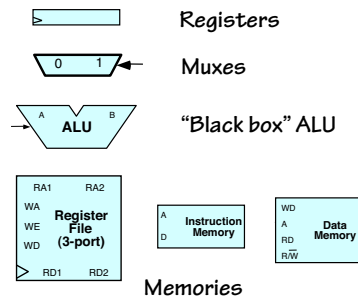
Approach: Incremental Featurism

Each instruction class can be implemented using a simple component repertoire. We'll try implementing data paths for each class individually, and merge them (using MUXes, etc).

Steps:

1. Operate instructions
2. Load & Store Instructions
3. Jump & Branch instructions
4. Exceptions
5. Merge data paths

Our Bag of Components:

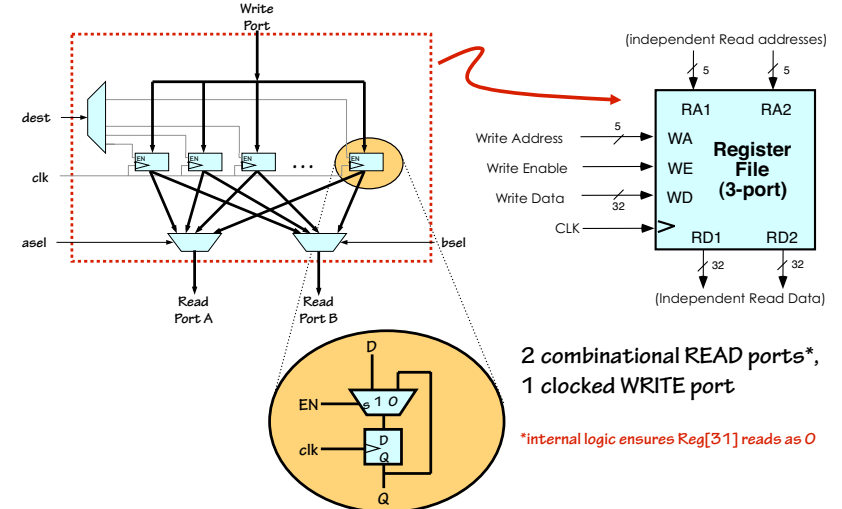


6.004 - Spring 2009

3/31/09

L14 - Building a Beta 5

Multi-Port Register Files



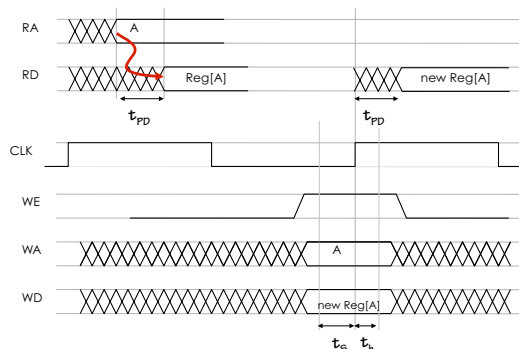
6.004 - Spring 2009

3/31/09

L14 - Building a Beta 6

Register File Timing

2 combinational READ ports, 1 clocked WRITE port



What if (say) $WA=RA1$???

RD1 reads "old" value of Reg[RA1] until next clock edge!

6.004 - Spring 2009

3/31/09

L14 - Building a Beta 7

Starting point: ALU Ops

32-bit (4-byte) ADD instruction:

10000000010000010000110000000000

OpCode Rc Ra Rb (unused)

Means, to BETA, $Reg[R4] \leftarrow Reg[R2] + Reg[R3]$

First, we'll need hardware to:

- Read next 32-bit instruction
- DECODE instruction: ADD, SUB, XOR, etc
- READ operands (Ra, Rb) from Register File;
- PERFORM indicated operation;
- WRITE result back into Register File (Rc).

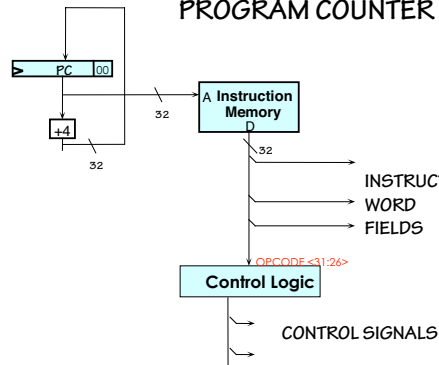
6.004 - Spring 2009

3/31/09

L14 - Building a Beta 8

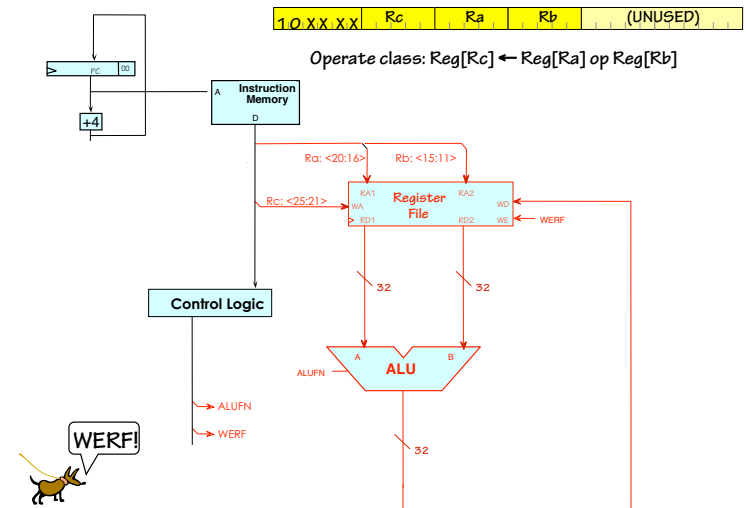
Instruction Fetch/Decode

- Use a counter to FETCH the next instruction:
PROGRAM COUNTER (PC)

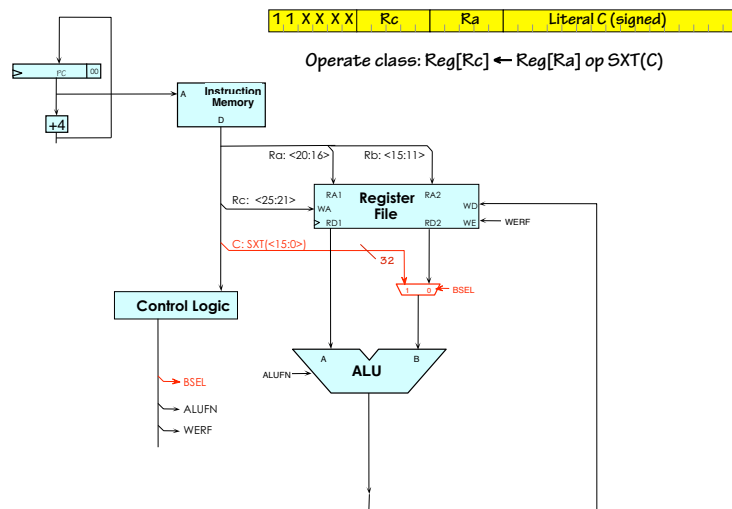


- use PC as memory address
- add 4 to PC, load new value at end of cycle
- fetch instruction from memory
 - use some instruction fields directly (register numbers, 16-bit constant)
 - use bits <31:26> to generate controls

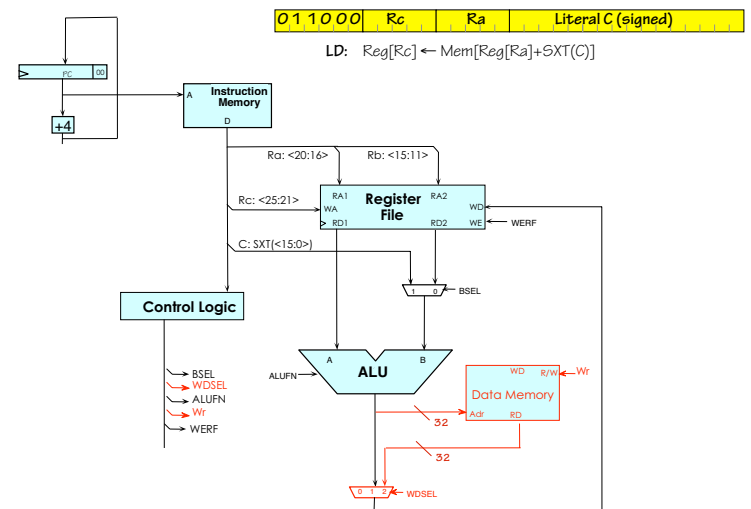
ALU Op Data Path



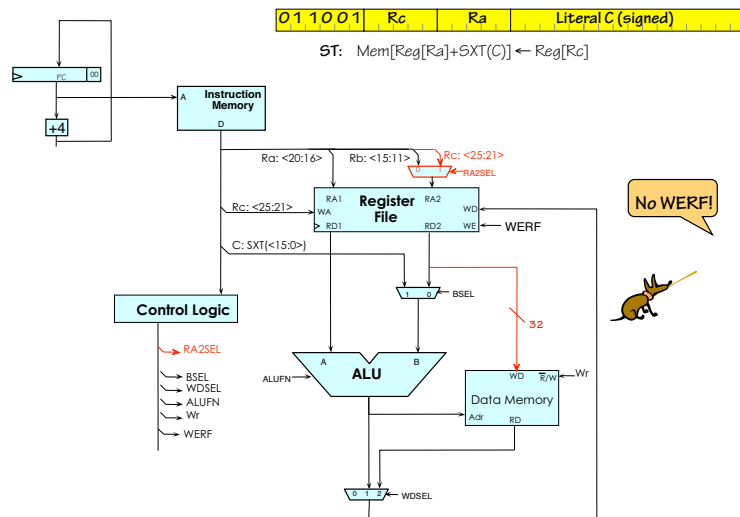
ALU Operations (w/constant)



Load Instruction



Store Instruction

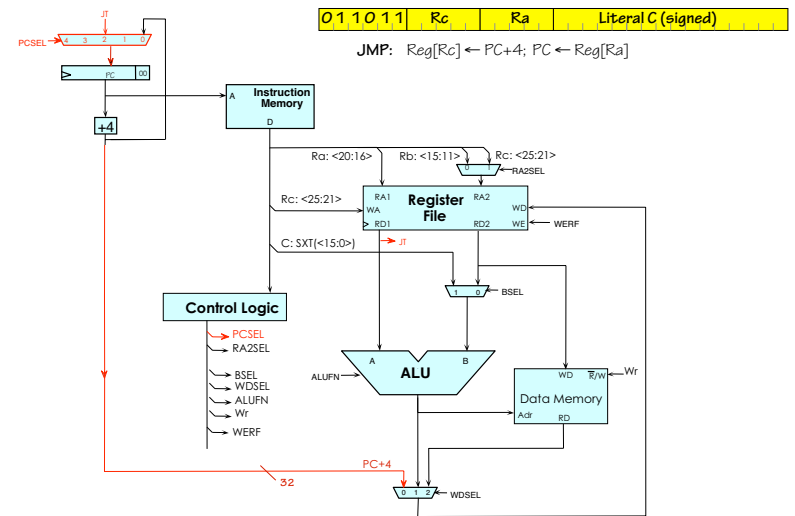


6.004 - Spring 2009

3/31/09

L14 - Building a Beta 13

JMP Instruction

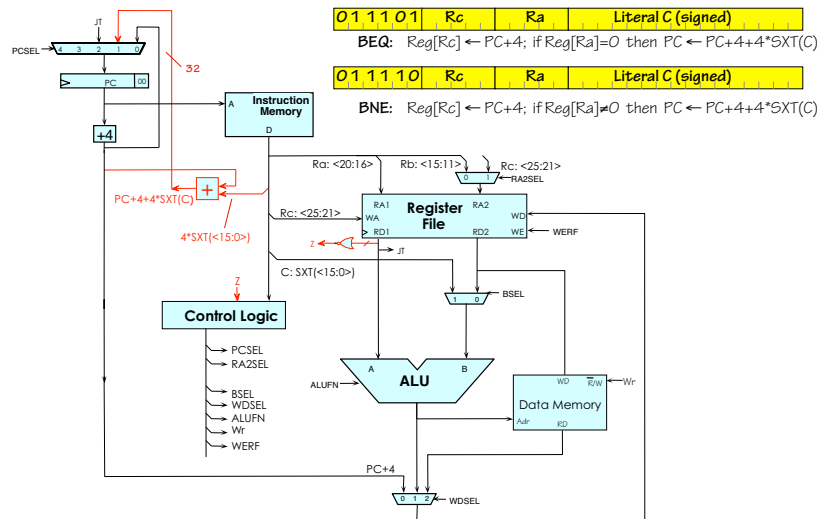


6.004 - Spring 2009

3/31/09

L14 - Building a Beta 14

BEQ/BNE Instructions

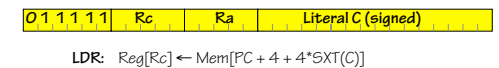


6.004 - Spring 2009

3/31/09

L14 - Building a Beta 15

Load Relative Instruction



Hey, WAIT A MINUTE. What's Load Relative good for anyway??? I thought

- Code is "PURE", i.e. READ-ONLY; and stored in a "PROGRAM" region of memory;
- Data is READ-WRITE, and stored either
 - On the STACK (local); or
 - In some GLOBAL VARIABLE region; or
 - In a global storage HEAP.

So why an instruction designed to load data that's "near" the instruction???

Addresses & other large constants

C: $X = X * 123456;$

BETA:

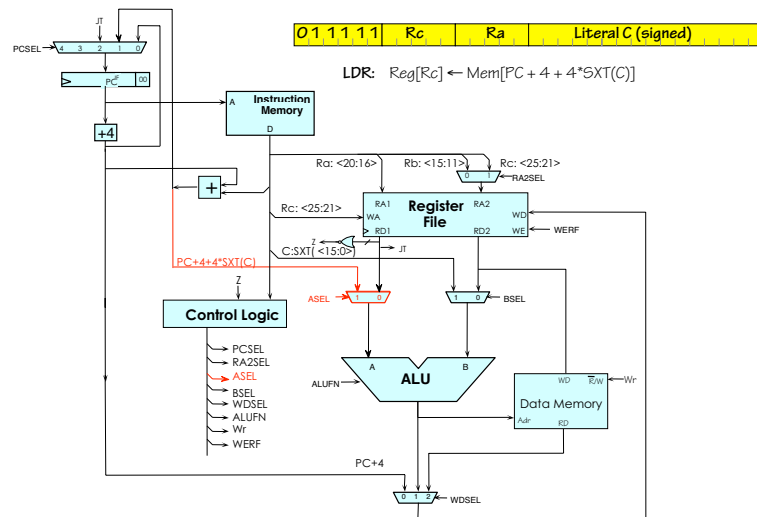
```
LD(X, r0)
LDR(c1, r1)
MUL(r0, r1, r0)
ST(r0, X)
...
c1: LONG(123456)
```

6.004 - Spring 2009

3/31/09

L14 - Building a Beta 16

LDR Instruction



6.004 - Spring 2009

3/31/09

L14 - Building a Beta 17

Exceptions

What if something BAD happens?

- Execution of an illegal op-code
- Reference to non-existent memory
- Divide by zero

Or, maybe, just something unanticipated...

- User hits a key
- A packet comes in via the network

GOAL: handle all these cases (and more) in *SOFTWARE*:

- Treat each such case as an (implicit) procedure call...
- Procedure handles problem, returns to interrupted program.
- *TRANSPARENT* to interrupted program!
- Important added capability: handlers for certain errors (illegal op-codes) can extend instruction set using software (Lab 7!).

6.004 - Spring 2009

3/31/09

L14 - Building a Beta 18

Exception Processing

Plan:

- Interrupt running program
- Invoke exception handler (like a procedure call)
- Return to continue execution.

We'd like **RECOVERABLE INTERRUPTS** for

- Synchronous events, generated by CPU or system
FAULTS (eg, Illegal Instruction, divide-by-0, illegal mem address)
TRAPS & system calls (eg, read-a-character)
- Asynchronous events, generated by I/O
(eg, key struck, packet received, disk transfer complete)

KEY: *TRANSPARENCY* to interrupted program.

- Most difficult for asynchronous interrupts

6.004 - Spring 2009

3/31/09

L14 - Building a Beta 19

Implementation...

How exceptions work:

- Don't execute current instruction
- Instead fake a "forced" procedure call
 - save current PC (actually current PC + 4)
 - load PC with *exception vector*
 - 0x4 for synch. exception, 0xB for asynch. exceptions

Question: where to save current PC + 4?

- Our approach: reserve a register (R30, aka XP)
- Prohibit user programs from using XP. Why?

Example: DIV unimplemented

LD (R31, A, R0)
LD (R31, B, R1)
DIV (R0, R1, R2)
ST (R2, C, R31)

Forced by hardware

IllOp:
PUSH (XP)

Fetch inst. at Mem[Reg[XP]-4]
check for DIV opcode, get reg numbers
perform operation in SW, fill result reg

POP (XP)
JMP (XP)

6.004 - Spring 2009

3/31/09

L14 - Building a Beta 20



3/31/09

L14 – Building a Beta 21

6.004 – Spring 2009

3/31/09

L14 – Building a Beta 22



3/31/09

L14 – Building a Beta 23



3/31/09

L14 – Building a Beta 24

MIT OpenCourseWare
<http://ocw.mit.edu>

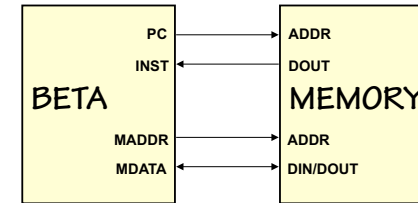
6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

The Memory Hierarchy

Lab #5 due tonight

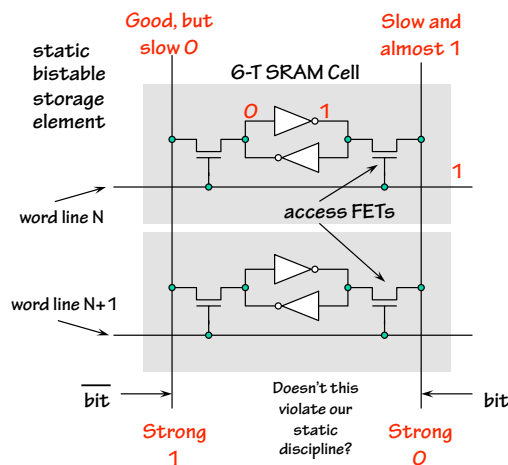
What we want in a memory



	Capacity	Latency	Cost
Register	100's of bits	20 ps	\$\$\$\$
SRAM	100's Kbytes	1 ns	\$\$\$
DRAM	100's Mbytes	40 ns	\$
Hard disk*	100's Gbytes	10 ms	¢
Want	1's Gbytes	1 ns	cheap

* non-volatile

SRAM Memory Cell



There are two bit-lines per column: one supplies the bit, the other it's complement.

On a Read Cycle:

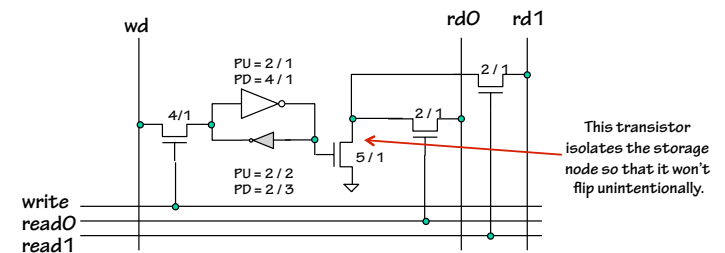
A single word line is activated (driven to "1"), and the access transistors enable the selected cells, and their complements, onto the bit lines.

Writes are similar to reads, except the bit-lines are driven with the desired value of the cell.

The writing has to "overpower" the original contents of the memory cell.

Multiport SRAMs

(a.k.a. Register Files)



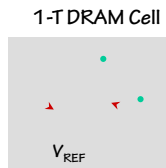
One can increase the number of SRAM ports by adding access transistors. By carefully sizing the inverter pair, so that one is strong and the other weak, we can assure that our WRITE bus will only fight with the weaker one, and the READs are driven by the stronger one - thus minimizing both access and write times.

1-T Dynamic Ram

Six transistors/cell may not sound like much, but they can add up quickly. What is the fewest number of transistors that can be used to store a bit?

Explicit storage capacitor

Can't we get rid of the explicit cap?

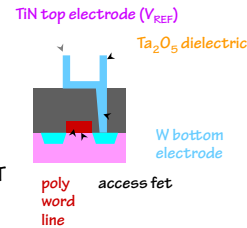


bit

C in storage capacitor determined by:

$$C = \frac{\epsilon A}{d}$$

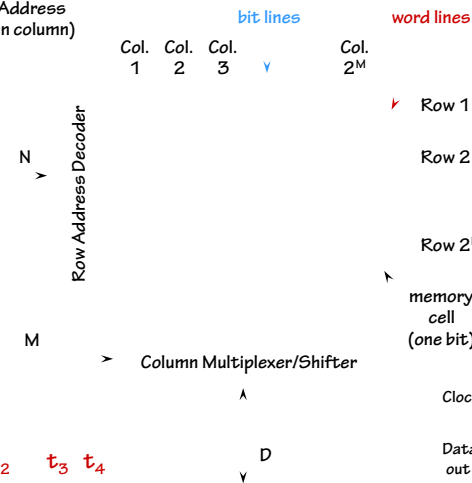
better dielectric more area
thinner film



Tricks for increasing throughput

but, alas, not latency

Multiplexed Address (row first, then column)



The first thing that should pop into your mind when asked to speed up throughput...

PIPELINING

Synchronous DRAM (SDRAM)

Double-clocked Synchronous DRAM (DDRAM)

Hard Disk Drives

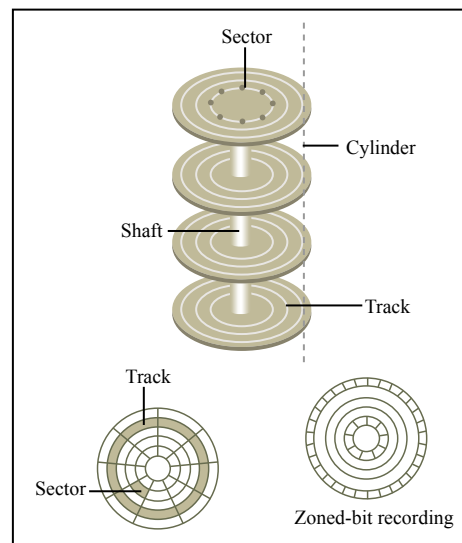


Figure by MIT OpenCourseWare.

Typical high-end drive:

- Average latency = 4 ms
- Average seek time = 9 ms
- Transfer rate = 20M bytes/sec
- Capacity = 100-500G byte
- Cost = ~\$ 1/Gbyte

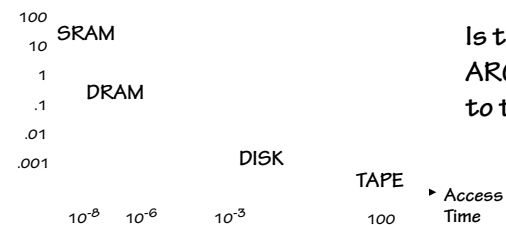
Quantity vs Quality...

Your memory system can be

- BIG and SLOW... or
- SMALL and FAST.

We've explored a range of circuit-design trade-offs.

\$/MB



Is there an ARCHITECTURAL solution to this DILEMMA?

Best of Both Worlds

What we WANT: A BIG, FAST memory!

We'd like to have a memory system that

- PERFORMS like 1 GBytes of SRAM; but
- COSTS like 1 GBytes of slow memory.

SURPRISE: We can (nearly) get our wish!

KEY: Use a hierarchy of memory technologies:



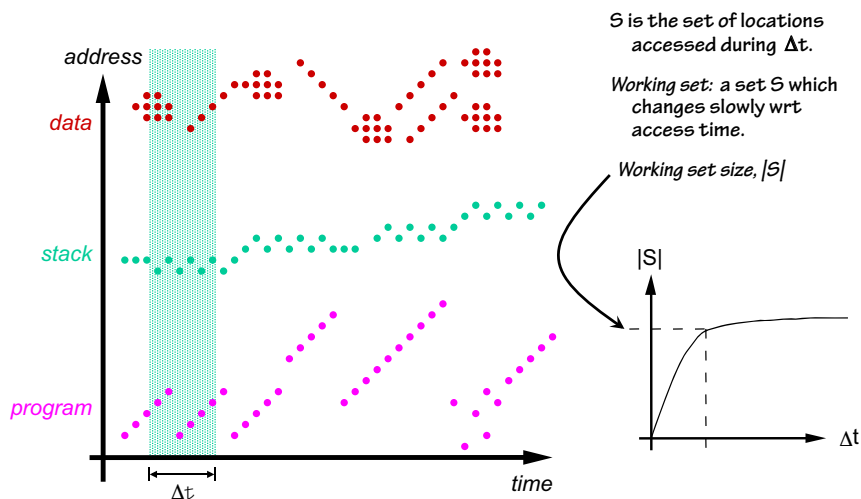
Key IDEA

- Keep the most often-used data in a small, fast SRAM (often local to CPU chip)
- Refer to Main Memory only rarely, for remaining data.
- The reason this strategy works: LOCALITY

Locality of Reference:

Reference to location X at time t implies that reference to location $X + \Delta X$ at time $t + \Delta t$ becomes more probable as ΔX and Δt approach zero.

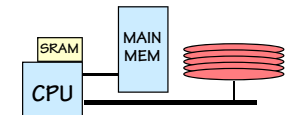
Memory Reference Patterns



Exploiting the Memory Hierarchy

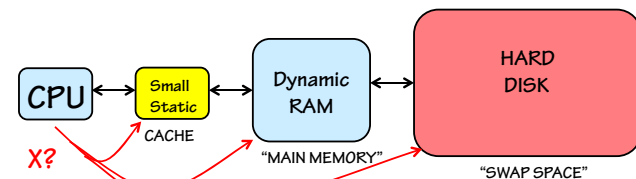
Approach 1 (Cray, others): Expose Hierarchy

- Registers, Main Memory, Disk each available as storage alternatives;
- Tell programmers: "Use them cleverly"

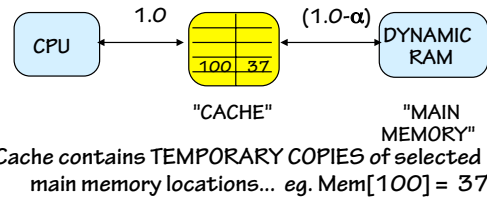


Approach 2: Hide Hierarchy

- Programming model: SINGLE kind of memory, single address space.
- Machine AUTOMATICALLY assigns locations to fast or slow memory, depending on usage patterns.



The Cache Idea: Program-Transparent Memory Hierarchy



GOALS:

- 1) Improve the **average access time**

α HIT RATIO: Fraction of refs found in CACHE.
 $(1-\alpha)$ MISS RATIO: Remaining references.

$$t_{ave} = \alpha t_c + (1-\alpha)(t_c + t_m) = t_c + (1-\alpha)t_m$$

- 2) Transparency (compatibility, programming ease)

Challenge:
make the
hit ratio as
high as
possible.

How High of a Hit Ratio?

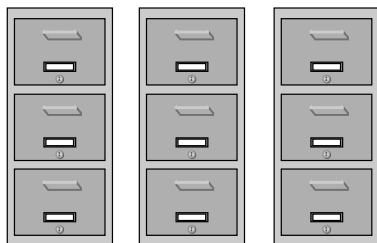
Suppose we can easily build an on-chip static memory with a 4 nS access time, but the fastest dynamic memories that we can buy for main memory have an average access time of 40 nS. How high of a hit rate do we need to sustain an average access time of 5 nS?

$$\alpha = 1 - \frac{t_{ave} - t_c}{t_m} = 1 - \frac{5 - 4}{40} = 97.5\%$$

The Cache Principle

Find "Bitdiddle, Ben"

5-Minute Access Time:



5-Second Access Time:



Figure by MIT OpenCourseWare.

ALGORITHM: Look nearby for the requested information first, if it's not there, check secondary storage

Basic Cache Algorithm

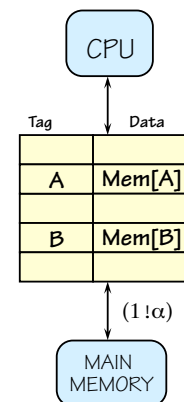
ON REFERENCE TO Mem[X]: Look for X among cache tags...

HIT: $X = TAG(i)$, for some cache line i

- READ: return DATA(i)
- WRITE: change DATA(i); Start Write to Mem(X)

MISS: X not found in TAG of any cache line

- REPLACEMENT SELECTION:
 - Select some line k to hold Mem[X] (Allocation)
- READ: Read Mem[X]
Set TAG(k)=X, DATA(k)=Mem[X]
- WRITE: Start Write to Mem(X)
Set TAG(k)=X, DATA(k)= new Mem[X]



QUESTION: How do we "search" the cache?

Associativity: Parallel Lookup

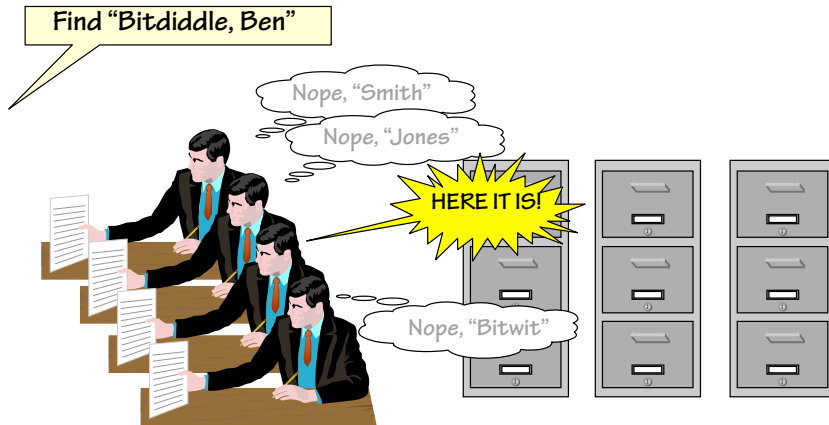
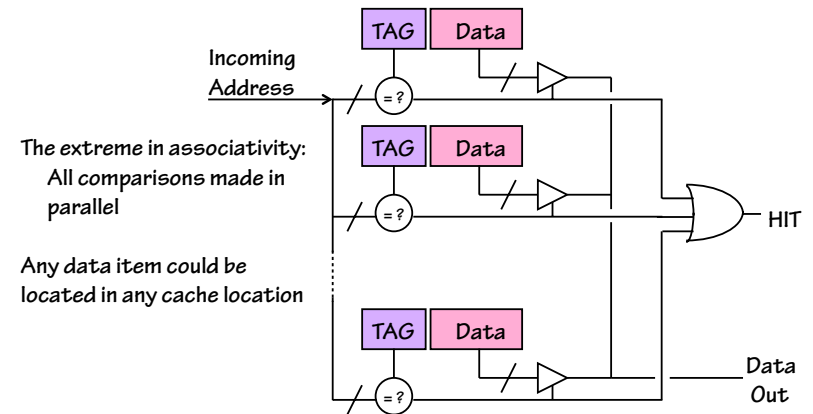
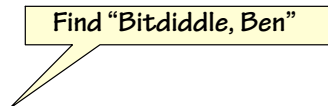


Figure by MIT OpenCourseWare.

Fully-Associative Cache



Direct-Mapped Cache (non-associative)



NO Parallelism:

Look in **JUST ONE** place,
determined by parameters of
incoming request (address bits)

... can use ordinary RAM as table

Need: Address Mapping Function!

Maps incoming **BIG** address to
small **CACHE** address... tells
us which **single** cache location
to use

Direct Mapped: just use a subset
of incoming address bits!

Collision when several addresses
map to same cache line.

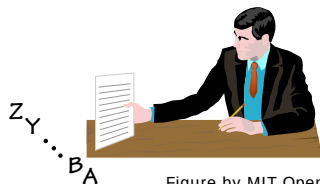
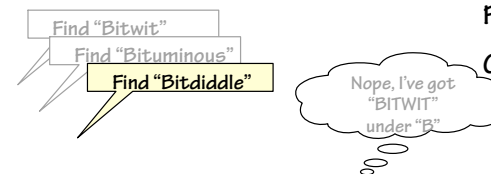


Figure by MIT OpenCourseWare.

The Problem with Collisions



PROBLEM:

Contention among B's... each
competes for same cache
line!

- CAN'T cache both
"Bitdiddle" & "Bitwit"

... Suppose B's tend
to come at once?

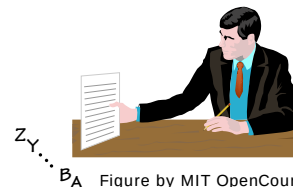
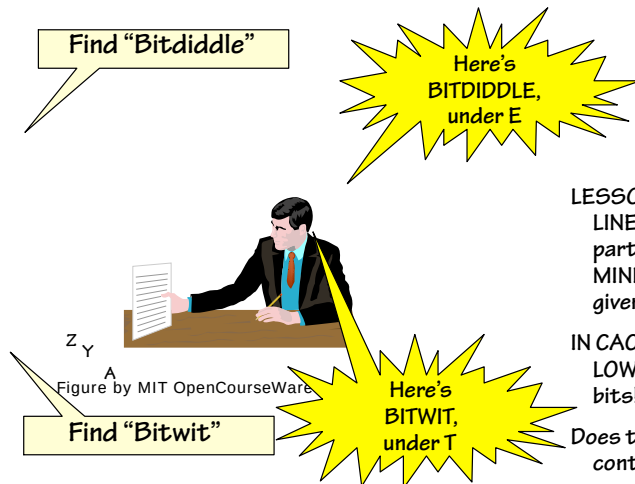


Figure by MIT OpenCourseWare.

BETTER IDEA:
File by **LAST** letter!

Optimizing for Locality:

selecting on statistically independent bits



LESSON: Choose CACHE LINE from independent parts of request to MINIMIZE CONFLICT given locality patterns...

IN CACHE: Select line by LOW ORDER address bits!

Does this ELIMINATE contention?

6.004 - Spring 2009

4/2/09

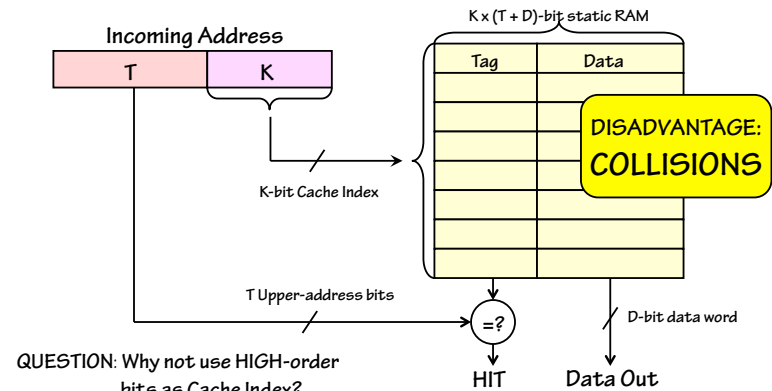
L15 - Memory Hierarchy 21

Direct Mapped Cache

Low-cost extreme:

Single comparator

Use ordinary (fast) static RAM for cache tags & data:



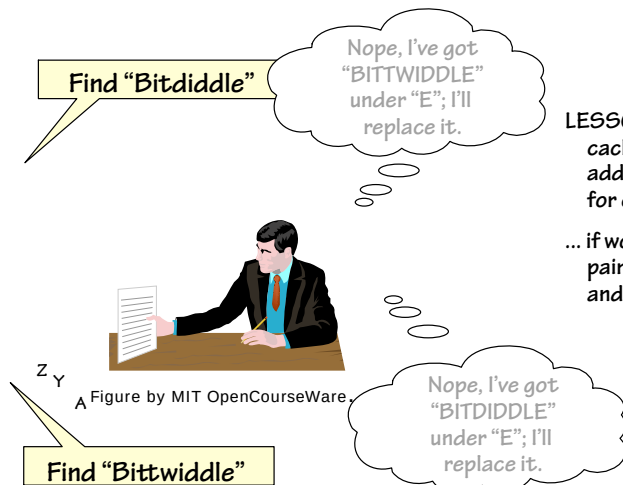
QUESTION: Why not use HIGH-order bits as Cache Index?

6.004 - Spring 2009

4/2/09

L15 - Memory Hierarchy 22

Contention, Death, and Taxes...



LESSON: In a non-associative cache, SOME pairs of addresses must compete for cache lines...

... if working set includes such pairs, we get THRASHING and poor performance.

6.004 - Spring 2009

4/2/09

L15 - Memory Hierarchy 23

Direct-Mapped Cache Contention

	Memory Address	Cache Line	Hit/Miss
Loop A: Pgm at 1024, data at 37:	1024	0	HIT
	37	37	HIT
	1025	1	HIT
	38	38	HIT
	1026	2	HIT
	39	39	HIT
Loop B: Pgm at 2048, data at 2048:	1024	0	MISS
	2048	0	MISS
	1025	1	MISS
	2049	1	MISS
	1026	2	MISS
	2050	2	MISS
...	1024	0	MISS
	...	...	...

Works GREAT here...

Assume 1024-line direct-mapped cache, 1 word/line. Consider tight loop, at steady state: (assume WORD, not BYTE, addressing)

... but not here!

We need some associativity, But not full associativity... Next lecture!

6.004 - Spring 2009

4/2/09

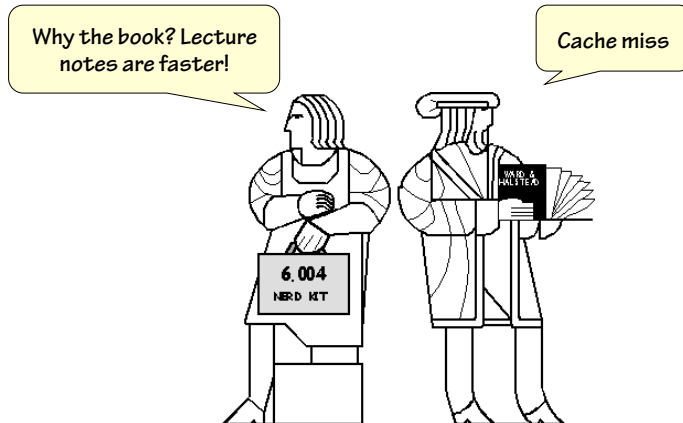
L15 - Memory Hierarchy 24

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

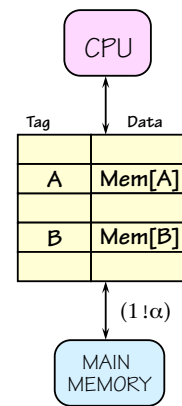
For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Cache Issues



Quiz #3 Friday!

Basic Cache Algorithm



ON REFERENCE TO Mem[X]:
Look for X among cache tags...

HIT: $X = \text{TAG}(i)$, for some cache line i

- READ: return DATA(i)
- WRITE: change DATA(i); Start Write to Mem(X)

MISS: X not found in TAG of any cache line

- REPLACEMENT SELECTION:
 - Select some line k to hold Mem[X] (Allocation)
- READ: Read Mem[X]
Set TAG(k)=X, DATA(k)=Mem[X]
- WRITE: Start Write to Mem(X)
Set TAG(k)=X, DATA(k)= new Mem[X]

Cache Design Issues

Associativity - a basic tradeoff between

- Parallel Searching (expensive) vs
- Constraints on which addresses can be stored where

Replacement Strategy:

- OK, we've missed. Gotta add this new address/value pair to the cache. What do we kick out?
 - Least Recently Used: discard the one we haven't used the longest.
 - Plausible alternatives, (e.g. random replacement).

Block Size:

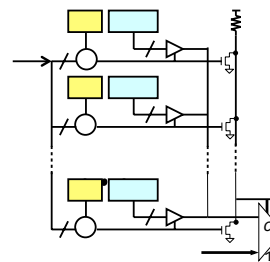
- Amortizing cost of tag over multiple words of data

Write Strategy:

- When do we write cache contents to main memory?

Associativity

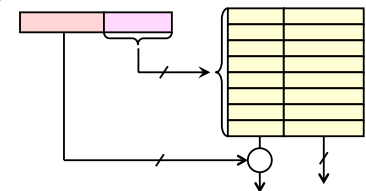
Lots...



Fully Associative

- expensive!
- flexible: any address can be cached in any line

... or NONE!



Direct Mapped

- cheap (ordinary SRAM)
- contention: addresses compete for cache lines

Direct-Mapped Cache Contention

Memory Address Cache Line Hit/Miss

Loop A:
Pgm at
1024,
data
at 37:

1024	0	HIT
37	37	HIT
1025	1	HIT
38	38	HIT
1026	2	HIT
39	39	HIT
1024	0	HIT

Loop B:
Pgm at
1024,
data
at
2048:

1024	0	MISS
2048	0	MISS
1025	1	MISS
2049	1	MISS
1026	2	MISS
2050	2	MISS
1024	0	MISS

Works
GREAT
here...

Assume 1024-line direct-mapped cache, 1 word/line.
Consider tight loop, at steady state:
(assume WORD, not BYTE, addressing)

... but not here!

We need some associativity,
But not full associativity...

Fully-assoc. vs. Direct-mapped

Fully-associative N-line cache:

- N tag comparators, registers used for tag/data storage (\$\$\$)
- Location A might be cached in any one of the N cache lines; no restrictions!
- Replacement strategy (e.g., LRU) used to pick which line to use when loading new word(s) into cache
- PROBLEM: Cost!

Direct-mapped N-line cache:

- 1 tag comparator, SRAM used for tag/data storage (\$)
- Location A is cached in a specific line of the cache determined by its address; address "collisions" possible
- Replacement strategy not needed: each word can only be cached in one specific cache line
- PROBLEM: Contention!

Cost vs Contention

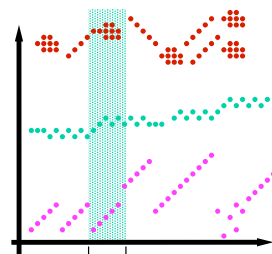
two observations...

1. Probability of collision diminishes with cache size...
... so lets build HUGE direct-mapped caches, using cheap SRAM!

2. Contention mostly occurs between independent "hot spots" --

- Instruction fetches vs stack frame vs data structures, etc
- Ability to simultaneously cache a few (2? 4? 8?) hot spots eliminates most collisions

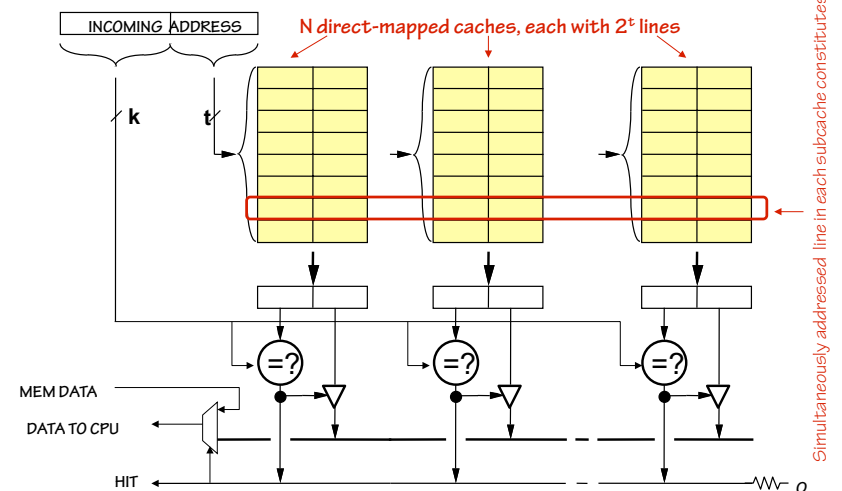
... so lets build caches that allow each location to be stored in some restricted **set** of cache lines, rather than in exactly one (direct mapped) or every line (fully associative).



Insight: an N-way set-associative cache affords modest parallelism

- parallel lookup (associativity): restricted to small set of N lines
- modest parallelism deals with most contention at modest cost
- can implement using N direct-mapped caches, running in parallel

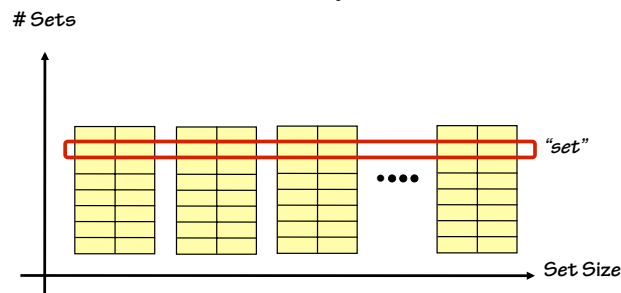
N-way Set-Associative Cache



Simultaneously addressed line in each subcache constitutes a **set**

Associativity

the birds-eye view



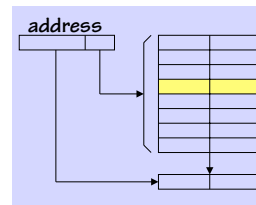
Can place caches in 2D space:

- Total lines = # Sets * Set Size
- # Sets = 1: Fully Associative
- Set Size = 1: Direct Mapped
- Set Size = N: N-way Set Associative

Associativity implies choices...

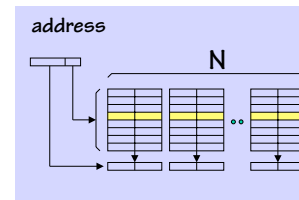
ISSUE: Replacement Strategy

Direct-mapped



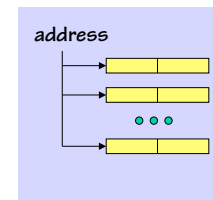
- compare addr with only one tag
- location A can be stored in exactly one cache line

N-way set associative



- compare addr with N tags simultaneously
- location A can be stored in exactly one set, but in any of the N cache lines belonging to that set

Fully associative



- compare addr with each tag simultaneously
- location A can be stored in any cache line

Replacement Strategy

LRU (Least-recently used)

- keeps most-recently used locations in cache
- need to keep ordered list of N items $\rightarrow N!$ orderings
 $\rightarrow O(\log_2 N!) = O(N \log_2 N)$ "LRU bits" + complex logic

Overhead is
 $O(N \log_2 N)$
bits/set

FIFO/LRR (first-in, first-out/least-recently replaced)

- cheap alternative: replace oldest item (dated by access time)
- within each set: keep one counter that points to victim line

Overhead is
 $O(\log_2 N)$
bits/set

Random (select replacement line using random, uniform distribution)

- no "pathological" reference streams causing worst-case results
- use pseudo-random generator to get reproducible behavior;
- use real randomness to prevent reverse engineering!

Overhead is
 $O(\log_2 N)$
bits/cache!

(0,1,2,3) Hit 2 $\rightarrow$ (2,0,1,3)
(2,0,1,3) Hit 1 $\rightarrow$ (1,2,0,3)
(1,2,0,3) Miss $\rightarrow$ (3,1,2,0)
(3,1,2,0) Hit 3 $\rightarrow$ (3,1,2,0)

Cache Benchmarking

Suppose this loop is entered with R3=4000:

ADR:	Instruction	I	D	
400:	LD (R3, 0, R0)	400		4000 + ...
404:	ADDC (R3, 4, R3)	404		
408:	BNE (R0, 400)	408		

GOAL: Given some cache design, simulate (by hand or machine) execution well enough to determine hit ratio.

1. Observe that the sequence of memory locations referenced is

400, 4000, 404, 408, 400, 4004, ...

We can use this simpler reference string, rather than the program, to simulate cache behavior.

2. We can make our life easier in many cases by converting to word addresses: 100, 1000, 101, 102, 100, 1001, ...

(Word Addr = (Byte Addr)/4)

Cache Simulation

4-line Fully-associative/LRU			4-line Direct-mapped		
Addr	Line#	Miss?	Addr	Line#	Miss?
100	0	M	100	0	M
1000	1	M	1000	0	M
101	2	M	101	1	M
102	3	M	102	2	M
100	0		100	0	M
1001	1	M	1001	1	M
101	2		101	1	M
102	3		102	2	
100	0		100	0	
1002	1	M	1002	2	M
101	2		101	1	
102	3		102	2	M
100	0		100	0	
1003	1	M	1003	3	M
101	2		101	1	
102	3		102	2	

6.004 - Spring 2009

4/7/09

L16 - Cache Issues 13

Associativity: Full vs 2-way

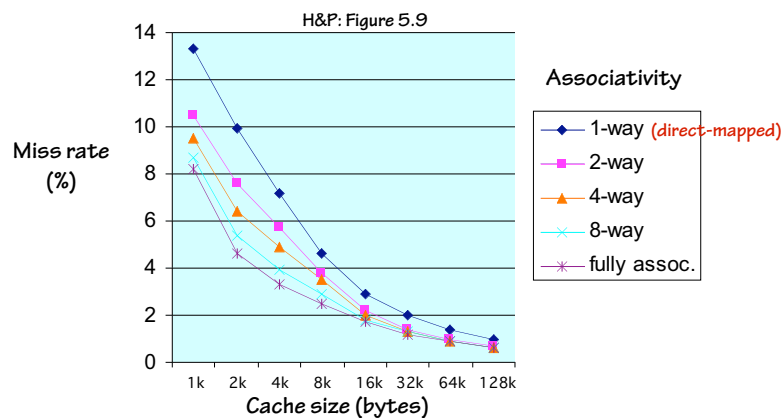
8-line Fully-associative, LRU			2-way, 8-line total, LRU		
Addr	Line#	Miss?	Addr	Set#/N	Miss?
100	0	M	100	0,0	M
1000	1	M	1000	0,1	M
101	2	M	101	1,0	M
102	3	M	102	2,0	M
100	0		100	0,0	
1001	4	M	1001	1,1	M
101	2		101	1,0	
102	3		102	2,0	
100	0		100	0,0	
1002	5	M	1002	2,1	M
101	2		101	1,0	
102	3		102	2,0	
100	0		100	0,0	
1003	6	M	1003	3,1	M
101	2		101	1,0	
102	3		102	2,0	

6.004 - Spring 2009

4/7/09

L16 - Cache Issues 14

Associativity vs. miss rate



- 8-way is (almost) as effective as fully-associative
- rule of thumb: N-line direct-mapped == N/2-line 2-way set assoc.

6.004 - Spring 2009

4/7/09

L16 - Cache Issues 15

Devil's Advocacy Games

Your company uses the cheaper FIFO cache, the competition uses LRU. Can you devise a benchmark to make your cache look better?

Assume 0x100 sets, 2-way...

Set 0 tags:

Adr

100

1000

100

2000

1000

BINGO!

2-way, LRU			2-way, FIFO		
	#0	#1		#0	#1
Set 0 tags:	1000	2000		2000	1000
Adr	Set, #	H/M		Set, #	H/M
100	0,0	M		0,0	M
1000	0,1	M		0,1	M
100	0,0	H		0,0	H
2000	0,1	M		0,0	M
1000	0,0	M		0,0	H

A carefully-designed benchmark can make *either* look better...

Pessim case: next adr referenced is the one just replaced!

Random replacement makes this game harder...

6.004 - Spring 2009

4/7/09

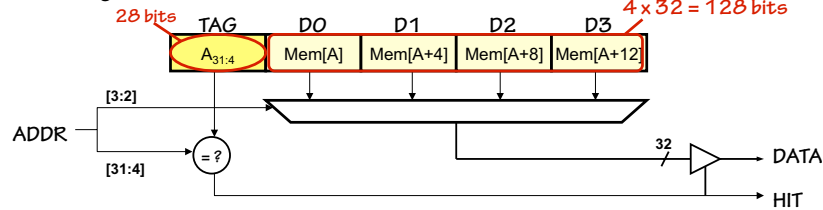
L16 - Cache Issues 16

Increasing Block Size

More Data/Tag

Overhead < ¼ bit of Tag per bit of data

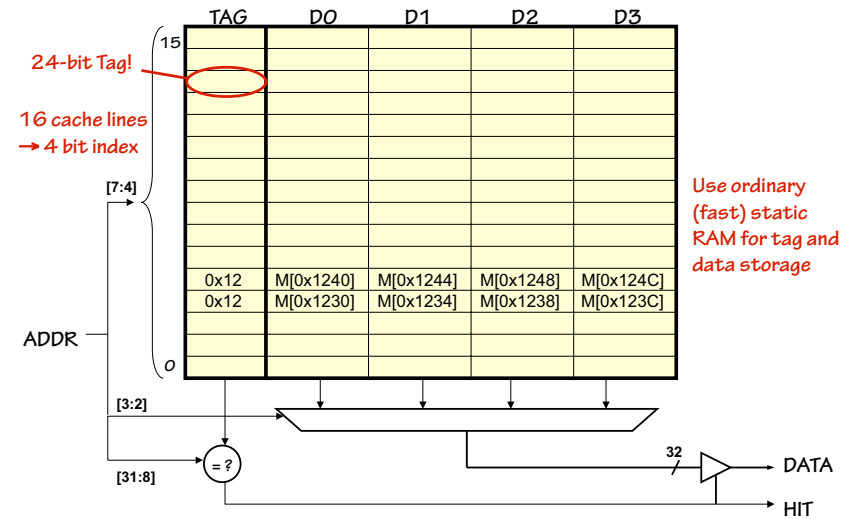
Enlarge each line in cache:



- **blocks** of 2^B words, on 2^B word boundaries
- always read/write 2^B word block from/to memory
- locality: access on word in block, others likely
- cost: some fetches of unaccessed words

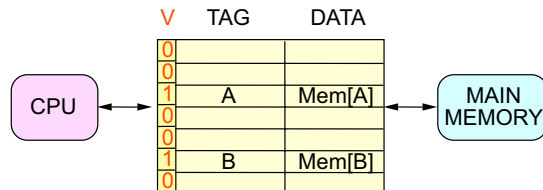
BIG WIN if there is a wide path to memory

4-word block, DM Cache



Only one comparator for entire cache!

Valid bits



Problem:

- Ignoring cache lines that don't contain anything of value... e.g., on
 - start-up
 - “Back door” changes to memory (eg loading program from disk)

Solution:

- Extend each TAG with **VALID bit**.
- Valid bit must be set for cache line to HIT.
- At power-up / reset : clear all valid bits
- Set valid bit when cache line is first replaced.
- Cache Control Feature: Flush cache by clearing all valid bits, Under program/external control.

Handling of WRITES

Observation: Most (90+%) of memory accesses are READs. How should we handle writes? Issues:

Write-through: CPU writes are cached, but also written to main memory (stalling the CPU until write is completed). Memory always holds “the truth”.

Write-behind: CPU writes are cached; writes to main memory may be buffered, perhaps pipelined. CPU keeps executing while writes are completed (in order) in the background.

Write-back: CPU writes are cached, but not immediately written to main memory. Memory contents can be “stale”.

Our cache thus far uses write-through.

Can we improve write performance?

Write-through

ON REFERENCE TO Mem[X]: Look for X among tags...

HIT: $X == TAG(i)$, for some cache line i

- READ: return DATA[i]
- WRITE: change DATA[i]; ~~Start Write to Mem[X]~~

MISS: X not found in TAG of any cache line

- REPLACEMENT SELECTION:
 - Select some line k to hold Mem[X]
- READ: Read Mem[X]
 - Set TAG[k] = X, DATA[k] = Mem[X]
- WRITE: ~~Start Write to Mem[X]~~
 - Set TAG[k] = X, DATA[k] = new Mem[X]

Write-back

ON REFERENCE TO Mem[X]: Look for X among tags...

HIT: $X = TAG(i)$, for some cache line i

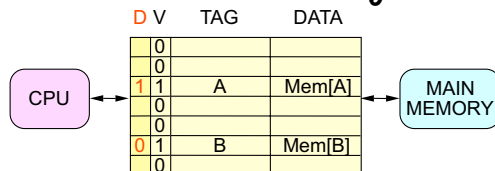
- READ: return DATA(i)
- WRITE: change DATA(i); ~~Start Write to Mem[X]~~

MISS: X not found in TAG of any cache line

- REPLACEMENT SELECTION:
 - Select some line k to hold Mem[X]
 - Write Back: Write Data(k) to Mem[Tag[k]]
- READ: Read Mem[X]
 - Set TAG[k] = X, DATA[k] = Mem[X]
- WRITE: ~~Start Write to Mem[X]~~
 - Set TAG[k] = X, DATA[k] = new Mem[X]

Is write-back worth the trouble? Depends on (1) cost of write; (2) consistency issues.

Write-back w/ "Dirty" bits



ON REFERENCE TO Mem[X]: Look for X among tags...

HIT: $X = TAG(i)$, for some cache line i

- READ: return DATA(i)
- WRITE: change DATA(i); ~~Start Write to Mem[X]~~ $D[i]=1$

MISS: X not found in TAG of any cache line

- REPLACEMENT SELECTION:
 - Select some line k to hold Mem[X]
 - If $D[k] == 1$ (Write Back) Write Data(k) to Mem[Tag[k]]
- READ: Read Mem[X]; Set TAG[k] = X, DATA[k] = Mem[X], $D[k]=0$
- WRITE: ~~Start Write to Mem[X]~~ $D[k]=1$
 - Set TAG[k] = X, DATA[k] = new Mem[X]

Caches: Summary

Associativity:

- Less important as size increases
- 2-way or 4-way usually plenty for typical program clustering; BUT additional associativity
 - Smooths performance curve
 - Reduces number of select bits (we'll see shortly how this helps)
- TREND: Invest in RAM, not comparators.

Replacement Strategy:

- BIG caches: any sane approach works well
- REAL randomness assuages paranoia!

Performance analysis:

- Tedious hand synthesis may build intuition from simple examples, BUT
- Computer simulation of cache behavior on REAL programs (or using REAL trace data) is the basis for most real-world cache design decisions.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Virtual Memory

You heard me right, kid.
TERABYTES of main
memory!

Quiz #3 Tomorrow!

Lessons from History...

There is only one mistake that can be made in computer design that is difficult to recover from—not having enough address bits for memory addressing and memory management.

Gordon Bell and Bill Strecker
speaking about the PDP-11 in 1976

A partial list of successful machines that eventually starved to death for lack of address bits includes the PDP 8, PDP 10, PDP 11, Intel 8080, Intel 8086, Intel 80186, Intel 80286, Motorola 6800, AMI 6502, Zilog Z80, Cray-1, and Cray X-MP.

Hennessey & Patterson

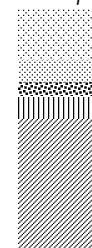
Why? Address size determines minimum width of anything that can hold an address: PC, registers, memory words, HW for address arithmetic (BR/JMP, LD/ST). When you run out of address space it's time for a new ISA!

Top 10 Reasons for a BIG Address Space

10. Keeping Micron and Rambus in business.
9. Unique addresses within every internet host.
8. Generating good 6.004 quiz problems.
7. Performing 32-bit ADD via table lookup
6. Support for meaningless advertising hype
5. Emulation of a Turing Machine's tape.
4. Bragging rights at geek parties.
3. Isolating ISA from IMPLEMENTATION
 - details of HW configuration shouldn't enter into SW design
2. Usage UNCERTAINTY
 - provide for run-time expansion of stack and heap
1. Programming CONVENIENCE
 - create regions of memory with different semantics: read-only, shared, etc.
 - avoid annoying bookkeeping

Squandering Address Space

Address Space



CODE, large monolithic programs (eg, Office, Netscape)....

- only small portions might be used
- add-ins and plug-ins
- shared libraries/DLLs
- ...

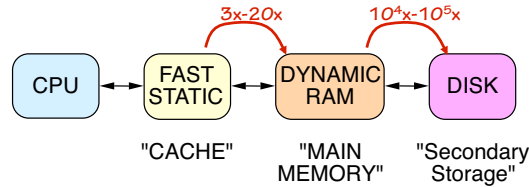
STACK: How much to reserve? (consider RECURSION!)

OBSERVATIONS:

- Can't BOUND each usage... without compromising use.
- Actual use is SPARSE
- Working set even MORE sparse

HEAP: N variable-size data records...
Bound N? Bound Size?

Extending the Memory Hierarchy



So, we've used SMALL fast memory + BIG slow memory to fake BIG FAST memory.

Can we combine RAM and DISK to fake DISK size at RAM speeds?

VIRTUAL MEMORY

- use of RAM as cache to much larger storage pool, on slower devices
- TRANSPARENCY - VM locations "look" the same to program whether on DISK or in RAM.
- ISOLATION of RAM size from software.

Virtual Memory

ILLUSION: Huge memory
(2^{32} bytes? 2^{64} bytes?)

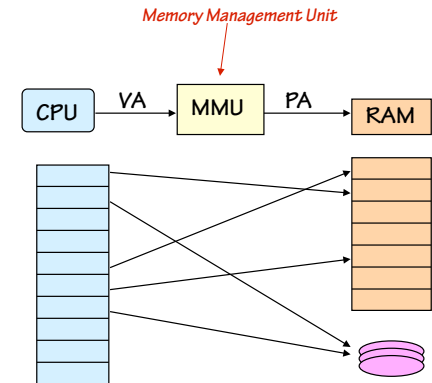
ACTIVE USAGE: small fraction
(2^{24} bytes?)

HARDWARE:

- 2^{30} (1 G) bytes of RAM
- 2^{37} (128 G) bytes of DISK...
- ... maybe more, maybe less!

ELEMENTS OF DECEIT:

- Partition memory into "Pages" (2K-4K-8K)
- MAP a few to RAM, others to DISK
- Keep "HOT" pages in RAM.



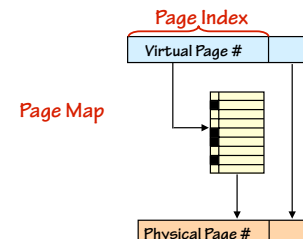
Demand Paging

"Bean - get in here immediately! And bring a mop!"

Basic idea:

- Start with all of VM on DISK ("swap area"), MMU "empty"
- Begin running program... each VA "mapped" to a PA
 - Reference to RAM-resident page: RAM accessed by hardware
 - Reference to a non-resident page: traps to software handler, which
 - Fetches missing page from DISK into RAM
 - Adjusts MMU to map newly-loaded virtual page directly in RAM
 - If RAM is full, may have to replace ("swap out") some little-used page to free up RAM for the new page.
- Working set incrementally loaded, gradually evolves...

Simple Page Map Design



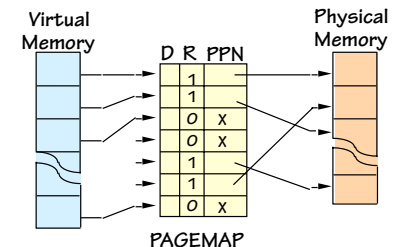
FUNCTION: Given Virtual Address,

- Map to PHYSICAL address

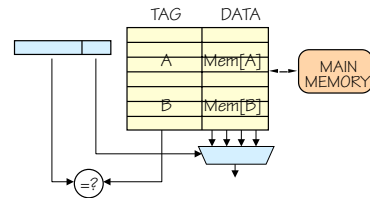
OR

- Cause **PAGE FAULT** allowing page replacement

Why use HIGH address bits to select page?
... LOCALITY.
Keeps related data on same page.



Virtual Memory vs. Cache

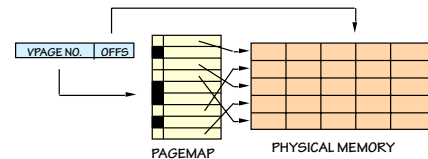


Cache:

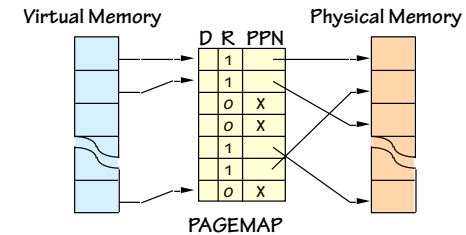
- Relatively short blocks
- Few entries: scarce resource
- miss time: 3x-20x hit times

Virtual memory:

- disk: long latency, fast xfer
 - miss time: $\sim 10^5 \times$ hit time
 - write-back essential!
 - large pages in RAM
- lots of entries: one for each page
- tags in page map, data in physical memory



Virtual Memory: the VI-1 view

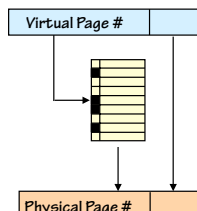


Pagemap Characteristics:

- One entry per **virtual** page!
- RESIDENT bit = 1 for pages stored in RAM, or 0 for non-resident (disk or unallocated). Page fault when R = 0.
- Contains PHYSICAL page number (PPN) of each resident page
- DIRTY bit says we've changed this page since loading it from disk (and therefore need to write it to disk when it's replaced)

Virtual Memory: the VI-3 view

Problem: Translate
VIRTUAL ADDRESS
to PHYSICAL ADDRESS



```
int VtoP(int VPageNo, int PO) {
    if (R[VPageNo] == 0)
        PageFault(VPageNo);
    return (PPN[VPageNo] << p) | PO;
}
// Multiply by 2^p, the page size

/* Handle a missing page... */
void PageFault(int VPageNo) {
    int i;

    i = SelectLRUPage();
    if (D[i] == 1)
        WritePage(DiskAdr[i], PPN[i]);
    R[i] = 0;

    PPN[VPageNo] = PPN[i];
    ReadPage(DiskAdr[VPageNo], PPN[i]);
    R[VPageNo] = 1;
    D[VPageNo] = 0;
}
```

The HW/SW Balance

IDEA:

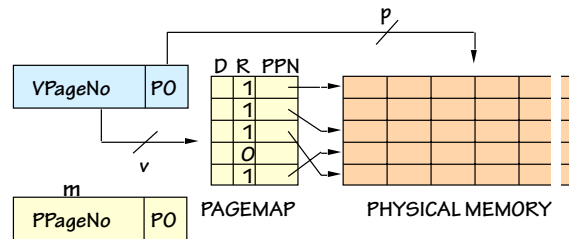
- devote **HARDWARE** to high-traffic, performance-critical path
- use (slow, cheap) **SOFTWARE** to handle exceptional cases

hardware	<pre>int VtoP(int VPageNo, int PO) { if (R[VPageNo] == 0) PageFault(VPageNo); return (PPN[VPageNo] << p) PO; }</pre>
software	<pre>/* Handle a missing page... */ void PageFault(int VPageNo) { int i = SelectLRUPage(); if (D[i] == 1) WritePage(DiskAdr[i], PPN[i]); R[i] = 0; PPN[VPageNo] = PPN[i]; ReadPage(DiskAdr[VPageNo], PPN[i]); R[VPageNo] = 1; D[VPageNo] = 0; }</pre>

HARDWARE performs address translation, detects page faults:

- running program interrupted ("suspended");
- PageFault(...) is forced;
- On return from PageFault; running program continues

Page Map Arithmetic

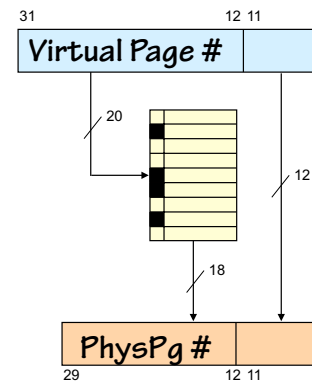


Wait... if v equals m ,
why have a pagemap at all?

- $(v + p)$ bits in virtual address
- $(m + p)$ bits in physical address
- 2^v number of VIRTUAL pages
- 2^m number of PHYSICAL pages
- 2^p bytes per physical page
- 2^{v+p} bytes in virtual memory
- 2^{m+p} bytes in physical memory
- $(m+2)2^v$ bits in the page map

Typical page size: 1K – 8K bytes
Typical $(v+p)$: 32 (or more) bits
Typical $(m+p)$: 30 – 32 bits
(1G – 4G)

Example: Page Map Arithmetic



SUPPOSE...

32-bit Virtual address

2^{12} page size (4 KB)

2^{30} RAM max (1 GB)

THEN:

Physical Pages = $2^{18} = 256K$

Virtual Pages = 2^{20}

Page Map Entries = $2^{20} = 1M$

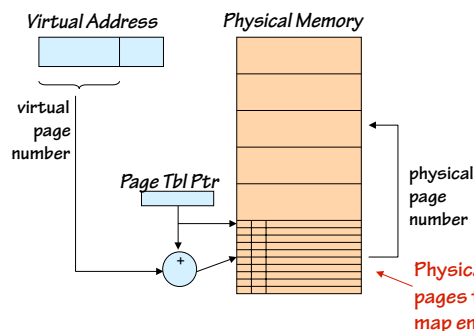
Bits in pagemap = $20 * 2^{20} = 20M$

Use SRAM for page map??? **OUCH!**

RAM-Resident Page Maps

SMALL page maps can use dedicated RAM... gets expensive for big ones!

SOLUTION: Move page map to MAIN MEMORY:

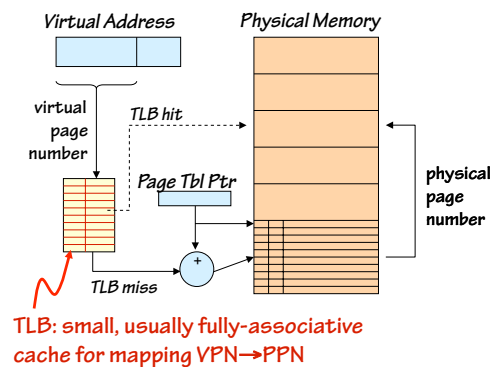


PROBLEM:
Each memory reference
now takes 2 accesses
to physical memory!

Translation Look-aside Buffer (TLB)

PROBLEM: 2x performance hit... each memory reference now takes 2 accesses!

SOLUTION: CACHE the page map entries



IDEA:
LOCALITY in memory
reference patterns →
SUPER locality in
reference to page map

VARIATIONS:

- sparse page map storage
- paging the page map!

Example: mapping VAs to PAs

Suppose

- virtual memory of 2^{32} bytes
- physical memory of 2^{24} bytes
- page size is 2^{10} (1 K) bytes

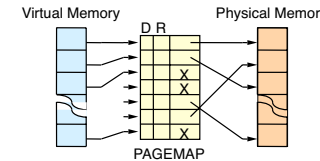
VPN	R	D	PPN
0	0	0	7
1	1	1	9
2	1	0	0
3	0	0	5
4	1	0	5
5	0	0	3
6	1	1	2
7	1	0	4
8	1	0	1
...			

1. How many pages can be stored in physical memory at once? $2^{24-10} = 2^{14}$
2. How many entries are there in the page table? 2^{22}
3. How many bits are necessary per entry in the page table? (Assume each entry has PPN, resident bit, dirty bit) 16
4. How many pages does the page table require? $2^{23} \text{ bytes} = 2^{13} \text{ pages}$
5. What's the largest fraction of VM that might be resident? $1/2^8$
6. A portion of the page table is given to the left. What is the physical address for virtual address $0x1804$?

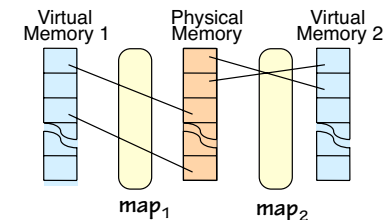
VPN=6 $\Rightarrow$ PPN=2 $\Rightarrow$ PA=0x804

Contexts

A context is a mapping of VIRTUAL to PHYSICAL locations, as dictated by contents of the page map:

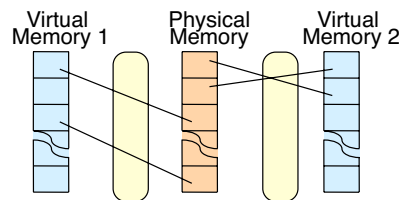


Several programs may be simultaneously loaded into main memory, each in its separate context:



“Context switch”:
reload the page map!

Contexts: A Sneak Preview



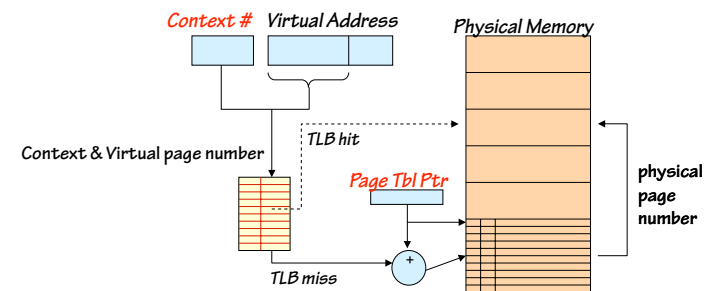
Every application can be written as if it has access to all of memory, without considering where other applications reside.

First Glimpse of a VIRTUAL MACHINE

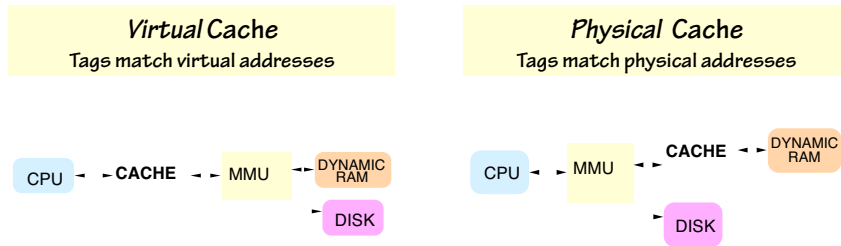
1. TIMESHARING among several programs --
 - Separate context for each program
 - OS loads appropriate context into pagemap when switching among pgms
 2. Separate context for OS “Kernel” (eg, interrupt handlers)...
 - “Kernel” vs “User” contexts
 - Switch to Kernel context on interrupt;
 - Switch back on interrupt return.
- HARDWARE SUPPORT: 2 HW pagemaps

Rapid Context Switching

Add a register to hold index of current context. To switch contexts: update Context # and Page Tbl Ptr registers. Don't have to flush TLB since each entry's tag includes context # in addition to virtual page number



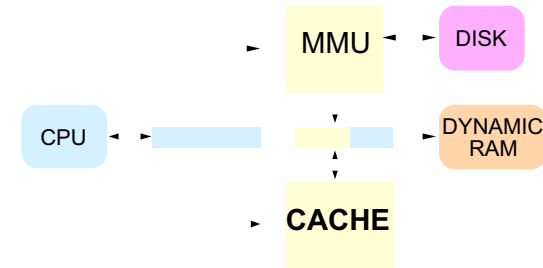
Using Caches with Virtual Memory



- Problem: cache invalid after context switch
- FAST: No MMU time on HIT

- Avoids stale cache data after context switch
- SLOW: MMU time on HIT

Best of both worlds



OBSERVATION: If cache line selection is based on *unmapped* page offset bits, RAM access in a physical cache can *overlap* page map access. Tag from cache is compared with physical page number from MMU.

Want “small” cache index → go with more associativity

Alternative memory structures?

Maybe we're hung up on the simple “address space” model. Some alternatives:

- Segments: named contiguous regions (Multics, x86, ...)
- Objects: Cons cells, arrays, ... (LISP machines, 432,)
- URIs (web)
- Triples/relations (LEAP, SAIL, RDF, ...)
- Associations
- Etc etc etc

All of these, and more, have been tried – with occasional success. But for the most part, we gravitate to that most venerable of Computer Science traditions:

Take a familiar model (*viz*, RAM).
Virtualize it.

Summary

Exploiting locality on a large scale...

- Programmers want a large, flat address space...
... but they'll use it sparsely, unpredictably!
- Key: Demand Page sparse working set into RAM from DISK
- IMPORTANT: Single-level pagemap, arithmetic, operation...
 - Access loaded pages via fast hardware path
 - Load virtual memory (RAM) on demand: page faults
- Various optimizations...
 - Moving pagemap to RAM, for economy & size
 - Translation Lookaside Buffer (TLB), to regain performance
 - Moving pagemap to DISK (or, equivalently, VM) for economy & size
- Cache/VM interactions: can cache physical or virtual locations

Semantic consequence:

- CONTEXT: a mapping between V and P addresses – we'll see again!

Challenge: Alternative models

- Will we just use bigger addresses when we outgrow our current ISAs?

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

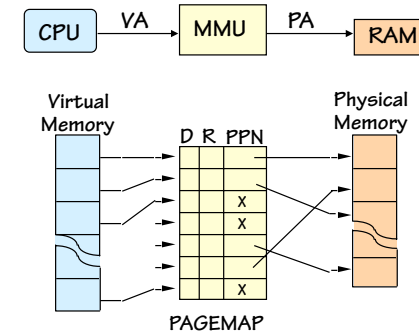
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Virtual Machines

Lab 6 due Thursday!

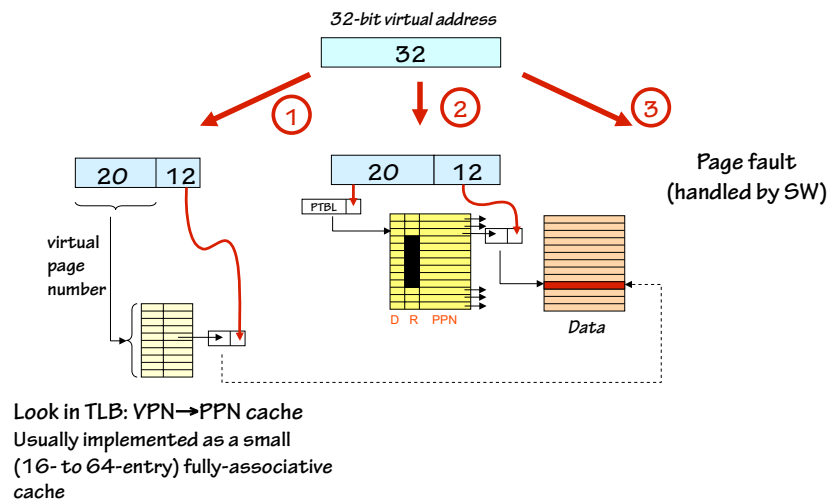
Review: Virtual Memory



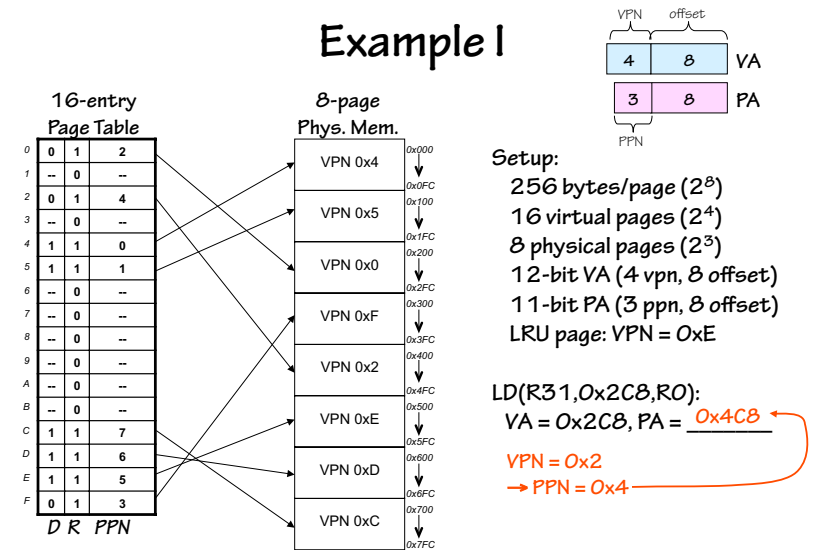
- Goal: create illusion of large virtual address space
- divide address into (VPN,offset), **map** to (PPN,offset) **or page fault**
 - **use high address** bits to select page: keep related data on same page
 - use cache (TLB) to speed up mapping mechanism—works well
 - **long disk latencies**: keep working set in physical memory, use write-back

MMU Address Translation

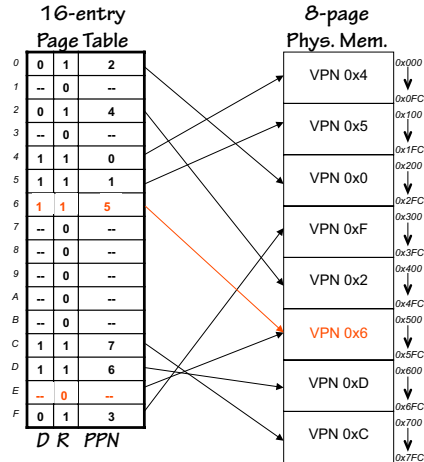
Typical Multi-level approach



Example I



Example II



Setup:

256 bytes/page (2^8)
 16 virtual pages (2^4)
 8 physical pages (2^3)
 12-bit VA (4 vpn, 8 offset)
 11-bit PA (3 ppn, 8 offset)
 LRU page: VPN = 0xE

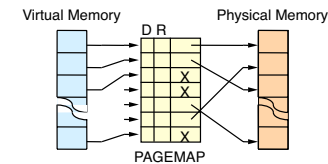
ST(BP,-4,SP), SP = 0x604
 VA = 0x600, PA = 0x500

VPN = 0x6
 → Not resident, it's on disk
 → Choose page to replace (LRU = 0xE)
 → D[0xE] = 1, so write 0x500-0x5FC to disk
 → Mark VPN 0xE as no longer resident
 → Read in page 0x6 from disk into 0x500-0x5FC
 → Set up page map for VPN 0x6 = PPN 0x5
 → PA = 0x500
 → This is a write so set D[0x6] = 1

Contexts

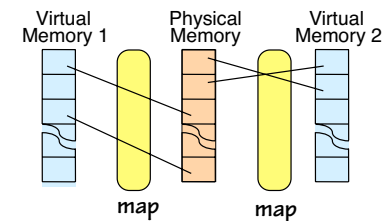
A **context** is an entire set of mappings from VIRTUAL to PHYSICAL page numbers as specified by the contents of the page map:

We might like to support multiple VIRTUAL to PHYSICAL Mappings and, thus, multiple Contexts.

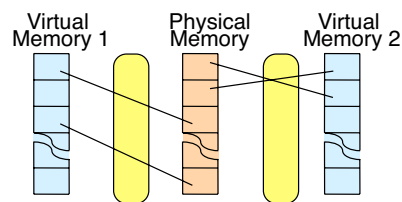


THE BIG IDEA: Several programs, each with their own context, may be simultaneously loaded into main memory!

"Context switch":
 reload the page map!



Power of Contexts: Sharing a CPU



Every application can be written as if it has access to all of memory, without considering where other applications reside.

More than Virtual Memory:
 A VIRTUAL MACHINE

1. TIMESHARING among several programs --

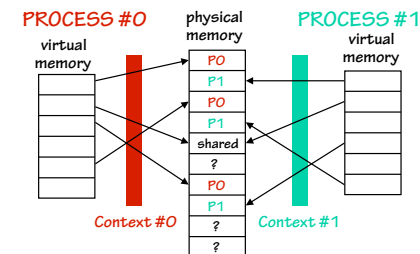
- Separate context for each program
- OS loads appropriate context into pagemap when switching among pgms

2. Separate context for Operating System "Kernel" (eg, interrupt handlers)...

- "Kernel" vs "User" contexts
- Switch to Kernel context on interrupt;
- Switch back on interrupt return.

TYPICAL HARDWARE SUPPORT: rapid context switch mechanism

Building a Virtual Machine



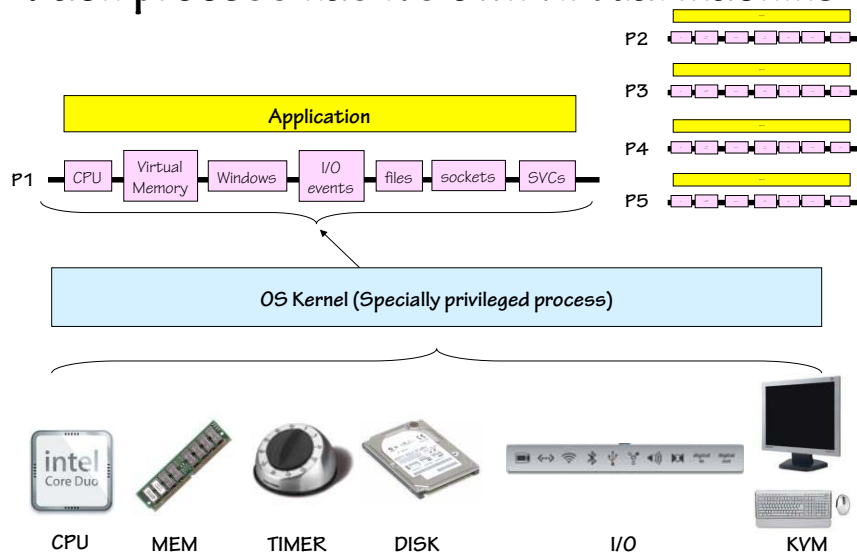
Goal: give each program its own "VIRTUAL MACHINE";
 programs don't "know" about each other...

New abstraction: a **process** which has its own

- machine state: R0, ..., R30
- context (virtual address space)
- PC, stack
- program (w/ shared code)
- virtual I/O devices (console...)

"OS Kernel" is a special, privileged process that oversees the other processes and handles real I/O devices, emulating virtual I/O devices for each process

Each process has its own virtual machine



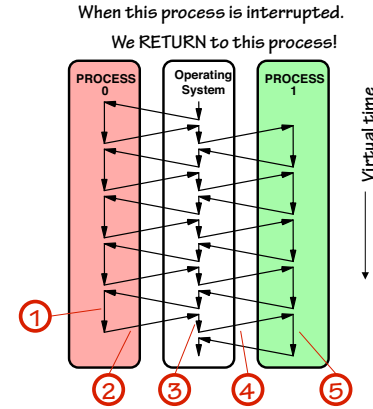
6.004 - Spring 2009

4/14/09

L18 - Virtual Machines 9

Processes:

Multiplexing the CPU



1. Running in process #0
2. Stop execution of process #0 either because of explicit *yield* or some sort of timer *interrupt*; trap to handler code, saving current PC in XP
3. First: save process #0 state (regs, context) Then: load process #1 state (regs, context)
4. "Return" to process #1: just like return from other trap handlers (ie., use address in XP) but we're returning from a *different* trap than happened in step 2!
5. Running in process #1

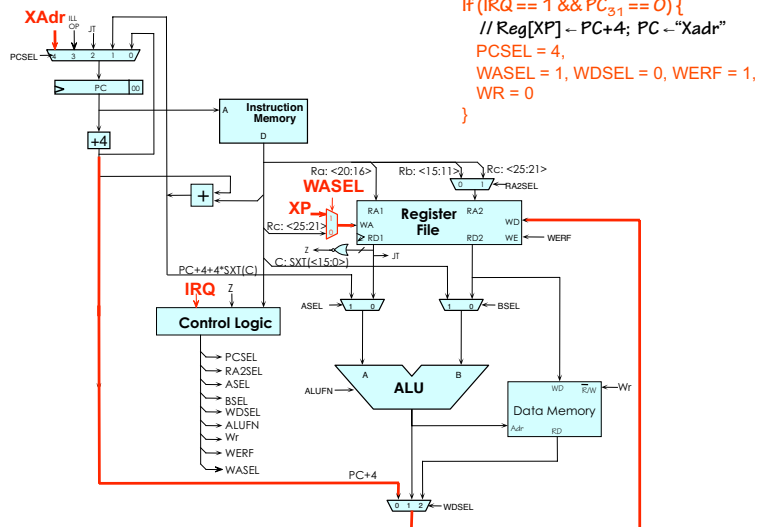
Key Technology: Interrupts.

6.004 - Spring 2009

4/14/09

L18 - Virtual Machines 10

Interrupt Hardware



```

If (IRQ == 1 && PC31 == 0) {
    // Reg[XP] ← PC+4; PC ← "Xadr"
    PCSEL = 4,
    WASEL = 1, WDESEL = 0, WERF = 1,
    WR = 0
}
    
```

6.004 - Spring 2009

4/14/09

L18 - Virtual Machines 11

Beta Interrupt Handling

Minimal Hardware Implementation:

- Check for Interrupt Requests (IRQs) before each instruction fetch.
- On IRQ_j:
 - copy PC into Reg[XP];
 - INSTALL_j*4 as new PC.

RESET → 0x80000000 0: BR (...)
 ILLOP → 0x80000000 4: BR (...)
 X_ADDR → 0x80000000 8: BR (...)
 12: BR (...)

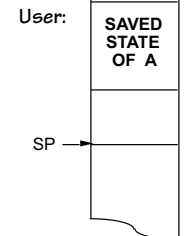
Handler Coding:

- Save state in "User" structure
- Call C procedure to handle the exception
- re-install saved state from "User"
- Return to Reg[XP]

TRANSPARENT to interrupted program!

WHERE to find handlers?

- BETA Scheme: WIRE IN a low-memory address for each exception handler entry point
- Common alternative: WIRE IN the address of a TABLE of handler addresses ("interrupt vectors")



6.004 - Spring 2009

4/14/09

L18 - Virtual Machines 12

External (Asynchronous) Interrupts

Example:

Operating System maintains current time of day (TOD) count. But...this value must be updated periodically in response to clock EVENTS, i.e. signal triggered by 60 Hz timer hardware.

Program A (Application)

- Executes instructions of the user program.
- Doesn't want to know about clock hardware, interrupts, etc!!
- Can incorporate TOD into results by "asking" OS.

Clock Handler

- GUTS: Sequence of instructions that increments TOD. Written in C.
- Entry/Exit sequences save & restore interrupted state, call the C handler. Written as assembler "stubs".

Interrupt Handler Coding

```
long TimeOfDay;
struct Mstate { int Regs[31]; } User;

/* Executed 60 times/sec */
Clock_Handler() {
    TimeOfDay = TimeOfDay+1;
    if (TimeOfDay % QUANTUM == 0) Scheduler();
}
```

Handler
(written in C)

```
Clock_h:
    ST(r0, User)      | Save state of
    ST(r1, User+4)    | interrupted
    ...               | app pgm...
    ST(r30, User+30*4)
    CMOVE(KStack, SP) | Use KERNEL SP
    BR(Clock_Handler,lp) | call handler
    LD(User, r0)        | Restore saved
    LD(User+4, r1)      | state.
    ...
    LD(User+30*4, r30)
    SUBC(XP, 4, XP)    | execute interrupted inst
    JMP(XP)            | Return to app.
```

"Interrupt stub"
(written in assy.)

Simple Timesharing Scheduler

```
struct Mstate {
    int Regs[31];
} User;

/* Structure to hold */
/* processor state */

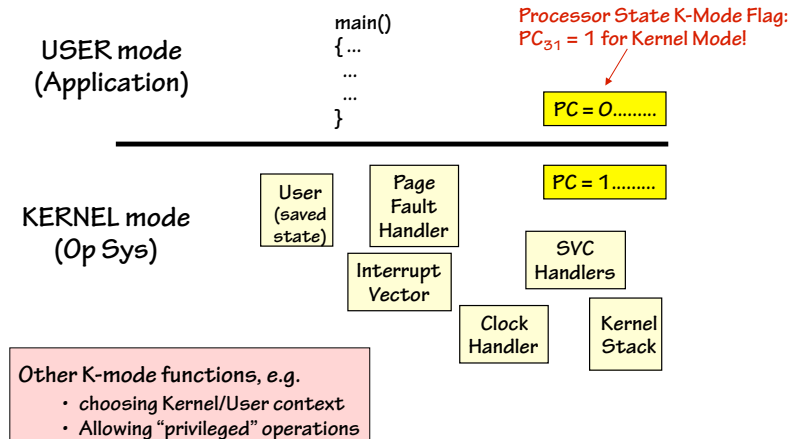
(PCB = Process Control Block)
struct PCB {
    struct MState State;
    Context PageMap;
    int DPYNum;
} ProcTbl[N];
/* Processor state */
/* VM Map for proc */
/* Console number */
/* one per process */

int Cur;
/* "Active" process */

Scheduler() {
    ProcTbl[Cur].State = User;
    Cur = (Cur+1)%N;
    User = ProcTbl[Cur].State;
    LoadUserContext(ProcTbl[Cur].Context);
}
```

Avoiding Re-entrance

Handlers which are interruptable are called **RE-ENTRANT**, and pose special problems... **Beta, like many systems, disallows reentrant interrupts!**
Mechanism: Uninterruptable "Kernel Mode" for OS:



Communicating with the OS

User-mode programs need to communicate with OS code:

Access virtual I/O devices

Communicate with other processes

...

But if OS Kernel is in another context (ie, not in user-mode address space) how do we get to it?

Solution:

Abstraction: a supervisor call (SVC) with args in registers – result in R0 or maybe user-mode memory

Implementation: use illegal instructions to cause an exception -- OS code will recognize these particular illegal instructions as a user-mode SVCs

Okay... show me how it works!

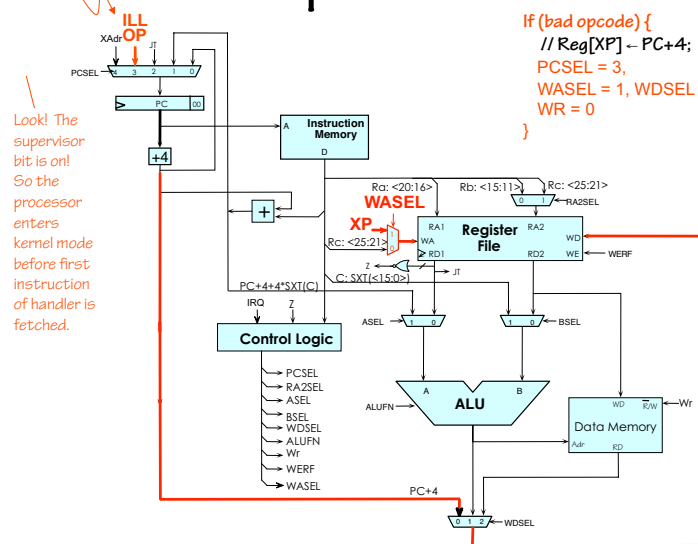
6.004 – Spring 2009

4/14/09

L18 – Virtual Machines 17

Exception Hardware

0x80000004



6.004 – Spring 2009

4/14/09

L18 – Virtual Machines 18

Code is from lab8.uasm

Exception Handling

```
. = 0x00000004
BR(I_IllOp) | on Illegal Instruction (eg SVC)

| Here's the SAVED STATE of the interrupted process, while we're
| processing an interrupt.
UserMState: STORAGE(32) | R0-R31... (PC is in XP!)

| Here are macros to SAVE and RESTORE state -- 31 registers -- from
| the above storage.

.macro SS(R) ST(R, UserMState+(4*R)) | (Auxiliary macro)
.macro SAVEDSTATE() {
SS(0) SS(1) SS(2) SS(3) SS(4) SS(5) SS(6) SS(7)
SS(8) SS(9) SS(10) SS(11) SS(12) SS(13) SS(14) SS(15)
SS(16) SS(17) SS(18) SS(19) SS(20) SS(21) SS(22) SS(23)
SS(24) SS(25) SS(26) SS(27) SS(28) SS(29) SS(30) }

.macro RS(R) LD(UserMState+(4*R), R) | (Auxiliary macro)
.macro RESTORESTATE() {
RS(0) RS(1) RS(2) RS(3) RS(4) RS(5) RS(6) RS(7)
RS(8) RS(9) RS(10) RS(11) RS(12) RS(13) RS(14) RS(15)
RS(16) RS(17) RS(18) RS(19) RS(20) RS(21) RS(22) RS(23)
RS(24) RS(25) RS(26) RS(27) RS(28) RS(29) RS(30) }
```

— This is where the HW sets the PC during an exception

Macros can be used like an in-lined procedure call

6.004 – Spring 2009

4/14/09

L18 – Virtual Machines 19

IllOp Handler

```
||||| Handler for Illegal Instructions
||||| (including SVCs)
|||||

I_IllOp:
    SAVESTATE() | Save the machine state.
    LD(KStack, SP) | Install kernel stack pointer.

    LD(XP, -4, r0) | Fetch the illegal instruction
    SHRC(r0, 26, r0) | Extract the 6-bit OPCODE
    SHLC(r0, 2, r0) | Make it a WORD (4-byte) index
    LD(r0, UUOTbl, r0) | Fetch UUOTbl[OPCODE]
    JMP(r0) | and dispatch to the UUO handler.

.macro UUO(ADR) LONG(ADR+0x80000000) | Auxiliary Macros
.macro BAD() UUO(UUOError)

UUOTbl: BAD() UUO(SVC_UUO) BAD() BAD()
        BAD() BAD() BAD() BAD()
        BAD() BAD() BAD() BAD()
        ... more table follows...
```

Don't trust the user's stack!

This is a 64-entry dispatch table. Each entry is an address of a "handler"

6.004 – Spring 2009

4/14/09

L18 – Virtual Machines 20

Actual Illops

||| Here's the handler for truly unused opcodes (not SVCs):
UUOError:

```
CALL(KWrMsg)           | Type out an error msg,
.text "Illegal instruction "
LD(xp, -4, r0)          | giving hex instr and location;
CALL(KHexPrt)
CALL(KWrMsg)
.text " at location 0x"
MOVE(xp, r0)
CALL(KHexPrt)
CALL(KWrMsg)
.text "!! ....."
HALT()                  | Then crash system.
```

These utility routines (Kxxx) don't follow our usual calling convention – they take their args in registers or from words immediately following the procedure call! They adjust LP to skip past any args before returning.

Supervisor Call Handler

SVC Instruction format



||| Sub-handler for SVCs, called from I_IllOp on SVC opcode:

```
SVC_UUO:
LD(XP, -4, r0)           | The faulting instruction.
ANDC(r0, 0x7, r0)        | Pick out low bits,
SHLC(r0, 2, r0)           | make a word index,
LD(r0, SVCTbl, r0)        | and fetch the table entry.
JMP(r0)

SVCTbl:
UUO(HaltH)               | SVC(0): User-mode HALT instruction
UUO(WrMsgH)              | SVC(1): Write message
UUO(WrChH)               | SVC(2): Write Character
UUO(GetKeyH)             | SVC(3): Get Key
UUO(HexPrtH)             | SVC(4): Hex Print
UUO(WaitH)               | SVC(5): Wait(S) , , , S in R3
UUO(SignalH)             | SVC(6): Signal(S), S in R3
UUO(YieldH)              | SVC(7): Yield()
```

Another
dispatch
table!

Handler for HALT SVC

||||| SVC Sub-handler for user-mode HALTs

```
HaltH: BR(I_Wait)        | SVC(0): User-mode HALT SVC
```

Looks like HALT should really be called LOOP!

||| Here's the common exit sequence from Kernel interrupt handlers:
||| Restore registers, and jump back to the interrupted user-mode
||| program.

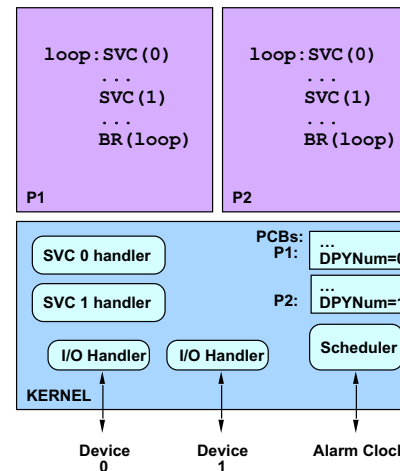
```
I_Rtn: RESTORESTATE()
kexit: JMP(XP)           | Good place for debugging breakpoint!
```

||| Alternate return from interrupt handler which BACKS UP PC,
||| and calls the scheduler prior to returning. This causes
||| the trapped SVC to be re-executed when the process is
||| eventually rescheduled...

```
I_Wait: LD(UserMState+(4*30), r0) | Grab XP from saved MState
SUBC(r0, 4, r0)                  | back it up to point to
ST(r0, UserMState+(4*30))        | SVC instruction
CALL(Scheduler)                  | Switch current process,
BR(I_Rtn)                        | and return to (some) user.
```

Fills UserMState from
PCB of next process to
run

OS organization



“Applications” are quasi-parallel
“PROCESSES”
on
“VIRTUAL MACHINES”,
each with:
• CONTEXT (virtual address space)
• Virtual I/O devices

O.S. KERNEL has:
• Interrupt handlers
• SVC (trap) handlers
• Scheduler
• PCB structures containing the
state of inactive processes

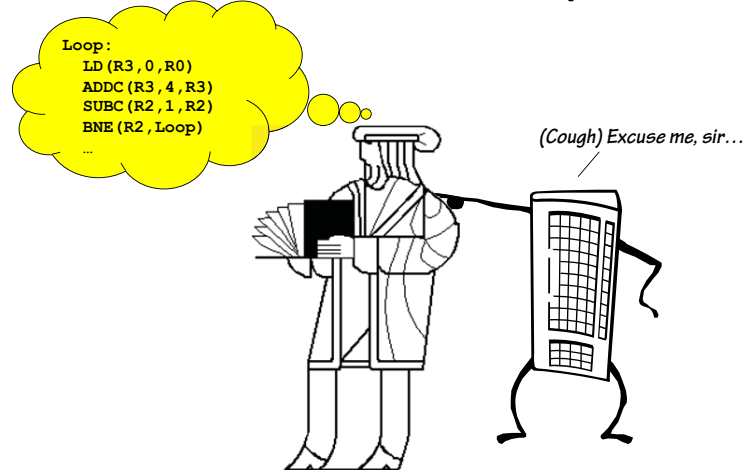
MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Devices & Interrupts



Lab #6 due tonight!

Why an OS?

What we've got:

- A Single Sequence Machine, capable of doing ONE thing at a time – one instruction, one I/O operation, one program.
- A universe of gadgets – e.g. I/O devices – that do similar things slightly differently.

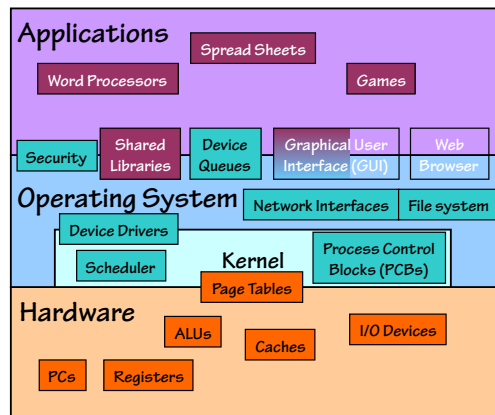
What we'd like:

- To listen to MP3s while reading email.
- To access disk, network, and screen “simultaneously”.
- To write a single program that does I/O with anybody's disk.

Plausible approaches:

- An infinite supply of identical computers with uniform, high-level peripherals for every conceivable purpose... or
- An illusion: Make one real computer look like many “virtual” ones.

Operating Systems



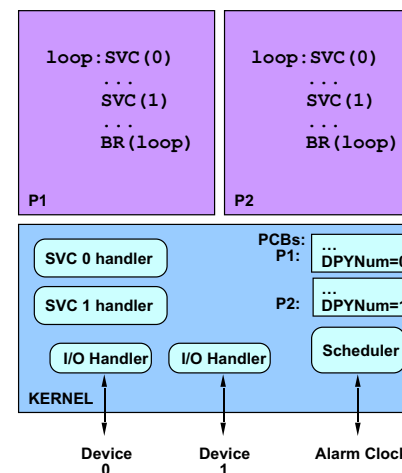
An OS is the Glue that holds a computer together.

- Mediates between competing requests
- Resolves names/bindings
- Maintains order/fairness

KERNEL - a RESIDENT portion of the O/S that handles the most common and fundamental service requests.

vir.tu.al \v*rch-(-)w*l, \v*r-ch*\ \v*r-ch*-wal-*t-e-\ \v*rch-(-)w*-le-, \v*rch-(-)le-\ aj [ME, possessed of certain physical virtues, fr. ML virtualis, fr. L virtus strength, virtue : being in essence or effect but not in fact - vir.tu.al.i.ty n

OS organization



“Applications” are quasi-parallel “PROCESSES”

on “VIRTUAL MACHINES”,

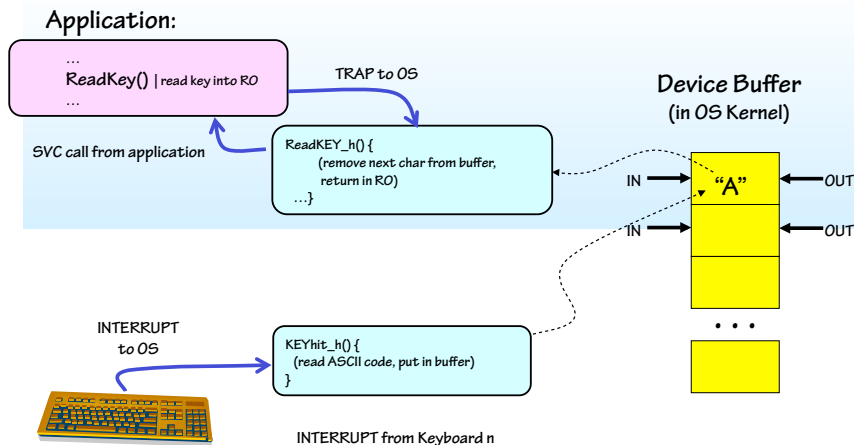
each with:

- CONTEXT (virtual address space)
- Virtual I/O devices

O.S. KERNEL has:

- Interrupt handlers
- SVC (trap) handlers
- Scheduler
- PCB structures containing the state of inactive processes

Asynchronous I/O Handling



6.004 - Spring 2009

4/16/09

L19 - Devices & Interrupts 5

Interrupt-based Asynchronous I/O

OPERATION: NO attention to Keyboard during normal operation

- on key strike: hardware asserts IRQ to request interrupt
- USER program interrupted, PC+4 of interrupted inst. saved in XP
- state of USER program saved on KERNEL stack;
- KeyboardHandler invoked, runs to completion;
- state of USER program restored; program resumes.

TRANSPARENT to USER program.

Keyboard Interrupt Handler (in O.S. KERNEL):

Assume each keyboard has an associated buffer

```

struct Device {
    char Flag, Data;
} Keyboard;

KEYhit_h() {
    Buffer[inptr] = Keyboard.Data;
    inptr = (inptr + 1) % BUFSIZE;
}

```

6.004 - Spring 2009

4/16/09

L19 - Devices & Interrupts 6

ReadKey SVC: Attempt #1

A *supervisor call* (SVC) is an instruction that transfers control to the kernel so it can satisfy some user request. Kernel returns to user program when request is complete.

First draft of a ReadKey SVC handler (supporting a Virtual Keyboard): returns next keystroke on a user's keyboard to that user's requesting application:

```

ReadKEY_h()
{
    int kbdnum = ProcTbl[Cur].DPYNum;
    while (BufferEmpty(kbdnum)) {
        /* busy wait loop */
    }
    User.Reg[0] = ReadInputBuffer(kbdnum);
}

```

Problem: Can't interrupt code running in the supervisor mode... so the buffer never gets filled.

6.004 - Spring 2009

4/16/09

L19 - Devices & Interrupts 7

ReadKey SVC: Attempt #2

A BETTER keyboard SVC handler:

```

ReadKEY_h()
{
    int kbdnum = ProcTbl[Cur].DPYNum;
    if (BufferEmpty(kbdnum)) {
        /* busy wait loop */
        User.Reg[XP] = User.Reg[XP] - 4;
    } else
        User.Reg[0] = ReadInputBuffer(kbdnum);
}

```

That's a funny way to write a loop

This one actually works!

Problem: The process just wastes its time-slice waiting for someone to hit a key...

6.004 - Spring 2009

4/16/09

L19 - Devices & Interrupts 8

ReadKey SVC: Attempt #3

EVEN BETTER: On I/O wait, YIELD remainder of quantum:

```
ReadKEY_h()
{
    int kbdnum = ProcTbl[Cur].DPYNum;
    if (BufferEmpty(kbdnum)) {
        User.Regs[XP] = User.Regs[XP] - 4;
        Scheduler();
    } else
        User.Regs[0] = ReadInputBuffer(kbdnum);
}
```

RESULT: Better CPU utilization!!

Does timesharing cause CPU use to be less efficient?

- COST: Scheduling, context-switching overhead; but
- GAIN: Productive use of idle time of one process by running another.

Sophisticated Scheduling

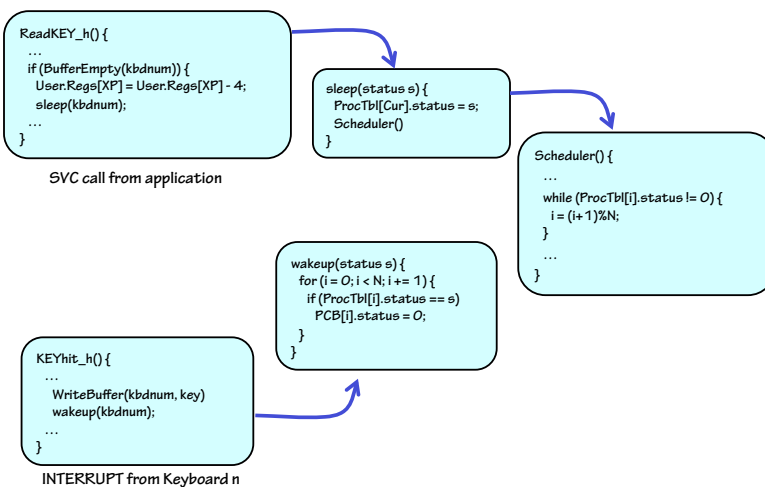
To improve efficiency further, we can avoid scheduling processes in prolonged I/O wait:

- Processes can be in **ACTIVE** or **WAITING** (“sleeping”) states;
- Scheduler cycles among **ACTIVE PROCESSES** only;
- Active process moves to **WAITING** status when it tries to read a character and buffer is empty;
- Waiting processes each contain a code (eg, in PCB) designating what they are waiting for (eg, keyboard N);
- Device interrupts (eg, on keyboard N) move any processes waiting on that device to **ACTIVE** state.

UNIX kernel utilities:

- `sleep(reason)` - Puts CurProc to sleep. “Reason” is an arbitrary binary value giving a condition for reactivation.
- `wakeup(reason)` - Makes active any process in `sleep(reason)`.

ReadKey SVC: Attempt #4



The Need for “Real Time”

Side-effects of CPU virtualization

- + abstraction of machine resources (memory, I/O, registers, etc.)
- + multiple “processes” executing concurrently
- + better CPU utilization
- Processing throughput is more variable

Our approach to dealing with the asynchronous world

- I/O - separate “event handling” from “event processing”

Difficult to meet “hard deadlines”

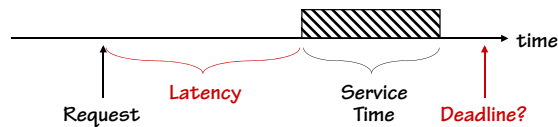
- control applications
- playing videos/MP3s

Real-time as an alternative to time-sliced or fixed-priority preemptive scheduling

Interrupt Latency

One way to measure the real-time performance of a system is **INTERRUPT LATENCY**:

- HOW MUCH TIME can elapse between an interrupt request and the START of its handler?

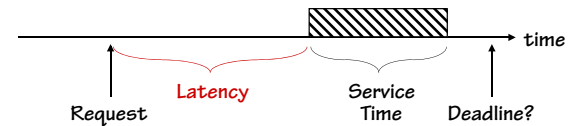


OFTEN bad things happen when service is delayed beyond some **deadline** - "real time" considerations:

Missed characters
System crashes
Nuclear meltdowns

} "HARD"
Real time constraints

Sources of Interrupt Latency



What causes interrupt latency:

But, this is application dependent!

- State save, context switch.
- Periods of uninterruptability:
 - Long, uninterruptable instructions -- eg block moves, multi-level indirection.
 - Explicitly disabled periods (eg for atomicity, during service of other interrupts).

We can consider this when we write our O/S

We can address this in our ISA

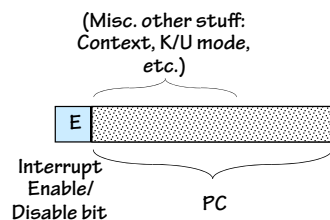
GOAL: BOUND (and minimize) interrupt latency!

- Optimize interrupt sequence context switch
- Make unbounded-time instructions INTERRUPTABLE (state in registers, etc).
- Avoid/minimize disable time
- Allow handlers to be interrupted, in certain cases (while still avoiding **reentrant** handlers!).

Interrupt Disable/Enable

INTERRUPT DISABLE BIT (part of processor status)... in PC:

Often in separate Processor Status Word ...



E=1: DISABLED
E=0: ENABLED

e.g.

- BETA K-mode bit (disables interrupts, other functions)
- Often separate bit/mechanism

TYPICAL OPERATION: (as with Beta K-mode bit):

- ONLY take interrupts if E=0; else defer.
- SAVE OLD E on interrupt, install new E from interrupt vector (along with PC, etc). New E=1 (to disable interrupts during handler).
- Run handler, with interrupts disabled.
- On JMP (at return from handler), saved state restored to processor, resuming interrupted program (with E=0 again).

Scheduling of Multiple Devices

"TOY" System scenario:	Actual w/c Latency	DEVICE	Service Time
	$500 + 400 = 900$	Keyboard	800
	$800 + 400 = 1200$	Disk	500
	$800 + 500 = 1300$	Printer	400

What is the WORST CASE latency seen by each device?

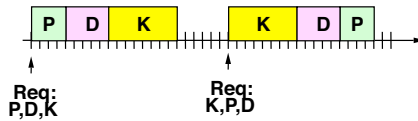


Assumptions:

- Infrequent interrupt requests (each happens only once/scenario)
 - Simultaneous requests might be served in ANY order.... Whence
 - Service of EACH device might be delayed by ALL others!
- ... can we improve this?

Weak (non-preemptive) Priorities

ISSUE: Processor becomes interruptable (at fetch of next instruction), several interrupt requests are pending. Which is served first?



WEAK PRIORITY ORDERING: Check in prescribed sequence, eg: DISK > PRINTER > KEYBOARD.

LATENCIES with WEAK PRIORITIES:

Service of each device might be delayed by:

- Service of 1 other (arbitrary) device, whose interrupt request was just honored; PLUS
- Service of ALL higher-priority devices.

Actual w/c Latency	DEVICE	Service Time
900	Keyboard	800
800	Disk	500
1300	Printer	400

vs 1200 -
Now delayed by only 1 service!

The Need for Preemption

Without preemption, ANY interrupt service can delay ANY other service request... the slowest service time constrains response to fastest devices. Often, tight deadlines can't be met using this scheme alone.

EXAMPLE: 800 uSec deadline (hence 300 uSec maximum interrupt latency) on disk service, to avoid missing next sector...

Priority	Latency w/preemption	Actual Latency	DEVICE	Serv. Time	Max. Delay
1	D, P	900	Keybrd	800	
3		~0	Disk	500	300
2	[D]	500	Printer	400	

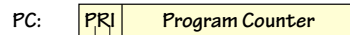
CAN'T SATISFY the disk requirement in this system using weak priorities!

need **PREEMPTION**: Allow handlers for LOWER PRIORITY interrupts to be interrupted by HIGHER priority requests!

Strong Priority Implementation

SCHEME:

- Expand E bit in PC to be a PRIORITY integer PRI (eg, 3 bits for 8 levels)
- ASSIGN a priority to each device.
- Prior to each instruction execution:
 - Find priority P_i of highest requesting device, say D_i
 - Take interrupt if and only if $P_i > \text{PRI}$, set $\text{PRI} = P_i$.



Strong priorities:

KEY: Priority in Processor state

Allows interruption of (certain) handlers

Allows preemption, but not reentrance

BENEFIT: Latency seen at high priorities UNAFFECTED by service times at low priorities.

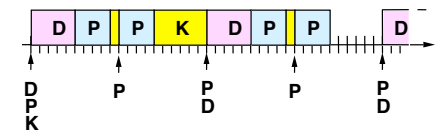
Recurring Interrupts

Consider interrupts which recur at bounded rates:

Actual Latency	DEVICE	P	Serv. Time	Max. Delay	Max. Freq
900	Keybrd	3	800		100/s
0	Disk	5	500	300	500/s
500	Printer	4	400		1000/s

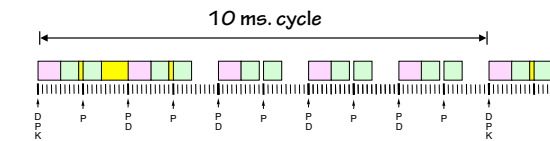
Note that interrupt LATENCIES don't tell the whole story—consider COMPLETION TIMES, eg for Keyboard in example to the right.

Keyboard service not complete until 3ms after request. Often **deadlines** used rather than max. delays.



Interrupt Load

How much CPU time is consumed by interrupt service?



Actual Latency	DEVICE	P	Serv. Time	Max. Delay	Max. Freq	% Load
900	Keybrd	3	800		100/s	$\frac{800u \cdot 100}{s} = 8\%$
0	Disk	5	500	300	500/s	$\frac{500u \cdot 500}{s} = 25\%$
500	Printer	4	400		1000/s	$\frac{400u \cdot 1000}{s} = 40\%$

Remaining fraction (27%) is left over for application; trouble if its <0!

Example: Ben visits ISS

International Space Station's on-board computer performs 3 tasks:

- guiding incoming supply ships to a safe docking
- monitoring gyros to keep solar panels properly oriented
- controlling air pressure in the crew cabin

Task	Period	Service time	Deadline
Supply ship guidance	30ms	5ms	25ms
Gyroscopes	40	10	20
Cabin pressure	100	? 10	100

16.6% $C, G = 10 + 10 + (5) = 25$
 25% $C = 10 + (10) = 20$
 10% $S, G = 5 + 10 + (10) = 25$

Assuming a **weak priority system**:

1. What is the maximum service time for “cabin pressure” that still allows all constraints to be met? **< 10 ms**
2. Give a weak priority ordering that meets the constraints **$G > SSG > CP$**
3. What fraction of the time will the processor spend idle? **48.33%**
4. What is the worst-case delay for each type of interrupt until completion of the corresponding service routine?

Example: Ben visits ISS (cont'd)

Our Russian collaborators don't like the sound of a “weak” priority interrupt system and lobby heavily to use a “strong” priority interrupt system instead.

Task	Period	Service time	Deadline
Supply ship guidance	30ms	5ms	25ms
Gyroscopes	40	10	20
Cabin pressure	100	? 50	100

16.6% $[G] 10 + 5$
 25% 10
 50% 100

Assuming a **strong priority system**, $G > SSG > CP$:

1. What is the maximum service time for “cabin pressure” that still allows all constraints to be met? **$100 - (3 \cdot 10) - (4 \cdot 5) = 50$**
2. What fraction of the time will the processor spend idle? **8.33%**
3. What is the worst-case delay for each type of interrupt until completion of the corresponding service routine?

Summary

Device interface – two parts:

- Device side: handle interrupts from device (transparent to apps)
- Application side: handle interrupts (SVCs) from application

Scheduler interaction:

- “Sleeping” (*inactive) processes waiting for device I/O
- Handler coding issues, looping thru User mode

Real Time constraints, scheduling, guarantees”

- Complex, hard scheduling problems – a black art!
- Weak (non-preemptive) vs Strong (preemptive) priorities help...
- Common real-world interrupt systems:
 - Fixed number (eg, 8 or 16) of strong priority levels
 - Each strong priority level can support many devices, arranged in a weak priority chain

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

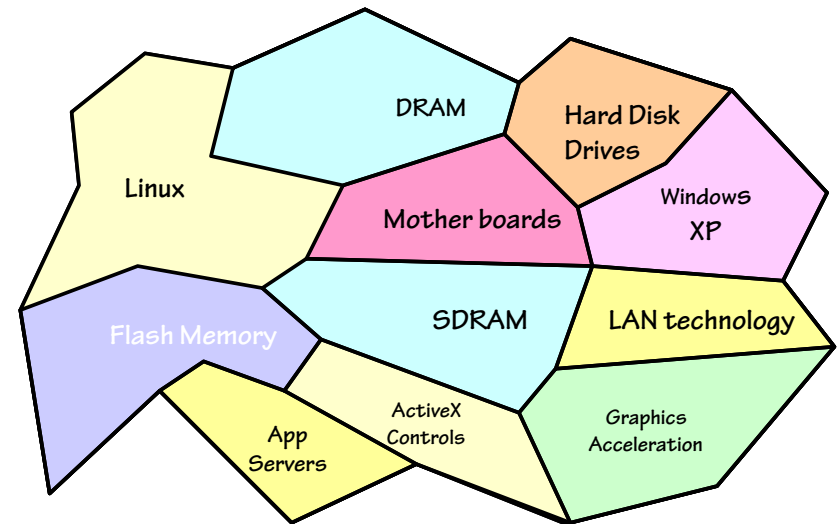
Interconnect & Communication

Space, Time, & stuff...

Quiz #4 tomorrow!

Computer System Technologies

What's the most important part of this picture?



Technology comes & goes; interfaces last forever

Interfaces typically deserve more engineering attention than the technologies they interface...

- **Abstraction:** should outlast many technology generations
- Often "virtualized" to extend beyond original function (e.g. memory, I/O, services, machines)
- Represent more potential value to their proprietors than the technologies they connect.

Interface sob stories:

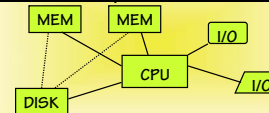
- Interface "warts": Windows "aux.c" bug, Big/little Endian wars
- IBM PC debacle

... and many success stories:

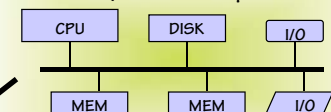
- IBM 360 Instruction set architecture; Postscript; Compact Flash; ...
- Backplane buses

System Interfaces & Modularity

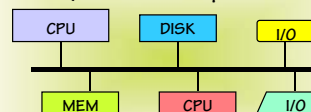
Ancient Times (Ad hoc connections)



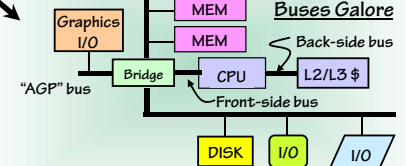
Late 60s (Processor-dependent Bus)



80s (Processor-independent Bus)



Today Buses Galore

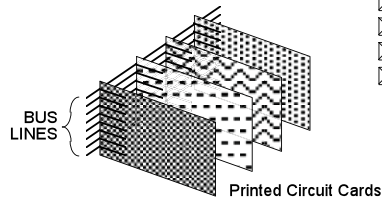


?

Interface Standard: Backplane Bus

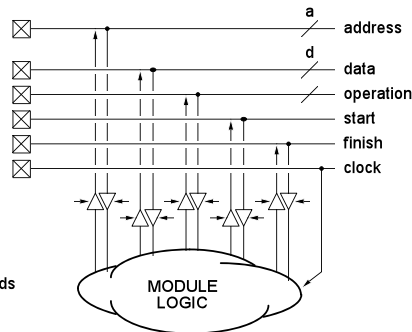
Modular cards that plug into
a common backplane:

CPU's
Memories
Bulk storage
I/O devices
S/W?



The backplane provides:

Power
Common system clock
Wires for communication



The Dumb Bus: ISA & EISA

Original primitive approach --

Just take the control
signals and data bus from
the CPU module, buffer it, and call
it a bus.

ISA bus (Original IBM PC bus) -

Pin out and timing is nearly identical
to the 8088 spec.

Ah, you forget,
Unibus, S-100,
SWTP 99S-50,
STB, MultiBus,
Apple 2E, ...
/

Pin	Signal	Pin	Signal
B1	Ground	A1	I/O Channel Check
B2	Reset Driver	A2	Data 7
B3	+5VDC	A3	Data 6
B4	Interrupt Request 9	A4	Data 5
B5	-VDC	A5	Data 4
B6	DMA Request 2	A6	Data 3
B7	+12 VDC	A7	Data 2
B8	Zero Wait State	A8	Data 1
B9	+12 VDC	A9	Data 0
B10	Ground	A10	I/O Channel Ready
B11	Real Memory Write	A11	Address Enable
B12	Real Memory Read	A12	Address 19
B13	Input/Output Write	A13	Address 18
B14	Input/Output Read	A14	Address 17
B15	DMA Acknowledge 3	A15	Address 16
B16	DMA Request 3	A16	Address 15
B17	DMA Acknowledge 1	A17	Address 14
B18	Refresh	A18	Address 13
B19	Ground	A19	Address 12
B20	Clock	A20	Address 11
B21	Interrupt Request 7	A21	Address 10
B22	Interrupt Request 6	A22	Address 9
B23	Interrupt Request 5	A23	Address 8
B24	Interrupt Request 4	A24	Address 7
B25	Interrupt Request 3	A25	Address 6
B26	DMA Acknowledge 2	A26	Address 5
B27	Terminal Count	A27	Address 4
B28	Address Latch Enable	A28	Address 3
B29	+5 VDC	A29	Address 2
B30	Oscillator	A30	Address 1
B31	Ground	A31	Address 0

Smarter "Processor Independent" Buses

<http://www.techfest.com/hardware/bus/pci.htm>

NuBus, PCI...

Isolate basic communication
primitives from processor
architecture:

- Simple read/write protocols
- Symmetric: any module can become "Master" (smart I/O, multiple processors, etc)
- Support for "plug & play" expansion

Goal: vendor-independent
interface standard

TERMINOLOGY -

⚡ PCI: initiator

BUS MASTER - a module that
initiates a bus transaction.
(CPU, disk controller, etc.)

⚡ PCI: target

BUS SLAVE - a module that
responds to a bus request.
(Memory, I/O device, etc.)

BUS CYCLE - The period from
when a transaction is
requested until it is served.

Buses, Interconnect... what's the big deal?

Aren't buses simply logic circuits with long wires?

Wires: circuit theorist's view:

Equipotential "nodes" of a
circuit.

Instant propagation of v, i
over entire node.

"space" abstracted out of
design model.

Time issues dictated by RLC
elements; wires are
timeless.

Wires: interconnect engineer's view:

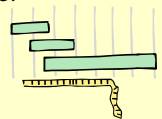
Transmission lines.

Finite signal propagation
velocity.

Space matters.

Time matters.

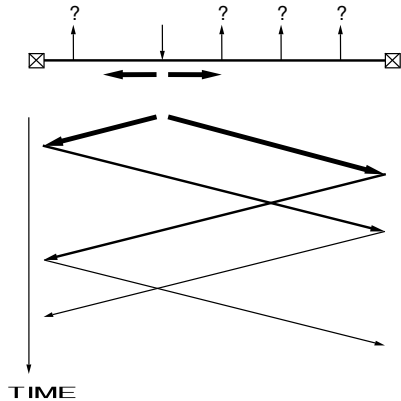
Reality matters.



Bus Lines as Transmission Lines

ANALOG ISSUES:

- Propagation times
 - Light travels about 1 ft / ns (about 7"/ns in a wire)
- Skew
 - Different points along the bus see the signals at different times
- Reflections & standing waves
 - At each interface (places where the propagation medium changes) the signal may reflect if the impedances are not matched.
 - Make a transition on a long line – may have to wait many transition times for echos to subside.



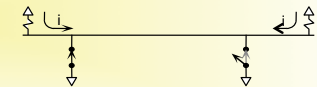
Coping with Analog Issues...

We'd like our bus to be *technology independent*...

- **Self-timed** protocols allow bus transactions to accommodate varying response times;
- **Asynchronous** protocols avoid the need to pick a (technology-dependent) clock frequency.

BUT... asynchronous protocols are vulnerable to analog-domain problems, like the infamous

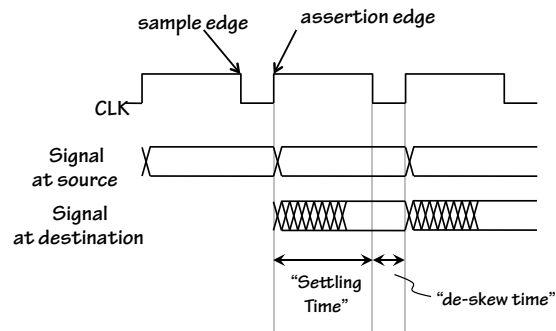
WIRED-OR GLITCH: what happens when a switch is opened???



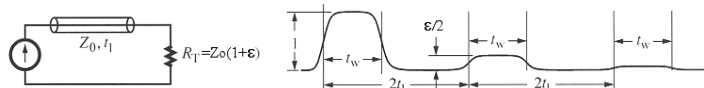
COMMON COMPROMISE: Synchronous, Self-Timed protocols

- Broadcast bus clock
- Signals sampled at "safe" times
- * DEAL WITH: noise, clock skew (wrt signals)

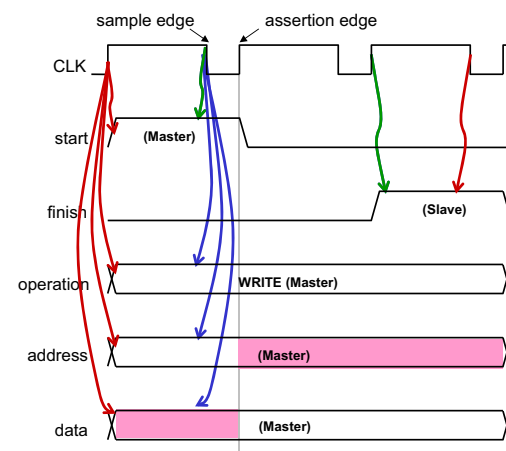
Synchronous Bus Clock Timing



Allow for several "round-trip" bus delays so that ringing can die down.



A Simple Bus Transaction



MASTER:

- 1) Chooses bus operation
- 2) Asserts an address
- 3) Waits for a slave to answer.

SLAVE:

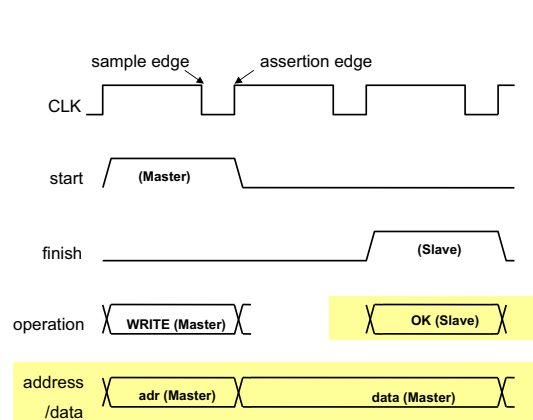
- 1) Monitors start
- 2) Check address
- 3) If meant for me
 - a) look at bus operation
 - b) do operation
 - c) signal finish of cycle

BUS:

- 1) Monitors start
- 2) Start count down
- 3) If no one answers before counter reaches 0 then "time out"

Multiplexed Bus: Write Transaction

More efficient use of shared wires



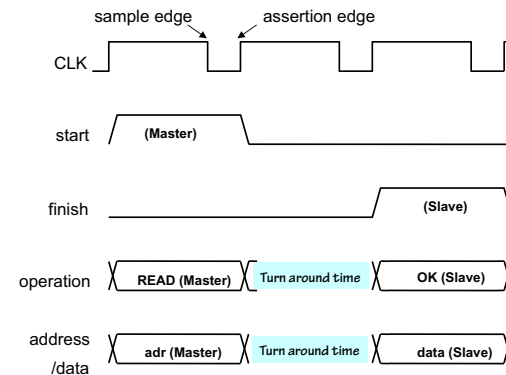
We let the address and data buses share the same wires.

Slave sends a status message by driving the operation control signals when it finishes. Possible indications:

- request succeeded
- request failed
- try again

A slave can stall the write by waiting several cycles before asserting the finish signal.

Multiplexed Bus: Read Transaction

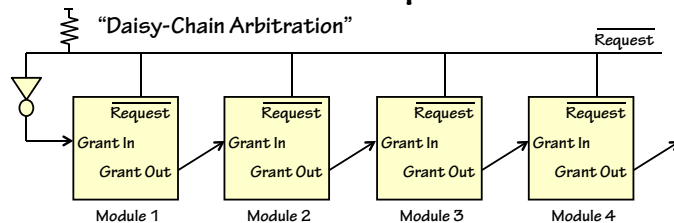


On reads, we allot one cycle for the bus to "turn around" (stop driving and begin receiving). It generally takes some time to read data anyway.

A slave can stall the read (for instance if the device is slow compared to the bus clock) by waiting several clocks before asserting the finish signal. These delays are sometimes called "WAIT-STATES"

Throughput: 3+ Clocks/word

Bus Arbitration: Multiple Bus Masters



ISSUES:

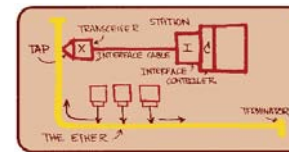
- **Fairness** - Given uniform requests, bus cycles should be divided evenly among modules (to each, according to their needs...)
- **Bounded Wait** - An upper bound on how long a module has to wait between requesting and receiving a grant
- **Utilization** - Arbitration scheme should allow for maximum bus performance
- **Scalability** - Fixed-cost per module (both in terms of arbitration H/W and arbitration time).

STATE OF THE ART ARBITRATION: N masters, log N time, log N wires.

Meanwhile, outside the box...

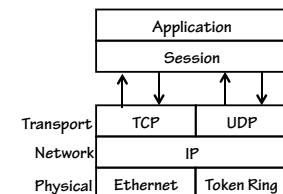
The Network as an interface standard

ETHERNET: In the mid-70's Bob Metcalf (at Xerox PARC, an MIT alum) devised a bus for networking computers together.



- Inspired by Aloha net (radio)
- COAX replaced "ether"
- Bit-serial (optimized for long wires)
- Variable-length "packets":
 - self-clocked data (no clock, skew!)
 - header (dest), data bits
- Issues: sharing, contention, arbitration, "backoff"

EMERGING IDEA: Protocol "layers" that isolate application-level interface from low-level physical devices:



Serial, point-to-point communications....

Becoming standard at all levels?

ETHERNET: Broadcast technology

- Sharing (contention) issues
- Multiple-drop-point issues...

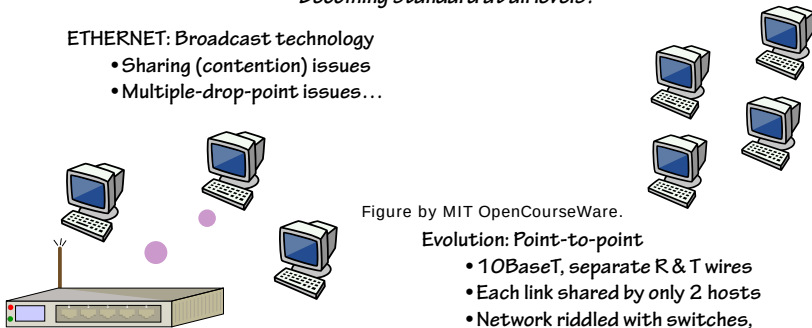


Figure by MIT OpenCourseWare.

Serial point-to-point bus replacements

- Multi Gbit/sec serial links!
- PCIe, Infiniband, SATA, ...
- Packets, headers
- Switches, routing
- Trend: localized, superfast, serial networks!

Figure by MIT OpenCourseWare.

Evolution: Point-to-point

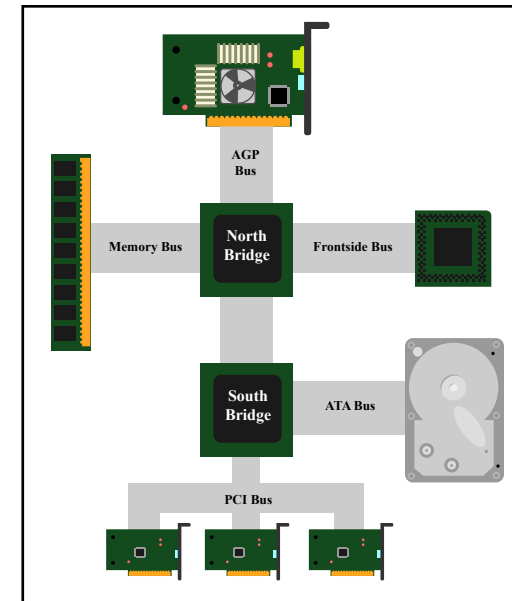
- 10BaseT, separate R & T wires
- Each link shared by only 2 hosts
- Network riddled with switches, routers

6.004 - Spring 2009

4/23/09

L20 - Communication 17

Buses & Bridges in Today's Computers



PCI Express x16...
(point-to-point)

Serial ATA
Firewire
USB 1.1/2.0
...

6.004 - Spring 2009

4/23/09

Figure by MIT OpenCourseWare.

L20 - Communication 18

Generalizing Buses...

Communication Topologies

1-dimensional approaches:

"Low cost networks" - constant cost/node

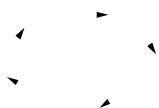
BUS

ONE step for random message delivery (but
only one message at a time)



RING

$\Theta(n)$ steps for random message delivery



6.004 - Spring 2009

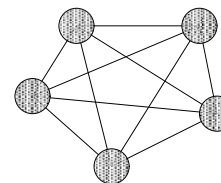
4/23/09

L20 - Communication 19

Quadratic-cost Topologies

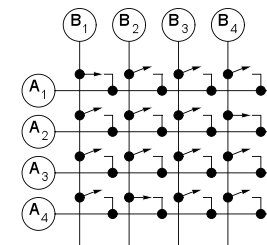
COMPLETE GRAPH:

Dedicated lines connecting each pair of
communicating nodes. $\Theta(n^2)$ simultaneous
communications.



CROSSBAR SWITCH:

- Switch dedicated between each pair of nodes
- Each A_i can be connected to one B_j at any time
- Special cases:
 - A = processors, B = memories
 - A, B same type of node
 - A, B same nodes (complete graph)

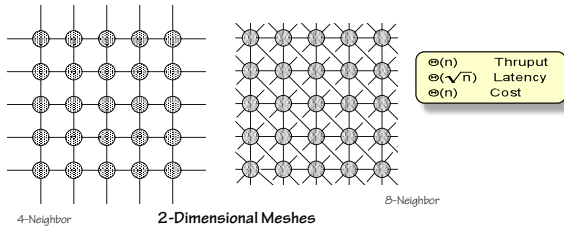


6.004 - Spring 2009

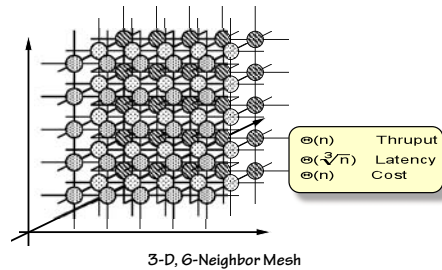
4/23/09

L20 - Communication 20

Mesh Topologies



Nearest-neighbor connectivity:
Point-to-point interconnect
- minimizes delays
- minimizes "analog" effects
Store-and-forward
(some overhead associated with communication routing)

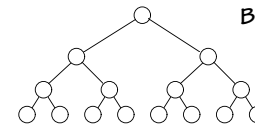
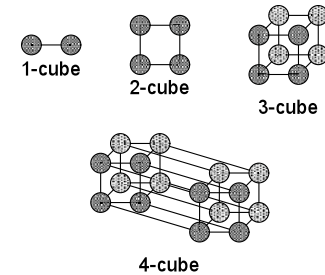


Logarithmic Latency Networks

HYPERCUBE (n-cube):

Cost = $\Theta(n \log n)$

Worst-case path length = $\Theta(\log n)$



BINARY TREE:

Maximum path length is $\Theta(\log n)$ steps;

Cost/node constant.

Communication Topologies: Latency

Theorist's view:

- Each point-to-point link requires one hardware unit.
- Each point-to-point communication requires one time unit.

Topology	\$	Theoretical Latency	Actual Latency
Complete Graph	$\Theta(n^2)$	$\Theta(1)$	$\geq \Theta(\sqrt[3]{n})$
Crossbar	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n)$
1D Bus	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
2D Mesh	$\Theta(n)$	$\Theta(\sqrt{n})$	
3D Mesh	$\Theta(n)$	$\Theta(\sqrt[3]{n})$	
Tree	$\Theta(n)$	$\Theta(\log n)$	$\geq \Theta(\sqrt[3]{n})$
N-cube	$\Theta(n \log n)$	$\Theta(\log n)$	$\geq \Theta(\sqrt[3]{n})$

IS IT REAL?

- Speed of Light: ~ 1 ns/foot (typical bus propagation: 5 ns/foot)
- Density limits: can a node shrink forever? How about Power, Heat, etc ... ?

OBSERVATION: Links on Tree, N-cube must grow with n ; hence time/link must grow.

Communications Futures

The Old Standbys:

- In box: Backplane buses: parallel, shared data paths
 - Arbitration, skew problems
- Local area: shared, single "ether" cable
 - Contention, collisions

New "switched fabric" tech (in & out of box):

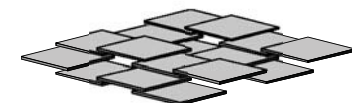
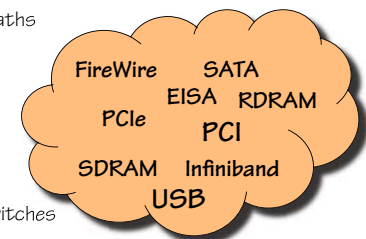
- Shared wires replaced by point-to-point serial
- Parallel data paths replaced by serial "packets"
- Communication network extended via active switches

Topological Invariants:

- Asymptotic performance/cost tradeoffs...
- Log-latency topologies: a useful fiction
- Best-case scaling with 3D mesh

Watch this space!

- Technologies: optical, proximity, ...
- 3D packaging, interconnect
- ???

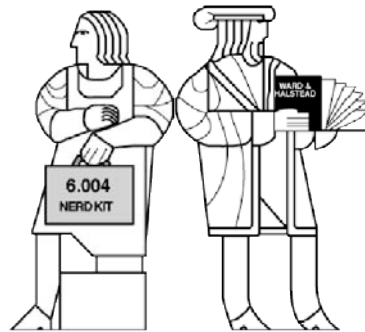


MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Processes, Synchronization, & Deadlock



Lab #7 due Thursday

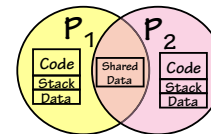
Interprocess Communication

Why communicate?

- Concurrency
- Asynchrony
- Processes as a programming primitive
- Data/Event driven

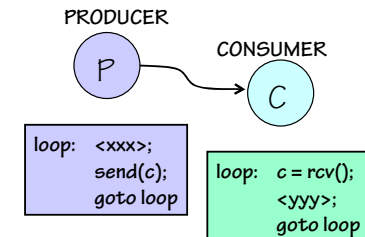
How to communicate?

- Shared Memory (overlapping contexts)...
- Supervisor calls
- Synchronization instructions, (hardware support)



Classic Example:

“PRODUCER-CONSUMER” Problem:



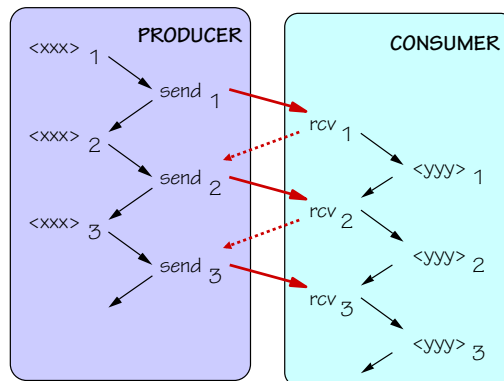
Real-World Examples:

UNIX pipeline,
Word processor/Printer Driver,
Preprocessor/Compiler,
Compiler/Assembler

Synchronous Communication

loop: <xxx>;
send(c);
goto loop

loop: c = rcv();
<yyy>;
goto loop



Precedence

Constraints:

$$\alpha \leq \beta$$

“ α precedes β ”

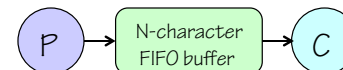
- Can't CONSUME data before it's PRODUCED

$$send_i \leq rcv_i$$

- Producer can't “OVERWRITE” data before it's consumed

$$rcv_i \leq send_i$$

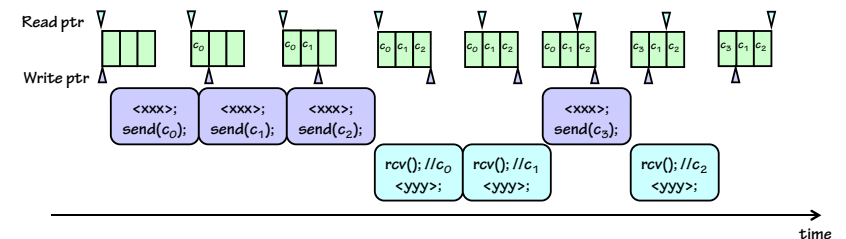
FIFO Buffering



RELAXES interprocess
synchronization constraints.
Buffering relaxes the following
OVERWRITE constraint to:

$$rcv_i \leq send_{i+N}$$

“Ring Buffer:”



Example: Bounded Buffer Problem

SHARED MEMORY:

```
char buf[N];          /* The buffer */
int in=0, out=0;
```

PRODUCER:

```
send(char c)
{
    buf[in] = c;
    in = (in+1) % N;
}
```

CONSUMER:

```
char rcv()
{
    char c;
    c = buf[out];
    out = (out+1) % N;
    return c;
}
```

**Problem: Doesn't enforce precedence constraints
(e.g. rcv() could be invoked prior to any send())**

Semaphores (Dijkstra)

Programming construct for synchronization:

- NEW DATA TYPE: *semaphore*, integer-valued
`semaphore s = K;           /* initialize s to K */`
- NEW OPERATIONS (defined on semaphores):
 - `wait(semaphore s)`
 stall current process if ($s \leq 0$), otherwise $s = s - 1$
 - `signal(semaphore s)`
 $s = s + 1$, (can have side effect of letting other processes proceed)
- SEMANTIC GUARANTEE: A semaphore s initialized to K enforces the constraint:

Often you will see
 $P(s)$ used for `wait(s)`
 and
 $V(s)$ used for `signal(s)`!
 $P = \text{"proberen" (test) or}$
 "pakken" (grab)
 $V = \text{"verhogen" (increase)}$

$$\text{signal}(s)_i \leq \text{wait}(s)_{i+K}$$

This is a *precedence* relationship, meaning that the $(i+K)^{\text{th}}$ call to `wait` cannot proceed before the i^{th} call to `signal` completes.

Semaphores for Resource Allocation

ABSTRACT PROBLEM:

- POOL of K resources
- Many processes, each needs resource for occasional uninterrupted periods
- MUST guarantee that at most K resources are in use at any time.

Semaphore Solution:

In shared memory:
`semaphore s = K;   /* K resources */`

In each process:
`...`
`wait(s);       /* Allocate one       */`
`...` `/* use it for a while */`
`signal(s);   /* return it to pool */`
`...`

Invariant: Semaphore value = number of resources left in pool

Bounded Buffer Problem w/Semaphores

SHARED MEMORY:

```
char buf[N];          /* The buffer */
int in=0, out=0;
semaphore chars=0;
```

PRODUCER:

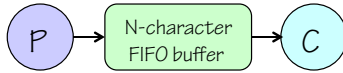
```
send(char c)
{
    buf[in] = c;
    in = (in+1) % N;
    signal(chars);
}
```

CONSUMER:

```
char rcv()
{
    char c;
    wait(chars);
    c = buf[out];
    out = (out+1) % N;
    return c;
}
```

RESOURCE managed by semaphore: Characters in FIFO.
 DOES IT WORK?

Flow Control Problems



Q: What keeps PRODUCER from putting N+1 characters into the N-character buffer?

A: Nothing.

Result: OVERFLOW. Randomness. Havoc. Smoke. Pain. Suffering. D's and F's in MIT courses.

WHAT we've got thus far:

$$\text{send}_i \leq \text{rcv}_i$$

WHAT we still need:

$$\text{rcv}_i \leq \text{send}_{i+N}$$

Bounded Buffer Problem w/ ^{more} Semaphores

SHARED MEMORY:

```
char buf[N];          /* The buffer */
int in=0, out=0;
semaphore chars=0, space=N;
```

PRODUCER:

```
send(char c)
{
    wait(space);
    buf[in] = c;
    in = (in+1)%N;
    signal(chars);
}
```

CONSUMER:

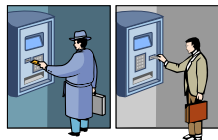
```
char rcv()
{
    char c;
    wait(chars);
    c = buf[out];
    out = (out+1)%N;
    signal(space);
    return c;
}
```

RESOURCEs managed by semaphore: Characters in FIFO, Spaces in FIFO

WORKS with **single** producer, consumer. But what about...

Simultaneous Transactions

Suppose you and your friend visit the ATM at exactly the same time, and remove \$ 50 from your account. What happens?



Debit(6004, 50) Debit(6004, 50)

```
Debit(int account, int amount)
{
    t = balance[account];
    balance[account] = t - amount;
}
```

What is *supposed* to happen?

Process #1

```
LD(R10, balance, R0)
SUB(R0, R1, R0)
ST(R0, balance, R10)
...
```

Process #2

```
...
LD(R10, balance, R0)
SUB(R0, R1, R0)
ST(R0, balance, R10)
```

NET: You have \$ 100, and your bank balance is \$ 100 less.

But, what if...

Process #1
LD(R10, balance, R0)
...
SUB(R0, R1, R0)
ST(R0, balance, R10)
...

Process #2

```
LD(R10, balance, R0)
SUB(R0, R1, R0)
ST(R0, balance, R10)
...
```

...
SUB(R0, R1, R0)
ST(R0, balance, R10)
...

NET: You have \$ 100 and your bank balance is \$ 50 less!

We need to be careful when writing concurrent programs. In particular, when modifying shared data.

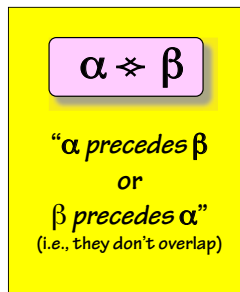
For certain code segments, called **CRITICAL SECTIONS**, we would like to assure that no two executions overlap.

This constraint is called **MUTUAL EXCLUSION**.

Solution: embed critical sections in wrappers (e.g., "transactions") that guarantee their **atomicity**, i.e. make them appear to be single, instantaneous operations.

Semaphores for Mutual Exclusion

```
semaphore lock = 1;
...
Debit(int account, int amount)
{
    wait(lock); /* Wait for exclusive access */
    t = balance[account];
    balance[account] = t - amount;
    signal(lock); /* Finished with lock */
}
```



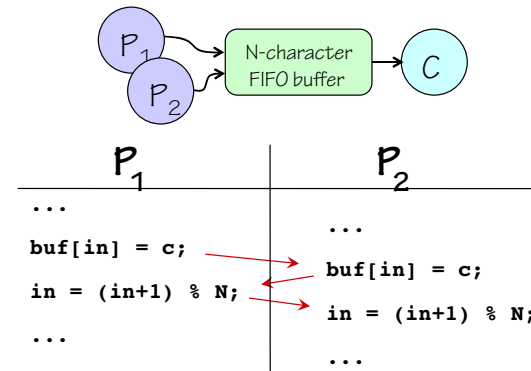
RESOURCE managed by “lock” semaphore: Access to critical section

ISSUES:

- Granularity of lock
 - 1 lock for whole balance database?
 - 1 lock per account?
 - 1 lock for all accounts ending in 004?
- Implementation of wait() and signal() functions

Producer/Consumer Atomicity Problems

Consider multiple PRODUCER processes:



BUG: Producers interfere with each other.

Bounded Buffer Problem w/ ^{even more} Semaphores

SHARED MEMORY:

```
char buf[N]; /* The buffer */
int in=0, out=0;
semaphore chars=0, space=N;
semaphore mutex=1;
```

PRODUCER:

```
send(char c)
{
    wait(space);
    wait(mutex);
    buf[in] = c;
    in = (in+1)%N;
    signal(mutex);
    signal(chars);
}
```

CONSUMER:

```
char rcv()
{
    char c;
    wait(chars);
    wait(mutex);
    c = buf[out];
    out = (out+1)%N;
    signal(mutex);
    signal(space);
    return c;
}
```

SHARED MEMORY:

```
char buf[N]; /* The buffer */
int in=0, out=0;
semaphore chars=0, space=N;
semaphore mutex=1;
```

PRODUCER:

```
send(char c)
{
    wait(space);
    wait(mutex);
    buf[in] = c;
    in = (in+1)%N;
    signal(mutex);
    signal(chars);
}
```

CONSUMER:

```
char rcv()
{
    char c;
    wait(chars);
    wait(mutex);
    c = buf[out];
    out = (out+1)%N;
    signal(mutex);
    signal(space);
    return c;
}
```

A single synchronization primitive that enforces both:

Precedence relationships:

$$\begin{aligned} \text{send}_i &\preceq \text{rcv}_i \\ \text{rcv}_i &\preceq \text{send}_{i+N} \end{aligned}$$

Mutual-exclusion relationships:
protect variables *in* and *out*

The Power of Semaphores

Semaphore Implementations

Semaphore implementation must address a basic arbitration problem:
how to choose among simultaneously waiting processes when a signal occurs. This involves some basic atomicity assumption in the implementation technology.

Approaches:

- SVC implementation, using atomicity of kernel handlers. Works in timeshared processor sharing a single uninterruptable kernel.
- Implementation by a special instruction (e.g. "test and set"), using atomicity of single instruction execution. Works with shared-bus multiprocessors supporting atomic read-modify-write bus transactions.
- Implementation using atomicity of individual read or write operations. Complex, clever, 2-phase scheme devised by Dijkstra. Unused in practice.

Bootstrapping: A simple lock ("binary semaphore") allows easy implementation of full semaphore support.

Semaphores as Supervisor Call

```
wait_h()
{
    int *addr;
    addr = User.Reg[R0]; /* get arg */
    if (*addr <= 0) {
        User.Reg[XP] = User.Reg[XP] - 4;
        sleep(addr);
    } else
        *addr = *addr - 1;
}

signal_h()
{
    int *addr;
    addr = User.Reg[R0]; /* get arg */
    *addr = *addr + 1;
    wakeup(addr);
}
```

Calling sequence:

```
...
|| put address of lock
|| into R0
CMOVE(lock, R0)
SVC(WAIT)
```

SVC call is not interruptible

since it is executed in
supervisory mode.

H/W support for Semaphores

TCLR(RA, literal, RC) test and clear location

PC ← PC + 4

EA ← Reg[Ra] + literal

Reg[Rc] ← MEM[EA]

MEM[EA] ← 0

Atomicity
Guaranteed, e.g. by
Bus protocols

Executed ATOMICALLY (cannot be interrupted)

Can easily implement mutual exclusion using binary semaphore

wait: TCLR(R31, lock, R0)

BEQ(R0, wait)

... critical section ...

CMOVE(1, R0)

ST(R0, lock, R31)

} wait(lock)

} signal(lock)

Synchronization: The Dark Side

The indiscriminate use of synchronization constraints can introduce its own set of problems, particularly when a process requires access to more than one protected resource.

```
Transfer(int account1, int account2, int amount)
{
    wait(lock[account1]);
    wait(lock[account2]);
    balance[account1] = balance[account1] - amount;
    balance[account2] = balance[account2] + amount;
    signal(lock[account2]);
    signal(lock[account1]);
}
```

DEAD-
LOCK!



Transfer(6001, 6004, 50)



Transfer(6004, 6001, 50)

Famous Toy Problem: Dining Philosophers

Philosophers think deep thoughts, but have simple secular needs. When hungry, a group of N philosophers will sit around a table with N chopsticks interspersed between them. Food is served, and each philosopher enjoys a leisurely meal using the chopsticks on either side to eat (2 sticks are required, to avoid an ancient Zen paradox about the sound of one stick feeding).

They are exceedingly polite and patient, and each follows the following dining protocol:

PHILOSOPHER'S ALGORITHM:

- Take (wait for) LEFT stick
- Take (wait for) RIGHT stick
- EAT until sated
- Replace both sticks

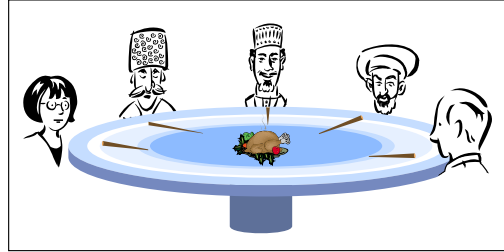


Figure by MIT OpenCourseWare.

Deadlock!

No one can make progress because they are all waiting for an unavailable resource

CONDITIONS:

- 1) Mutual exclusion - only one process can hold a resource at a given time
- 2) Hold-and-wait - a process holds allocated resources while waiting for others
- 3) No preemption - a resource can not be removed from a process holding it
- 4) Circular Wait

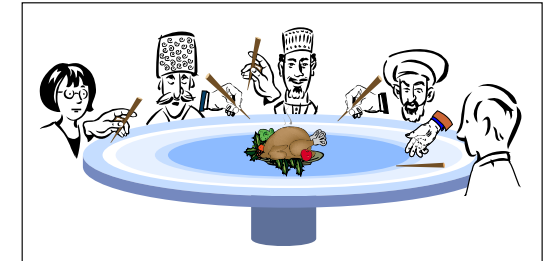


Figure by MIT OpenCourseWare.

SOLUTIONS:
Avoidance
-or-
Detection and Recovery

One Solution

KEY: Assign a unique number to each chopstick, request resources in globally consistent order:

New Algorithm:

- Take LOW stick
- Take HIGH stick
- EAT
- Replace both sticks.

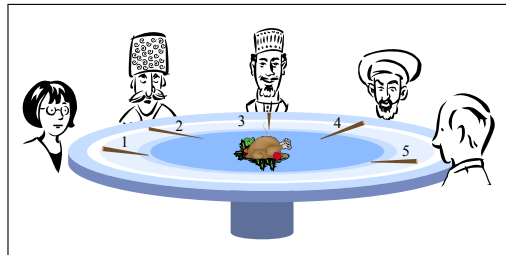


Figure by MIT OpenCourseWare.

SIMPLE PROOF:

Deadlock means that each philosopher is waiting for a resource held by some other philosopher ...

But, the philosopher holding the highest numbered chopstick can't be waiting for any other philosopher (no hold-and-wait) ...

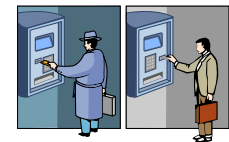
Thus, there can be no deadlock

Dealing with Deadlocks

Cooperating processes:

- Establish a fixed ordering to shared resources and require all requests to be made in the prescribed order

```
Transfer(int account1, int account2, int amount)
{
    int a, b;
    if (account1 > account2) { a = account1; b = account2; } else { a = account2; b = account1; }
    wait(lock[a]);
    wait(lock[b]);
    balance[account1] = balance[account1] - amount;
    balance[account2] = balance[account2] + amount;
    signal(lock[b]);
    signal(lock[a]);
}
```



Unconstrained processes:

- O/S discovers circular wait & kills waiting process
- Transaction model
- Hard problem

Transfer(6001, 6004, 50) Transfer(6004, 6001, 50)

Summary

Communication among asynchronous processes requires synchronization....

- Precedence constraints: a partial ordering among operations
- Semaphores as a mechanism for enforcing precedence constraints
- Mutual exclusion (critical sections, atomic transactions) as a common compound precedence constraint
- Solving Mutual Exclusion via binary semaphores
- Synchronization *serializes* operations, limits parallel execution.

Many alternative synchronization mechanisms exist!

Deadlocks:

- Consequence of undisciplined use of synchronization mechanism
- Can be avoided in special cases, detected and corrected in others.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

Spring 2009

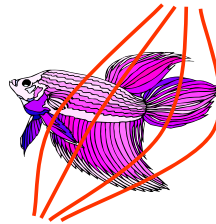
For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Pipelining the Beta

bet-ta ('be-t& n. Any of various species of small, brightly colored, long-finned freshwater fishes of the genus *Betta*, found in southeast Asia.

be-ta ('bA-t&, 'bE-) n. **1.** The second letter of the Greek alphabet. **2.** The exemplary computer system used in 6.004.

I don't think they mean the fish...



Lab #7 due Tonight!



6.004 - Spring 2009

4/30/09

modified 4/27/09 11:17

L22 - Pipelining the Beta 1

CPU Performance

We've got a working Beta... can we make it fast?

$$MIPS = \frac{Freq}{CPI}$$

MIPS = Millions of Instructions/Second

Freq = Clock Frequency, MHz

CPI = Cycles per Instruction

To Increase MIPS:

1. DECREASE CPI.

- RISC *simplicity* reduces CPI to 1.0.

- CPI below 1.0? Tough... you'll see multiple instruction issue machines in 6.823.

2. INCREASE Freq.

- Freq limited by delay along longest combinational path; hence

- **PIPELINING** is the key to improved performance through fast clocks.

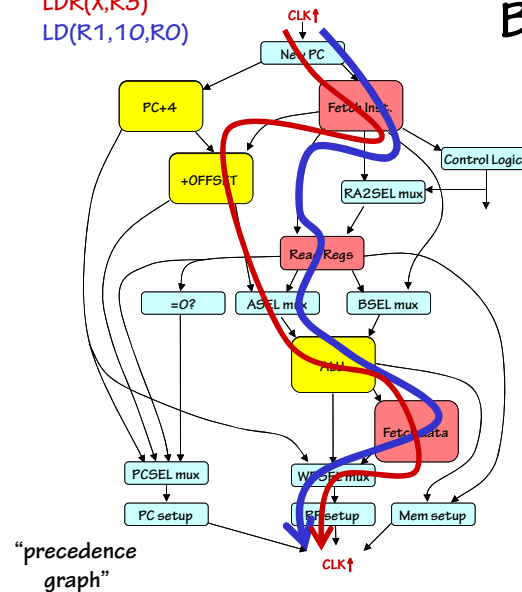
6.004 - Spring 2009

4/30/09

L22 - Pipelining the Beta 2

Beta Timing

LDR(X,R3)
LD(R1,10,R0)



Wanted:
longest paths

Complications:

- some apparent paths aren't "possible"
- operations have variable execution times (eg, ALU)
- time axis is not to scale (eg, $t_{PD, MEM}$ is very big!)

6.004 - Spring 2009

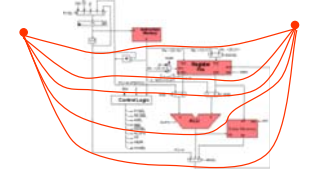
4/30/09

L22 - Pipelining the Beta 3

Why isn't this a 20-minute lecture?

We've learned how to pipeline combinational circuits.

What's the big deal?



1. The Beta isn't combinational...

- Explicit state in register file, memory;
- Hidden state in PC.

2. Consecutive operations - instruction executions - interact:

- Jumps, branches dynamically change instruction sequence
- Communication through registers, memory

Our goals:

- Move slow components into separate pipeline stages, running clock faster
- Maintain instruction semantics of unpipelined Beta as far as possible

6.004 - Spring 2009

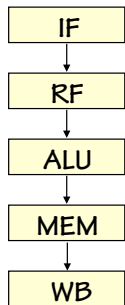
4/30/09

L22 - Pipelining the Beta 4

Ultimate Goal: 5-Stage Pipeline

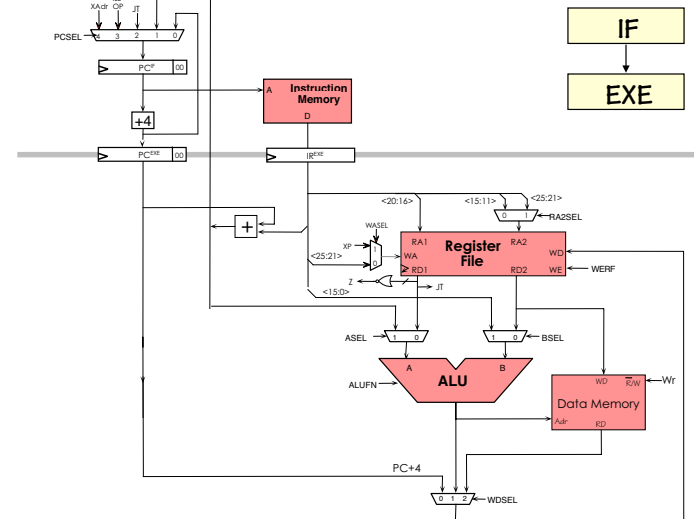
GOAL: Maintain (nearly) 1.0 CPI, but increase clock speed to barely include slowest components (mems, regfile, ALU)

APPROACH: structure processor as 5-stage pipeline:



- Instruction Fetch stage:** Maintains PC, fetches one instruction per cycle and passes it to
- Register File stage:** Reads source operands from register file, passes them to
- ALU stage:** Performs indicated operation, passes result to
- Memory stage:** If it's a LD, use ALU result as an address, pass mem data (or ALU result if not LD) to
- Write-Back stage:** writes result back into register file.

First Steps: A Simple 2-Stage Pipeline



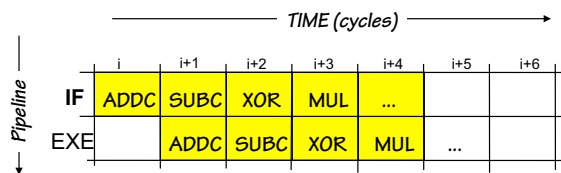
2-Stage Pipelined Beta Operation

Consider a sequence of instructions:

```

...
ADDC(r1, 1, r2)
SUBC(r1, 1, r3)
XOR(r1, r5, r1)
MUL(r2, r6, r0)
...
    
```

Executed on our 2-stage pipeline:

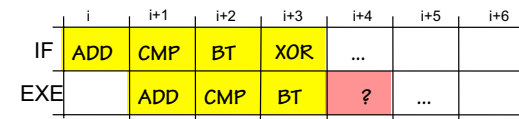


Pipeline Control Hazards

BUT consider instead:

```

LOOP: ADD(r1, r3, r3)
      CMPLC(r3, 100, r0)
      BT(r0, LOOP)
      XOR(r3, -1, r3)
      MUL(r1, r2, r2)
      ...
    
```



This is the cycle where the branch decision is made... but we've already fetched the following instruction which should be executed only if branch is not taken!

Branch Delay Slots

PROBLEM: One (or more) following instructions have been pre-fetched by the time a branch is taken.

POSSIBLE SOLUTIONS:

1. Make hardware “annul” instructions following branches which are taken, e.g., by disabling WERF and WR.

2. “Program around it”. Either

- a) Follow each BR with a NOP instruction; or
- b) Make compiler clever enough to move USEFUL instructions into the branch delay slots
 - i. Always execute instructions in delay slots
 - ii. Conditionally execute instructions in delay slots

NOP = ‘no-operation’, e.g.
ADD(R31, R31, R31)

Branch Alternative 1

Make the hardware annul instructions in the branch delay slots of a taken branch.

LOOP: ADD(r1, r3, r3)
CMPLC(r3, 100, r0)
BT(r0, LOOP)
XOR(r3, -1, r3)
MUL(r1, r2, r2)
...

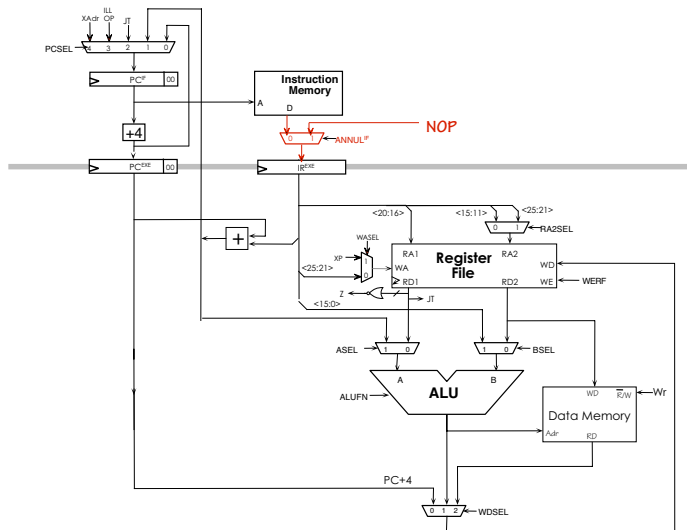
	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	BT	XOR	ADD	CMP	BT
EXE		ADD	CMP	BT	XOR	ADD	CMP



NOP

Pros: same program runs on both unpipelined and pipelined hardware
Cons: in SPEC benchmarks 14% of instructions are taken branches → 12% of total cycles are annulled

Branch Annulment Hardware



Branch Alternative 2a

Fill branch delay slots with NOP instructions (i.e., the software equivalent of alternative 1)

LOOP: ADD(r1, r3, r3)
CMPLC(r3, 100, r0)
BT(r0, LOOP)
NOP()
XOR(r3, -1, r3)
MUL(r1, r2, r2)
...

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	BT	NOP	ADD	CMP	BT
EXE		ADD	CMP	BT	NOP	ADD	CMP



Branch taken

Pros: same as alternative 1
Cons: NOPs make code longer; 12% of cycles spent executing NOPs

Branch Alternative 2b(i)

Put USEFUL instructions in the branch delay slots; remember they will be executed whether the branch is taken or not

```
LOOP: ADD(r1, r3, r3)
LOOPx: CMPLC(r3, 100, r0)
      BT(r0, LOOPx)
      ADD(r1, r3, r3)
      SUB(r3, r1, r3)
      XOR(r3, -1, r3)
      MUL(r1, r2, r2)
      ...
```

We need to add this silly instruction to UNDO the effects of that last ADD

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	BT	ADD	CMP	BT	ADD
EXE		ADD	CMP	BT	ADD	CMP	BT

↑ Branch taken

Pros: only two “extra” instructions are executed (on last iteration)
Cons: finding “useful” instructions that are *always* executed is difficult; clever rewrite may be required. Program executes differently on naïve unpipelined implementation.

Branch Alternative 2b(ii)

Put USEFUL instructions in the branch delay slots; annul them if branch *doesn't* behave as predicted

```
LOOP: ADD(r1, r3, r3)
LOOPx: CMPLC(r3, 100, r0)
      BT.taken(r0, LOOPx)
      ADD(r1, r3, r3)
      XOR(r3, -1, r3)
      MUL(r1, r2, r2)
      ...
```

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	BT	ADD	CMP	BT	ADD
EXE		ADD	CMP	BT	ADD	CMP	BT

↑ Branch taken

Pros: only one instruction is annulled (on last iteration); about 70% of branch delay slots can be filled with useful instructions
Cons: Program executes differently on naïve unpipelined implementation; not really useful with more than one delay slot.

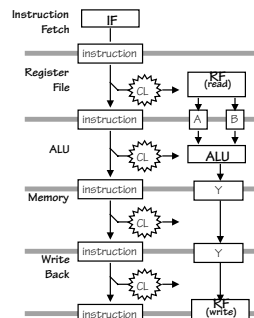
Architectural Issue: Branch Decision Timing

BETA approach:

- SIMPLE branch condition logic ...
Test for $\text{Reg}[Ra] = 0!$
- ADVANTAGE: early decision, single delay slot

ALTERNATIVES:

- Compare-and-branch...
(eg, if $\text{Reg}[Ra] > \text{Reg}[Rb]$)
- MORE powerful, but
- LATER decision (hence more delay slots)



Suppose decision were made in the ALU stage ... then there would be 2 branch delay slots (and instructions to annul!)

Wow! I guess those guys really were thinking when they made up all those instructions

4-Stage Beta Pipeline

Treat register file as two separate devices: combinational READ, clocked WRITE at end of pipe.

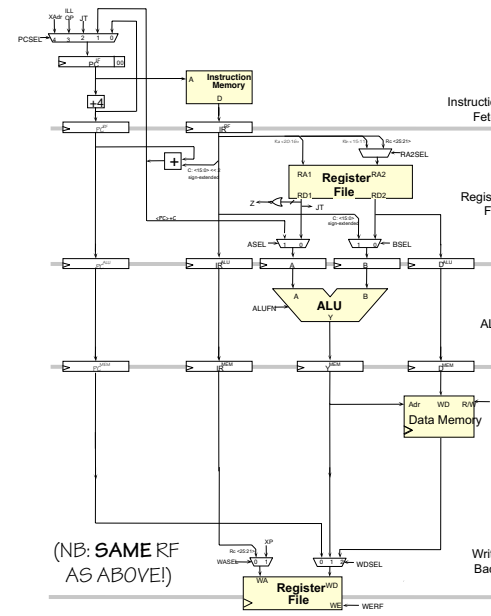
What other information do we have to pass down pipeline?

PC (return addresses)

instruction fields

(decoding)

What sort of improvement should expect in cycle time?

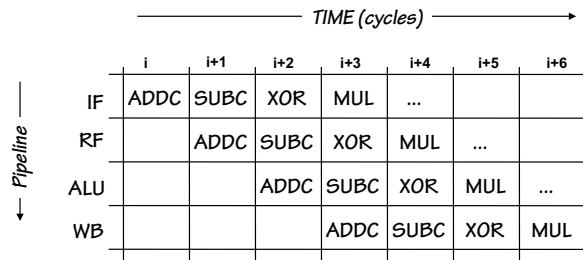


4-Stage Beta Operation

Consider a sequence of instructions:

```
...
ADDC(r1, 1, r2)
SUBC(r1, 1, r3)
XOR(r1, r5, r1)
MUL(r2, r6, r0)
...
```

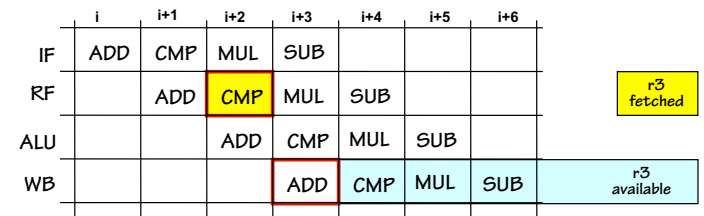
Executed on our 4-stage pipeline:



Pipeline “Data Hazard”

BUT consider instead:

```
ADD(r1, r2, r3)
CMPLC(r3, 100, r0)
MULC(r1, 100, r4)
SUB(r1, r2, r5)
```



Oops! CMP is trying to read Reg[R3] during cycle i+2 but ADD doesn't write its result into Reg[R3] until the end of cycle i+3!

Data Hazard Solution 1

“Program around it”

... document weird semantics, declare it a software problem.

- Breaks sequential semantics!
- Costs code efficiency.

EXAMPLE: Rewrite

```
ADD(r1, r2, r3)          ADD(r1, r2, r3)
CMPLC(r3, 100, r0)       MULC(r1, 100, r4)
MULC(r1, 100, r4)        SUB(r1, r2, r5)
SUB(r1, r2, r5)          CMPLC(r3, 100, r0)
```

as

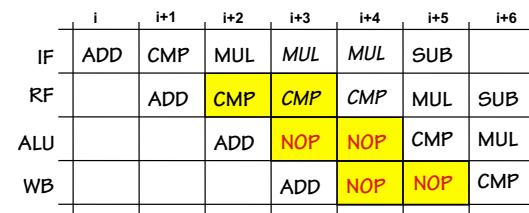
How often can we do this?

Programmer's fallback: Insert NOPs (sigh!)

Data Hazard Solution 2

Stall the pipeline:

Freeze IF, RF stages for 2 cycles, inserting NOPs into ALU-stage instruction register



Drawback: NOPs mean “wasted” cycles

Data Hazard Solution 3

Bypass (aka forwarding) Paths:

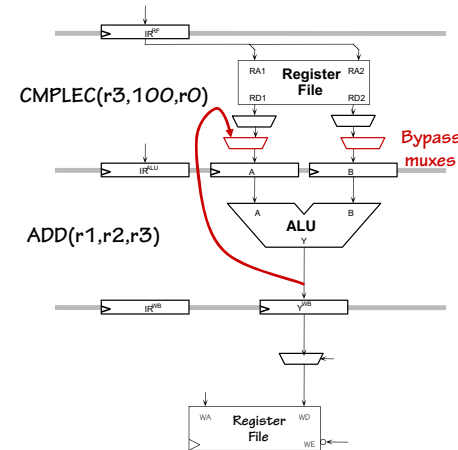
Add extra data paths & control logic to re-route data in problem cases.

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	MUL	SUB			
RF		ADD	CMP	MUL	SUB		
ALU			ADD	CMP	MUL	SUB	
WB				ADD	CMP	MUL	SUB

r3 available

Idea: the result from the ADD which will be written into the register file at the end of cycle i+3 is actually available at output of ALU during cycle i+2 – just in time for it to be used by CMP in the RF stage!

Bypass Paths (I)

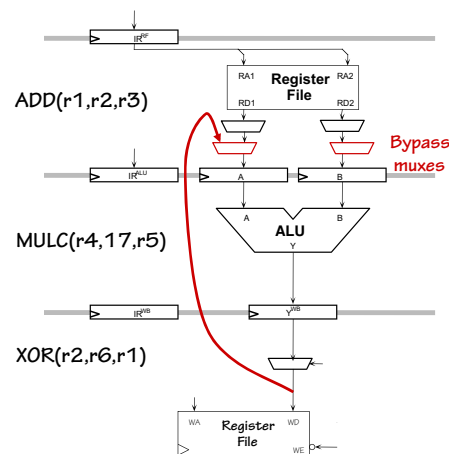


SELECT this BYPASS path if
 $OpCode^{RF} = \text{reads } Ra$
and $OpCode^{ALU} = OP, OPC$
and $Ra^{RF} = Rc^{ALU}$

i.e., instructions which use
 ALU to compute result

and $Ra^{RF} \neq R31$

Bypass Paths (II)



SELECT this BYPASS path if
 $OpCode^{RF} = \text{reads } Ra$
and $Ra^{RF} \neq R31$
and not using ALU bypass
and $WERF = 1$
and $Ra^{RF} = Wa$

But why can't we get
 it from the register file?
 It's being written this cycle!

Next Time



MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures

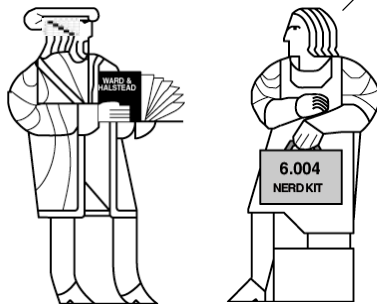
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Pipeline Issues

This pipeline stuff makes my head hurt!

Maybe it's that dumb hat



Home Stretch:

Lab #8 due Thursday 5/7;

Quiz 5 FRIDAY 5/8!

Recalling Data Hazards

PROBLEM: Subsequent instructions can reference the contents of a register well before the pipeline stage where the register is written.

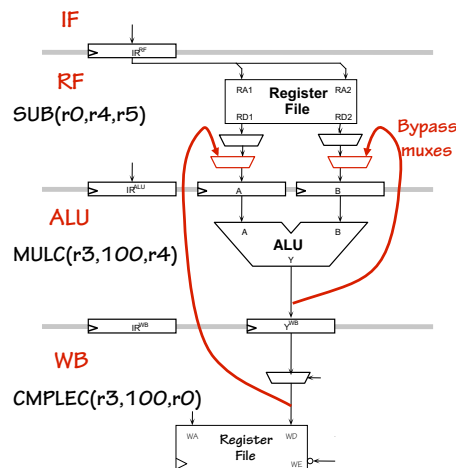
ADD(r1, r2, r3)
CMPLC(r3, 100, r0)
MULC(r3, 100, r4)
SUB(r0, r4, r5)

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	CMP	MUL	SUB			
RF		ADD	CMP	MUL	SUB		
ALU			ADD	CMP	MUL	SUB	
WB				ADD	CMP	MUL	SUB

SOLUTION #1: Deal with it in SOFTWARE; expose the pipeline for all to see.

SOLUTION #2: Add special hardware to maintain the sequential execution semantics of the ISA.

Bypass Paths



Add special data paths, called **BYPASSES**, that route the results of the ALU and WB stages to the RF stage, thus substituting the register's old contents with a value that will be written to that register at some point in the future.

Detection of these cases has to be incorporated into the decoding logic of the RF stage, which basically looks at the instructions in the ALU and WB stage to see if their destination register matches a source register reference.

But there are some problems that **BYPASSING CAN'T FIX!**

Load Hazards

Consider LOADS:

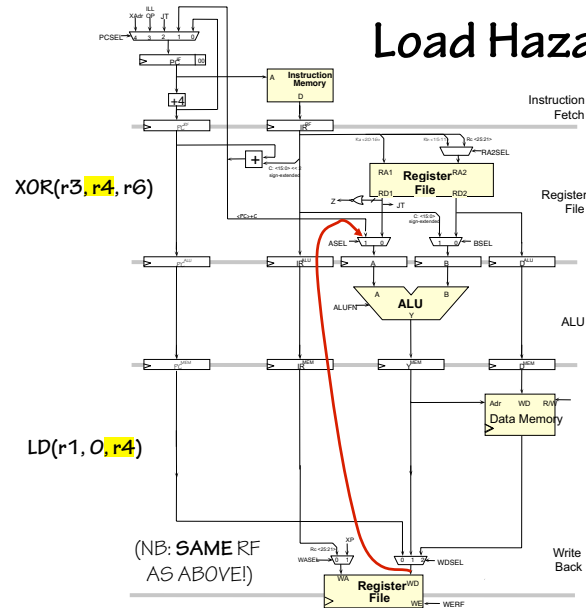
Can we fix all these problems using bypass paths?

LD(r1, 0, r4)
ADD(r4, r1, r5)
XOR(r3, r4, r6)

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	LD	ADD	XOR				
RF		LD	ADD	XOR			
ALU			LD	ADD	XOR		
WB				LD	ADD	XOR	

The hazard between the XOR and the LD can be addressed by our established bypass paths...

Load Hazard (easy)



6.004 – Spring 2009

5/5/09

L23 – Pipeline Issues 5

The XOR operand r4 can simply be bypassed from the output of the memory in the WB stage to the RF stage... by our normal bypass path.

Structural Data Hazard

The XOR hazard is pretty easy, but...

```
LD(r1, 0, r4)
ADD(r1, r4, r5)
XOR(r3, r4, r6)
```

How do we fix this one?

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	LD	ADD	XOR				
RF		LD	ADD	XOR			
ALU			LD	ADD	XOR		
WB				LD	ADD	XOR	

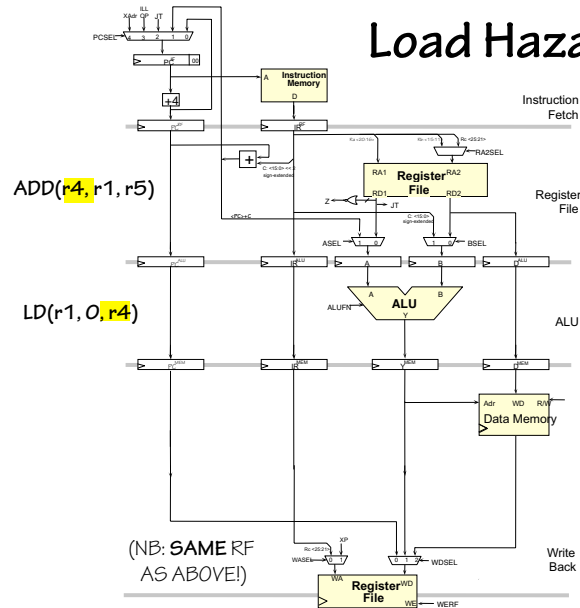
In a 4-stage pipeline, for a LD instruction fetched during clock i, the data from memory isn't returned from memory until late into cycle i+3. Bypassing can fix the XOR but not ADD!

6.004 – Spring 2009

5/5/09

L23 – Pipeline Issues 6

Load Hazard (hard)



6.004 – Spring 2009

5/5/09

L23 – Pipeline Issues 7

The r4 operand to the ADD instruction hasn't yet been fetched from memory.

It exists NOWHERE on our data paths – we can't solve this problem by bypassing!

Load Delay

Bypassing can't fix the problem with ADD since the data simply isn't available! We have to add some pipeline interlock hardware to stall ADD's execution.

```
LD(r1, 0, r4)
ADD(r1, r4, r5)
XOR(r3, r4, r6)
```

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	LD	ADD	XOR	XOR			
RF		LD	ADD	ADD	XOR		
ALU			LD	NOP	ADD	XOR	
WB				LD	NOP	ADD	XOR

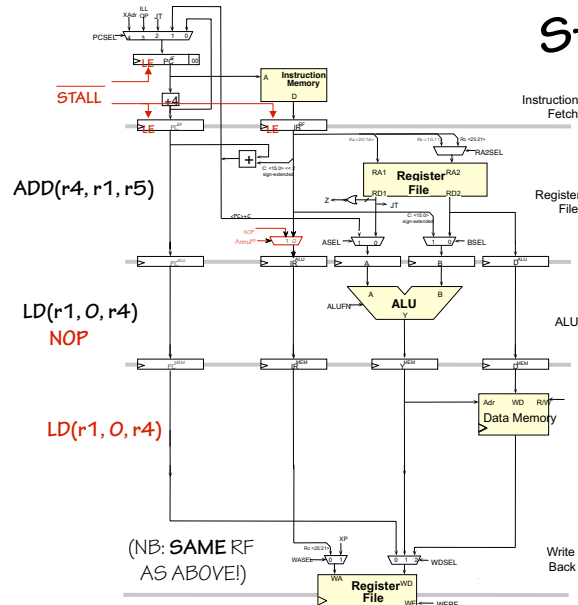
If the compiler knows about a machine's load delay, it can often rearrange code sequences to eliminate such hazards. Many compilers provide machine-specific *instruction scheduling*.

6.004 – Spring 2009

5/5/09

L23 – Pipeline Issues 8

Stall Logic



6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 9

- (1) Freeze IF, RF stages
- (2) Introduce NOP into ALU stage
- (3) Wait until operand is available

Memory Timing & Pipelining

But, but, what about FASTER processors?

FACT: Processors have become very fast relative to memories!
And this gap continues to grow...

Do we just increase the clock period to accommodate this bottleneck component?

ALTERNATIVE: Longer pipelines.

1. Add "MEMORY WAIT" stages between START of read operation & return of data.

2. Build pipelined memories, so that multiple (say, N) memory transactions can be in progress at once.

These steps add load delay slots; hence

3. Stall pipeline on unbypassable load delays.

A 4-Stage pipeline requires READ access in less than one clock.

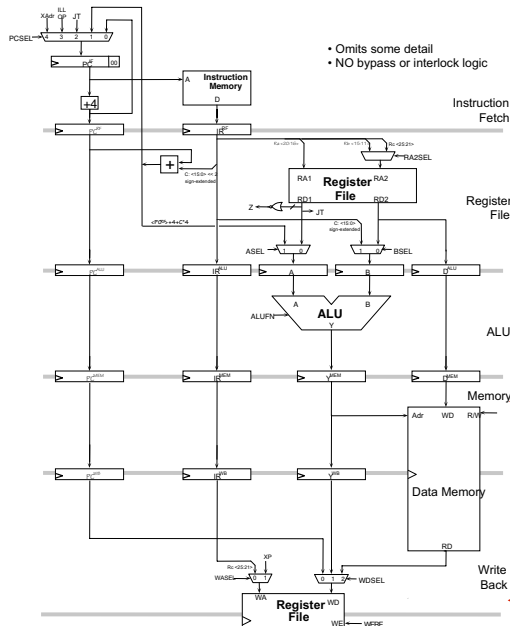
A 5-Stage pipeline would allow nearly two clocks...

6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 10

5-stage Pipeline



6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 11

Address available right after instruction enters Memory pipe stage

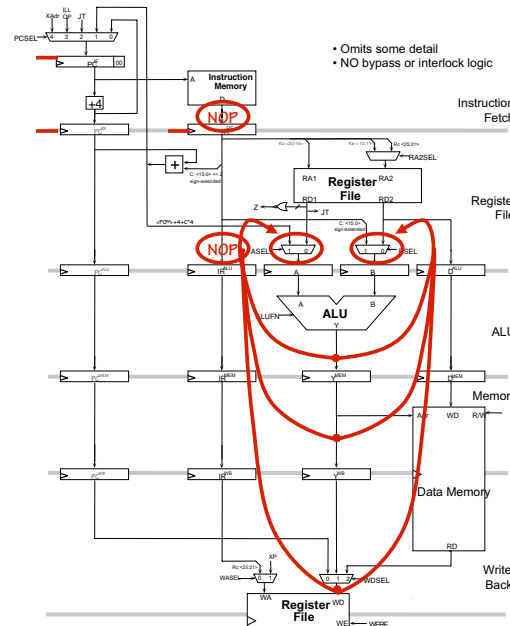
almost 2 clock cycles

Data needed right before rising clock edge at end of Write Back pipe stage

5-stage pipeline

We wanted a simple, clean pipeline but...

- added IR/IF mux to annul branch-slot instructions
- added A/B bypass muxes to get data before it's written to regfile
- added LE/muxes to freeze IF/RF stage so we can wait for LD to reach WB stage



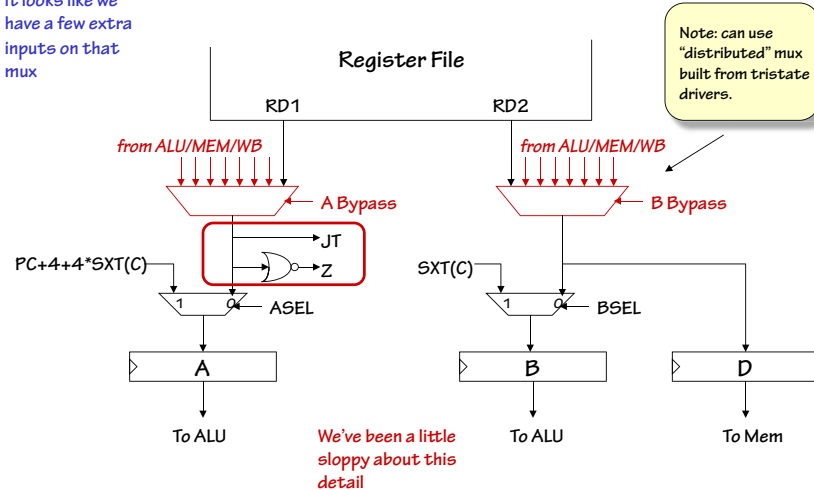
6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 12

RF-stage Bypass Details

1, 2, 3, 4... Hum,
it looks like we
have a few extra
inputs on that
mux

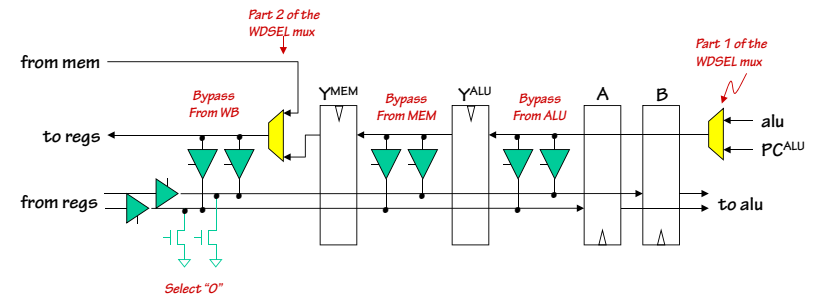


6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 13

Bypass Implementation



To reduce the amount of bypass logic, the WDSSEL mux has been split: choice between ALU and PC+4 is made in ALU stage, choice between ALU/PC and MEMDATA is made in WB stage.

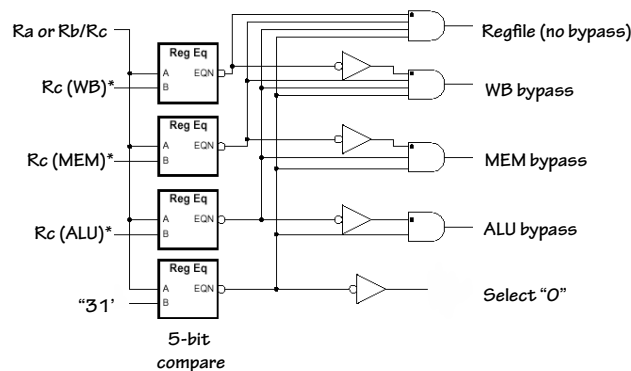
6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 14

Bypass Logic

Beta Bypass logic (need two copies for A/B data):



* If instruction is a ST (doesn't write into regfile), set RC for ALU/MEM/WB to R31

6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 15

Linkage Register Write Timing

The code: Assume Reg[LP] = 100...

ADD(r31, r31, LP)

BR(f, LP)

| Reg[LP] = .+4

x: SUB(LP, 1, LP)

...

f: XOR(LP, r31, r0)

OR(r31, LP, r1)

ADD(r31, LP, r2)

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	BR	SUB	XOR	OR	ADD	
RF		ADD	BR	NOP	XOR	OR	ADD
ALU			ADD	BR	NOP	XOR	OR
MEM				ADD	BR	NOP	XOR
WB					ADD	BR	NOP

BR Decision Time

ADD writes

BR writes

Can we make
XOR's regfile
access work
by bypassing?

6.004 - Spring 2009

5/5/09

L23 - Pipeline Issues 16

Unused Opcode Traps

IDEA: TRAP illegal instructions to a special routine in the Operating System, which can

- Interpret them in software; or
- Print humane error report.

| User program:

```
...
BAD(...) | Illegal instr.
r: ...
```

IMPLEMENTATION: On Bad Opcode (discovered in RF Stage):

- Select IllOp adr as next PC
- Annul instruction in IF stage
- Substitute BNE(r31,0,XP) for bad instruction - will (eventually) store PC+4 into XP ... need bypass paths to make XP usable immediately by code at IllOp!

| Operating System: UUO Handler

```
IllOp: ST(r0,...) | Save a reg,
LD(xp,-4,r0) | Fetch bad instr
...
LD(...,r0) | Restore regs,
JMP(xp) | Return to pgm.
```

6.004 - Spring 2009

5/5/09

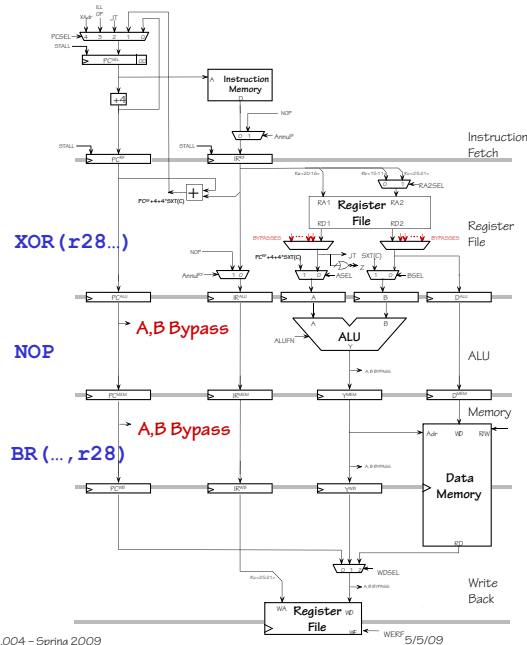
L23 - Pipeline Issues 18

BR/JMP PC bypass

For BR/JMP, R_c value is taken from PC, not ALU.

So we have to add bypass paths for PC^{ALU} and PC^{MEM}.

PC^{WB} is already taken care of if we bypass WB stage from output of WDSSEL mux.



6.004 - Spring 2009

L23 - Pipeline Issues 17

Taking Exception...

In general, we'd like to annul ALL instructions following one that causes a trap or fault:

FREEZE state at time of exception, for inspection by handler code.

ILLEGAL INSTRUCTIONS are recognizable in RF stage of pipe; are ALL faults & traps?

CONSIDER:

ARITHMETIC EXCEPTIONS: divide by zero, etc.

- Caught by ALU subsystem, during processing of data in ALU stage

MEMORY FAULTS: Program reference to illegal memory location...

- Caught by MEMORY subsystem, during processing of address input in MEM stage

6.004 - Spring 2009

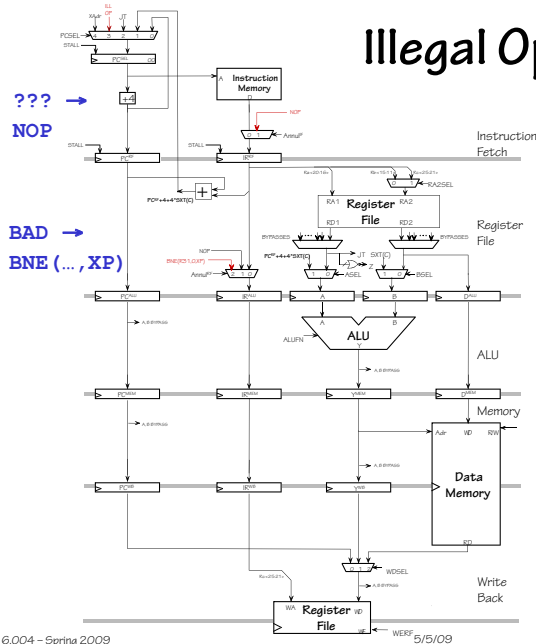
5/5/09

L23 - Pipeline Issues 20

Illegal Opcode Traps

Bad opcode decoded in RF stage:

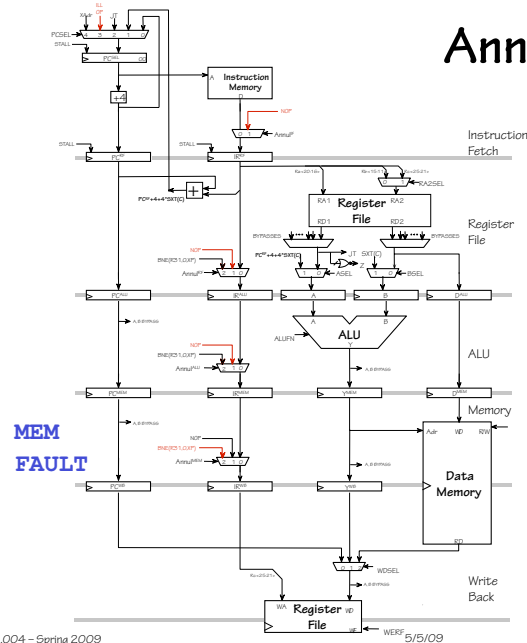
- PC ← address of IllOp handler
- Annul instruction in IF
- Force BNE(R31,0,XP) in RF stage → will save PC+4 in XP when it reaches WB stage



6.004 - Spring 2009

L23 - Pipeline Issues 19

Annulment Logic



Fault in ??? stage:

- $PC \leftarrow$ address of fault handler
- Force BNE(R31,O,XP) in ??? stage $\rightarrow$ will save PC+4 in XP when it reaches WB stage
- Annul all following instructions (those earlier in the pipeline): called "flushing the pipe"

Asynchronous I/O Interrupts

This should be easy.

Take, for example,

| The interrupted code:

```
...
ADD (...)
SUB (...)
MUL (...)
XOR (...)
...
```

*Interrupt
Taken
HERE*

| The interrupt handler:

```
xh:  OR (...)
...
    JMP (xp)
```

Suppose key struck, interrupt requested (via IRQ) during the fetch of ADD. Then let's

- Select XAdr (handler) as next PC
- Leave ADD in pipeline; NO annulment!
- Code handler to return to SUB instruction.

Can this work???

Let's find out...

Asynchronous Interrupt Timing

```
...
ADD (...)
SUB (...)
MUL (...)
XOR (...)
...
xh:  OR (...) | interrupt handler
...
    JMP (xp)
```

*Interrupt
Occurs PC = XAdr??*

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	OR	...	...			
RF		ADD	OR	...			
ALU			ADD	OR	...		
MEM				ADD	OR	...	
WB					ADD	OR	...

PROBLEM: When does old PC+4 get written to XP???

Making Interrupts Work

Alternative: When taking interrupt,

- ANNUL instruction in IF stage... BUT

instead of changing it to a NOP, change it to BNE(r31,O,XP)

This will cause PC+4 of annulled instruction to be written to XP!

- CODE HANDLER to return to Reg[XP]-4 (since the annulled instruction is never executed)

	i	i+1	i+2	i+3	i+4	i+5	i+6
IF	ADD	OR	...	...			
RF		BNE	OR	...			
ALU			BNE	OR	...		
MEM				BNE	OR	...	
WB					BNE	OR	

Interrupt taken

“Smart” Interrupt Handler

| The interrupted code:

```

...
ADD (...) ← Interrupt taken HERE,
SUB (...)   ADD instruction annulled
MUL (...)
XOR (...)

```

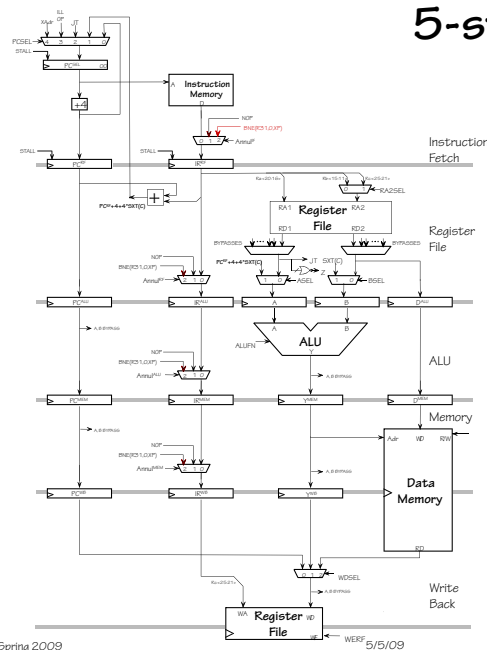
| The interrupt handler:

xh: OR (. . .)

```

...
SUBC(xp,4,xp)      | Adjust XP so
we
JMP(xp)            | return to annulled
                   | instruction (ADD)

```



5-stage Pipeline: Final Version

- Can annul instruction at each stage
- Can force instruction to BNE(R31,O,XP) in each stage → will save PC+4 in XP when it reaches WB stage
- Can stall IF and RF stages while waiting for LD result to reach WB
- Can bypass results from ALU, MEM and WB back to RF

Pipeline Review

Simple unpipelined Beta:

- 1 cycle/instruction
- long cycle time: mem+regs+alu+mem

2-Stage pipeline:

- increased throughput ($< 2x$)
- introduced branch delay slots
 - Choice of executing or annulling inst. after branch

5-stage pipeline:

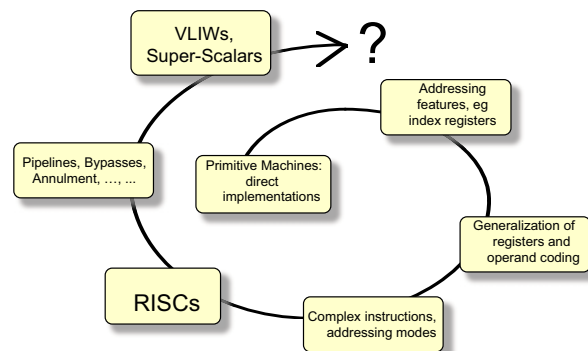
- increased throughput (3x???)
- branch delay slots
- delayed register writeback (3 cycles)
 - Add bypass paths (10) to access correct value

- memory data available only in WB stage
 - Introduce NOPs at IR^{ALU}, stall IF and RF stages until LD result ready
- handle RF/ALU/MEM stage exceptions
 - Save PC+4 in XP (fake a BR)
 - annul following insts. (those earlier in pipeline)
- implement interrupts
 - Throw away IF inst., save PC+4 in XP, fix return
- extra HW due to pipelining
 - Registers to hold values between stages
 - Data bypass muxes in RF stage
 - Inst. muxes for "rewriting" code to annul or save PC

RISC = Simplicity???

“The P.T. Barnum World’s Tallest Dwarf Competition”

World's Most Complex RISC?



MIT OpenCourseWare
<http://ocw.mit.edu>

6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Parallel Processing

The Home Stretch

TODAY 5/7: **Lab 8 (LAST!) due**

Friday 5/8 section: LAST QUIZ (#5)!

Tu 5/12: Wrapup (LAST!) Lecture!

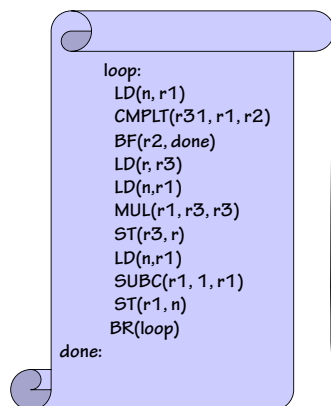
Wednesday 5/13:

NO SECTION MEETINGS!

- **Optional DESIGN PROJECT due**
- **ALL (late) Assignments due**
- Immense Satisfaction/Rejoicing/Relief/Celebration/Wild Partying.

Taking a step back

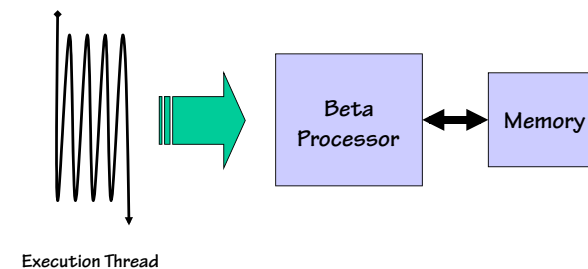
Static Code



Dynamic
Execution Path
aka "Thread of Execution"

Path Length = number of instructions along path

We have been building machines to execute
one thread (quickly)



$$\text{Time} = \frac{\text{Path Length} \times \text{Clocks-per-Instruction}}{\text{Clocks-per-second}}$$

Can we make $CPI < 1$?

...Implies we can complete
more than one instruction each clock cycle!

Two Places to Find Parallelism

Instruction Level (ILP) – Fetch and issue
groups of independent instructions within a
thread of execution

Thread Level (TLP) – Simultaneously execute multiple execution streams

Instruction-Level Parallelism

Sequential Code

```

loop:
LD(r, r1)
CMPLT(r31, r1, r2)
BF(r2, done)
LD(r, r3)
LD(n, r1)
MUL(r1, r3, r3)
ST(r3, r)
LD(n, r4)
SUBC(r4, 1, r4)
ST(r4, n)
BR(loop)
done:

```

“Safe” Parallel Code

loop:
~~LD(r3, r1)~~
~~CMPLT(r3, r1, r2)~~
~~BF(r2, done)~~
 LD(r3, r3) ~~LD(n, r1)~~ ~~LD(n, r4)~~
 MUL(r1, r3, r3) ~~SUBC(r4, 1, r4)~~
 ST(r3, r) ST(r4, n) BR(loop)

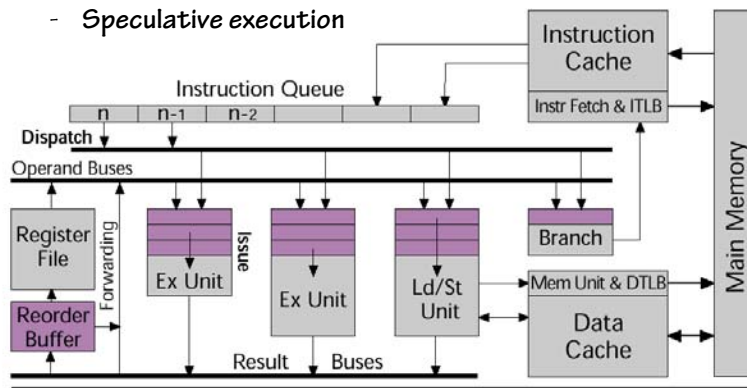
done:

What if I tried to
 do multiple
 iterations at
 once?

This is okay, but smarter coding does better in this example!

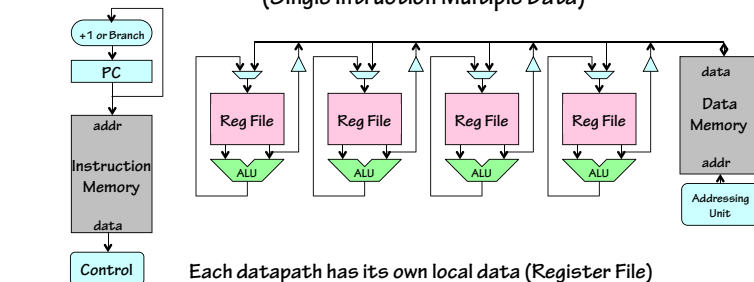
Superscalar Parallelism

- Popular now, but the limits are near (8-issue)
- Multiple instruction dispatch
- Speculative execution



SIMD Processing

(Single Instruction Multiple Data)



This sort of construct is also becoming popular on modern uniprocessors

Each datapath has its own local data (Register File)

All data paths execute the same instruction

Conditional branching is difficult...

(What if only one CPU has $R1 = 0$?)

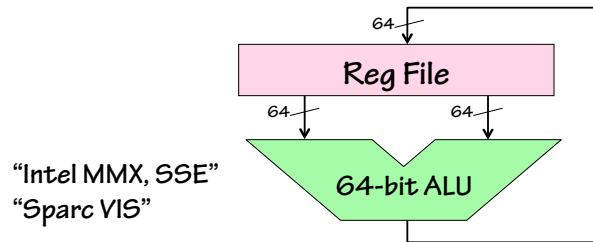
Conditional operations are common in SIMD machines

if (flag1) $R_c = R_a \text{ <op> } R_b$

Global ANDing or ORing of flag registers are used for high-level control

Model: “hide” parallelism in primitives (eg, vector operations)

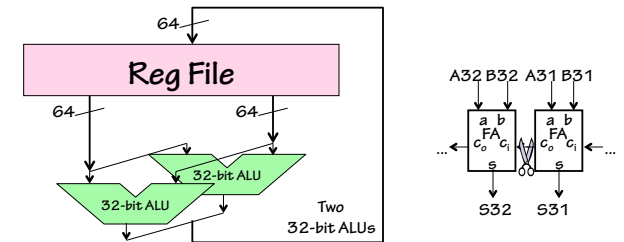
SIMD Coprocessing Units



"Intel MMX, SSE"
"Sparc VIS"

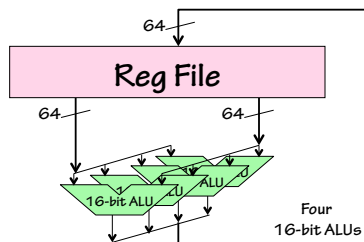
SIMD data path added to a traditional CPU core
Register-only operands
Core CPU handles memory traffic
Partitionable Datapaths for variable-sized
"PACKED OPERANDS"

SIMD Coprocessing Units



SIMD data path added to a traditional CPU core
Register-only operands
Core CPU handles memory traffic
Partitionable Datapaths for variable-sized
"PACKED OPERANDS"

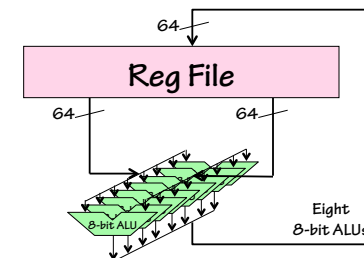
SIMD Coprocessing Units



Nice data size for:
Graphics,
Signal Processing,
Multimedia Apps,
etc.

SIMD data path added to a traditional CPU core
Register-only operands
Core CPU manages memory traffic
Partitionable Datapaths for variable-sized
"PACKED OPERANDS"

SIMD Coprocessing Units



MMX instructions:
PADDB - add bytes
PADDD - add 32-bit words
PSUB{B,W,D} - subtract
PMULTLW - multiply low
PMULTHW - multiply high
PMADDW - multiply & add
PACK -
UNPACK -
PAND -
PORD -

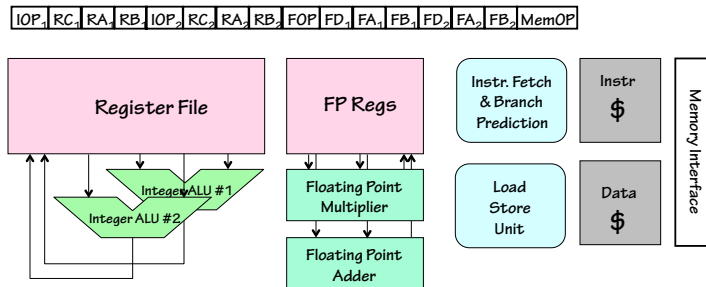
SIMD data path added to a traditional CPU core
Register-only operands
Core CPU manages memory traffic
Partitionable Datapaths for variable-sized
"PACKED OPERANDS"

VLW Variant of SIMD Parallelism

(Very Long Instruction Word)

A single-WIDE instruction controls multiple heterogeneous datapaths.

Exposes parallelism to compiler (S/W vs. H/W)



6.004 Spring 2009

5/7/09

L24-- Parallel Processing 13

Multiple Instruction Streams: MIMD

Exploiting Thread Level Parallelism

All processors share a common main memory

Leverages existing CPU designs

Easy to map "Processes (threads)" to "Processors"

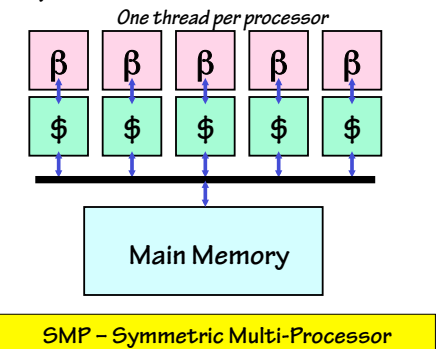
Share data and program

Communicate through shared memory

Upgradeable

Problems:

Scalability
Synchronization

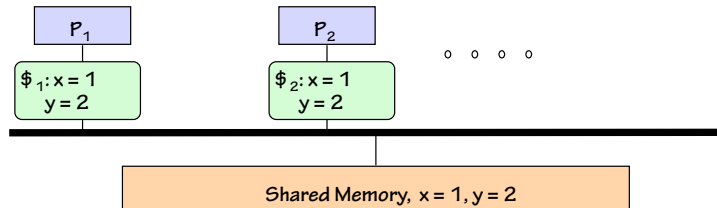


6.004 Spring 2009

5/7/09

L24-- Parallel Processing 14

Hmmm....does it even work?



Consider the following trivial processes running on P_1 and P_2 :

Process A

$x = 3;$
 $\text{print}(y);$

Process B

$y = 4;$
 $\text{print}(x);$

What are the Possible Outcomes?

Process A

$x = 3;$
 $\text{print}(y);$

$\$1: x = 1, 3$
 $y = 2$

Process B

$y = 4;$
 $\text{print}(x);$

$\$2: x = 1$
 $y = 2, 4$

Plausible execution sequences:

SEQUENCE	A prints	B prints
$x=3; \text{print}(y); y=4; \text{print}(x);$	2	1
$x=3; y=4; \text{print}(y); \text{print}(x);$	2	1
$x=3; y=4; \text{print}(x); \text{print}(y);$	2	1
$y=4; x=3; \text{print}(x); \text{print}(y);$	2	1
$y=4; x=3; \text{print}(y); \text{print}(x);$	2	1
$y=4; \text{print}(x); x=3; \text{print}(y);$	2	1

Hey, we get the same answer every time... Let's go build it!

6.004 Spring 2009

5/7/09

L24-- Parallel Processing 15

6.004 Spring 2009

5/7/09

L24-- Parallel Processing 16

Uniprocessor Outcome

But, what are the possible outcomes if we ran Process A and Process B on a **single timed-shared processor**?

Process A

```
x = 3;
print(y);
```

Process B

```
y = 4;
print(x);
```

Plausible Uniprocessor execution sequences:

SEQUENCE	A prints	B prints
x=3; print(y); y=4; print(x);	2	3
x=3; y=4; print(y); print(x);	4	3
x=3; y=4; print(x); print(y);	4	3
y=4; x=3; print(x); print(y);	4	3
y=4; x=3; print(y); print(x);	4	3
y=4; print(x); x=3; print(y);	4	1

Notice that the outcome 2, 1 does not appear in this list!

Sequential Consistency

Semantic constraint:

Result of executing N parallel programs should correspond to *some* interleaved execution on a single processor.

Shared Memory

```
int x=1, y=2;
```

Process A

```
x = 3;
print(y);
```

Process B

```
y = 4;
print(x);
```

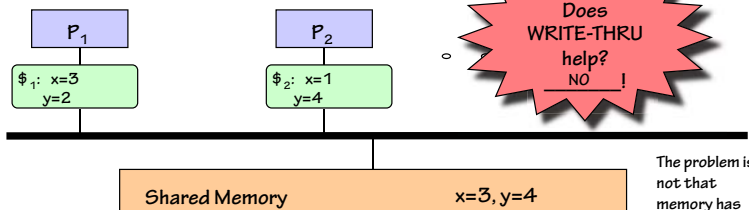
Weren't caches supposed to be invisible to programs?

Possible printed values: 2, 3; 4, 3; 4, 1.
(each corresponds to at least one interleaved execution)

IMPOSSIBLE printed values: 2, 1
(corresponds to NO valid interleaved execution).

Cache Incoherence

PROBLEM: "stale" values in cache ...



Process A

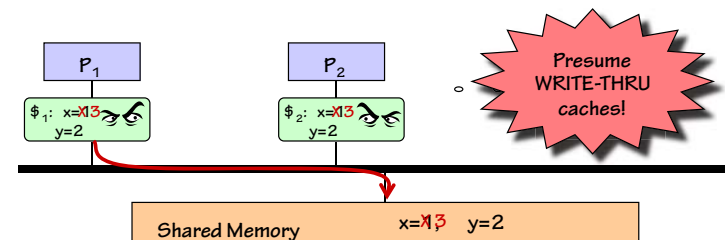
```
x = 3;
print(y);
```

Process B

```
y = 4;
print(x);
```

Q: How does B know that A has changed the value of x?

"Snoopy" Caches



IDEA:

- P₁ writes 3 into x; write-thru cache causes bus transaction.
- P₂, snooping, sees transaction on bus. INVALIDATES or UPDATES its cached x value.

Snoopy Cache Design

Two-bit STATE in cache line encodes one of M, E, S, I states ("MESI" cache):

INVALID: cache line unused.

SHARED ACCESS: read-only, valid, not dirty. Shared with other read-only copies elsewhere. Must invalidate other copies before writing.

EXCLUSIVE: exclusive copy, not dirty. On write becomes modified.

MODIFIED: exclusive access; read-write, valid, dirty. Must be written back to memory eventually; meanwhile, can be written or read by local processor.

Current state	Read Hit	Read Miss, Snoop Hit	Read Miss, Snoop Miss	Write Hit	Write Miss	Snoop for Read	Snoop for Write
Modified	Modified	Invalid (Wr-Back)	Invalid (Wr-Back)	Modified	Invalid (Wr-Back)	Shared (Push)	Invalid (Push)
Exclusive	Exclusive	Invalid	Invalid	Modified	Invalid	Shared	Invalid
Shared	Shared	Invalid	Invalid	Modified (Invalidate)	Invalid	Shared	Invalid
Invalid	X	Shared (Fill)	Exclusive (Fill)	X	Modified (Fill-Inv)	X	X



Who needs Sequential Consistency, anyway?

ALTERNATIVE MEMORY SEMANTICS:

"WEAK" consistency

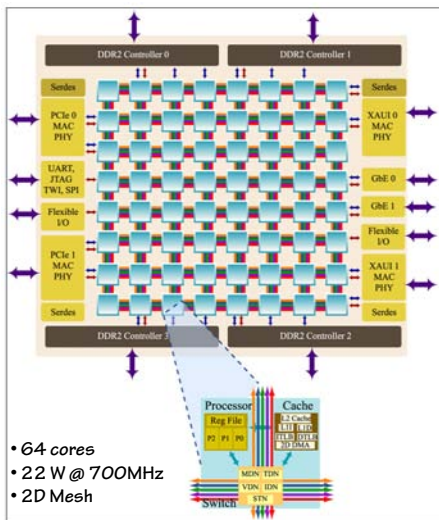
EASIER GOAL: Memory operations from each processor appear to be performed in order issued by that processor;

Memory operations from different processors may overlap in arbitrary ways (not necessarily consistent with any interleaving).

ALTERNATIVE APPROACH:

- Weak consistency, by default;
- MEMORY BARRIER instruction: stalls processor until all previous memory operations have completed.

MIMD Multicore Arrays



- 64 cores
- 22 W @ 700MHz
- 2D Mesh

- Can Leverage existing CPU designs / development tools
- H/W focuses on communication 2-D Mesh / cache hierarchy/ ...)
- S/W focuses on partitioning, extracting parallelism
- "speculative execution" hacks

- 16 cores/32 thr
- 250W @ 2.3GHz
- Transactional Mem
- Thread Speculation
- "scout" threads

<http://www.tilera.com/>
Figure by MITOpenCourseWare.

Parallel Processing Summary

Prospects for future CPU architectures:

Pipelining - Well understood, but mined-out

Superscalar - At its practical limits

SIMD - Limited use for special applications

VLIW - Returns controls to S/W... but inflexible

Prospects for future Computer System architectures:

Single-thread limits: forcing multicores, parallelism

Brains work well, with dismal clock rates ... parallelism?

Needed: NEW models, NEW ideas, NEW approaches

FINAL ANSWER: Its up to YOUR generation!

MIT OpenCourseWare
<http://ocw.mit.edu>

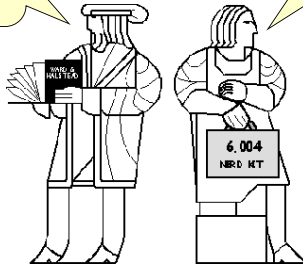
6.004 Computation Structures
Spring 2009

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

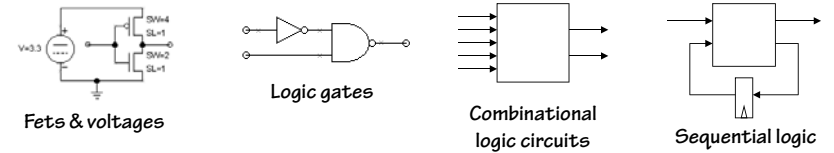
Computer Architecture: Exciting Times Ahead!

Prediction is very
difficult, especially
about the future.
-- Neils Bohr

The best way to
predict the future is
to invent it.
-- Alan Kay



You've mastered a lot...

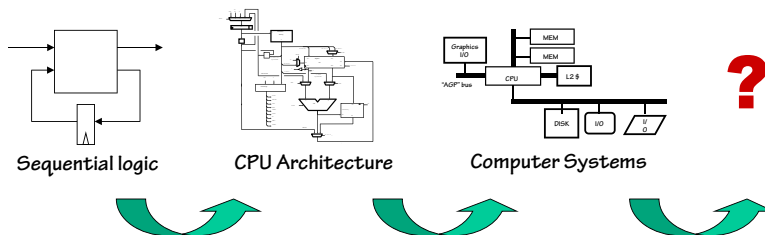


Combinational contract:
♦ discrete-valued inputs
♦ complete in/out spec.
♦ static discipline

Acyclic connections
Summary specification
Design:
♦ sum-of-products
♦ simplification
♦ muxes, ROMs, PLAs

Storage & state
Dynamic discipline
Finite-state machines
Metastability
Throughput & latency
Pipelining

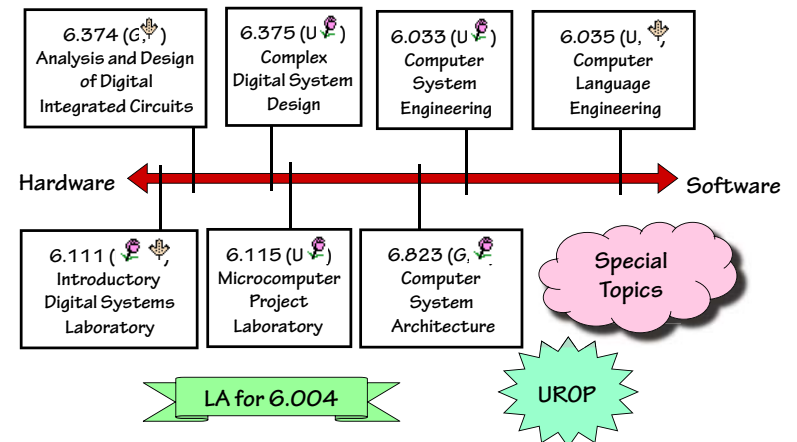
... a WHOLE lot ...



Computing Theory
Instruction Set Architectures
Beta implementation
Pipelined Beta
Software conventions
Memory architectures

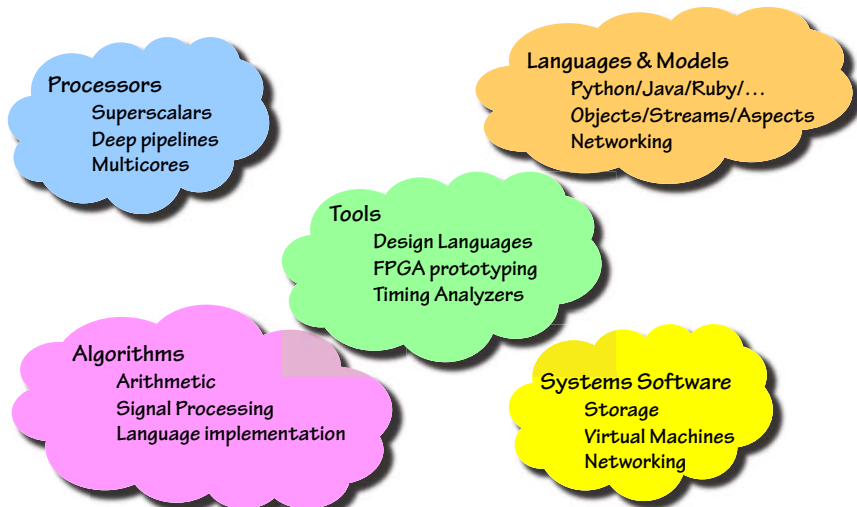
Interconnect
Virtual machines
Interprocess communication
Operating Systems
Real time, Interrupts
Parallel Processing

What's next? Some follow-on options...



Things to look forward to...

6.004 is only an appetizer!



Verilog example: Beta Register File

```

// 2-read, 1-write 32-location register file
module regfile(ral,rd1,ra2,rd2,clk,werf,wa,wd);
    input [4:0] ral;           // address for read port 1 (Reg[RA])
    output [31:0] rd1;         // read data for port 1
    input [4:0] ra2;           // address for read port 2 (Reg[RB], Reg[RC] for ST)
    output [31:0] rd2;         // read data for port 2
    input clk;
    input werf;                // write enable, active high
    input [4:0] wa;             // address for write port (Reg[RC])
    input [31:0] wd;            // write data

    reg [31:0] registers[31:0]; // the register file itself (local)

    // read paths are combinational, check for reads from R31
    assign rd1 = (ral == 31) ? 0 : registers[ral];
    assign rd2 = (ra2 == 31) ? 0 : registers[ra2];

    // write port is active only when WERF is asserted
    always @(posedge clk)
        if (werf) registers[wa] <= wd;
endmodule
  
```

```

module pc(clk,reset,pcsel,offset,jump_addr,
          branch_addr,pc,pc_plus_4);
    input clk;
    input reset;                // forces PC to 0x80000000
    input [2:0] pcsel;           // selects source of next PC
    input [15:0] offset;         // inst[15:0]
    input [31:0] jump_addr;      // from Reg[RA], used in JMP instruction
    output [31:0] branch_addr;   // send to datapath for LDR instruction
    output [31:0] pc;            // used as address for instruction fetch
    output [31:0] pc_plus_4;     // saved in regfile during branches, JMP, traps

    reg [31:0] pc;
    wire [30:0] pcinc;
    wire [31:0] npc;

    // the Beta PC increments by 4, but won't change supervisor bit
    assign pcinc = pc + 4;
    assign pc_plus_4 = {pc[31],pcinc};

    // branch address = PC + 4 + 4*sxt(offset)
    assign branch_addr = {0,pcinc + {{13{offset[15]}},offset[15:0],2'b00}};

    assign npc = reset ? 32'h80000000 :
        (pcsel == 0) ? {pc[31],pcinc} : // normal
        (pcsel == 1) ? {pc[31],branch_addr[30:0]} : // branch
        (pcsel == 2) ? {pc[31] & jump_addr[31],jump_addr[30:0]} : // jump
        (pcsel == 3) ? 32'h80000004 : 32'h80000008; // illop, trap

    // pc register, pc[31] is supervisor bit and gets special treatment
    always @(posedge clk) pc <= npc;
endmodule
  
```

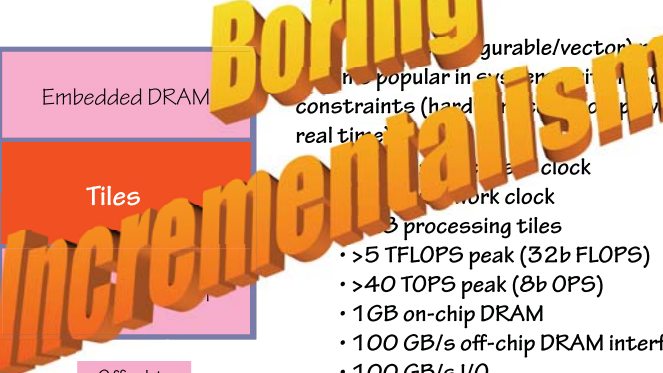
PC

The Crystal Ball

some trends in computer evolution

- Technology shrinks
 - 30% linear shrink/generation
 - Cheaper, faster, lower power
- Multicores (SMP, Tiled NUMA, ...)
- Superscalar/SMT pipelines
- Power management
- Reconfigurable processing/interconnect
- VLIW, SIMD influences

where software is the main expense



Boring Incrementalisms

- Configurable/vectorizable
- Not popular in systems with hard constraints (hard real time, copy to hard real time)
- 100 MHz clock
- 100 MHz work clock
- 3 processing tiles
- >5 TFLOPS peak (32b FLOPS)
- >40 TOPS peak (8b OPS)
- 1GB on-chip DRAM
- 100 GB/s off-chip DRAM interface
- 100 GB/s I/O
- $25 \times 25 \text{ mm}^2$ in $0.045 \mu\text{m}$ CMOS

L25 - Wrapup Lecture 13

Will computers always look
and operate the way
computers do today?

- Well-defined system “state”
- Programming
- Silicon-based logic
- Logic at all

Synchronous Clocked Systems

MOSFET transistors

Boolean Logic

L25 - Wrapup Lecture 14

Is there an engineering discipline for building goal-oriented systems from goal-oriented components?

- ▶ Adaptive
- ▶ Memory

L25 – Wrapup Lecture 15

- 1) The most reliable, sustainable, efficient, and smartest??? machines that we know of are biological
- 2) Fined tuned through millions of years of evolution
- 3) The assembly, repair, and operation “instructions” for multi-billion element machines are “digitally” encoded in a single molecule
- 4) We are just beginning to understand the “gates” and the “machine language”

L25 – Wrapup Lecture 16

DNA Chips

(DNA probes or microarrays)

Leverages VLSI fabrication techniques
(photolithography)

Use PCRs (polymerase chain reactions) to
make an exponential number of DNA
copies

Mechanically bind specific “tagged” gene
sequences onto a patterned substrate

Expose to bath of denatured nucleotides
(separated and diced up pieces of DNA)

Look for phosphorescent markers

Medical applications are obvious, but what
does it have to do with computation?

We can reliably reslice and
recombine (state machines?)

Questions:
What inputs satisfy
 $f(x_1, x_2, \dots, x_N) = 1$.

6.004 – Spring 2009

5/12/09

L25 – Wrapup Lecture 17

Can we Program Microbes?

DNA = program

Protein synthesis = gates?

$$F(n) = n * F(n-1); \\ F(0) = 1$$

Can we “engineer” organisms
to perform computations
for us?

Can we make a “standard cell library” offering digital
building blocks from DNA sequences?

This is alien thinking for biologist, but standard fare
for systems designers

6.004 – Spring 2009

5/12/09

L25 – Wrapup Lecture 18

Computing at the limit

At the particle level nature behaves very strangely...

Far separated particles can be entangled

- electron spins
- photon polarizations
- magnetic fields

They can be simultaneously in either state
(so long as you don't look).

The act of looking at them (measuring, or observing them)
forces the entangled particle into one of its states.

Strangely enough, it is believed that we can use such
entangled particles in computations w/o disturbing them.

6.004 – Spring 2009

5/12/09

L25 – Wrapup Lecture 19

Quantum Computing?

Classic computers perform operations on strings of bits (0s
and 1s).

A quantum computer would be able to compute on bits
(qubits) that can be simultaneously in either state.

Classic computer:
(with a dumb algorithm)
Search through all 2^{20}
permutations

$$F(0 < x < 2^{20}) = x * 371$$

Quantum computer:
Insert 20 qubits, select
the desired answer, then
look back and see what
the qubits resolved to...

$$F(?) = 197001$$

6.004 – Spring 2009

5/12/09

L25 – Wrapup Lecture 20

The Dilemma

- We have no clue how to build a practical quantum computer
- Currently, quantum computing is merely a fantasy of theoreticians
- What other problems can a quantum computer solve more efficiently than a classic computer?

A SUBTLE Reminder:

Turing, Church, Post,
Kleene, and Markov
really “invented” most
of modern day computer
science long before a
“practical” implementation.

6004: The Big Lesson

You’ve built, debugged, understood a complex
computer from FETs to OS... what have you
learned?

Engineering Abstractions:

- Understanding of their technical underpinnings
- Respect for their value
- Techniques for using them

But, most importantly:

- The self assurance to discard them, in favor of new abstractions!

Good engineers use abstractions;

GREAT engineers create them!

THE END!

Pens, pencils, paper
they attempt to solve problems
that teachers set forth.

The only problem
with Haiku is that you just
get started and then

